

Machine Learning Using Python

Foundations, Algorithms, Applications, and Reproducible Research

Itauma Itauma

2026-08-28

Table of contents

Preface	30
About This Book	30
Why a Second Edition?	30
What Is New?	31
From Learning Algorithms to Doing Data Science	31
A Philosophy of Machine Learning	31
Machine Learning in the Age of Generative AI	32
From Learner to Practitioner and Researcher	32
Acknowledgments	32
Edition Statement	33
 How to Use This Book	 34
Who This Book Is For	34
What You Should Know Before Beginning	34
The Architecture of the Book	34
Four Reading Pathways	35
Foundational Path	35
Practitioner Path	35
Researcher Path	35
Modern AI Path	35
Three Application Tracks	36
Health	36
Education	36
Open ML	36
A Longitudinal Computer Vision Thread	36
From Algorithm to Research Question	37
How Chapters Are Designed	37
Notation and Conventions	37
Software and Computational Environment	38
Using Generative AI While Learning	38
Reproducibility From the Beginning	38
Teaching and Self-Study Pathways	38
A Final Orientation	39

I	Part I — Foundations	40
1	Machine Learning in the Age of Data and AI	41
1.1	Learning Objectives	41
1.2	Why This Matters	41
1.3	A Conceptual Map of Modern Machine Learning	41
1.4	What Does It Mean for a Machine to Learn?	42
1.5	The Vocabulary of an ML Problem	43
1.6	Ways Machine Learning Systems Learn	43
1.6.1	Supervised Learning	43
1.6.2	Unsupervised Learning	44
1.6.3	Semi-Supervised Learning	44
1.6.4	Self-Supervised Learning	44
1.6.5	Reinforcement Learning	44
1.7	Machine Learning Workloads: Where Computer Vision Fits	44
1.8	Why Generalization Matters	45
1.8.1	A First Generalization Experiment	45
1.9	Statistics You Need	47
1.10	Prediction, Explanation, Inference, and Causation	48
1.11	Three Learning and Research Tracks	48
1.11.1	Health Track — NHANES	48
1.11.2	Education Track — IPEDS and College Scorecard	48
1.11.3	Open ML Benchmark Track	48
1.12	A Disciplined Machine Learning Workflow	49
1.13	Try It	49
1.14	Hands-On Lab	49
1.14.1	Computer Vision Preview	50
1.15	Research Studio	50
1.16	AI Pair Programmer	50
1.17	Responsible Machine Learning Begins Before Modeling	50
1.18	From Model Fitting to Research	51
1.18.1	A Research Ladder	51
1.19	Practitioner Track	51
1.20	Researcher Track	51
1.21	Knowledge Check	52
1.22	Professor’s Challenge	52
1.23	Reproducibility Check	52
1.24	Key Takeaways	52
1.25	Looking Ahead	53
2	Python for Machine Learning	54
2.1	Learning Objectives	54

2.2	Why This Matters	54
2.2.1	Connection to Chapter 1	55
2.3	Why Python?	55
2.4	Your Machine Learning Workspace	55
2.5	Reproducible Python Environments	56
2.6	Python You Actually Need	57
2.6.1	Variables and Objects	57
2.6.2	Lists and Dictionaries	57
2.6.3	Conditions	57
2.6.4	Functions	58
2.7	NumPy: Thinking in Arrays	58
2.8	Images Are Arrays Too	59
2.9	pandas: Working with Tabular Data	60
2.9.1	Selecting and Filtering	62
2.10	From a DataFrame to (X) and (y)	62
2.11	Data Types Are Analytical Decisions	63
2.12	Missing Data	63
2.13	Exploratory Visualization	63
2.14	Data Provenance	64
2.15	Three Data Tracks in Practice	65
2.15.1	Open ML Benchmark Track	65
2.15.2	Health Track	66
2.15.3	Education Track	66
2.16	Beyond the Algorithm: Code as Evidence	66
2.17	Writing Reusable Analytical Code	66
2.18	Debugging: Read the Evidence	67
2.19	Git and GitHub	67
2.20	Quarto: Narrative + Computation + Evidence	68
2.21	Try It	68
2.22	Hands-On Lab: Build a Reproducible Mini-Project	68
2.23	AI Pair Programmer	69
2.24	Practitioner Track	69
2.25	Researcher Track	69
2.26	Professor's Challenge: Audit the Analysis	69
2.27	Research Studio	70
2.28	Knowledge Check	70
2.29	Reproducibility Check	71
2.30	Key Takeaways	71
3	Statistics and Mathematics for Machine Learning	72
3.1	Learning Objectives	72
3.2	Why This Matters	72
3.3	From Data to Mathematical Representation	73

3.4	Describing Data: Center and Spread	73
3.4.1	Mean	74
3.4.2	Variance and Standard Deviation	74
3.5	Visual Intuition: Distribution and Variation	75
3.6	Probability: A Language for Uncertainty	76
3.6.1	Expected Value	77
3.7	Probability Distributions	77
3.7.1	A Normal Distribution in Python	77
3.8	Population, Sample, Parameter, Statistic	78
3.8.1	Sampling Variation by Simulation	78
3.9	Standard Error and Uncertainty	79
3.10	Prediction, Inference, and Different Kinds of Questions	80
3.10.1	Predictive Signal Without a Unique Explanation	80
3.11	Covariance and Correlation	83
3.12	Standardization: Putting Features on Comparable Scales	84
3.13	Linear Algebra: The Language of Many Models	85
3.13.1	Vectors	85
3.13.2	Matrix-Vector Multiplication	86
3.14	Distance and Similarity	86
3.14.1	Why Scaling Changes Distance	87
3.15	Functions, Loss, and Learning	87
3.16	Derivatives: Measuring Local Change	88
3.17	Gradients: Change in Multiple Dimensions	88
3.18	Gradient Descent	89
3.18.1	Watch Optimization Happen	89
3.18.2	Learning Rate	90
3.19	From Intuition to Equation to Python to ML	90
3.20	Beyond the Algorithm: The Map Is Not the Territory	91
3.21	Three Tracks: Same Mathematics, Different Meaning	92
3.21.1	Health Track	92
3.21.2	Education Track	92
3.21.3	Open ML Benchmark Track	92
3.22	Try It	92
3.23	Hands-On Lab: Mathematics You Can See	93
3.24	AI Pair Programmer	93
3.25	Practitioner Track	94
3.26	Researcher Track	94
3.27	Professor's Challenge: When Mathematics Misleads	94
3.28	Research Studio	95
3.29	Knowledge Check	95
3.30	Reproducibility Check	96
3.31	Key Takeaways	96
3.32	Looking Ahead	97

II Part II — The Machine Learning Workflow 98

4 Framing Machine Learning Problems	99
4.1 Learning Objectives	99
4.2 Why This Matters	99
4.2.1 Connection to the Foundations	99
4.3 Start With the Problem, Not the Algorithm	100
4.4 A Machine Learning Problem Specification	100
4.5 Unit of Analysis	100
4.5.1 Try It: Find the Unit	101
4.6 Defining the Target	101
4.7 From Construct to Operational Target	101
4.7.1 Three Tracks: Same Problem, Different Proxy	102
4.7.2 Construct Validity as an ML Design Problem	102
4.7.3 Try It: Audit the Target	103
4.8 Prediction Time and Horizon	103
4.8.1 The Availability Test	103
4.9 Leakage Begins With Framing	104
4.10 Prediction Is Not Intervention	104
4.11 Different Analytical Purposes	104
4.12 Computer Vision Is a Workload, Not a Problem Specification	104
4.13 Baselines: What Must the Model Beat?	105
4.14 Error Costs and Thresholds	106
4.15 Evaluation Must Match Intended Use	106
4.16 The Problem Framing Canvas	106
4.17 Three Tracks	107
4.17.1 Health	107
4.17.2 Education	107
4.17.3 Open Benchmark	107
4.18 A Small Framing Experiment	108
4.19 Beyond the Algorithm: What Should We Choose to Predict?	109
4.20 Responsible Framing	109
4.21 AI Pair Programmer	109
4.22 Hands-On Lab: Build an ML Problem Specification	110
4.23 Practitioner Track	110
4.24 Researcher Track	110
4.25 Research Studio: From Topic to Research Question	110
4.26 Professor's Challenge: The 95% Accurate Model	111
4.27 Knowledge Check	111
4.28 Reproducibility Check	111
4.29 Key Takeaways	112

5	Data Preparation and Feature Engineering	113
5.1	Learning Objectives	113
5.2	Why This Matters	113
5.3	From Problem Specification to Data Pipeline	114
5.4	Preserve Raw Data	114
5.5	Inspect Before Cleaning	114
5.6	Data Quality Is Contextual	116
5.7	Data-Centric Machine Learning	116
5.7.1	Dimensions of Data Quality	117
5.7.2	Small Experiment: Better Model or Better Data?	118
5.8	Missing Data	120
5.8.1	Why <code>dropna()</code> Is Not a Strategy	120
5.8.2	Imputation	121
5.9	The Evaluation Boundary	121
5.10	Train/Test Splitting	122
5.11	Numerical Features	123
5.12	Categorical Features	123
5.12.1	One-Hot Encoding	123
5.12.2	Ordinal Variables	124
5.13	Identifiers Are Not Ordinary Features	124
5.14	Datetime Features	124
5.15	Feature Engineering: Representation as Modeling	124
5.16	Computer Vision Data: Representation Is Preprocessing	125
5.16.1	Flattening Changes the Representation	125
5.16.2	Vision Leakage Can Be Subtle	126
5.16.3	Augmentation Belongs Inside the Training Process	126
5.17	Interactions and Nonlinearity	127
5.18	Outliers: Error, Rarity, or Signal?	127
5.19	ColumnTransformer: Different Columns, Different Transformations	127
5.20	Pipeline: Preprocessing and Model as One Workflow	128
5.21	Visual Experiment: Leakage Inflates Performance	129
5.22	Three Data Tracks	131
5.22.1	Health Track	131
5.22.2	Education Track	131
5.22.3	Open ML Benchmark Track	131
5.23	Beyond the Algorithm: Data Are Made, Not Merely Found	131
5.24	AI Pair Programmer: Audit the Preprocessing	132
5.25	Hands-On Lab: Build a Leakage-Safe Pipeline	132
5.25.1	Computer Vision Extension	133
5.26	Practitioner Track	133
5.27	Researcher Track	133
5.28	Research Studio: Build a Data Dictionary and Feature Ledger	133
5.29	Professor's Challenge: The Perfect Preprocessing Notebook	134

5.30 Knowledge Check	135
5.31 Reproducibility Check	135
5.32 Key Takeaways	135
5.33 Looking Ahead	136

III Part III — Supervised Learning 137

6 Linear Regression: From Relationship to Prediction 138

6.1 Learning Objectives	138
6.2 Why This Matters	138
6.2.1 Connection to the Workflow	139
6.3 The Simple Linear Model	139
6.4 See the Line	139
6.5 Ordinary Least Squares	141
6.5.1 Why the Estimate Is a Minimizer	141
6.6 Matrix Form	141
6.7 Multiple Linear Regression	142
6.8 A Complete Predictive Workflow	142
6.9 Regression Metrics	143
6.9.1 Mean Absolute Error	143
6.9.2 Root Mean Squared Error	143
6.9.3 Coefficient of Determination	143
6.10 Always Compare With a Baseline	143
6.11 Residuals: What the Model Failed to Explain	144
6.12 Statistical Assumptions and Predictive Goals	145
6.13 Inference With statsmodels	146
6.14 Same Model, Different Question	147
6.14.1 Weak Predictive Gain Despite a Real Relationship	147
6.15 Nonlinearity	150
6.16 Interactions	150
6.17 Multicollinearity	151
6.18 Ridge Regression	151
6.19 Lasso Regression	151
6.20 Why Scaling Matters for Regularization	152
6.21 Bias–Variance Intuition	152
6.22 Visual Experiment: Coefficient Shrinkage	152
6.23 Coefficients Are Not Automatically Causes	154
6.24 Computer Vision Connection: Regression From Images	155
6.25 Three Tracks	155
6.25.1 Health Track	155
6.25.2 Education Track	156
6.25.3 Open ML Benchmark Track	156

6.26	Beyond the Algorithm: Is Simplicity a Limitation or a Strength?	156
6.27	AI Pair Programmer	156
6.28	Hands-On Lab: OLS to Regularization	157
6.29	Practitioner Track	157
6.30	Researcher Track	157
6.31	Research Studio	158
6.32	Professor's Challenge: The Significant Model	158
6.33	Knowledge Check	159
6.34	Reproducibility Check	159
6.35	Key Takeaways	160
7	Classification: From Probability to Decision	161
7.1	Learning Objectives	161
7.2	Why This Matters	161
7.3	Why Not Ordinary Linear Regression?	162
7.4	Visual Intuition: Logistic Function	162
7.5	Coefficients and Odds Ratios	163
7.6	Leakage-Safe Logistic Regression	163
7.7	Probability Is Not Yet a Decision	164
7.8	Confusion Matrix	164
7.9	Core Metrics	165
7.10	Prevalence Changes Meaning	166
7.11	Thresholds Create Tradeoffs	166
7.12	ROC Analysis	167
7.13	Precision-Recall Analysis	169
7.14	Discrimination Is Not Calibration	170
7.15	From Probability to Decision	171
7.15.1	Why 0.50 Is Not Universal	172
7.15.2	Threshold Choice Under Asymmetric Costs	172
7.15.3	Computer Vision Learning Experience: Handwritten-Digit Classification	174
7.15.4	Think Like a Data Scientist	177
7.16	Three Tracks	178
7.17	Threshold Selection as Decision Analysis	178
7.18	Class Imbalance	180
7.19	Multiclass Classification	180
7.20	Three Tracks	180
7.20.1	Health Track	180
7.20.2	Education Track	180
7.20.3	Open ML Benchmark Track	180
7.21	Beyond the Algorithm: A Probability Is Not a Decision	181
7.22	AI Pair Programmer	181
7.23	Hands-On Lab: Probability to Decision	181
7.24	Research Studio	182

7.25	Professor’s Challenge: Which Model Is Better?	182
7.26	Knowledge Check	182
7.27	Reproducibility Check	183
7.28	Key Takeaways	183
8	Decision Trees and Random Forests: Learning by Partitioning	184
8.1	Learning Objectives	184
8.2	Why This Matters	184
8.3	From Linear Boundaries to Partitions	185
8.4	Anatomy of a Tree	185
8.5	Classification Impurity	186
8.5.1	Gini Impurity	186
8.5.2	Entropy	186
8.6	Split Improvement	187
8.7	Calculate Impurity	187
8.8	Fit a Classification Tree	188
8.9	Visualize the Tree	188
8.10	Regression Trees	189
8.11	Why Trees Overfit	190
8.12	Controlling Tree Complexity	191
8.13	Cost-Complexity Pruning	192
8.14	Instability: A Tree’s Hidden Weakness	192
8.15	Bagging: Many Unstable Models Can Become Stable Together	193
8.16	Interpretable Does Not Mean Stable	193
8.16.1	Bootstrap the Root Split	193
8.17	Random Forests	196
8.18	Fit a Random Forest	196
8.19	Why a Forest Can Generalize Better	197
8.20	Out-of-Bag Evaluation	197
8.21	Feature Importance: Important in What Sense?	198
8.22	Permutation Importance	199
8.23	Compare Importance Measures	200
8.24	Partial Dependence: Model Behavior, Not Causal Effect	201
8.25	Computer Vision Learning Experience: Pixel Importance Is Model Dependence	202
8.26	Single Tree Versus Forest	205
8.27	Three Tracks	205
8.27.1	Health Track	205
8.27.2	Education Track	205
8.27.3	Open ML Benchmark Track	205
8.28	Beyond the Algorithm: What Does “Important” Mean?	206
8.29	AI Pair Programmer: Explain My Forest	206
8.30	Hands-On Lab: From One Tree to a Forest	207
8.31	Practitioner Track	207

8.32	Researcher Track	207
8.33	Research Studio	207
8.34	Professor’s Challenge: The Most Important Predictor	208
8.35	Knowledge Check	208
8.36	Reproducibility Check	209
8.37	Key Takeaways	210
9	Support Vector Machines: Margins, Geometry, and Kernels	211
9.1	Learning Objectives	211
9.2	Why This Matters	211
9.3	Classification as Geometry	212
9.4	Many Boundaries Can Separate the Same Data	213
9.5	Hard-Margin SVM	213
9.6	Support Vectors	214
9.7	Visual Experiment: Maximum-Margin Classification	214
9.8	Soft Margins: Allowing Imperfection	216
9.8.1	Intuition for C	217
9.9	Hinge Loss	217
9.10	Why Scaling Matters	218
9.11	Leakage-Safe SVM Pipeline	219
9.12	The Decision Function Is Not a Probability	220
9.13	When a Straight Boundary Is Not Enough	221
9.14	Feature Maps	223
9.15	The Kernel Trick	223
9.16	Representation Can Change the Difficulty of the Problem	224
9.16.1	A Simple Feature Map	224
9.16.2	Kernel as an Implicit Representation	227
9.16.3	Bridge to Representation Learning	227
9.17	Common Kernels	228
9.17.1	Linear Kernel	228
9.17.2	Polynomial Kernel	228
9.17.3	Radial Basis Function Kernel	228
9.18	Understanding γ	228
9.19	Visual Experiment: The Kernel Changes the Boundary	229
9.20	Visual Experiment: C and γ	232
9.21	Compare Logistic Regression, Linear SVM, and RBF SVM	234
9.22	Computer Vision Learning Experience: An RBF SVM on Handwritten Digits	235
9.22.1	What Did the Kernel Add?	236
9.23	Computational Considerations	237
9.24	SVM for Regression	237
9.25	Three Tracks	238
9.25.1	Health Track	238
9.25.2	Education Track	238

9.25.3	Open ML Benchmark Track	238
9.26	Beyond the Algorithm: Change the Representation, Change the Problem . . .	239
9.27	AI Pair Programmer: The Kernel Made It True	239
9.28	Hands-On Lab: From Hyperplane to Kernel	240
9.29	Practitioner Track	240
9.30	Researcher Track	240
9.31	Research Studio	241
9.32	Professor’s Challenge: The Perfect Kernel	241
9.33	Knowledge Check	242
9.34	Reproducibility Check	242
9.35	Key Takeaways	243
10	Ensemble Learning: From Many Models to Stronger Predictions	244
10.1	Learning Objectives	244
10.2	Why This Matters	244
10.3	The Ensemble Principle	245
10.4	Four Major Ensemble Strategies	245
10.4.1	Bagging	245
10.4.2	Boosting	246
10.4.3	Voting	246
10.4.4	Stacking: Learn the Combination Rule	246
10.5	Bagging Revisited	246
10.6	Correlated Errors Limit the Wisdom of the Crowd	247
10.7	Boosting: Learn From What Is Still Wrong	247
10.8	AdaBoost	247
10.9	Fit AdaBoost	248
10.10	Why “Focus on Mistakes” Can Be Dangerous	249
10.11	Gradient Boosting: Optimization in Function Space	249
10.12	Squared-Error Example	250
10.13	Visual Experiment: Sequential Residual Correction	250
10.14	Learning Rate and Number of Trees	252
10.15	Fit Gradient Boosting	253
10.16	Modern Histogram-Based Gradient Boosting	253
10.17	The Modern Boosting Ecosystem	254
10.17.1	What Actually Differs?	254
10.18	Boosting and Overfitting	255
10.19	Compare Several Supervised Models	256
10.20	Voting Ensembles	257
10.20.1	Hard Voting	257
10.20.2	Soft Voting	257
10.21	Fit a Soft-Voting Ensemble	257
10.22	Stacking	258
10.22.1	The Leakage Danger	259

10.23	Fit a Stacking Classifier	259
10.24	Diversity: More Than Different Algorithm Names	260
10.25	Visual Experiment: When Averaging Helps	260
10.26	Calibration Still Matters	262
10.27	Feature Importance Returns—With the Same Warning	263
10.28	Computer Vision Learning Experience: A Small Vision Model Comparison	264
10.28.1	Do Not Turn the Table Into the Conclusion	266
10.29	Three Tracks	266
10.29.1	Health Track	266
10.29.2	Education Track	267
10.29.3	Open ML Benchmark Track	267
10.30	Beyond the Algorithm: When Does the Crowd Become Wise?	267
10.31	AI Pair Programmer: Build Me the Ultimate Ensemble	268
10.32	Hands-On Lab: Bagging to Stacking	268
10.33	Practitioner Track	269
10.34	Researcher Track	269
10.35	Research Studio	269
10.36	Professor’s Challenge: The 12-Model Super Ensemble	270
10.37	Knowledge Check	270
10.38	Reproducibility Check	271
10.39	Key Takeaways	272
10.40	Looking Ahead	272

IV Part IV — Unsupervised Learning 273

11 Clustering: Discovering Structure Without Labels 274

11.1	Learning Objectives	274
11.2	Why This Matters	274
11.2.1	From Supervised to Unsupervised Learning	275
11.3	From Prediction to Structure Discovery	275
11.4	What Does Similar Mean?	276
11.4.1	Euclidean Distance	276
11.4.2	Manhattan Distance	276
11.5	Distance by Hand	276
11.6	Why Scaling Matters	277
11.7	A First Clustering Dataset	277
11.8	Are Clusters Discovered or Constructed?	278
11.8.1	The Same Observations, Different Representations	279
11.8.2	Clusters Are Analytical Claims	282
11.9	k -Means Clustering	283
11.10	The k -Means Algorithm	283
11.11	Fit k -Means	283

11.12	Initialization Matters	285
11.13	How Many Clusters?	286
11.14	Within-Cluster Sum of Squares and the Elbow	286
11.15	Silhouette Analysis	287
11.16	Internal Validation Is Not Scientific Validation	289
11.17	Cluster Profiles	290
11.18	Hierarchical Clustering	290
11.18.1	Agglomerative Strategy	291
11.19	Linkage Strategies	291
11.19.1	Single Linkage	291
11.19.2	Complete Linkage	291
11.19.3	Average Linkage	291
11.19.4	Ward Linkage	291
11.20	Dendrogram	291
11.21	Agglomerative Clustering in scikit-learn	293
11.22	A Different Definition of Structure: Density	294
11.23	Why k -Means Can Struggle	295
11.24	DBSCAN	297
11.25	Fit DBSCAN	298
11.26	Compare Three Notions of a Cluster	299
11.27	Cluster Stability	300
11.28	External Variables and Cluster Usefulness	301
11.29	Can Clustering Find Groups That Are Not Really There?	301
11.30	Computer Vision Learning Experience: Clustering Images Without Labels	303
11.31	Three Tracks	304
11.31.1	Health Track	304
11.31.2	Education Track	304
11.31.3	Open ML Benchmark Track	304
11.32	Beyond the Algorithm: Do Clusters Exist Before We Name Them?	305
11.33	AI Pair Programmer: Name These Clusters	305
11.34	Hands-On Lab: What Counts as a Cluster?	306
11.35	Practitioner Track	306
11.36	Researcher Track	307
11.37	Research Studio	307
11.38	Professor's Challenge: Four Types of Students	307
11.39	Knowledge Check	308
11.40	Reproducibility Check	309
11.41	Key Takeaways	309
12	Dimensionality Reduction: Compressing Data Without Losing the Story	311
12.1	Learning Objectives	311
12.2	Why This Matters	311

12.3	Feature Selection Versus Feature Extraction	312
12.3.1	Feature Selection	312
12.3.2	Feature Extraction	313
12.4	Why High Dimensions Become Difficult	313
12.5	A Representation Problem	313
12.6	Principal Component Analysis	313
12.6.1	Begin With Variance	314
12.6.2	Covariance	314
12.6.3	Covariance by Hand	314
12.6.4	PCA as a Rotation of the Coordinate System	315
12.6.5	Eigenvectors and Eigenvalues	316
12.6.6	A Geometric PCA Example	316
12.6.7	Principal Component Scores	318
12.6.8	Loadings	319
12.6.9	Scaling Can Change the Answer	319
12.6.10	PCA on a Real Dataset	320
12.6.11	Scree Plot	320
12.6.12	Cumulative Explained Variance	321
12.6.13	Explained Variance Is Not Importance	322
12.6.14	Visualizing Observations With PCA	323
12.6.15	Interpreting Loadings	324
12.6.16	Reconstruction	325
12.6.17	Reconstruction Error Across Dimensions	326
12.6.18	PCA in Predictive Modeling	327
12.6.19	Does PCA Improve Prediction?	328
12.7	Dimensionality Reduction Is an Information Choice	329
12.7.1	PCA Preserves Variance, Not Meaning	329
12.7.2	Reconstruction Error Makes the Tradeoff Visible	330
12.7.3	Visualization Is Not the Data	332
12.8	Nonlinear Embeddings	332
12.8.1	t-SNE	332
12.8.2	t-SNE Example	333
12.8.3	Perplexity	334
12.8.4	t-SNE Is Primarily a Visualization Tool	335
12.8.5	UMAP	335
12.9	PCA, t-SNE, or UMAP?	336
12.10A	Visualization Can Manufacture Confidence	337
12.11	Stress-Test the Representation	337
12.12	Computer Vision Learning Experience: Projecting Image Space	337
12.13	Three Tracks	340
12.13.1	Health Track	340
12.13.2	Education Track	340
12.13.3	Open ML Benchmark Track	340

12.14	Beyond the Algorithm: What Must Be Forgotten?	341
12.15	AI Pair Programmer: I Found Three Patient Types	341
12.16	Hands-On Lab: Compress, Visualize, Predict	342
12.17	Practitioner Track	342
12.18	Researcher Track	343
12.19	Research Studio	343
12.20	Professor's Challenge: The Two-Dimensional Truth	343
12.21	Knowledge Check	344
12.22	Reproducibility Check	345
12.23	Key Takeaways	345
12.24	Looking Ahead	346

V Part V — Modern Machine Learning 347

13 Neural Networks: Learning Representations Through Layers 348

13.1	Learning Objectives	348
13.2	Why This Matters	348
13.2.1	From Representation to Representation Learning	349
13.3	From Logistic Regression to a Neuron	349
13.4	A Neuron by Hand	349
13.5	Hidden Layers and Representation Learning	350
13.6	Why Nonlinearity Is Essential	350
13.7	Activation Functions	351
13.7.1	Sigmoid	351
13.7.2	Hyperbolic Tangent	351
13.7.3	ReLU	351
13.7.4	Softmax	351
13.8	Visualize Common Activations	351
13.9	Forward Propagation by Hand	354
13.10	Neural Networks as Representation Learning	354
13.10.1	From Feature Engineering to Learned Features	355
13.10.2	What Might a Hidden Representation Learn?	355
13.10.3	Learned Does Not Mean Understood	356
13.10.4	Representation Quality Depends on the Objective	356
13.10.5	Bridge Forward	357
13.11	Architecture	357
13.12	Loss Functions Give the Network a Goal	357
13.13	Gradient Descent	358
13.14	Gradient Descent Demonstration	358
13.15	The Chain Rule	359
13.16	Backpropagation	360
13.17	Numerical Gradient Check	360

13.18	Batches and Epochs	361
13.19	Vanishing and Exploding Gradients	361
13.20	Initialization	361
13.21	Training and Validation Curves	362
13.22	Build an MLP With scikit-learn	363
13.23	Inspect the Learned Architecture	364
13.24	Compare With a Simpler Baseline	365
13.25	Calibration	365
13.26	Learned Representations	367
13.27	Representation Learning Does Not Eliminate Feature Engineering	367
13.28	Universal Approximation—Carefully Interpreted	367
13.29	Neural Networks for Regression	367
13.30	When Should We Use Deep Learning?	368
13.31	Deep Learning Ecosystem	369
13.32	Computer Vision Learning Experience: From Pixels to a Neural Classifier	369
13.32.1	Why Convolution Changed Computer Vision	370
13.33	Three Tracks	371
13.33.1	Health Track	371
13.33.2	Education Track	371
13.33.3	Open ML Benchmark Track	371
13.34	Beyond the Algorithm: Where Did Human Judgment Go?	371
13.35	AI Pair Programmer: Make It Deeper	372
13.36	Hands-On Lab: From Neuron to Network	372
13.37	Practitioner Track	373
13.38	Researcher Track	373
13.39	Research Studio	373
13.40	Professor’s Challenge: Deep Learning Must Be Better	373
13.41	Knowledge Check	374
13.42	Reproducibility Check	374
13.43	Key Takeaways	375
14	Natural Language Processing: From Words to Meaningful Representations	376
14.1	Learning Objectives	376
14.2	Why Language Is Different	377
14.3	The Unit of Analysis	377
14.4	Core Vocabulary	378
14.5	Tokenization	378
14.6	A Simple Tokenization Demonstration	379
14.7	Text Normalization	380
14.8	Stop Words	380
14.9	Stemming and Lemmatization	381
14.10	Bag-of-Words Foundations	381
14.11	From Documents to Counts	381

14.12	CountVectorizer	381
14.13	Sparsity	382
14.14	What Bag of Words Forgets	383
14.15	N-Grams Restore Some Local Order	383
14.16	TF-IDF	384
14.17	Why Counts Are Not Enough	384
14.18	Term Frequency	384
14.19	Inverse Document Frequency	385
14.20	TF-IDF Weight	385
14.21	TF-IDF in Python	385
14.22	Cosine Similarity	386
14.23	Document Similarity	386
14.24	Text Classification	387
14.25	A Public Benchmark	387
14.26	A Reproducible Teaching Corpus	387
14.27	Leakage-Safe NLP Pipeline	388
14.28	Evaluate the Classifier	389
14.29	Inspect Lexical Signals	390
14.30	Spurious Lexical Cues	392
14.31	Distributed Representations	392
14.32	From Sparse Counts to Dense Vectors	392
14.33	The Distributional Idea	393
14.34	One-Hot Versus Dense Representation	393
14.35	Word2Vec Intuition	394
14.36	Static Embeddings Have a Limitation	394
14.37	Attention	395
14.38	Why Context Requires Interaction	395
14.39	Scaled Dot-Product Attention	395
14.40	Step 1: Compare Queries and Keys	396
14.41	Step 2: Scale	396
14.42	Step 3: Softmax	396
14.43	Step 4: Weighted Values	396
14.44	Attention by Hand	397
14.45	Attention Is Not Automatically Explanation	399
14.46	Transformers	399
14.47	The Transformer Idea	399
14.48	Multi-Head Attention	399
14.49	Positional Information	400
14.50	Encoder-Style Models	400
14.51	Decoder-Style Models	400
14.52	Encoder-Decoder Models	401
14.53	From NLP to Foundation Models	401
14.54	Research-Grade NLP	401

14.55	Split by the Real Generalization Boundary	401
14.56	Duplicate and Near-Duplicate Text	402
14.57	Temporal Leakage	402
14.58	Construct Validity	402
14.59	Fairness and Language Variation	403
14.60	From NLP to Multimodal Learning	403
14.60.1	Shared Spaces and Cross-Modal Alignment	404
14.60.2	Learning Experience: Design a Multimodal Study	404
14.61	Three Tracks	404
14.61.1	Health Track	404
14.61.2	Education Track	405
14.61.3	Open ML Benchmark Track	405
14.62	Public Research Dataset Studio	405
14.63	Beyond the Algorithm: Does a Vector Understand a Word?	406
14.64	Text Is Not the Same as Meaning	407
14.65	AI Pair Programmer: The Model Understands Students	407
14.66	Hands-On Lab: Build a Text Classifier	408
14.67	Practitioner Track	408
14.68	Researcher Track	409
14.69	Research Studio	409
14.70	Professor’s Challenge: Predicting Student Success From Essays	410
14.71	Knowledge Check	410
14.72	Reproducibility Check	411
14.73	Key Takeaways	412
15	Generative AI: From Prediction to Intelligent Systems	413
15.1	Learning Objectives	413
15.2	Why Generative AI Changes the Conversation	413
15.3	Discriminative Versus Generative Models	414
15.4	From Chapter 14 to Chapter 15	415
15.5	Tokens Are the Model’s Working Symbols	415
15.6	Logits and Probabilities	416
15.7	A Tiny Next-Token Distribution	416
15.8	Learning to Predict Tokens	417
15.9	Cross-Entropy Training Objective	417
15.10	Perplexity	418
15.11	Why Pretraining Is Powerful	418
15.12	Foundation Models	419
15.13	Generation and Sampling	419
15.14	Greedy Decoding	419
15.15	Temperature	419
15.16	Temperature Demonstration	420
15.17	Top-k Sampling	422

15.18	Nucleus Sampling	422
15.19	Sampling Is Not Creativity in a Simple Sense	422
15.20	Prompting and In-Context Learning	422
15.21	The Prompt Defines a Temporary Task Context	422
15.22	Zero-Shot Prompting	423
15.23	Few-Shot Prompting	423
15.24	In-Context Learning Is Not Ordinary Fine-Tuning	423
15.25	Prompt Engineering Versus Problem Engineering	424
15.26	Retrieval-Augmented Generation	424
15.27	Why Retrieval?	424
15.28	Vector Retrieval	424
15.29	A Tiny Retrieval Demonstration	425
15.30	RAG Is a System, Not a Single Model	426
15.31	Retrieval Does Not Guarantee Truth	427
15.32	RAG Evaluation	427
15.32.1	Retrieval Quality	427
15.32.2	Answer Quality	427
15.32.3	Faithfulness or Groundedness	427
15.32.4	Citation Quality	428
15.33	Fine-Tuning and Adaptation	428
15.34	Fine-Tuning	428
15.35	Instruction Tuning	428
15.36	Preference and Alignment Training	428
15.37	RAG or Fine-Tuning?	429
15.38	Hallucination and Uncertainty	429
15.39	What Is a Hallucination?	429
15.40	Why Fluent Errors Are Dangerous	430
15.41	Abstention	430
15.42	Calibration for Generative Systems	430
15.43	Tool Use and Agents	431
15.44	Language Models Can Be Embedded in Larger Systems	431
15.45	A Simple Tool-Use Loop	431
15.46	What Is an Agent?	431
15.47	Agentic Risk Grows With Action Space	432
15.48	Prompt Injection	432
15.49	Evaluating Generative AI	432
15.50	There Is No Single Accuracy Metric	432
15.51	Exact-Match Evaluation	433
15.52	Semantic Metrics	433
15.53	Human Evaluation	433
15.54	LLM-as-Judge	434
15.55	Pairwise Evaluation	434
15.56	Evaluation as an Experiment	434

15.57	Generative AI for Research	435
15.58	AI Can Support the Research Workflow	435
15.59	AI Should Not Become an Invisible Coauthor	435
15.60	Citation Fabrication Test	436
15.61	AI-Generated Code	436
15.62	Fluent Generation Is Not Evidence	436
15.63	Generative AI Beyond Text: Images and Multimodal Systems	437
15.63.1	Multimodal Generation	437
15.63.2	Evaluating Generated Images	438
15.63.3	Computer Vision Learning Experience	438
15.64	Three Tracks	438
15.65	Health Track	438
15.66	Education Track	439
15.67	Open ML Benchmark Track	440
15.68	Beyond the Algorithm: What Does It Mean for a Model to Know?	440
15.69	AI Pair Programmer: Build Me an Autonomous Researcher	441
15.70	Hands-On Lab: Build a Small RAG Evaluation	441
15.71	Practitioner Track	442
15.72	Researcher Track	442
15.73	Research Studio	442
15.74	Professor’s Challenge: The AI Tutor Experiment	443
15.75	Knowledge Check	444
15.76	Reproducibility Check	445
15.77	Key Takeaways	446
15.78	Looking Ahead	447

VI Part VI — Research, Responsibility, and Reproducibility 448

16 Model Evaluation: From Performance Metrics to Scientific Evidence 449

16.1	Learning Objectives	449
16.2	What Does It Mean for a Model to Work?	450
16.2.1	From Building Systems to Evaluating Evidence	450
16.3	Evaluation Begins With the Claim	451
16.4	Generalization Error	451
16.5	Training Error Is Optimistic	451
16.6	Three Data Roles	452
16.7	The Test Set Is Not a Development Tool	452
16.8	Classification Evaluation	452
16.9	Confusion Matrix	452
16.10	Accuracy	453
16.11	Sensitivity	453
16.12	Specificity	453

16.13	Precision	454
16.14	F1 Score	454
16.15	Compute Classification Metrics	454
16.16	ROC Curve	456
16.17	ROC-AUC	457
16.18	Plot the ROC Curve	457
16.19	Precision-Recall Curve	458
16.20	ROC-AUC Versus PR-AUC	460
16.21	Calibration	460
16.22	Discrimination Is Not Calibration	460
16.23	Brier Score	460
16.24	Calibration Curve	461
16.25	Calibration Depends on Context	462
16.26	Decision Thresholds	462
16.27	Probability Is Not the Decision	462
16.28	Threshold Tradeoffs	463
16.29	Visualize Threshold Tradeoffs	464
16.30	Error Costs	465
16.31	Regression Evaluation	466
16.32	Mean Absolute Error	466
16.33	Mean Squared Error	466
16.34	Root Mean Squared Error	466
16.35	Coefficient of Determination	466
16.36	Metric Choice Encodes Values	466
16.37	Evaluation Is an Argument About Future Performance	467
16.38	Resampling and Cross-Validation	467
16.39	Why a Single Split Is Unstable	467
16.40	Cross-Validation Demonstration	468
16.41	Standard Deviation Across Folds Is Not a Confidence Interval	469
16.42	Repeated Cross-Validation	469
16.43	Hyperparameter Tuning and Nested Cross-Validation	470
16.44	Selection Creates Optimism	470
16.45	Nested Cross-Validation	470
16.46	Nested CV Example	471
16.47	Uncertainty in Performance	473
16.48	A Metric Without Uncertainty Is Incomplete	473
16.49	Bootstrap Confidence Interval for Test-Set AUC	473
16.50	Visualize Bootstrap Performance	475
16.51	Comparing Models	476
16.52	Tiny Differences Need Context	476
16.53	Paired Evaluation	477
16.54	Statistical Significance Is Not Practical Significance	477
16.55	Validation Beyond Random Splits	478

16.56	Internal Validation	478
16.57	Temporal Validation	478
16.58	Geographic or Site Validation	478
16.59	External Validation	478
16.60	External Validation Is Not a Ceremonial Final Step	478
16.61	Distribution Shift	479
16.62	The Data-Generating Process Can Change	479
16.63	Covariate Shift	479
16.64	Label Shift	479
16.65	Concept Shift	480
16.66	Examples	480
16.67	Subgroup Evaluation	480
16.68	Overall Performance Can Hide Important Differences	480
16.69	Evaluate Relevant Groups	481
16.70	Small Groups Produce Noisy Estimates	481
16.71	Subgroup Metrics Do Not Solve Fairness	481
16.72	Evaluation Under Dependence	482
16.73	Observations Are Often Clustered	482
16.74	Split at the Generalization Unit	482
16.75	Leaderboards and Adaptive Overfitting	482
16.76	Public Test Sets Can Become Training Data Indirectly	482
16.77	Benchmark Performance Is Not Scientific Universality	483
16.78	Computer Vision Learning Experience: When Does a Vision Model Generalize?	483
16.78.1	Visual Domain Shift	484
16.78.2	Beyond Overall Accuracy	484
16.79	Three Tracks	485
16.80	Health Track	485
16.81	Education Track	485
16.82	Open ML Benchmark Track	485
16.83	Beyond the Algorithm: Evidence Is Always Conditional	486
16.84	AI Pair Programmer: Declare the Winner	487
16.85	Hands-On Lab: Evaluation Audit	487
16.86	Practitioner Track	488
16.87	Researcher Track	488
16.88	Research Studio	488
16.89	Professor’s Challenge: The 94% Accurate Early-Warning System	489
16.90	Knowledge Check	490
16.91	Reproducibility Check	491
16.92	Key Takeaways	492
16.93	Looking Ahead	492
17	Responsible Machine Learning: From Prediction to Consequence	494
17.1	Learning Objectives	494

17.2 Can We Build It? Should We Use It?	495
17.2.1 From Performance to Consequence	495
17.3 From Model Evaluation to Responsible Use	496
17.4 Responsibility Is a System Property	496
17.5 Stakeholders, Harms, and Power	497
17.6 Map the Stakeholders	497
17.7 Types of Harm	497
17.7.1 Allocative harm	497
17.7.2 Quality-of-service harm	497
17.7.3 Representational harm	497
17.7.4 Privacy harm	497
17.7.5 Autonomy harm	498
17.7.6 Procedural harm	498
17.8 Benefit Is Also Unequally Distributed	498
17.9 Where Bias Enters the Pipeline	498
17.10 Historical Bias	498
17.11 Representation Bias	498
17.12 Measurement Bias	499
17.13 Label Bias	499
17.14 Aggregation Bias	499
17.15 Deployment Bias	499
17.16 Feedback Loops	499
17.17 Fairness as a Measurement Problem	500
17.18 Protected or Sensitive Groups	500
17.19 Demographic Parity	500
17.20 Equal Opportunity	500
17.21 Equalized Odds	501
17.22 Predictive Parity	501
17.23 A Small Fairness Audit	501
17.24 Visualize Group Metrics	502
17.25 Fairness Criteria Can Conflict	503
17.26 Different Definitions Encode Different Normative Goals	504
17.27 Base Rates Matter	504
17.28 Fairness Is Not “Make Every Number Equal”	504
17.29 Individual, Group, and Procedural Fairness	504
17.30 Group Fairness	504
17.31 Individual Fairness	505
17.32 Procedural Fairness	505
17.33 Intersectionality	505
17.34 One Attribute at a Time Can Hide Harm	505
17.35 Statistical Caution	505
17.36 Explainability and Interpretability	506
17.37 Why Explain?	506

17.38	Global Versus Local Explanation	506
17.39	Permutation Importance	506
17.40	Demonstration	507
17.41	Importance Is Not Causality	510
17.42	Explanations Can Also Mislead	510
17.43	Privacy, Confidentiality, and Security	510
17.44	These Concepts Are Related but Different	510
17.45	Data Minimization	511
17.46	Reidentification Risk	511
17.47	Sensitive Inference	511
17.48	Differential Privacy	511
17.49	Privacy as a Formal Guarantee	511
17.50	Interpreting Epsilon	512
17.51	Privacy Is Not Merely a Technical Parameter	512
17.52	Human Oversight	512
17.53	“Human in the Loop” Is Not Enough	512
17.54	Selective Automation	513
17.55	Contestability	513
17.56	Governance Across the ML Lifecycle	513
17.57	Governance Begins Before Modeling	513
17.58	Purpose and Scope	514
17.59	Data Governance	514
17.60	Model Governance	514
17.61	Deployment Governance	514
17.62	Model Cards	515
17.63	Documentation as an Engineering Artifact	515
17.64	Algorithmic Impact Assessment	515
17.65	Evaluate the System Before Deployment	515
17.66	Feedback Loops and Performative Prediction	516
17.67	Predictions Can Change Outcomes	516
17.68	Policy Changes the Meaning of Data	516
17.69	Generative AI Inside Responsible ML Systems	517
17.70	Generative Components Add New Failure Modes	517
17.71	RAG Does Not Eliminate Responsibility	517
17.72	Agentic Systems Increase Consequence	517
17.73	Responsible Computer Vision: What Does the Camera Make Legible?	518
17.73.1	Representation and Measurement	518
17.73.2	Privacy and Surveillance	519
17.73.3	Human Oversight in Visual Decisions	519
17.73.4	Generative and Synthetic Visual Content	519
17.73.5	Responsible Vision Audit	519
17.74	Three Tracks	520
17.75	Health Track	520

17.76	Education Track	520
17.77	Open ML Benchmark Track	521
17.78	Beyond the Algorithm: Prediction Is an Exercise of Power	521
17.79	AI Pair Programmer: Remove the Sensitive Attribute	522
17.80	Hands-On Lab: Responsible ML Audit	523
17.81	Practitioner Track	523
17.82	Researcher Track	523
17.83	Research Studio	524
17.84	Responsibility Begins Before the Model	524
17.84.1	Fairness Metrics Do Not Choose Values for Us	525
17.85	Professor’s Challenge: The Student Success Score	525
17.86	Knowledge Check	526
17.87	Reproducibility Check	527
17.88	Key Takeaways	528
17.89	Looking Ahead	529
18	Reproducible Machine Learning: From Analysis to Scientific Knowledge	530
18.1	Learning Objectives	530
18.2	A Result Is More Than a Number	531
18.2.1	From Responsible Claims to Reproducible Evidence	531
18.3	Reproducibility Is a Chain	532
18.4	Four Related Ideas	532
18.4.1	Reproducibility	532
18.4.2	Replication	532
18.4.3	Robustness	532
18.4.4	Generalizability	533
18.5	The Research Compendium	533
18.6	Organize the Project Around the Research Process	533
18.7	Reproducibility Is More Than Rerunning Code	534
18.7.1	From Student Submission to Scholarly Artifact	534
18.8	Final Three-Track Culmination	535
18.9	Final Reflection	535
18.10	Separate Source From Derived Artifacts	535
18.11	Data Provenance	536
18.12	Every Dataset Has a History	536
18.13	Create a Data README	536
18.14	Hashes Can Identify File Versions	537
18.15	Randomness and Determinism	537
18.16	Random Seeds Improve Repeatability	537
18.17	A Seed Is Not a Reproducibility Guarantee	538
18.18	Record the Computational Environment	538
18.19	Software Is Part of the Method	538
18.20	Inspect the Current Environment	538

18.21	Dependency Files	539
18.22	Do Not Confuse Environment Capture With Scientific Validity	539
18.23	Pipelines as Reproducible Methods	540
18.24	Preprocessing Is Part of the Model	540
18.25	Experiment Configuration	541
18.26	Separate Configuration From Results	541
18.27	Store Machine-Readable Metadata	542
18.28	Results Should Point Back to Configuration	542
18.29	Version Control With Git	543
18.30	Git Preserves Project History	543
18.31	A Commit Should Represent a Meaningful Change	543
18.32	Git Does Not Replace Documentation	544
18.33	GitHub and Remote Repositories	544
18.34	Why Use a Remote Repository?	544
18.35	Do Not Commit Secrets	544
18.36	Quarto as a Reproducible Publishing System	545
18.37	Narrative and Computation Belong Together	545
18.38	Literate Programming	545
18.39	A Figure Should Have Provenance	545
18.40	Notebooks Versus Scripts	547
18.41	Exploration and Production Serve Different Purposes	547
18.42	Beware Hidden Notebook State	547
18.43	Saving Model Artifacts	547
18.44	Models Are Derived Artifacts	547
18.45	Serialization Has Limits	549
18.46	Automated Workflows	549
18.47	Reproducibility Improves When Steps Are Explicit	549
18.48	One-Command Reproduction	549
18.49	Testing Data and ML Pipelines	550
18.50	Tests Are Not Only for Software Engineers	550
18.51	Example Data Assertions	550
18.52	Tests Cannot Prove the Research Question Is Good	550
18.53	Provenance of Figures, Tables, and Claims	551
18.54	Every Result Should Be Traceable	551
18.55	Archiving and Persistent Identifiers	551
18.56	A Repository Is Living; A Release Is Fixed	551
18.57	Zenodo and DOI Archiving	552
18.58	Concept DOI and Version DOI	552
18.59	OSF as a Research Hub	552
18.60	DOI Does Not Mean Peer Reviewed	553
18.61	Citation and Credit	553
18.62	Make Software and Data Citable	553
18.63	Cite Data as Research Objects	553

18.64	Reproducibility and Restricted Data	554
18.65	Open Science Does Not Mean “Upload Everything”	554
18.66	Never Publish Data Merely to Satisfy a Reproducibility Checklist	554
18.67	Robustness and Multiverse Thinking	554
18.68	One Analysis Path Is Rarely the Only Reasonable Path	554
18.69	Specification Curves and Multiverse Analyses	555
18.70	Reproducibility Is Not Replication	555
18.71	The Distinction Matters	555
18.72	Reproducible Computer Vision: Preserve the Visual Evidence Chain	556
18.72.1	Preserve the Split, Not Just the Seed	556
18.72.2	Restricted Images	557
18.72.3	Vision Reproducibility Challenge	557
18.73	Three Tracks	558
18.74	Health Track	558
18.75	Education Track	558
18.76	Open ML Benchmark Track	559
18.77	Beyond the Algorithm: Reproducibility as Intellectual Humility	559
18.78	AI Pair Programmer: Make My Study Reproducible	560
18.79	Hands-On Lab: Build a Research Compendium	560
18.80	Practitioner Track	561
18.81	Researcher Track	561
18.82	Research Studio	561
18.83	Professor’s Challenge: The Irreproducible Breakthrough	562
18.84	Capstone: From Machine Learning Project to Research Artifact	563
18.85	Health	564
18.86	Education	564
18.87	Open ML	564
18.88	Knowledge Check	565
18.89	Reproducibility Check	566
18.90	Key Takeaways	567

VII Resources 568

Teaching with This Book 569

Recurring Learning Experiences	569
The Three Application Tracks	570
A 7-Week Graduate Course	570
Recommended 7-Week Assessment Pattern	571
A 14-Week Graduate Course	571
Suggested Semester Project Milestones	572
A Research Seminar	572
Seminar Learning Cycle	573

From Application to Contribution	573
Guided Self-Study	574
Self-Study Evidence Portfolio	574
Teaching Computer Vision Across the Book	574
Using Generative AI in Instruction	575
Reproducibility as a Course-Level Expectation	575
Designing Assessments	576
A Suggested Evaluation Lens	576
Closing Perspective	577
About the Author	578
Teaching and Mentorship	579
Beyond the Classroom	579
Author Information	579
Citation, Edition, and License	580
Publication Identity	580
Preferred Citation	580
Versioning	581
Repository and Reproducibility	581
Archival DOI	582
License	582
How to Cite Code or a Specific Release	582
Corrections and Updates	583
Citation Principle	583
References	584

Preface

Foundations, Algorithms, Applications, and Reproducible Research

About This Book

Machine Learning Using Python is for readers who want to move beyond calling machine-learning functions toward understanding what models do, why they work, how they fail, and what evidence is required before their predictions should be trusted.

The book combines mathematical reasoning, statistical foundations, executable Python, real-world applications, responsible machine learning, and reproducible research.

$$\text{Machine Learning} = \text{Algorithms} + \text{Data} + \text{Evidence} + \text{Judgment}$$

Machine learning is not merely the construction of predictive models. It is the disciplined use of data, computation, and evidence to reason under uncertainty.

Why a Second Edition?

The First Edition introduced machine learning using Python through accessible explanations and practical examples. The Second Edition responds not simply to new algorithms, but to a changed practice of data science.

It is no longer enough to ask whether a model predicts accurately. We must also ask whether the problem was formulated appropriately, whether evaluation supports the claim, whether performance generalizes, what happens when predictions affect people, and whether another researcher can reproduce the evidence.

The Second Edition therefore expands the book into a graduate-level treatment of machine-learning foundations, algorithms, applications, responsible practice, and reproducible computational research.

What Is New?

Major additions include stronger statistical and mathematical foundations; explicit problem formulation; reproducible data preparation; deeper supervised and unsupervised learning; neural networks; natural language processing; generative AI; calibration, uncertainty, and validation; responsible ML; reproducible research; three longitudinal application tracks; hands-on laboratories; knowledge checks; Professor's Challenges; AI Pair Programmer activities; and research-oriented exercises.

The goal is not to expose readers to the largest possible collection of algorithms. It is to develop the judgment required to formulate, implement, evaluate, question, and communicate machine-learning work.

From Learning Algorithms to Doing Data Science

A classroom workflow may look like:

load data → fit model → predict → score.

A mature workflow is closer to:

question → measurement → data → assumptions → model
→ evaluation → interpretation → decision → reproducible evidence.

Readers will therefore be asked not only to write code, but to defend analytical choices.

A Philosophy of Machine Learning

Algorithms optimize objectives that humans specify. A loss function can tell an optimizer what numerical quantity to minimize. It cannot determine whether the target is meaningful, whether the data represent the intended population, whether an intervention is justified, or whether a resulting decision is fair.

Technical reasoning asks, *How does the algorithm work?*

Critical reasoning asks, *What must be true for this result to mean what we claim it means?*

Responsible reasoning asks, *What happens if someone acts on this prediction?*

All three belong to data science.

Machine Learning in the Age of Generative AI

This book treats generative AI neither as something learners must avoid nor as an epistemic authority.

AI Output + Human Verification + Domain Knowledge + Empirical Evidence

AI Pair Programmer activities ask readers to inspect, test, critique, and improve AI-generated reasoning and code.

From Learner to Practitioner and Researcher

A reader may progress through:

$$\text{Learner} \rightarrow \text{Coder} \rightarrow \text{Modeler} \rightarrow \text{Critical Evaluator} \rightarrow \begin{cases} \text{Practitioner} \\ \text{Researcher} \end{cases}$$

Both destinations require responsible reasoning and reproducibility.

Acknowledgments

A book develops through conversations—with students, colleagues, collaborators, mentors, reviewers, and communities of scholars and practitioners.

I am grateful to the students whose questions continually sharpen the distinction between knowing how to run an algorithm and understanding why an analytical choice is justified. I am equally grateful to colleagues, collaborators, and mentors whose scholarship, critique, encouragement, and example have shaped how I think about data science, research, teaching, and evidence.

The Second Edition also reflects the work of the broader open-source and scientific-computing communities whose tools make transparent, reproducible, and accessible computational learning possible.

Most of all, I acknowledge every reader who approaches machine learning with curiosity and humility—the willingness to build models, question assumptions, examine evidence, and revise conclusions when the evidence requires it.

Edition Statement

This is the **Second Edition** of *Machine Learning Using Python: Foundations, Algorithms, Applications, and Reproducible Research*.

The Second Edition expands the original work into an 18-chapter graduate-level treatment that integrates machine-learning foundations, algorithms, real-world applications, computer vision, modern AI, responsible practice, and reproducible research.

Formal citation, licensing, version, repository, and archival information are provided with the publication resources accompanying this book.

How to Use This Book

Machine learning can be learned in several ways. This book supports readers who need a systematic introduction, practitioners strengthening their modeling judgment, researchers preparing empirical studies, and readers concentrating on modern AI.

Who This Book Is For

The book is designed for graduate students, data enthusiasts, analysts, researchers, instructors, professionals, and self-directed learners who want to move from Python examples toward defensible analytical practice.

What You Should Know Before Beginning

Some Python familiarity is useful, but Chapter 2 develops the programming concepts used throughout the book. Introductory statistics is helpful, while Chapter 3 develops the statistical and mathematical foundations needed for later chapters.

Mathematics is used to clarify algorithms rather than create unnecessary abstraction. When an equation appears, ask:

1. What quantities does it relate?
2. What mechanism or assumption does it represent?
3. How does it connect to the executable implementation?

The Architecture of the Book

The Second Edition contains **18 chapters organized into six parts**:

1. **Foundations** — Chapters 1–3.
2. **Machine Learning Workflow** — Chapters 4–5.
3. **Supervised Learning** — Chapters 6–10.
4. **Unsupervised Learning** — Chapters 11–12.
5. **Modern Machine Learning** — Chapters 13–15.

6. Research, Responsibility, and Reproducibility — Chapters 16–18.

Foundations → Workflow → Models → Modern AI → Evaluation → Responsibility → Reproducible Research.

The progression is cumulative rather than merely sequential. Later chapters repeatedly return to ideas introduced earlier—problem formulation, measurement, leakage, representation, uncertainty, generalization, responsibility, and reproducibility—so that increasingly sophisticated models are evaluated within a consistent analytical framework.

Four Reading Pathways

Foundational Path

For systematic study, read Chapters 1–18 sequentially.

Practitioner Path

A possible route is Chapters 1, 4, 5, selected algorithm chapters, and Chapters 16–18. Evaluation, responsibility, and reproducibility remain professional competencies rather than optional research topics.

Researcher Path

A possible route is Chapters 1, 3–5, relevant method chapters, and Chapters 16–18. Give particular attention to problem formulation, leakage, uncertainty, external validation, provenance, and reproducibility.

Modern AI Path

A possible route is Chapters 1, 3, 5, 13–15, and 16–18. Powerful models still require rigorous evaluation and responsible use.

Three Application Tracks

Health

The Health Track uses health outcomes, wellness, risk prediction, public health, clinical reasoning, and responsible health analytics.

Education

The Education Track uses learning, student success, persistence, educational measurement, institutional analytics, and responsible learning analytics.

Open ML

The Open ML Track uses public benchmark data, general analytical problems, methodological comparisons, and applications beyond the two primary domains.

You Do Not Have to Choose One Track

Follow one track longitudinally for domain continuity, or move among all three to compare how the same machine-learning principle changes across contexts.

A Longitudinal Computer Vision Thread

Computer vision appears throughout the book as a recurring machine-learning workload rather than as a single isolated chapter. Readers encounter images as data, classical supervised models, unsupervised visual structure, dimensionality reduction, neural representations, multi-modal learning, generative vision, evaluation under visual domain shift, responsible use, and reproducible visual workflows.

This progression reinforces a broader lesson:

A Modality Changes the Evidence Problem, Not the Need for Judgment

The goal is not to turn the book into a specialized computer-vision text. It is to help readers experience how one important data modality interacts with the same principles of representation, evaluation, responsibility, and reproducibility developed across the rest of the book.

From Algorithm to Research Question

Applying an algorithm is not automatically a research contribution.

Fit Random Forest to a public dataset and report accuracy.

is an application.

Research asks what substantive or methodological uncertainty the analysis resolves.

Algorithm Application $\neq$ Research Contribution

Research Studio activities help readers make that transition.

How Chapters Are Designed

Recurring elements include **Learning Objectives**, mathematical development, **Hands-On Practice**, the three application tracks, **AI Pair Programmer**, **Professor's Challenge**, **Practitioner Track**, **Researcher Track**, **Research Studio**, **Knowledge Check**, **Reproducibility Check**, and **Key Takeaways**. Not every feature appears in every chapter; features are used when they advance the learning objective.

Notation and Conventions

Notation	Meaning
n	number of observations
p	number of predictors/features
$X \in \mathbb{R}^{n \times p}$	feature/design matrix
$\mathbf{x}_i$	feature vector for observation i
y_i	observed outcome
$\hat{y}_i$	predicted outcome
$\hat{p}_i$	predicted probability
β	regression coefficients
θ	generic model parameters
$\hat{\theta}$	estimated parameters
$\mathcal{L}(\theta)$	loss/objective function
$P(Y X)$	conditional probability
$E[X]$	expected value
$\text{Var}(X)$	variance

Additional notation is introduced where needed.

Software and Computational Environment

Examples primarily use Python, NumPy, pandas, Matplotlib, and scikit-learn, with additional tools introduced when required. Exact dependencies should be obtained from the repository rather than hard-coded into prose.

The computational environment is part of the method.

Using Generative AI While Learning

Generative AI can explain code, propose implementations, identify errors, and suggest alternatives. It can also invent APIs, fabricate citations, create leakage, choose inappropriate metrics, and produce persuasive but incorrect explanations.

A useful workflow is:

Ask → Inspect → Test → Verify → Revise → Document.

Reproducibility From the Beginning

As you work through the book, document data sources, preserve source data, record transformations, set seeds where appropriate, use leakage-safe pipelines, track dependencies, use version control, and generate figures and tables from code.

Chapter 18 brings these practices together into a complete research workflow.

Teaching and Self-Study Pathways

A dedicated **Teaching with This Book** resource accompanies the Second Edition. It provides suggested pathways for:

- a 7-week graduate course;
- a 14-week graduate course;
- a research seminar;
- guided self-study.

These pathways are designed to help instructors and independent learners sequence chapters, laboratories, application tracks, reproducibility activities, and research-oriented assignments without changing the book's 18-chapter structure.

A Final Orientation

Do not measure progress only by algorithms memorized. Learn to ask:

- What problem am I solving?
- What evidence would convince me?
- What assumptions am I making?
- What does the model fail to capture?
- Who is affected by the result?
- Can someone else reproduce what I did?

Those questions turn algorithms into analytical discipline.

Part I

Part I — Foundations

1 Machine Learning in the Age of Data and AI

1.1 Learning Objectives

By the end of this chapter, you should be able to:

- define machine learning in terms of data, models, loss, and generalization;
- distinguish AI, machine learning, deep learning, foundation models, and generative AI;
- distinguish supervised, unsupervised, semi-supervised, self-supervised, and reinforcement learning;
- identify observations, features, targets, predictions, parameters, and hyperparameters;
- explain why performance on unseen data is central to machine learning;
- distinguish prediction, explanation, inference, and causation;
- formulate problems using health, education, and benchmark-data tracks;
- distinguish model fitting from a potential research contribution.

1.2 Why This Matters

Machine learning supports predictive systems, scientific discovery, decision support, and modern AI. The ability to call `fit()` is not the same as understanding machine learning.

Central Question

How do we build models that learn from data—and how do we know they will work appropriately on data they have never seen?

1.3 A Conceptual Map of Modern Machine Learning

Artificial intelligence (AI) is the broadest concept here. **Machine learning (ML)** is a major approach to AI in which models learn patterns from data (T. M. Mitchell 1997). **Deep learning** uses multilayer neural networks to learn representations (I. Goodfellow, Bengio, and Courville 2016). **Foundation models** are trained at scale on broad data and adapted to downstream tasks (Bommasani et al. 2021). **Generative AI** creates new content such as text, images, audio, video, or code.

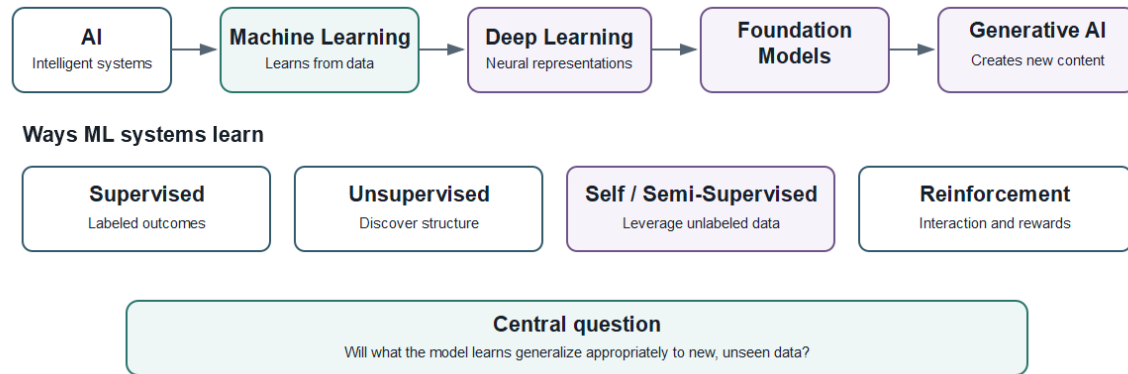


Figure 1.1: Conceptual map of AI and modern machine learning.

Modern generative systems do not make classical machine learning obsolete. Regression, trees, ensembles, clustering, and related techniques remain central to structured problems in health, education, finance, operations, and science.

1.4 What Does It Mean for a Machine to Learn?

We say that a machine learns, but the word deserves examination. A machine learning system does not necessarily understand a phenomenon in the way a human researcher understands it. It identifies patterns in representations of observations and uses those patterns to perform a task.

This distinction raises an important question: When a model predicts accurately, what has it actually learned?

A model may discover a stable relationship, exploit a proxy variable, reproduce a historical pattern, or capitalize on an artifact of data collection. Predictive success alone cannot tell us which explanation is true.

This is why machine learning is concerned with evidence and not just computation. We must ask what the data represent, what the model has access to, where the pattern came from, whether it generalizes, and what conclusions the evidence permits.

A useful habit for the data scientist is therefore:

Do not ask only, “Does the model work?” Ask, “What does its success allow me to know?”

Suppose we observe

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n.$$

A supervised model estimates

$$f_\theta : \mathcal{X} \rightarrow \mathcal{Y},$$

and predicts

$$\hat{y} = f_\theta(x).$$

Many algorithms learn parameters by minimizing a loss:

$$\hat{\theta} = \arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n L(y_i, f_\theta(x_i)).$$

Read the Equation Before Solving It

For now, read the pattern conceptually:

data → **model** → **prediction** → **error** → **learning**

Chapter 3 develops the mathematics.

1.5 The Vocabulary of an ML Problem

An **observation** is the unit represented by a row of modeling data. A **feature** is an input variable available to the model. A **target** is the outcome a supervised model attempts to predict. A **prediction**, $\hat{y}$, is the model's estimate.

A **parameter** is learned from training data. A **hyperparameter** controls an aspect of the learning process and is generally selected through validation.

1.6 Ways Machine Learning Systems Learn

1.6.1 Supervised Learning

Uses labeled outcomes. Major tasks are **regression** and **classification**.

1.6.2 Unsupervised Learning

Searches for structure without a predefined target. Common tasks include clustering, dimensionality reduction, and anomaly detection.

1.6.3 Semi-Supervised Learning

Combines a smaller labeled dataset with a larger collection of unlabeled observations.

1.6.4 Self-Supervised Learning

Constructs learning signals from the data themselves and is important in modern representation learning.

1.6.5 Reinforcement Learning

An agent interacts with an environment, takes actions, receives feedback, and learns a policy intended to maximize cumulative reward (Sutton and Barto 2018).

1.7 Machine Learning Workloads: Where Computer Vision Fits

Learning paradigms describe **how a system learns**. Workloads describe **what kind of problem or data the system works with**. These are different dimensions.

For example, **computer vision** is a major machine-learning workload concerned with extracting useful information from images and video (Szeliski 2022). A vision system may use supervised, self-supervised, semi-supervised, or other learning paradigms.

Common computer-vision tasks include:

- **image classification** — assign a label to an image;
- **object detection** — identify and locate objects;
- **image segmentation** — assign labels to pixels or regions;
- **image retrieval and similarity** — find visually related images;
- **visual anomaly detection** — identify unusual visual patterns;
- **image generation and transformation** — create or modify visual content.

Other important ML workloads include natural-language processing, forecasting, recommendation, anomaly detection, ranking, and multimodal learning.

The distinction matters:

Learning Paradigm $\neq$ Workload

A computer-vision application is not automatically “deep learning,” although modern vision systems often use neural networks. Classical ML can also operate on image features, and later chapters will show how representation learning changes what can be learned from visual data.

Computer Vision Learning Thread

Computer vision will appear as a recurring learning experience in this book. We begin with images as numerical arrays, then revisit visual representation, classification, neural networks, evaluation, responsible use, and reproducibility.

The goal is not merely to run a vision model. It is to understand how an image becomes data, how representations affect learning, and what evidence is required before a visual prediction should be trusted.

1.8 Why Generalization Matters

A model can memorize training observations and perform poorly on new data. This is **overfitting**.

We distinguish **training data**, used to estimate parameters; **validation data**, used for model selection and tuning; and **test data**, protected for final evaluation.

1.8.1 A First Generalization Experiment

This experiment fits decision trees of increasing complexity to a nonlinear classification problem.

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

X, y = make_moons(n_samples=800, noise=0.30, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(
```

```

    X, y, test_size=0.30, random_state=42, stratify=y
)

depths = range(1, 21)
train_scores, test_scores = [], []

for depth in depths:
    model = DecisionTreeClassifier(max_depth=depth, random_state=42)
    model.fit(X_train, y_train)
    train_scores.append(model.score(X_train, y_train))
    test_scores.append(model.score(X_test, y_test))

plt.figure(figsize=(8, 5))
plt.plot(depths, train_scores, marker="o", label="Training accuracy")
plt.plot(depths, test_scores, marker="o", label="Test accuracy")
plt.xlabel("Maximum tree depth")
plt.ylabel("Accuracy")
plt.title("Model Complexity and Generalization")
plt.legend()
plt.grid(alpha=0.2)
plt.show()

```

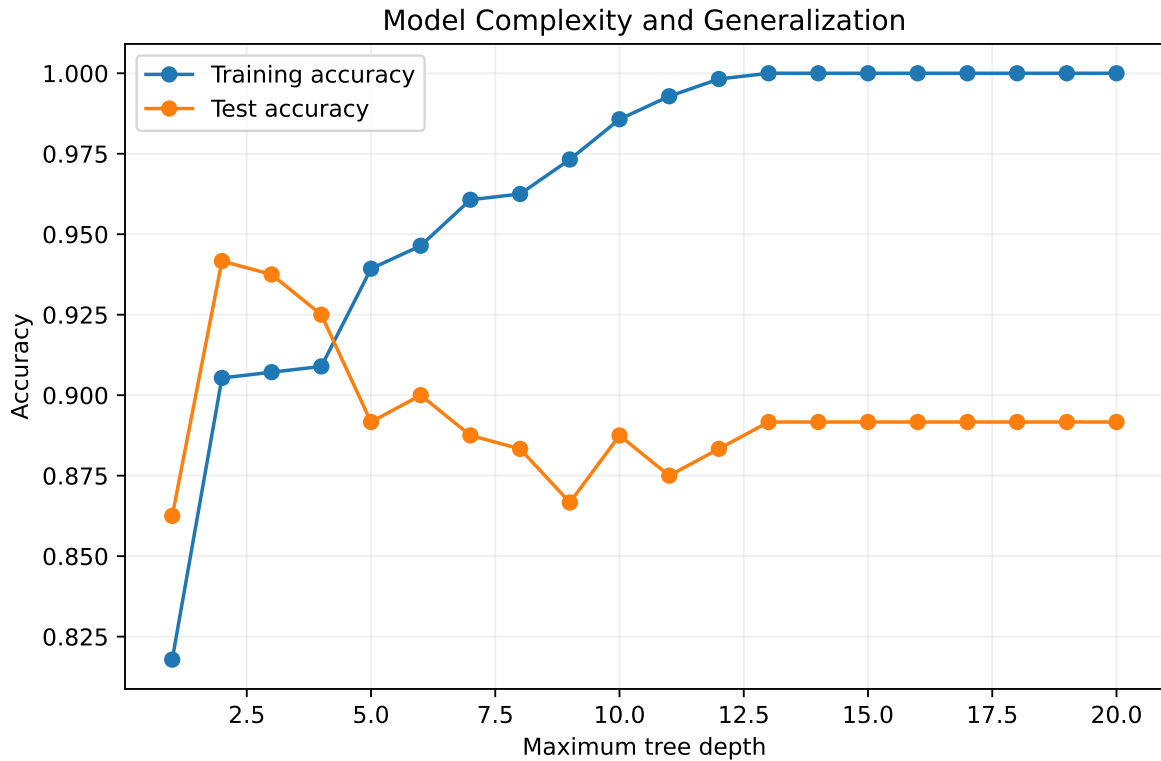


Figure 1.2: Training and test accuracy as decision-tree complexity increases.

As flexibility increases, training performance tends to rise. Test performance can improve initially and then flatten or deteriorate.

Key Idea

The best model is not the model that performs best on training data. It is the model that generalizes appropriately to new data for its intended use.

1.9 Statistics You Need

Foundation	Why it matters
Probability	uncertainty and probabilistic prediction
Distributions	variables, errors, sampling, assumptions
Descriptive statistics	center, spread, relationships
Sampling and estimation	populations, uncertainty, generalization

Foundation	Why it matters
Correlation and covariance	relationships among variables
Linear algebra	vectors, matrices, transformations
Functions	mappings from inputs to outputs
Derivatives and gradients	optimization and parameter learning

You do not need to master these before continuing. Chapter 3 develops them as needed.

1.10 Prediction, Explanation, Inference, and Causation

A model can predict well without identifying causal relationships.

Predictive Importance Is Not Causal Importance

High feature importance does **not** establish that intervening on that feature will change the outcome.

1.11 Three Learning and Research Tracks

1.11.1 Health Track — NHANES

Use individual-level public health data for themes such as cardiometabolic risk, physical activity, sleep, nutrition, biomarkers, and health outcomes (National Center for Health Statistics 2024).

1.11.2 Education Track — IPEDS and College Scorecard

Use public postsecondary data for themes such as retention, enrollment, completion, institutional characteristics, finance, and student outcomes (National Center for Education Statistics 2026).

1.11.3 Open ML Benchmark Track

Use UCI, OpenML, scikit-learn, and other suitable benchmarks to isolate algorithm behavior, compare methods, debug pipelines, and reproduce experiments.

A dataset can be excellent for understanding an algorithm without being sufficient for a novel research contribution.

1.12 A Disciplined Machine Learning Workflow

1. Define the substantive problem.
2. Specify the unit of analysis and target.
3. Identify information available at prediction time.
4. Understand data provenance and limitations.
5. Design the evaluation strategy.
6. Establish a simple baseline.
7. Prepare data without leakage.
8. Fit candidate models.
9. Evaluate on appropriate unseen data.
10. Inspect errors, uncertainty, and subgroup behavior.
11. Document limitations and intended use.
12. Make the analysis reproducible.

1.13 Try It

Classify each problem as supervised, unsupervised, self-supervised, or reinforcement learning:

1. Predict systolic blood pressure.
2. Group universities with similar profiles.
3. Predict fraudulent transactions.
4. Learn representations from a large unlabeled text corpus.
5. Train an agent in a simulated environment.

1.14 Hands-On Lab

Using the generalization experiment:

1. Identify the depth with the highest test accuracy.
2. Compare it with the deepest tree's training accuracy.
3. Explain why the deepest tree is not automatically preferable.
4. Change `noise=0.30` to `0.15` and `0.45`.
5. Explain how the data-generating process changes the result.

1.14.1 Computer Vision Preview

Use `sklearn.datasets.load_digits` to inspect a small collection of handwritten-digit images.

Without building a sophisticated vision model yet:

1. load the dataset;
2. inspect the shape of the image array;
3. display several images with their labels;
4. explain what one observation, one feature, and the target mean in this setting;
5. reshape one image into a feature vector and describe what information is preserved and what spatial structure becomes less explicit;
6. identify whether digit recognition is a workload, a learning paradigm, or both.

We will return to computer vision after developing the mathematical, data-preparation, classification, and neural-network foundations needed to evaluate it responsibly.

1.15 Research Studio

Choose one track and specify the substantive problem, unit of analysis, target, feature families, prediction horizon, baseline, evaluation strategy, generalization concern, potential contribution, and claims the analysis cannot support.

1.16 AI Pair Programmer

Audit the AI, Not Just the Code

Ask an AI assistant to explain why a decision tree can achieve perfect training accuracy while performing worse on test data. Audit the response for correct treatment of generalization, test-set use, causation, and claims requiring verification.

1.17 Responsible Machine Learning Begins Before Modeling

Responsible ML begins with problem formulation, data, target construction, representation, evaluation, and intended use—not after the model has been built (National Institute of Standards and Technology 2023).

Ask who benefits, who bears error costs, whether the target is defensible, whether populations are represented, whether inequities may be reproduced, whether evaluation matches intended use, and whether another analyst can reproduce the result.

1.18 From Model Fitting to Research

The Research Distinction

Using machine learning does not, by itself, make a study machine-learning research.

A contribution comes from the question, method, data, evaluation, setting, generalization evidence, reproducibility, insight, or methodological innovation—not merely from using an algorithm.

1.18.1 A Research Ladder

Level 1 — Reproduction: reproduce an existing analysis or benchmark.

Level 2 — Extension: defensibly change a population, period, outcome, representation, or dataset.

Level 3 — Rigorous Benchmarking: compare methods under a transparent common protocol.

Level 4 — Methodological Contribution: develop or materially adapt a method, representation, or evaluation procedure.

Level 5 — Substantive Contribution: use ML to generate credible new knowledge about an important problem.

1.19 Practitioner Track

Focus on problem formulation, preprocessing, baselines, model selection, evaluation, maintainability, monitoring, and reproducibility.

1.20 Researcher Track

Focus additionally on theory, sampling, measurement, uncertainty, external validity, temporal and subgroup generalization, benchmarking, novelty, and limitations.

1.21 Knowledge Check

1. What does it mean for a model to generalize?
2. Why can perfect training accuracy be undesirable?
3. How is a feature different from a target?
4. How does self-supervised learning differ from supervised learning?
5. Why should test data be protected during development?
6. Why does predictive importance not establish causality?
7. Why evaluate a simple baseline?
8. When might a benchmark dataset be preferable to NHANES or IPEDS?
9. What distinguishes a parameter from a hyperparameter?
10. What distinguishes a model result from a research contribution?

1.22 Professor's Challenge

A team predicts student non-retention with **92% accuracy** and concludes that the model should be deployed to identify students who need intervention.

Evaluate the conclusion as both a data scientist and researcher. Consider prevalence, naive baselines, sensitivity and specificity, error consequences, calibration, leakage, temporal validation, subgroup performance, target validity, intended use, and the difference between predicting risk and estimating intervention effects.

Conclude with the additional evidence you would require before supporting deployment.

1.23 Reproducibility Check

Verify that you can explain the intended prediction problem, observation, target, information available at prediction time, protection of evaluation data, data provenance, supported and unsupported claims, and what another analyst needs to reproduce the work.

1.24 Key Takeaways

- Machine learning is about learning useful patterns and evaluating generalization.
- Training performance is not evidence of real-world performance.
- AI, ML, deep learning, foundation models, and generative AI are related but distinct.
- Computer vision is a major ML workload that can use multiple learning paradigms.
- Statistics and mathematics provide the language for understanding learning and uncertainty.

- Prediction, explanation, inference, and causation require different evidence.
- Benchmark and domain research datasets serve different purposes.
- Responsible and reproducible ML begins with problem formulation.
- Using an ML algorithm does not by itself create a research contribution.

1.25 Looking Ahead

Chapter 1 established machine learning as a disciplined process of learning from data rather than merely selecting algorithms. Chapter 2 turns that perspective into practice by developing the Python foundations used throughout the book to represent data, express computational logic, inspect results, and build reproducible machine-learning workflows.

2 Python for Machine Learning

2.1 Learning Objectives

By the end of this chapter, you should be able to:

- configure a reproducible Python workspace for machine learning;
- use the Python language features most relevant to data science;
- work with NumPy arrays and pandas DataFrames;
- move from a DataFrame to feature matrix X and target vector y ;
- distinguish storage types from measurement meaning;
- create purposeful exploratory visualizations;
- write reusable functions and debug systematically;
- document data provenance and transformations;
- use Git, GitHub, Quarto, and AI assistance as parts of a reproducible workflow;
- audit code that runs successfully but is analytically questionable.

2.2 Why This Matters

A machine learning model is only as trustworthy as the computational process that produced it.

Python makes it easy to move from an idea to executable analysis. That speed is valuable, but code can run successfully while embodying poor assumptions, undocumented transformations, leakage, fragile paths, or irreproducible dependencies.

Central Question

How do we move from exploratory Python code to a trustworthy, reproducible computational workflow for machine learning?

2.2.1 Connection to Chapter 1

Chapter 1 framed machine learning as a complete evidence-building process rather than an exercise in calling algorithms. This chapter provides the computational language for that process. Python will be used not only to fit models, but also to represent data, automate transformations, visualize evidence, document decisions, and reproduce analyses across the chapters that follow.

2.3 Why Python?

Python combines a readable general-purpose language with a large scientific-computing ecosystem.

Library	Primary role
NumPy	numerical arrays and vectorized computation (Harris et al. 2020)
pandas	tabular data manipulation
Matplotlib	foundational plotting (Hunter 2007)
Plotly	interactive visualization when interactivity adds value
scikit-learn	preprocessing, modeling, evaluation, pipelines (Pedregosa et al. 2011)
SciPy	scientific and numerical routines (Virtanen et al. 2020)
statsmodels	statistical modeling and inference in selected applications

The goal is not to memorize library names. It is to understand how the tools participate in a coherent analytical workflow.

2.4 Your Machine Learning Workspace

Our primary workflow uses **VS Code**, **Python**, **Quarto**, **Git**, and **GitHub**. Quarto can execute Python code embedded in Markdown so generating code remains connected to tables, figures, and interpretation (Quarto 2026).

A small project might look like:

```
my-ml-project/  
  data/  
    raw/  
    processed/  
  figures/  
  src/  
    prepare_data.py  
  analysis.qmd  
  README.md  
  requirements.txt  
  .gitignore
```

Preserve the Evidence

Raw data should normally remain raw. Write code that transforms source data into processed data rather than manually editing the source.

2.5 Reproducible Python Environments

Different projects can require different package versions. Python's `venv` module creates isolated environments (Python Software Foundation 2026).

```
python -m venv .venv
```

Windows PowerShell:

```
.venv\Scripts\Activate.ps1
```

macOS/Linux:

```
source .venv/bin/activate
```

Install packages:

```
python -m pip install numpy pandas matplotlib scikit-learn jupyter
```

Record dependencies:


```
python -m pip freeze > requirements.txt
```

Recreate them:

```
python -m pip install -r requirements.txt
```

A project should not depend silently on whatever happens to be installed on one computer.

2.6 Python You Actually Need

2.6.1 Variables and Objects

```
course = "Machine Learning"
edition = 2
threshold = 0.80
is_reproducible = True
type(course), type(edition), type(threshold), type(is_reproducible)
```

(str, int, float, bool)

2.6.2 Lists and Dictionaries

```
features = ["age", "bmi", "sleep_hours", "activity_minutes"]

study = {
    "dataset": "NHANES",
    "unit_of_analysis": "participant",
    "target": "systolic_blood_pressure"
}
```

2.6.3 Conditions

```
score = 0.87
interpretation = "meets illustrative threshold" if score >= 0.80 else "below threshold"
interpretation
```

'meets illustrative threshold'

2.6.4 Functions

```
def mean_absolute_error_manual(actual, predicted):  
    errors = [abs(a - p) for a, p in zip(actual, predicted)]  
    return sum(errors) / len(errors)  
  
mean_absolute_error_manual([10, 12, 14], [11, 10, 15])
```

1.3333333333333333

Functions convert repeated analytical logic into reusable, auditable units.

2.7 NumPy: Thinking in Arrays

```
import numpy as np  
  
x = np.array([2.0, 4.0, 6.0, 8.0])  
2 * x
```

array([4., 8., 12., 16.])

Vectorized operations allow us to operate on arrays as mathematical objects.

```
X_demo = np.array([  
    [1.2, 3.1],  
    [2.4, 1.8],  
    [3.0, 4.2]  
)  
  
X_demo.shape
```

(3, 2)

This connects to

$$X \in \mathbb{R}^{n \times p},$$

where n is the number of observations and p the number of features.

Code and Mathematics Describe the Same Object

If `X.shape` returns `(800, 12)`, your feature matrix has $n = 800$ observations and $p = 12$ features.

2.8 Images Are Arrays Too

Tabular data are not the only data machine-learning systems consume. A grayscale image can be represented numerically as a two-dimensional array, while a color image is commonly represented with an additional channel dimension.

For an image of height (H) and width (W),

$$I \in \mathbb{R}^{H \times W}$$

for grayscale data, while a color image can be represented as

$$I \in \mathbb{R}^{H \times W \times C},$$

where (C) is the number of channels.

```
from sklearn.datasets import load_digits
import matplotlib.pyplot as plt

digits = load_digits()

image = digits.images[0]
label = digits.target[0]

image.shape, label
```

```
((8, 8), np.int64(0))
```

```
plt.figure(figsize=(4, 4))
plt.imshow(image, cmap="gray")
plt.title(f"Target label: {label}")
plt.axis("off")
plt.show()
```

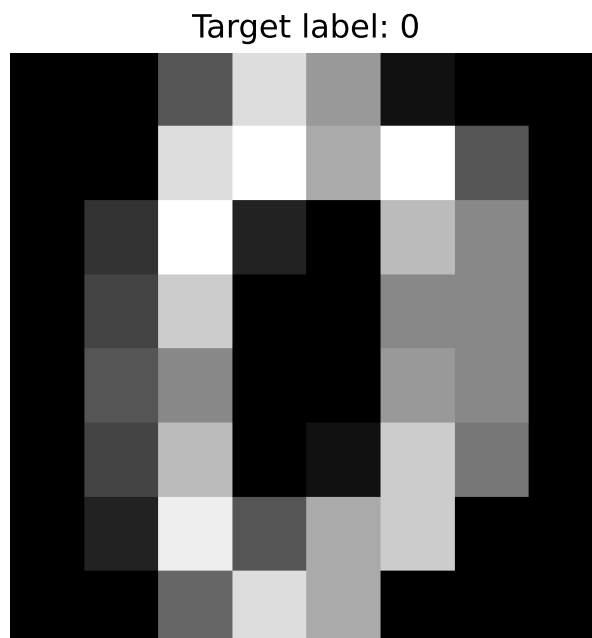


Figure 2.1: A handwritten digit represented as a numerical image array.

This is an important bridge to computer vision: an image is visually meaningful to a person, but a learning algorithm receives a numerical representation.

Flattening an $(H \times W)$ image produces a vector with (HW) values:

$$I \in \mathbb{R}^{H \times W} \rightarrow x_i \in \mathbb{R}^{HW}.$$

That transformation preserves pixel values but makes the original two-dimensional neighborhood structure less explicit. Later chapters will examine why representation matters so much for visual learning.

2.9 pandas: Working with Tabular Data

pandas provides labeled tabular data structures for data manipulation and analysis (pandas development team 2026).

```
import pandas as pd

students = pd.DataFrame({
```

```

    "study_hours": [4, 7, 2, 9, 6],
    "attendance": [0.82, 0.94, 0.71, 0.97, 0.88],
    "exam_score": [72, 91, 65, 95, 84]
})

```

```
students
```

	study_hours	attendance	exam_score
0	4	0.82	72
1	7	0.94	91
2	2	0.71	65
3	9	0.97	95
4	6	0.88	84

Inspect before transforming:

```
students.shape
```

```
(5, 3)
```

```
students.dtypes
```

```

study_hours      int64
attendance        float64
exam_score        int64
dtype: object

```

```
students.describe()
```

	study_hours	attendance	exam_score
count	5.000000	5.000000	5.000000
mean	5.600000	0.864000	81.400000
std	2.701851	0.103586	12.660964
min	2.000000	0.710000	65.000000
25%	4.000000	0.820000	72.000000
50%	6.000000	0.880000	84.000000
75%	7.000000	0.940000	91.000000

	study_hours	attendance	exam_score
max	9.000000	0.970000	95.000000

Ask what one row represents, which columns are identifiers, which variable is the target, whether units are documented, and whether missingness has meaning.

2.9.1 Selecting and Filtering

```
students[["study_hours", "exam_score"]]
```

	study_hours	exam_score
0	4	72
1	7	91
2	2	65
3	9	95
4	6	84

```
students.loc[students["attendance"] >= 0.90]
```

	study_hours	attendance	exam_score
1	7	0.94	91
3	9	0.97	95

2.10 From a DataFrame to (X) and (y)

```
X = students[["study_hours", "attendance"]]
y = students["exam_score"]

print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (5, 2)
y shape: (5,)
```

Construct these intentionally. Leaving the target—or a post-outcome proxy—inside X can create leakage.

2.11 Data Types Are Analytical Decisions

A ZIP code can be stored as an integer without being a quantitative measurement. The same distinction applies to IDs, diagnosis codes, ordinal responses, dates, and text.

Storage Type	Measurement Meaning
Never decide how to model a variable solely from <code>df.dtypes</code> . Ask what the variable represents.	

2.12 Missing Data

```
example = pd.DataFrame({
    "age": [22, 35, np.nan, 51],
    "income": [42000, np.nan, 61000, 79000]
})

example.isna().sum()
```

```
age      1
income   1
dtype: int64
```

Deleting every incomplete row or replacing every missing value with a mean is not automatically defensible. Later chapters will treat imputation within pipelines. Here, develop the habit of inspecting and documenting missingness before acting.

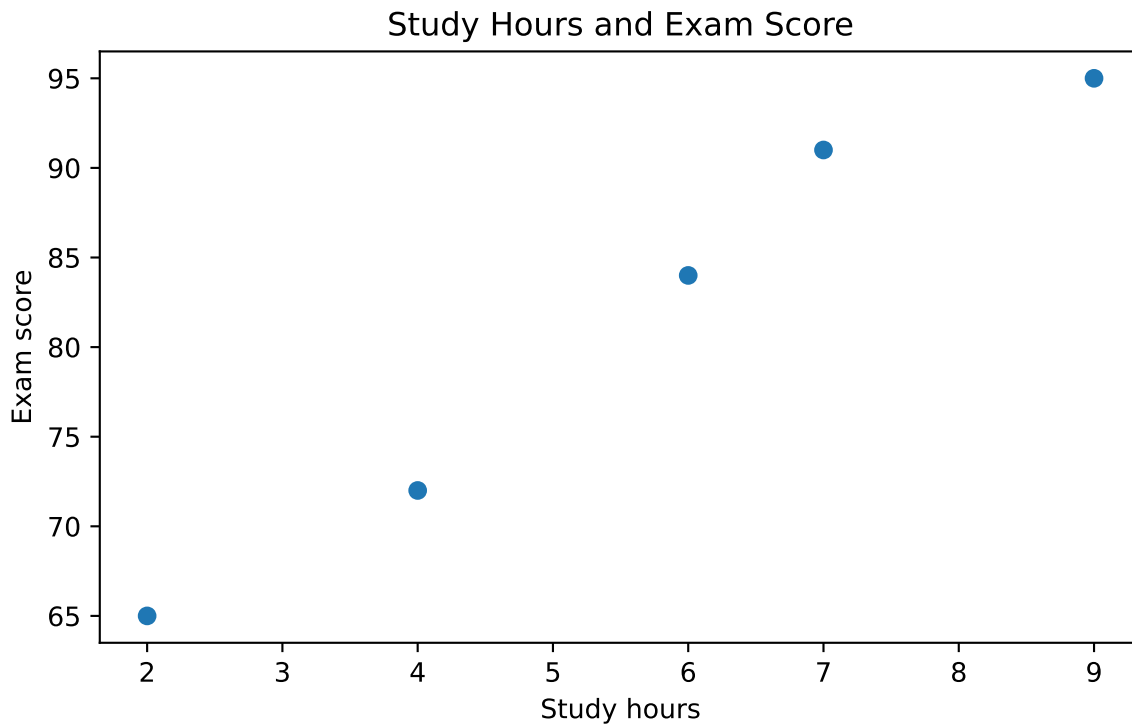
2.13 Exploratory Visualization

Your current course material uses Plotly with Gapminder, Iris, and Tips to introduce bar, line, scatter, and histogram graphics. We retain the pedagogical value while shifting the emphasis from **making charts** to **answering questions**.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(7, 4))
plt.scatter(students["study_hours"], students["exam_score"])
```

```
plt.xlabel("Study hours")
plt.ylabel("Exam score")
plt.title("Study Hours and Exam Score")
plt.show()
```



Ask:

What feature of the data am I trying to understand?

Interactive Plotly graphics are valuable when interaction materially improves understanding. Stable static figures are often preferable for publication artifacts.

2.14 Data Provenance

A line such as `pd.read_csv("data.csv")` conceals the dataset's history.

Record at least:

Field	What to record
Source	authoritative provider
Dataset	official dataset name
Location/API	source endpoint
Retrieval date	when acquired
Version/cycle	release or survey cycle
Unit of analysis	what one row represents
Transformations	raw-to-processed steps
License/terms	applicable use conditions

Provenance Principle

If an analytical decision can change the result, that decision belongs in the reproducible record.

2.15 Three Data Tracks in Practice

2.15.1 Open ML Benchmark Track

```
from sklearn.datasets import load_wine

wine = load_wine(as_frame=True)
wine.frame.head()
```

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavanoid_p
0	14.23	1.71	2.43	15.6	127.0	2.80	3.06	0.28
1	13.20	1.78	2.14	11.2	100.0	2.65	2.76	0.26
2	13.16	2.36	2.67	18.6	101.0	2.80	3.24	0.30
3	14.37	1.95	2.50	16.8	113.0	3.85	3.49	0.24
4	13.24	2.59	2.87	21.0	118.0	2.80	2.69	0.39

Benchmark data are excellent for learning workflows and algorithms. Convenience alone does not create research novelty.

2.15.2 Health Track

NHANES provides public demographic, examination, laboratory, behavioral, and other health data (National Center for Health Statistics 2024). Researchers must understand survey cycles, variable documentation, units, eligibility, missing codes, and the intended population.

2.15.3 Education Track

IPEDS and related public postsecondary data support institutional analyses involving enrollment, completions, graduation, finances, staffing, and other characteristics (National Center for Education Statistics 2026). The unit of analysis may be an institution or institution-year rather than a person.

The Python syntax can be similar across tracks. The scientific meaning is not.

2.16 Beyond the Algorithm: Code as Evidence

What Does Code Allow Us to Know?

In data science, code is not merely a set of instructions to a computer. It becomes part of the argument supporting an analytical claim.

If a paper reports that a model achieved an AUC of 0.87, that statement depends on decisions embedded in code: exclusions, missing-data treatment, preprocessing order, data splitting, feature construction, and evaluation.

Reproducible code does not guarantee a correct conclusion. But without a transparent computational record, it becomes harder to determine why the conclusion should be trusted.

Could another knowledgeable analyst reconstruct the path from raw evidence to reported claim?

2.17 Writing Reusable Analytical Code

```
def summarize_numeric_column(df, column):  
    return {  
        "n": df[column].notna().sum(),  
        "missing": df[column].isna().sum(),  
        "mean": df[column].mean(),  
        "median": df[column].median()
```

```
}  
  
summarize_numeric_column(students, "exam_score")
```

```
{'n': np.int64(5),  
 'missing': np.int64(0),  
 'mean': np.float64(81.4),  
 'median': np.float64(84.0)}
```

As an analysis stabilizes, repeated logic should migrate into functions.

2.18 Debugging: Read the Evidence

A disciplined process is:

1. read the final error message;
2. identify the line that failed;
3. inspect the relevant object;
4. form a hypothesis;
5. test the smallest correction;
6. verify that the correction solves the analytical problem, not merely the syntax.

A Traceback Is Evidence

Do not treat an error message as noise to paste immediately into an AI assistant. Read it first.

2.19 Git and GitHub

A minimal workflow:

```
git status  
git add .  
git commit -m "Document Chapter 2 analysis workflow"  
git push
```

Do not commit credentials, private data, or `.venv`.

2.20 Quarto: Narrative + Computation + Evidence

Quarto can execute Python within Markdown so code, output, prose, equations, and citations remain connected (Quarto 2026).

```
#| label: fig-example
#| fig-cap: "A meaningful caption"
#| warning: false
```

The goal is to generate outputs reproducibly rather than copy them manually into a report.

2.21 Try It

Using the Wine dataset:

1. determine the number of observations and variables;
2. identify the target;
3. inspect data types;
4. calculate descriptive statistics for three features;
5. create one plot that answers a specific question;
6. interpret it without making a causal claim.

2.22 Hands-On Lab: Build a Reproducible Mini-Project

Create:

```
chapter02-lab/
  data/
    raw/
    processed/
  figures/
  src/
  analysis.qmd
  README.md
  requirements.txt
  .gitignore
```

Then create `.venv`, install packages, load a dataset, document provenance, inspect it, define X and y , create purposeful figures, move one repeated transformation into a function, render the Quarto analysis, and commit the project.

The goal is not yet the best predictive model. It is a workflow another analyst can understand and reproduce.

2.23 AI Pair Programmer

Ask an AI coding assistant to generate a function that summarizes missing values in a DataFrame. Audit whether it reports counts and percentages correctly, handles edge cases, mutates data, and makes claims consistent with the code.

AI-Generated Code Can Run and Still Be Wrong

Successful execution is not validation.

2.24 Practitioner Track

Focus on environments, project structure, arrays, DataFrames, visualization, feature/target construction, reusable functions, debugging, and version control.

2.25 Researcher Track

Focus additionally on provenance, measurement meaning, preservation of raw data, auditable transformations, and a reproducible path from source evidence to reported output.

2.26 Professor's Challenge: Audit the Analysis

Consider:

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

df = pd.read_csv("student_data.csv")
df = df.dropna()
```

```

X = df.drop(columns=["student_id"])
y = df["retained"]

X_train, X_test, y_train, y_test = train_test_split(X, y)

model = RandomForestClassifier()
model.fit(X_train, y_train)

print(model.score(X_test, y_test))

```

The code may run. Audit it before rewriting it.

Consider target leakage, undocumented row deletion, class imbalance, random vs stratified/grouped/temporal splitting, random seeds, post-outcome predictors, categorical representation, the meaning of `score()`, reliance on one split, provenance, population, and environment reproducibility.

2.27 Research Studio

Choose one track and create a data-intake memo containing dataset/source, analytical purpose, unit of analysis, target, feature families, measurement concerns, missingness, provenance, raw/processed structure, reproducible transformations, one leakage risk, and one question the dataset alone cannot answer.

2.28 Knowledge Check

1. Why use a virtual environment?
2. What does `X.shape == (500, 12)` mean?
3. How do NumPy arrays and pandas DataFrames differ conceptually?
4. Why can an integer column represent a category?
5. Why should raw data usually remain unchanged?
6. What is data provenance?
7. Why is `dropna()` not automatically defensible?
8. Why use functions for repeated analytical logic?
9. What does Quarto contribute to reproducibility?
10. Why is code that executes without error not necessarily analytically correct?

2.29 Reproducibility Check

Verify that your project reveals the Python environment, dependencies, data source, raw/processed distinction, transformations, construction of X and y , figure-generation code, result-generation code, and version history.

2.30 Key Takeaways

- Python is most useful within a disciplined analytical workflow.
- Virtual environments isolate dependencies.
- NumPy and pandas provide complementary data representations.
- Variable meaning matters more than storage type.
- Data provenance is part of analytical evidence.
- Visualization should answer questions rather than decorate reports.
- Functions, debugging, Git, and Quarto improve transparency.
- AI coding assistance should be audited rather than trusted by default.
- Code that runs is not necessarily code that supports a valid conclusion.
- The computational path from evidence to claim matters.

3 Statistics and Mathematics for Machine Learning

3.1 Learning Objectives

By the end of this chapter, you should be able to:

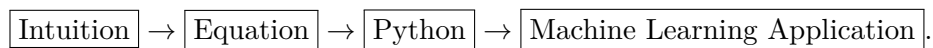
- explain how descriptive statistics summarize a sample without fully describing its data-generating process;
- interpret probability as a language for uncertainty;
- distinguish population quantities, sample statistics, parameters, and estimates;
- explain why sampling variation matters for machine learning;
- calculate and interpret mean, variance, standard deviation, covariance, and correlation;
- standardize features and explain why scaling matters for selected algorithms;
- represent observations, features, and model coefficients as vectors and matrices;
- calculate and interpret common distance measures and dot products;
- explain a loss function as a mathematical representation of model error;
- interpret derivatives and gradients as information about local change;
- implement gradient descent for a simple optimization problem;
- connect statistical and mathematical abstractions to machine-learning decisions;
- recognize that mathematical precision does not eliminate measurement, sampling, or modeling assumptions.

3.2 Why This Matters

Machine learning can be used by calling library functions without understanding the mathematics underneath them. But technical fluency becomes much deeper when we can explain **why** an algorithm behaves as it does.

Statistics helps us reason about variation, uncertainty, sampling, and relationships. Linear algebra gives us a compact language for representing many observations and features at once. Calculus helps us understand how model parameters can be adjusted to reduce error. Optimization connects those ideas to learning.

This chapter is therefore not a detached mathematics review. Every concept will follow a recurring path:



Central Question

What mathematical and statistical ideas allow us to understand what a machine-learning algorithm is actually doing?

3.3 From Data to Mathematical Representation

Suppose our data contain n observations and p features. We can represent the feature matrix as

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix} \in \mathbb{R}^{n \times p}.$$

The target can be represented as

$$y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}.$$

This notation is not merely decorative. It allows us to describe entire learning algorithms compactly.

3.4 Describing Data: Center and Spread

Consider a sample $x_1, \dots, x_n$.

3.4.1 Mean

The sample mean is

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i.$$

```
import numpy as np

scores = np.array([72, 75, 81, 84, 88, 90, 95])
scores.mean()
```

```
np.float64(83.57142857142857)
```

The mean is a useful summary, but it does not reveal the entire distribution.

3.4.2 Variance and Standard Deviation

Sample variance is

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2.$$

The sample standard deviation is

$$s = \sqrt{s^2}.$$

```
sample_variance = np.var(scores, ddof=1)
sample_sd = np.std(scores, ddof=1)

sample_variance, sample_sd
```

```
(np.float64(67.61904761904763), np.float64(8.223080178318076))
```

Variance measures squared dispersion; standard deviation returns that dispersion to the original measurement units.

Same Mean, Different Data

Two datasets can have the same mean and dramatically different variability, shape, outliers, and relationships. Summary statistics compress information. Compression is useful—but never neutral.

3.5 Visual Intuition: Distribution and Variation

```
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)
narrow = rng.normal(loc=0, scale=1, size=1000)
wide = rng.normal(loc=0, scale=3, size=1000)

plt.figure(figsize=(8, 5))
plt.hist(narrow, bins=35, alpha=0.55, label="SD 1")
plt.hist(wide, bins=35, alpha=0.45, label="SD 3")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.title("Similar Centers, Different Variation")
plt.legend()
plt.show()
```

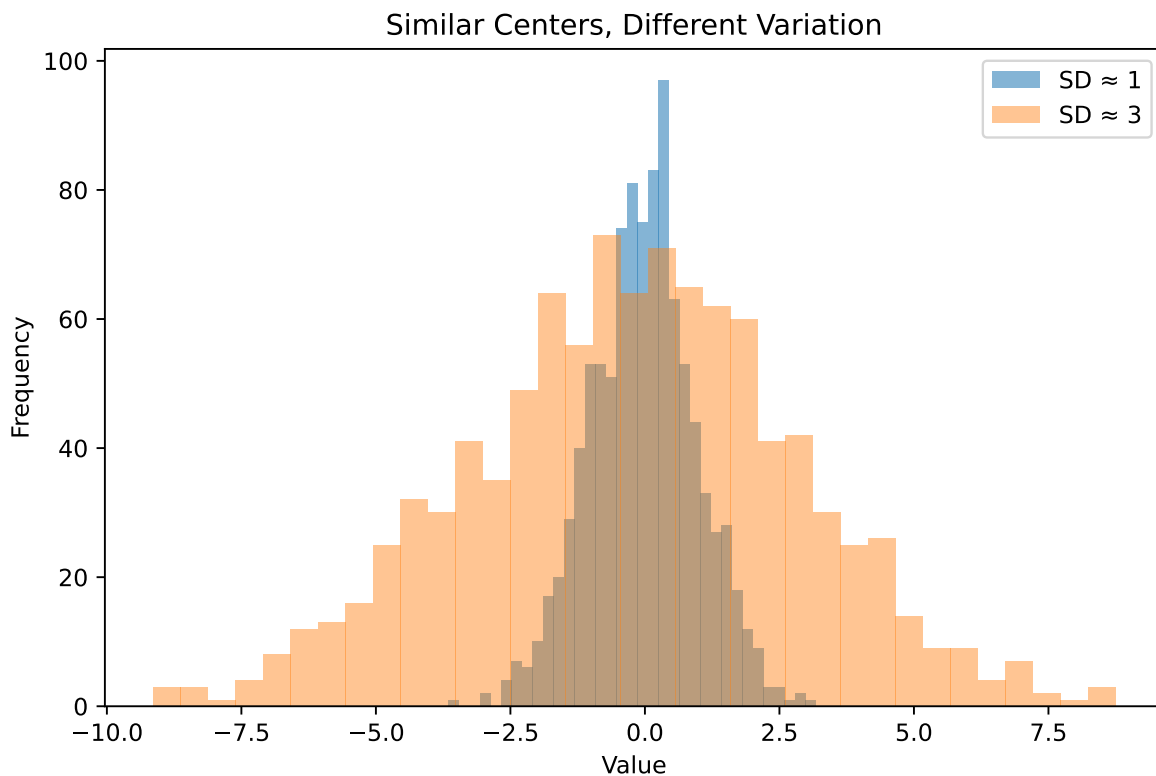


Figure 3.1: Two samples can have similar centers while differing substantially in spread.

Variation matters in ML because algorithms respond not only to where variables are centered but also to their scales and distributions.

3.6 Probability: A Language for Uncertainty

For an event A ,

$$0 \leq P(A) \leq 1.$$

If A and B are events, conditional probability is

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}, \quad P(B) > 0.$$

Conditional reasoning appears throughout machine learning. A classifier may estimate a probability such as

$$P(Y = 1 \mid X = x).$$

That is a statement about uncertainty conditional on the information represented by x .

3.6.1 Expected Value

For a discrete random variable X ,

$$E[X] = \sum_x xP(X = x).$$

Expected value is a probability-weighted average. Many machine-learning objectives can be interpreted as attempts to minimize expected loss.

3.7 Probability Distributions

A probability distribution describes how probability is allocated across possible values of a random variable. SciPy provides extensive tools for continuous and discrete probability distributions as well as sample statistics and statistical tests (SciPy Community 2026).

Common distributions we will encounter include:

- Bernoulli — binary outcomes;
- Binomial — counts of successes across trials;
- Normal — continuous measurements and error models;
- Poisson — event counts;
- empirical distributions — distributions represented directly by observed data.

3.7.1 A Normal Distribution in Python

```
from scipy.stats import norm

probability = norm.cdf(1.96) - norm.cdf(-1.96)
probability
```

```
np.float64(0.950004209703559)
```

The code computes the probability that a standard normal random variable falls between -1.96 and 1.96.

The important lesson is not memorizing 1.96. It is understanding that probability models provide mathematical representations of uncertainty.

3.8 Population, Sample, Parameter, Statistic

Suppose we want to know the mean sleep duration of a target population.

The **population** is the full set of units about which we want to make a statement.

A **sample** is the subset observed.

A **parameter** is a population quantity such as μ .

A **statistic** is a sample quantity such as $\bar{x}$.

We use statistics to learn about parameters, but the statistic changes from sample to sample.

3.8.1 Sampling Variation by Simulation

```
rng = np.random.default_rng(42)

population = rng.normal(loc=7.0, scale=1.5, size=100_000)
sample_means = []

for _ in range(2000):
    sample = rng.choice(population, size=100, replace=True)
    sample_means.append(sample.mean())

plt.figure(figsize=(8, 5))
plt.hist(sample_means, bins=35)
plt.axvline(np.mean(population), linestyle="--", label="Population mean")
plt.xlabel("Sample mean")
plt.ylabel("Frequency")
plt.title("Repeated Samples Produce Different Estimates")
plt.legend()
plt.show()
```

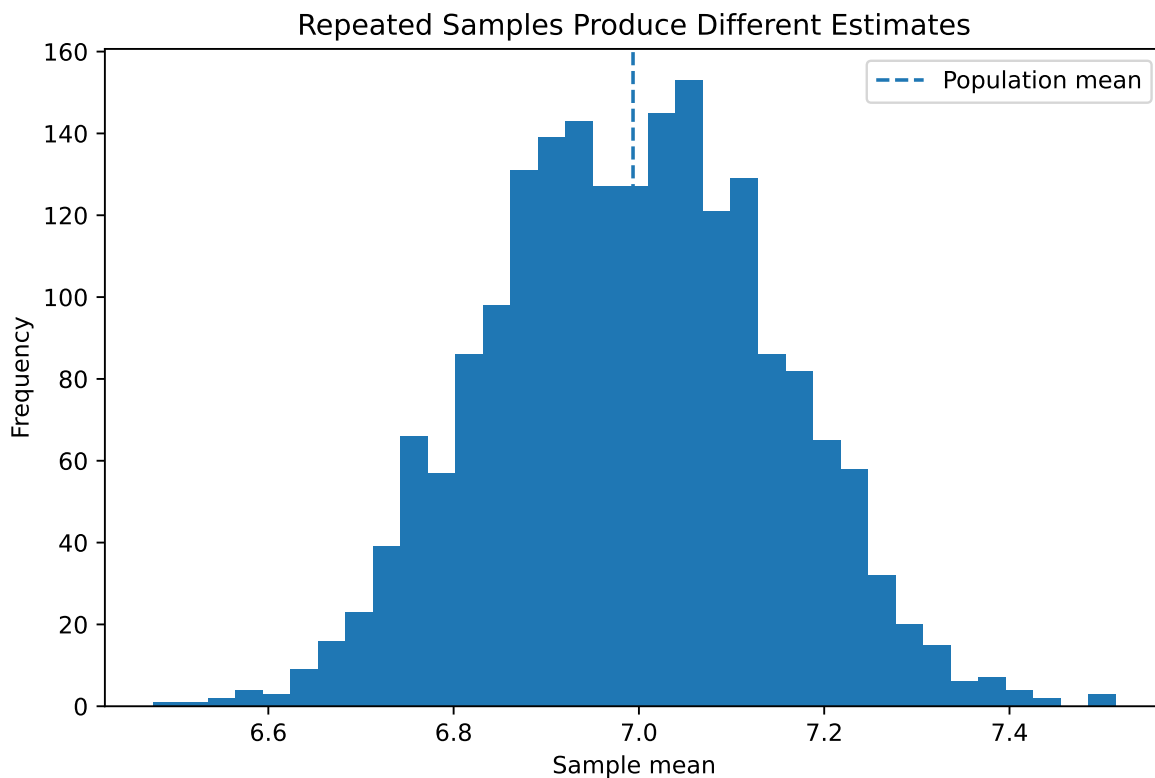


Figure 3.2: Sampling distribution of the mean across repeated samples.

The same sampling process does not produce exactly the same estimate every time.

Connection to Machine Learning

Training data are samples. Model performance, fitted parameters, feature importance, and selected hyperparameters can therefore vary with the sample. A single train/test split is one realization—not a law of nature.

3.9 Standard Error and Uncertainty

For an independent sample, the estimated standard error of the mean is commonly written

$$SE(\bar{x}) = \frac{s}{\sqrt{n}}.$$

As n grows, the standard error generally becomes smaller, all else equal.

But large samples do not automatically solve poor measurement, biased sampling, leakage, confounding, or dataset shift.

3.10 Prediction, Inference, and Different Kinds of Questions

Statistics and machine learning may use the same data and even the same models while asking different questions. In particular, explanatory and predictive modeling pursue distinct goals and can imply different modeling choices (Shmueli 2010).

Prediction: How accurately can information in (X) predict (Y) for observations not used to fit the model?

Statistical inference: What does the observed sample permit us to conclude about a population parameter or relationship?

Causal inference: What would happen to (Y) if we intervened to change (X)?

Prediction $\neq$ Statistical Inference $\neq$ Causal Inference

A model can predict well even when individual coefficients are difficult to interpret. Conversely, a precisely estimated association may add little predictive value. The evidence required depends on the claim.

3.10.1 Predictive Signal Without a Unique Explanation

Correlated predictors can preserve predictive signal while making allocation of that signal across individual coefficients unstable.

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
import numpy as np
import matplotlib.pyplot as plt

rng_pi = np.random.default_rng(42)
n_pi = 600
x1_pi = rng_pi.normal(size=n_pi)
x2_pi = x1_pi + rng_pi.normal(scale=0.12, size=n_pi)
y_pi = 3 * x1_pi + rng_pi.normal(scale=1.0, size=n_pi)
X_pi = np.column_stack([x1_pi, x2_pi])
```



```

Xtr_pi, Xte_pi, ytr_pi, yte_pi = train_test_split(
    X_pi, y_pi, test_size=0.30, random_state=42
)

model_pi = LinearRegression().fit(Xtr_pi, ytr_pi)
pred_pi = model_pi.predict(Xte_pi)
rmse_pi = np.sqrt(mean_squared_error(yte_pi, pred_pi))

coef_pi = []
for _ in range(250):
    idx_pi = rng_pi.choice(len(Xtr_pi), len(Xtr_pi), replace=True)
    coef_pi.append(
        LinearRegression().fit(Xtr_pi[idx_pi], ytr_pi[idx_pi]).coef_
    )

coef_pi = np.asarray(coef_pi)

plt.figure(figsize=(8, 5))
plt.scatter(coef_pi[:, 0], coef_pi[:, 1], alpha=0.45, s=22)
plt.xlabel(r"Estimated coefficient for $x_1$")
plt.ylabel(r"Estimated coefficient for $x_2$")
plt.title("Correlated Predictors and Coefficient Instability")
plt.show()

print(f"Test RMSE: {rmse_pi:.3f}")
print("Fitted coefficients:", model_pi.coef_)

```

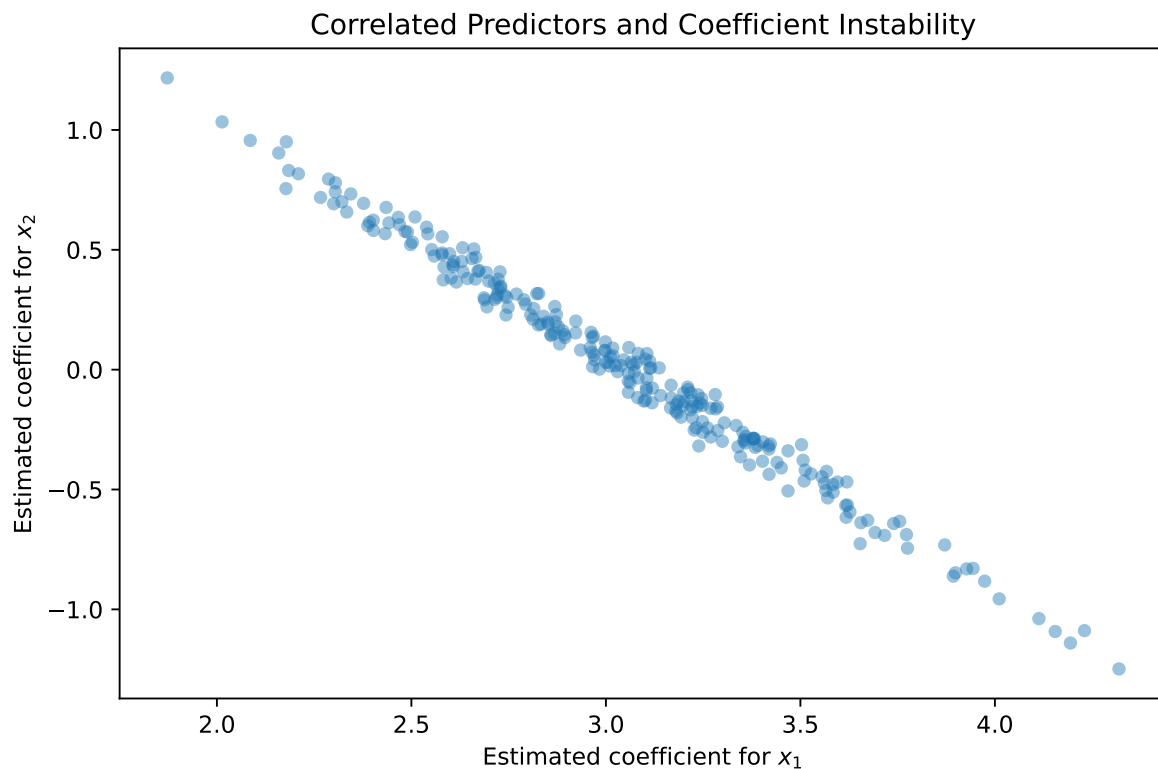


Figure 3.3: Correlated predictors can preserve predictive signal while individual coefficient estimates vary across bootstrap samples.

Test RMSE: 0.965

Fitted coefficients: [2.9705437 0.08362775]

The model can retain useful predictive information while repeated samples redistribute the apparent contribution between (x_1) and (x_2).

Stable prediction $\nRightarrow$ stable interpretation of every parameter

Uncertainty Is Information

Uncertainty is not an embarrassment that disappears when a model becomes sophisticated. Sampling variation, measurement error, model specification, hyperparameter selection, and distribution shift can all matter.

Ask not only **What is my estimate?** but also **How much could this conclusion change under another defensible sample, split, specification, or population?**

Research Reality Check

Strong predictive performance does not, by itself, demonstrate that predictors cause the outcome, coefficients are stable scientific effects, the model reveals the true data-generating mechanism, or the same relationships hold elsewhere.

Match the strength of the claim to the design and strength of the evidence.

3.11 Covariance and Correlation

Covariance describes how two variables vary together:

$$\text{Cov}(X, Y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}).$$

Correlation standardizes covariance:

$$r_{XY} = \frac{\text{Cov}(X, Y)}{s_X s_Y}.$$

For Pearson correlation,

$$-1 \leq r \leq 1.$$

```
study_hours = np.array([2, 3, 5, 6, 8, 9])
exam_score = np.array([61, 66, 74, 78, 88, 92])

np.corrcoef(study_hours, exam_score)[0, 1]
```

```
np.float64(0.999252397692809)
```

Correlation can be useful for exploration, feature analysis, and understanding redundancy. It does not establish causation.

3.12 Standardization: Putting Features on Comparable Scales

A standard score is

$$z_i = \frac{x_i - \bar{x}}{s}.$$

In machine-learning implementations, scaling statistics should be learned from the **training data** and then applied to later data. scikit-learn's `StandardScaler` centers features and scales them using statistics learned during `fit` (scikit-learn developers 2026e).

```
from sklearn.preprocessing import StandardScaler

X_scale_demo = np.array([
    [18, 22000],
    [25, 48000],
    [40, 76000],
    [55, 110000]
])

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_scale_demo)

X_scaled
```

```
array([[ -1.15736053, -1.28397747],
       [-0.66635909, -0.48913427],
       [ 0.38578684,  0.3668507 ],
       [ 1.43793278,  1.40626103]])
```

Why can scaling matter? Many learning objectives depend on distances, dot products, gradients, or regularization. A feature whose numerical scale is orders of magnitude larger than another can disproportionately influence such calculations. scikit-learn explicitly notes this issue for algorithms involving RBF kernels and L1/L2 regularization (scikit-learn developers 2026e).

Statistical Calculation Can Become Leakage

Do **not** compute scaling parameters using the entire dataset before evaluation. The test set must not influence training-time preprocessing.

We will solve this systematically with pipelines in Chapter 5.

3.13 Linear Algebra: The Language of Many Models

NumPy provides operations for dot products, matrix multiplication, norms, decompositions, and other linear-algebra routines (NumPy Developers 2026).

3.13.1 Vectors

A feature vector for observation (i) might be

$$x_i = \begin{bmatrix} x_{i1} \\ x_{i2} \\ x_{i3} \end{bmatrix}.$$

A coefficient vector might be

$$\beta = \begin{bmatrix} \beta_1 \\ \beta_2 \\ \beta_3 \end{bmatrix}.$$

Their dot product is

$$x_i^\top \beta = \sum_{j=1}^p x_{ij} \beta_j.$$

```
x = np.array([2.0, 5.0, 1.5])
beta = np.array([0.8, -0.2, 1.1])

np.dot(x, beta)
```

```
np.float64(2.25)
```

This operation appears throughout linear models, neural networks, similarity calculations, and many other methods.

3.13.2 Matrix-Vector Multiplication

For many observations,

$$\hat{y} = X\beta.$$

```
X_demo = np.array([
    [1.0, 2.0],
    [1.0, 4.0],
    [1.0, 6.0]
])

beta = np.array([3.0, 1.5])

X_demo @ beta
```

```
array([ 6.,  9., 12.])
```

The same equation produces predictions for all observations at once.

3.14 Distance and Similarity

For vectors x and z , Euclidean distance is

$$d(x, z) = \sqrt{\sum_{j=1}^p (x_j - z_j)^2}.$$

```
x = np.array([1.0, 2.0])
z = np.array([4.0, 6.0])

np.linalg.norm(x - z)
```

```
np.float64(5.0)
```

Distance is central to methods such as nearest neighbors, clustering, and some anomaly-detection approaches.

3.14.1 Why Scaling Changes Distance

```
person_a = np.array([20, 30_000])
person_b = np.array([50, 31_000])

np.linalg.norm(person_a - person_b)
```

```
np.float64(1000.4498987955369)
```

The income difference dominates the age difference numerically. This does not necessarily mean income is substantively more important.

Distance Is a Modeling Choice

Before asking which observations are “close,” ask:

Close according to which representation, scale, and metric?

3.15 Functions, Loss, and Learning

For observation (i), a supervised model can be written as

$$\hat{y}_i = f_{\theta}(x_i).$$

To learn θ , we need a way to quantify error.

For regression, mean squared error is

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

```
y_true = np.array([3.0, 5.0, 7.0])
y_pred = np.array([2.5, 5.5, 8.0])

mse = np.mean((y_true - y_pred) ** 2)
mse
```

```
np.float64(0.5)
```

The loss function translates the vague instruction “fit the data well” into a mathematical objective.

That translation has consequences. Different loss functions value errors differently.

3.16 Derivatives: Measuring Local Change

For a function $f(\theta)$, the derivative

$$\frac{df}{d\theta}$$

describes how the function changes locally as θ changes.

Consider

$$J(\theta) = (\theta - 3)^2.$$

Then

$$\frac{dJ}{d\theta} = 2(\theta - 3).$$

If $\theta > 3$, the derivative is positive. If $\theta < 3$, it is negative. At $\theta = 3$, the derivative is zero.

3.17 Gradients: Change in Multiple Dimensions

For parameters

$$\theta = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_p \end{bmatrix},$$

the gradient is

$$\nabla J(\theta) = \begin{bmatrix} \frac{\partial J}{\partial \theta_1} \\ \frac{\partial J}{\partial \theta_2} \\ \vdots \\ \frac{\partial J}{\partial \theta_p} \end{bmatrix}.$$

The gradient points in the direction of steepest local increase. To reduce a differentiable objective, we move in the opposite direction.

3.18 Gradient Descent

A basic update is

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t),$$

where η is the **learning rate**.

3.18.1 Watch Optimization Happen

```
theta = -4.0
learning_rate = 0.15
history = [theta]

for _ in range(20):
    gradient = 2 * (theta - 3)
    theta = theta - learning_rate * gradient
    history.append(theta)

grid = np.linspace(-5, 7, 300)
loss = (grid - 3) ** 2

plt.figure(figsize=(8, 5))
plt.plot(grid, loss, label=r"$J(\theta)=(\theta-3)^2$")
plt.scatter(history, [(h - 3) ** 2 for h in history], label="Gradient descent steps")
plt.xlabel(r"$\theta$")
plt.ylabel("Loss")
plt.title("Learning by Following the Gradient")
plt.legend()
plt.show()
```

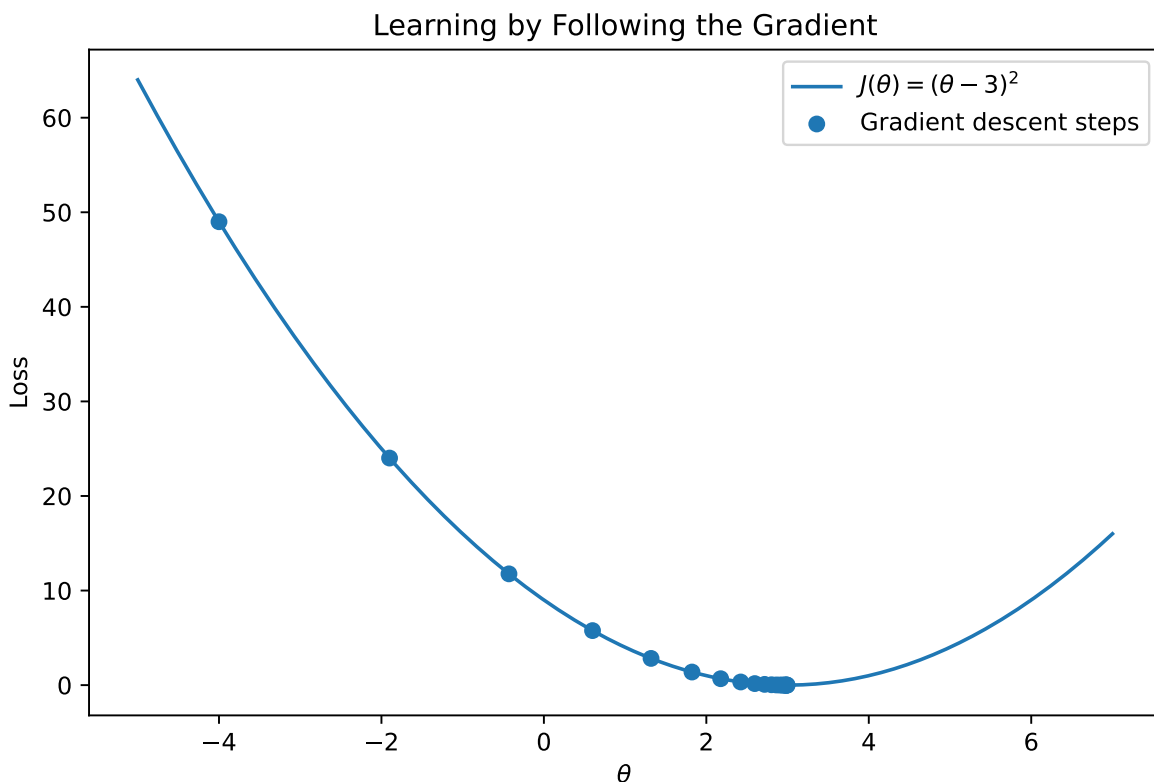


Figure 3.4: Gradient descent iteratively approaches the minimum of a simple loss function.

The algorithm does not “know” that the answer is 3. It repeatedly uses local information about the slope to update the parameter.

3.18.2 Learning Rate

If η is too small, learning can be slow. If it is too large, updates may overshoot or diverge.

This simple idea will reappear in neural networks and other optimization-based methods.

3.19 From Intuition to Equation to Python to ML

Consider feature standardization.

Intuition: features measured on very different scales may distort distance- or optimization-based learning.

Equation:

$$z = \frac{x - \mu}{\sigma}.$$

Python:

```
scaler = StandardScaler()
```

ML application: support vector machines, regularized linear models, nearest-neighbor methods, neural networks, and other methods can be sensitive to feature scale.

This four-part pattern is how we will use mathematics throughout the book.

3.20 Beyond the Algorithm: The Map Is Not the Territory

Mathematical Precision and Substantive Truth

A model can be mathematically precise and still represent the world poorly.

When we write a person as a vector,

$$x = [x_1, x_2, \dots, x_p],$$

we have not represented the person. We have represented selected measurements about that person.

When we minimize a loss function, we have not defined everything society or a scientific discipline values. We have defined a mathematical criterion.

Abstraction is one of the great powers of data science because it allows complex phenomena to become computationally tractable. But abstraction also removes information.

The disciplined data scientist therefore asks two questions:

1. **Is the mathematics correct?**
2. **Is the mathematical representation appropriate for the phenomenon and decision?**

The first question concerns computation. The second requires domain knowledge, measurement, judgment, and sometimes ethics.

The map can be extraordinarily useful without becoming the territory.

3.21 Three Tracks: Same Mathematics, Different Meaning

3.21.1 Health Track

Suppose systolic blood pressure and age are positively correlated in a sample. The correlation summarizes an observed relationship. It does not establish that changing age would cause blood pressure to change in an intervention-like sense.

Scaling biomarkers may help an algorithm computationally, but it does not make different biomarkers substantively equivalent.

3.21.2 Education Track

Institutional tuition, enrollment, retention, and completion may have dramatically different scales. Standardization can help certain algorithms, but the meaning of a one-standard-deviation difference depends on the variable and population.

3.21.3 Open ML Benchmark Track

Benchmark datasets are useful for isolating mathematical behavior. We can intentionally rescale one feature and observe how a distance-based algorithm changes. This makes benchmark data especially useful for controlled learning experiments.

3.22 Try It

For the vector

$$x = [2, 4, 6, 8, 10],$$

1. calculate the mean;
2. calculate the sample variance;
3. calculate the sample standard deviation;
4. standardize each value;
5. verify that the standardized values have mean approximately zero;
6. explain why this transformation changes units but not ordering.

Then reproduce your calculations with NumPy.

3.23 Hands-On Lab: Mathematics You Can See

Use a public or built-in dataset with several numeric features.

1. inspect feature means and standard deviations;
2. create distributions for two features;
3. calculate a correlation matrix;
4. select two observations and calculate Euclidean distance;
5. standardize the features using **training data only**;
6. recalculate the distance;
7. explain why it changed;
8. fit a simple k-nearest-neighbors classifier before and after scaling;
9. compare performance;
10. explain why the result is algorithm-dependent rather than proof that scaling always improves ML.

3.24 AI Pair Programmer

Ask an AI assistant:

“Explain why standardization improves every machine-learning algorithm.”

The premise is intentionally flawed.

Audit the response:

- Does it challenge the word **every**?
- Does it distinguish distance/gradient-sensitive models from tree-based models?
- Does it explain why scaling must be learned from training data?
- Does it confuse standardization with normalization?
- Does it claim that standardized features become normally distributed?
- Which claims require verification?

Mathematical Language Can Make Wrong Answers Sound More Convincing

Equations and technical vocabulary do not substitute for valid reasoning.

3.25 Practitioner Track

A practitioner should be able to:

- summarize distributions and relationships;
- identify when scale matters;
- reason about vectors, matrices, and distances;
- interpret loss functions;
- understand optimization at a conceptual level;
- recognize sampling variability in model results.

3.26 Researcher Track

A researcher should additionally be able to:

- distinguish samples from target populations;
- reason about uncertainty and sampling;
- separate association from causation;
- connect mathematical representations to measurement assumptions;
- recognize that estimates and model metrics vary across samples;
- justify mathematical choices in relation to the research question
- distinguish predictive, inferential, and causal questions before interpreting model output;
- report uncertainty appropriate to the scientific claim.

3.27 Professor's Challenge: When Mathematics Misleads

A research team compares two classifiers for predicting a health outcome.

Model A has an accuracy of 0.91.

Model B has an accuracy of 0.88.

The team concludes:

“Model A is mathematically superior and should therefore be used.”

Critique the conclusion.

Your analysis should address:

- outcome prevalence;
- sampling variation;
- uncertainty in the performance estimates;

- sensitivity and specificity;
- calibration;
- asymmetric error costs;
- subgroup performance;
- external validity;
- whether the same test observations were used for both models;
- whether preprocessing was fitted only on training data;
- whether a three-percentage-point difference is practically meaningful;
- whether mathematical superiority can be defined by a single metric.

Conclude by explaining what additional evidence is required before choosing between the models.

3.28 Research Studio

Choose one of the three tracks.

Identify a candidate research problem and write a mathematical specification containing:

1. unit of analysis;
2. target variable;
3. feature matrix X ;
4. target vector y ;
5. at least three numeric features and their units;
6. at least one potential scaling issue;
7. one expected association;
8. one association that should **not** be interpreted causally;
9. a candidate loss or evaluation function;
10. the target population to which you hope the result will generalize;
11. one source of sampling uncertainty;
12. one way the mathematical representation simplifies the real phenomenon.

3.29 Knowledge Check

1. What is the difference between a parameter and a statistic?
2. Why does sample variance use squared deviations?
3. What does standard deviation communicate that variance does not?
4. What does $P(Y = 1 \mid X = x)$ represent?
5. Why do repeated samples produce different estimates?
6. What does correlation fail to establish?
7. Why can feature scaling matter for Euclidean distance?

8. What does a dot product do?
9. What does a loss function contribute to machine learning?
10. What information does a derivative provide?
11. Why does gradient descent move opposite the gradient?
12. Why can a mathematically correct model still support a poor substantive conclusion?
13. How can a model predict well while individual coefficient interpretations remain unstable?
14. Why does strong out-of-sample prediction not establish causality?
15. What does uncertainty communicate beyond a point estimate?

3.30 Reproducibility Check

Before leaving this chapter, verify that your analysis records:

- whether statistics use population or sample conventions;
- random seeds for simulations;
- feature units;
- scaling procedures;
- which data were used to estimate scaling parameters;
- distance metrics;
- loss functions;
- optimization settings such as learning rate;
- code used to produce mathematical figures and numerical results.

3.31 Key Takeaways

- Statistics provides a language for variation, relationships, sampling, and uncertainty.
- Probability represents uncertainty mathematically.
- Samples provide imperfect information about populations.
- Correlation measures association, not causation.
- Scaling changes numerical representation and can materially affect some algorithms.
- Linear algebra compactly represents observations, features, parameters, and predictions.
- Distance depends on representation, scale, and metric.
- Loss functions translate modeling goals into mathematical objectives.
- Derivatives and gradients describe local change.
- Gradient descent uses local information to iteratively reduce a loss.
- Mathematical correctness is necessary but not sufficient for scientific or practical validity.
- Predictive, inferential, and causal questions require different forms of evidence.
- Strong prediction does not establish stable parameter interpretation or causality.
- Uncertainty communicates limits and stability of analytical conclusions.

3.32 Looking Ahead

The mathematical and statistical ideas in this chapter become useful only when they are connected to a well-defined analytical problem. Chapter 4 therefore shifts from foundations to formulation: defining the unit of analysis, target, predictors, prediction horizon, decision context, baseline, and evidence required before modeling begins.

Part II

Part II — The Machine Learning Workflow

4 Framing Machine Learning Problems

4.1 Learning Objectives

By the end of this chapter, you should be able to:

- translate a substantive problem into a machine-learning specification;
- distinguish a prediction target from the broader construct it represents;
- define unit of analysis, prediction time, prediction horizon, and target population;
- identify features legitimately available at prediction time;
- distinguish prediction, forecasting, ranking, explanation, and causal questions;
- select meaningful baselines and evaluation criteria;
- reason about asymmetric errors and decision thresholds;
- identify target leakage before model development;
- distinguish predictive performance from decision usefulness.

4.2 Why This Matters

A sophisticated algorithm cannot rescue a poorly framed problem.

Projects often begin too late: “Which model should we use?” A stronger project asks: **What problem exists? What decision or knowledge need does it create? What exactly should be predicted, for whom, at what point in time, using which information, and how will success be judged?**

Central Question

Before choosing an algorithm, how do we translate a real-world problem into a machine-learning problem that is actually worth solving?

4.2.1 Connection to the Foundations

Chapters 1–3 established what machine learning is, the Python environment in which we will work, and the mathematical and statistical ideas that support model reasoning. We now move

from foundations to the first major workflow decision: translating a substantive question into a machine-learning problem that can be evaluated with appropriate data and evidence.

4.3 Start With the Problem, Not the Algorithm

Compare:

“I want to use Random Forest to study student retention.”

with:

“At the end of a student’s second week, can we identify students at elevated risk of not returning the following term using only information available by that prediction time?”

The second formulation exposes unit, target, timing, horizon, information availability, intervention context, error costs, and generalization.

4.4 A Machine Learning Problem Specification

We will use

$$\mathcal{P} = (U, Y, X_t, t, H, L, E, C),$$

where U is unit of analysis; Y the target; X_t information legitimately available at prediction time t ; H the horizon; L the loss/utility objective; E the evaluation design; and C constraints and context.

This is a pedagogical framework, not a universal mathematical law.

Problem Specification Before Model Specification

Do not begin with $f_\theta(X)$. Begin by deciding what X and Y are allowed to mean.

4.5 Unit of Analysis

One row might represent a patient, student, university, transaction, document, institution-year, patient-visit, or student-course enrollment. Changing the unit can change the question.

4.5.1 Try It: Find the Unit

Identify the defensible unit for:

1. 30-day patient readmission;
2. next-year university enrollment;
3. news-topic classification;
4. fraudulent transactions;
5. course-level student passing.

4.6 Defining the Target

“Student success” is not yet a target. Operationalizations might include next-term retention, six-year graduation, first-year credits earned, or course completion.

Target	Construct
A measurable target is often a proxy for something broader. Predicting the proxy accurately does not prove that the full construct has been captured.	

4.7 From Construct to Operational Target

Many machine-learning problems begin with an idea that cannot be observed directly: learning, health need, engagement, readiness, quality, satisfaction, or resilience. This connects machine-learning design to the tradition of construct validity and to contemporary measurement modeling in computational systems (Cronbach and Meehl 1955; Jacobs and Wallach 2021).

A model cannot ingest a construct directly. The construct passes through operational decisions:

Construct → Operational Definition → Measurement → Recorded Variable → ML Target
--

Each arrow introduces assumptions.

If an education study wants to predict **learning** but uses final course grade, it implicitly proposes

Learning $\approx$ Final Course Grade.

Yet grades can also reflect assessment design, attendance policies, late penalties, prior preparation, revision opportunities, and instructor practices. The grade may still be useful; the question is whether it is defensible for the intended claim.

4.7.1 Three Tracks: Same Problem, Different Proxy

Track	Construct	Possible proxy	Validity question
Health	health need	healthcare utilization	Does utilization also reflect access and care-seeking behavior?
Education	learning	final course grade	Does the grade combine learning with policy and opportunity?
Open ML	customer satisfaction	support contacts	Do contacts indicate dissatisfaction, engagement, complexity, or access?

Let (C) denote the construct and (Y) the operational target. Strong prediction of (Y) does not establish

$$Y = C.$$

4.7.2 Construct Validity as an ML Design Problem

Before fitting a model, ask:

1. What broader construct matters?
2. Why should this target represent it?
3. What dimensions does the target omit?
4. What other processes influence the recorded target?
5. Does the target mean the same thing across groups, sites, or time?
6. Could optimizing the proxy distort the substantive goal?

Research Reality Check

An AUC of 0.95 says nothing by itself about whether the target validly represents the construct named in the research question.

Excellent prediction of the wrong operational target is still excellent prediction of the wrong target.

4.7.3 Try It: Audit the Target

For one proposed ML problem, identify: 1. the construct; 2. the operational definition; 3. the recorded target; 4. two other processes that could influence the target; 5. an alternative operationalization; 6. how conclusions might change under that alternative.

Target form	Example	Typical task
continuous	systolic blood pressure	regression
binary	retained/not retained	classification
multiclass	diagnostic category	multiclass classification
count	emergency visits	count/regression
time-to-event	time until readmission	survival analysis
ordered category	risk level	ordinal modeling

4.8 Prediction Time and Horizon

Let t be when a prediction is produced and H how far beyond t the outcome occurs.

t = end of Week 2, H = retention to next term.

4.8.1 The Availability Test

Would I actually know this value at the moment the model is supposed to make its prediction?

If not, it may create leakage. Leakage can produce systematically overoptimistic estimates of model performance and has become an important reproducibility concern in machine-learning-based science (Kapoor and Narayanan 2023).

4.9 Leakage Begins With Framing

Predicting course completion using prior GPA and first-week attendance may be legitimate. Using final course grade for an early-warning model is not.

Temporal Integrity

A model should not learn from information that arrives after the decision point it is meant to support.

4.10 Prediction Is Not Intervention

A high predicted attrition risk does not identify which intervention will help. A health risk predictor does not prove that modifying an important feature will cause risk to fall.

Prediction concerns $P(Y | X)$. Intervention requires different causal reasoning (Hernán and Robins 2020).

Risk Factor Treatment Recommendation

A predictor can identify risk without identifying an effective intervention.

4.11 Different Analytical Purposes

Prediction estimates unknown outcomes. **Forecasting** predicts future values with temporal structure. **Classification** estimates categories/probabilities. **Ranking** orders observations. **Discovery** characterizes structure. **Explanation** investigates relationships or mechanisms. **Causal estimation** asks what would happen under alternative interventions.

4.12 Computer Vision Is a Workload, Not a Problem Specification

“Use computer vision” is not yet a machine-learning problem.

A vision workload can contain very different targets, units, outputs, and evaluation requirements:

Vision task	Possible unit	Target/output	Example evaluation concern
image classification	image	class label	class imbalance and subgroup performance
object detection	image/object	class + bounding box	localization and detection quality
semantic segmentation	pixel/region	class per pixel	boundary quality and class imbalance
visual anomaly detection	image/region	anomaly score	rare-event validation
image retrieval	query-image pair	similarity/ranking	relevance and ranking quality

A health study might classify a medical image, an education application might analyze hand-written work, and an Open ML experiment might classify benchmark images. The workload does not determine whether the study is scientifically valid.

The same framing questions still apply:

- What does one observation represent?
- What is the target or output?
- When is the prediction made?
- Which visual information is legitimately available then?
- What is the baseline?
- What errors matter?
- To what cameras, sites, populations, devices, or acquisition conditions should performance generalize?

Research Reality Check

High image-classification accuracy does not establish that a vision system will generalize to new devices, lighting conditions, institutions, populations, or image-acquisition protocols. In computer vision, **domain shift can be visual**.

4.13 Baselines: What Must the Model Beat?

```
import numpy as np
y = np.array([0] * 90 + [1] * 10)
max(np.mean(y == 0), np.mean(y == 1))
```

```
np.float64(0.9)
```

Always predicting the majority class already achieves 90% accuracy here.

Baselines Create Meaning

“Accuracy = 0.91” is incomplete. “Accuracy = 0.91 versus a 0.90 naive baseline” tells a different story.

4.14 Error Costs and Thresholds

A predicted probability becomes an action only after a decision rule is applied. A default threshold such as 0.50 is not automatically appropriate.

A simple expected-cost representation is

$$\text{Expected Cost} = C_{FP}P(FP) + C_{FN}P(FN).$$

This does not imply every consequence should be monetized. It makes explicit that errors can matter differently.

4.15 Evaluation Must Match Intended Use

Evaluation may require random, stratified, grouped, temporal, geographic, or external validation. Random splitting can mislead when people repeat, institutions repeat across years, future prediction is the goal, or deployment occurs elsewhere.

Ask:

To whom, where, and when do we hope this model will generalize?

4.16 The Problem Framing Canvas

Element	Question
Substantive problem	What real-world problem exists?
Stakeholder	Who needs the evidence?
Decision/use	What could happen because of the output?
Unit	What does one row represent?

Element	Question
Target	What exactly is predicted?
Construct	What broader phenomenon do we actually care about?
Operationalization	Why should this target represent that construct?
Proxy limitations	What does the target omit or confound?
Prediction time	When is prediction made?
Horizon	How far ahead is the outcome?
Available features	What is known then?
Baseline	What simple strategy must be beaten?
Error costs	Which mistakes matter?
Evaluation	How will generalization be tested?
Target population	To whom should results generalize?
Constraints	Privacy, fairness, latency, resources?
Intended use	What is the model designed to support?
Unsupported claims	What must not be concluded?

4.17 Three Tracks

4.17.1 Health

Rather than “use NHANES to predict diabetes,” specify population, operational diabetes criterion, information available at the analytical time point, validation design, and intended claim.

4.17.2 Education

Rather than “use IPEDS to predict retention,” specify institution-level unit, outcome period, prediction time, available prior-period features, and temporal validation.

4.17.3 Open Benchmark

Rather than “compare five classifiers on Wine,” specify a common evaluation protocol and a pedagogical question such as how scaling and model family affect out-of-sample performance.

4.18 A Small Framing Experiment

```
import matplotlib.pyplot as plt

positive_prevalence = np.linspace(0.01, 0.50, 50)
majority_accuracy = 1 - positive_prevalence

plt.figure(figsize=(8, 5))
plt.plot(positive_prevalence, majority_accuracy)
plt.xlabel("Positive-class prevalence")
plt.ylabel("Accuracy of always predicting negative")
plt.title("A Naive Classifier Can Look Accurate")
plt.ylim(0.45, 1.01)
plt.grid(alpha=0.2)
plt.show()
```

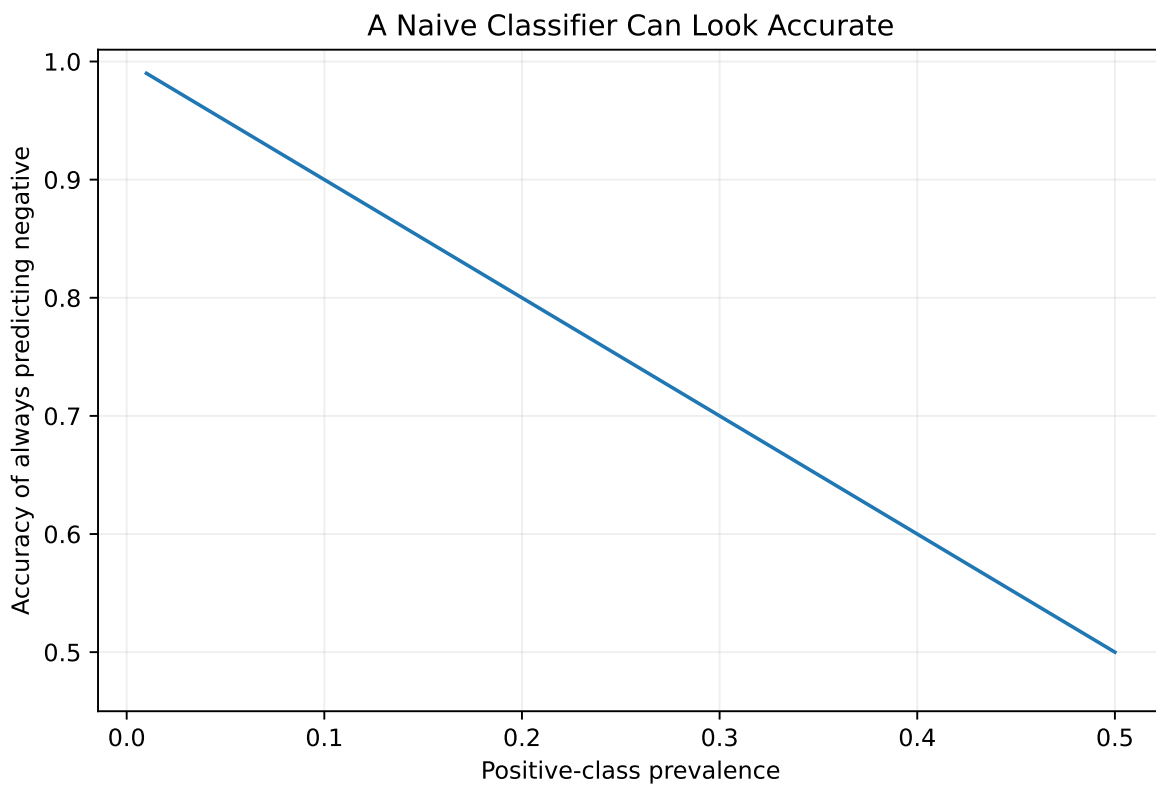


Figure 4.1: Majority-class accuracy increases as the positive outcome becomes rarer.

A rare outcome can make a useless classifier appear impressive. Evaluation metrics must be chosen in context.

4.19 Beyond the Algorithm: What Should We Choose to Predict?

Predictability Is Not Permission

Machine learning encourages us to ask what can be predicted. A more mature question is whether the target is appropriate to predict for the intended purpose.

A target is a choice. When we operationalize “risk,” “success,” “quality,” “readiness,” or “potential,” that choice can influence who receives resources, scrutiny, attention, or opportunity.

Ask:

1. **Can this be predicted?**
2. **Can it be predicted reliably?**
3. **Should it be predicted for this purpose?**
4. **What decisions should—and should not—follow?**

The algorithm enters only after these questions begin to have defensible answers.

4.20 Responsible Framing

Responsible AI considerations belong in design, development, deployment, use, and evaluation (National Institute of Standards and Technology 2023). Consider legitimacy, problematic historical targets, sensitive proxies, uneven error consequences, over-trust, scope creep, and whether a simpler non-ML solution is preferable.

Sometimes the Best ML Decision Is Not to Build an ML Model

A rule, descriptive analysis, experiment, process redesign, or policy change may be more appropriate than prediction.

4.21 AI Pair Programmer

Prompt an AI assistant: “Build a machine-learning model to predict college student success.”

Audit whether it clarifies success, unit, prediction time, horizon, decision, population, feature availability, leakage, baseline, error costs, evaluation, ethics, and causal versus predictive claims. Rewrite the prompt so model selection cannot sensibly begin before framing is complete.

4.22 Hands-On Lab: Build an ML Problem Specification

Choose one track—or a computer-vision workload within a track—and express it as

$$\mathcal{P} = (U, Y, X_t, t, H, L, E, C).$$

Include substantive problem, stakeholder, unit, target, prediction time/horizon, feature-availability rules, leakage example, baseline, metrics, error costs, validation design, target population, constraints, and at least three unsupported claims. **No model fitting is required.**

4.23 Practitioner Track

Define the operational decision; specify target, timing, horizon, and information; identify leakage; choose baselines; align evaluation with deployment; reason about error costs; document intended use.

4.24 Researcher Track

Additionally distinguish constructs from targets, define generalization populations, separate predictive from causal claims, justify evaluation as research design, and explain contribution beyond model comparison.

4.25 Research Studio: From Topic to Research Question

Transform a broad topic such as cardiovascular health, student retention, institutional outcomes, or classifier benchmarking into a substantive gap/decision problem, population, unit, target, timing, research question, evaluation design, contribution, and unsupported causal claim.

A useful predictive template is:

Among [population/unit], how well can [information available at time (t)] predict [target] over [horizon], and how well does performance generalize across [time/group/site/context]?

Use this to expose assumptions, not as a mandatory formula.

- one construct-to-target validity argument;
- one alternative operationalization and how it could change the study.

4.26 Professor's Challenge: The 95% Accurate Model

A university reports 95% dropout-prediction accuracy and recommends automatic mandatory interventions. Only 6% dropped out; `final_term_status` is in the data; repeated student records may cross a random split; no baseline is reported; 0.50 is used as the default threshold; error costs and subgroup performance are ignored; years are randomly mixed; and feature importance is called “causes.”

Critique accuracy, leakage, unit ambiguity, temporal validity, baseline, threshold, error costs, subgroup evaluation, causation, intervention logic, evaluation design, and the problem statement.

4.27 Knowledge Check

1. Why frame before selecting an algorithm?
2. What is a unit of analysis?
3. Construct versus operational target?
4. Prediction time versus horizon?
5. What is the availability test?
6. Why is post-outcome information leakage?
7. Why does prediction not identify intervention effects?
8. Why use a baseline?
9. How can prevalence make accuracy misleading?
10. Why might 0.50 be inappropriate?
11. What does deployment-aligned evaluation mean?
12. Why might framing end with no ML model?
13. Why is construct validity relevant even when predictive performance is high?
14. Distinguish a construct, operational definition, and recorded target.
15. How can optimizing a proxy conflict with the substantive goal?

4.28 Reproducibility Check

Version the substantive problem, stakeholder, intended use, unit, target, prediction time, horizon, feature-availability rule, exclusions, baseline, error assumptions, metrics, validation design, target population, constraints, and unsupported claims.

If framing changes, document **what changed and why**.

4.29 Key Takeaways

- A sophisticated algorithm cannot rescue a poorly framed problem.
- Targets operationalize constructs; they are not the constructs themselves.
- Prediction time determines legitimate information.
- Leakage can often be identified before coding.
- Prediction does not establish causation or intervention effectiveness.
- Baselines contextualize metrics.
- Thresholds and error costs should reflect use.
- Evaluation design is part of problem formulation.
- Generalization must be defined across people, places, times, or settings.
- Technical predictability does not determine whether something should be predicted.
- Sometimes the best decision is not to build an ML model.
- Target selection is a measurement decision as well as a modeling decision.
- Predictive accuracy for a proxy does not establish validity for the broader construct.

5 Data Preparation and Feature Engineering

5.1 Learning Objectives

By the end of this chapter, you should be able to:

- distinguish raw, analytical, training, validation, and test data;
- explain why preprocessing must respect the evaluation boundary;
- diagnose missingness, duplicates, invalid values, and inconsistent representations;
- distinguish numerical, nominal, ordinal, datetime, text, and identifier variables;
- apply scaling, imputation, and categorical encoding appropriately;
- explain feature engineering as the construction of model representations;
- use `Pipeline` and `ColumnTransformer` to prevent common forms of leakage;
- explain why preprocessing choices should be learned from training data;
- evaluate whether an engineered feature would actually exist at prediction time;
- build reproducible transformations rather than manually modifying source data;
- connect domain knowledge to health, education, and benchmark feature design;
- audit AI-generated preprocessing code for technically subtle errors.

5.2 Why This Matters

Most machine-learning algorithms do not receive “reality.” They receive a representation of reality produced by a data pipeline.

That representation may contain missing values, inconsistent categories, measurement artifacts, duplicated records, inappropriate identifiers, post-outcome information, or transformations learned from the wrong observations.

A model can therefore fail before the algorithm itself has done anything wrong.

Central Question

How do we transform raw data into model-ready evidence without contaminating the evaluation or distorting the meaning of the variables?

5.3 From Problem Specification to Data Pipeline

Chapter 4 defined the problem as

$$\mathcal{P} = (U, Y, X_t, t, H, L, E, C).$$

Chapter 5 operationalizes X_t .

The critical subscript is t : features must be legitimately available when the prediction is made.

A useful conceptual pipeline is

$$D_{\text{raw}} \rightarrow D_{\text{validated}} \rightarrow D_{\text{split}} \rightarrow T_{\text{train}} \rightarrow X_{\text{model}}.$$

Transformations T that learn from data are estimated using the training portion rather than the complete dataset.

5.4 Preserve Raw Data

Raw data should normally remain immutable within the analytical workflow.

```
data/  
  raw/  
  interim/  
  processed/
```

Instead of manually changing cells, write transformations that can be rerun.

Reproducibility Principle

A transformation that matters to the analysis should exist as code, not as an undocumented manual edit.

5.5 Inspect Before Cleaning

```
import numpy as np
import pandas as pd

df = pd.DataFrame({
    "age": [22, 35, 35, np.nan, 51, 64],
    "program": ["MS", "PhD", "PhD", "MS", "DBA", "DBA"],
    "study_hours": [6, 8, 8, 4, np.nan, 10],
    "retained": [1, 1, 1, 0, 0, 1]
})

df
```

	age	program	study_hours	retained
0	22.0	MS	6.0	1
1	35.0	PhD	8.0	1
2	35.0	PhD	8.0	1
3	NaN	MS	4.0	0
4	51.0	DBA	NaN	0
5	64.0	DBA	10.0	1

Start with structure, missingness, duplicates, and summaries.

```
df.shape, df.dtypes
```

```
((6, 4),
 age          float64
 program      object
 study_hours  float64
 retained     int64
 dtype: object)
```

```
df.isna().sum()
```

```
age          1
program      0
study_hours  1
retained     0
dtype: int64
```

```
df.duplicated().sum()
```

```
np.int64(1)
```

```
df.describe(include="all")
```

	age	program	study_hours	retained
count	5.000000	6	5.000000	6.000000
unique	NaN	3	NaN	NaN
top	NaN	MS	NaN	NaN
freq	NaN	2	NaN	NaN
mean	41.400000	NaN	7.200000	0.666667
std	16.288032	NaN	2.280351	0.516398
min	22.000000	NaN	4.000000	0.000000
25%	35.000000	NaN	6.000000	0.250000
50%	35.000000	NaN	8.000000	1.000000
75%	51.000000	NaN	8.000000	1.000000
max	64.000000	NaN	10.000000	1.000000

The goal is not to “clean everything.” The goal is to understand which observations and variables are credible for the specified problem.

5.6 Data Quality Is Contextual

A value can be syntactically valid and substantively impossible: negative age, 25 hours of sleep per day, graduation before enrollment, an identifier treated as a continuous quantity, inconsistent laboratory units, or categories whose definitions changed across years.

Validation therefore combines computational checks with domain knowledge.

5.7 Data-Centric Machine Learning

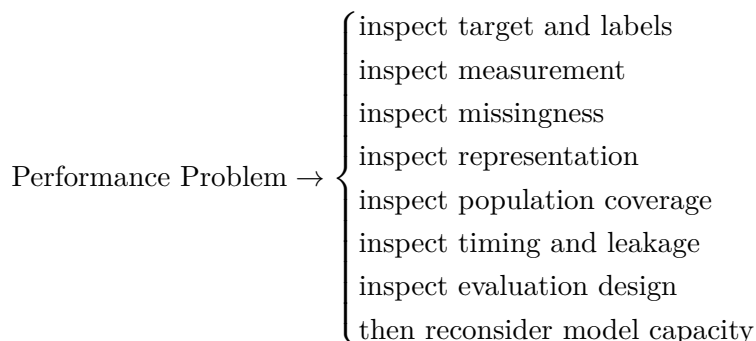
A common reflex is

Disappointing performance → try a more sophisticated algorithm.

Sometimes that is appropriate. Often it is premature.

A data-centric perspective asks whether better evidence could matter more than greater model complexity. Research on data cascades in high-stakes AI similarly shows how upstream data problems can propagate through downstream systems (Sambasivan et al. 2021). Potential improvements include better labels, measurement, population coverage, temporal alignment, category consistency, feature representation, and removal of corrupted or duplicated observations.

A useful diagnostic sequence is



5.7.1 Dimensions of Data Quality

Dimension	Diagnostic question
Label quality	Is the target recorded consistently and correctly?
Measurement validity	Do variables represent the intended phenomena?
Completeness	What is missing, for whom, and why?
Consistency	Are units, categories, and definitions stable?
Uniqueness	Are duplicate observations present?
Timeliness	Would the information exist at prediction time?
Coverage	Are important populations, sites, periods, or conditions underrepresented?
Provenance	Can we explain how each variable was produced?

A large dataset can still be weak for a particular problem.

5.7.2 Small Experiment: Better Model or Better Data?

```
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
import numpy as np
import matplotlib.pyplot as plt

X_dc, y_dc = make_classification(
    n_samples=1800,
    n_features=12,
    n_informative=7,
    n_redundant=2,
    class_sep=1.0,
    random_state=42
)

Xtr_dc, Xte_dc, ytr_dc, yte_dc = train_test_split(
    X_dc, y_dc, test_size=0.30, random_state=42, stratify=y_dc
)

rng_dc = np.random.default_rng(42)
y_noisy_dc = ytr_dc.copy()
flip_dc = rng_dc.choice(
    len(y_noisy_dc),
    size=int(0.20 * len(y_noisy_dc)),
    replace=False
)
y_noisy_dc[flip_dc] = 1 - y_noisy_dc[flip_dc]

log_noisy_dc = LogisticRegression(max_iter=1000).fit(Xtr_dc, y_noisy_dc)
rf_noisy_dc = RandomForestClassifier(
    n_estimators=300, random_state=42
).fit(Xtr_dc, y_noisy_dc)
log_clean_dc = LogisticRegression(max_iter=1000).fit(Xtr_dc, ytr_dc)

scores_dc = [
    accuracy_score(yte_dc, log_noisy_dc.predict(Xte_dc)),
    accuracy_score(yte_dc, rf_noisy_dc.predict(Xte_dc)),
    accuracy_score(yte_dc, log_clean_dc.predict(Xte_dc))
]
```

```

]

plt.figure(figsize=(8, 5))
plt.bar(
    ["Logistic\nnoisy labels", "Random forest\nnoisy labels", "Logistic\nclean labels"],
    scores_dc
)
plt.ylabel("Test accuracy")
plt.ylim(0, 1)
plt.title("Model Complexity Cannot Always Rescue Weak Training Evidence")
plt.show()

scores_dc

```

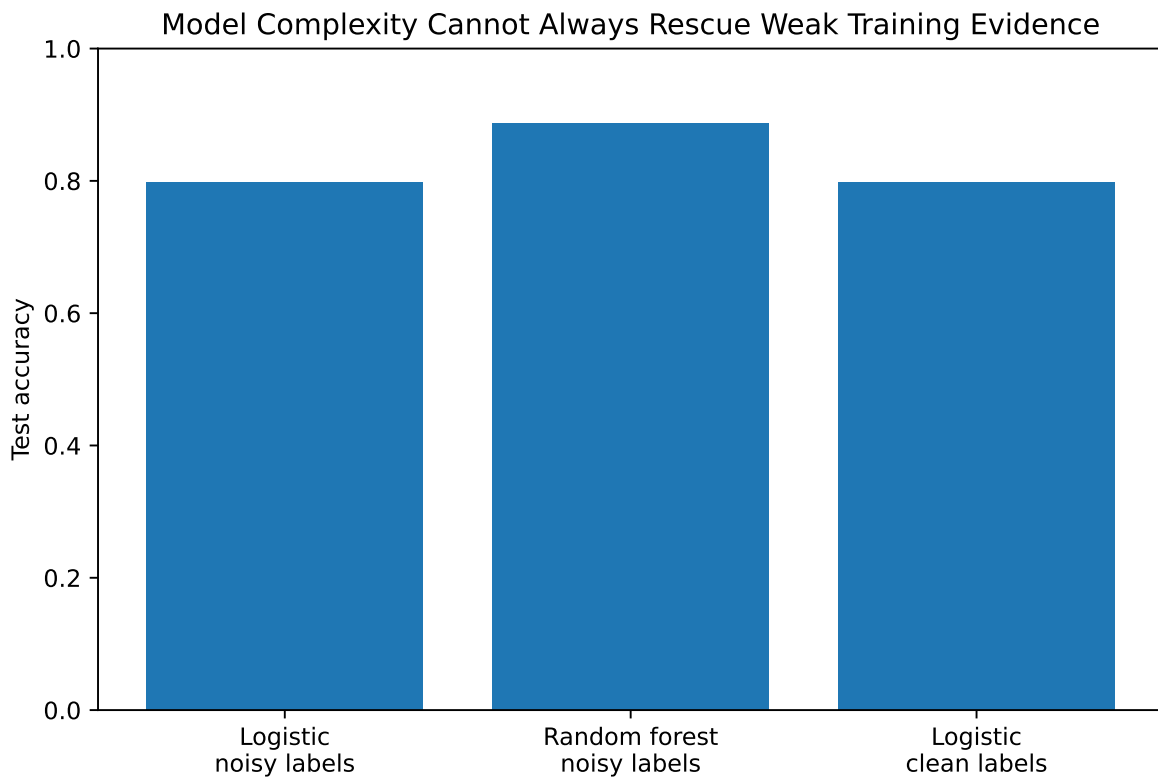


Figure 5.1: When training labels are corrupted, increasing model complexity may help less than improving the training evidence.

This is a controlled illustration, not a universal law. The lesson is diagnostic: **before escalating**

model complexity, investigate whether the bottleneck is the information supplied to the model.

Data Quality Sets an Information Boundary

A model cannot recover information that was never measured, repair an invalid target merely by optimizing harder, or guarantee coverage of populations absent from the data.

Model quality is constrained by the information quality of the data-generating process.

Research Reality Check

If several algorithms perform poorly, do not immediately conclude that the phenomenon is “not predictable.” Examine target reliability, measurement, timing, missingness, coverage, label noise, feature availability, sample adequacy, and evaluation design.

Failure of tested representations and algorithms is not proof that the underlying phenomenon contains no predictable signal.

5.8 Missing Data

Missing values may arise because a variable was not measured, not applicable, refused, structurally unavailable, lost, or recorded under a special code. The missingness process matters for valid inference; missing values should therefore not be treated as a purely mechanical cleaning problem (Rubin 1976).

Before imputation, ask:

1. How much is missing?
2. For which variables?
3. For which observations or groups?
4. Why might it be missing?
5. Is missingness itself informative?
6. Would this variable be available at deployment?

5.8.1 Why `dropna()` Is Not a Strategy

```
df = df.dropna()
```


This can change the analytical sample and therefore the population represented by the analysis.

5.8.2 Imputation

For a numerical feature, simple median imputation can be written

$$x_i^* = \begin{cases} x_i, & x_i \text{ observed} \\ \tilde{x}_{\text{train}}, & x_i \text{ missing.} \end{cases}$$

The training subscript matters.

5.9 The Evaluation Boundary

Suppose

$$D = D_{\text{train}} \cup D_{\text{test}}.$$

The test set approximates unseen data. Preprocessing statistics should not be learned from it.

Incorrect:

```
X_scaled = scaler.fit_transform(X)
X_train, X_test = train_test_split(X_scaled, y)
```

Better:

```
X_train, X_test, y_train, y_test = train_test_split(X, y)
scaler.fit(X_train)
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Best for repeatable workflows: place learned preprocessing and modeling inside a pipeline.

Leakage Can Be Quiet

The target does not need to appear explicitly in the training features for leakage to occur. Test-set information used to estimate preprocessing parameters also contaminates

evaluation. Leakage can make reported performance substantially more optimistic than performance under a valid evaluation design (Kapoor and Narayanan 2023).

5.10 Train/Test Splitting

For demonstration, create a slightly larger mixed-type dataset so stratification is stable.

```
rng = np.random.default_rng(42)
n = 120

ml_df = pd.DataFrame({
    "age": rng.integers(20, 66, size=n).astype(float),
    "program": rng.choice(["MS", "PhD", "DBA"], size=n),
    "study_hours": rng.normal(7, 2, size=n).clip(0),
})

ml_df.loc[rng.choice(n, 8, replace=False), "age"] = np.nan
ml_df.loc[rng.choice(n, 10, replace=False), "study_hours"] = np.nan

signal = (
    ml_df["age"].fillna(40) * -0.015
    + ml_df["study_hours"].fillna(7) * 0.35
    + (ml_df["program"] == "PhD") * 0.4
)
prob = 1 / (1 + np.exp(-(signal - 1.5)))
ml_df["retained"] = rng.binomial(1, prob)

X = ml_df[["age", "program", "study_hours"]]
y = ml_df["retained"]
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,
    random_state=42,
    stratify=y
)

X_train.shape, X_test.shape
```

((90, 3), (30, 3))

A random stratified split is pedagogically convenient here. Chapter 4 established that grouped, temporal, geographic, or external validation may be required by the actual problem.

5.11 Numerical Features

Common transformations include imputation, standardization, robust scaling, transformations, interaction terms, and substantively justified rates or ratios.

Standardization is

$$z = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}.$$

5.12 Categorical Features

Many algorithms require numeric representations of categories.

5.12.1 One-Hot Encoding

```
pd.get_dummies(ml_df["program"], dtype=int).head()
```

	DBA	MS	PhD
0	0	0	1
1	1	0	0
2	0	0	1
3	0	0	1
4	1	0	0

For a fitted ML workflow, `OneHotEncoder` can learn category structure from training data and handle later unknown categories.

5.12.2 Ordinal Variables

An ordinal variable has order but not necessarily equal intervals:

Never < Sometimes < Often < Always.

Encoding these as 0, 1, 2, 3 imposes numerical structure. That is an analytical assumption, not merely a formatting decision.

5.13 Identifiers Are Not Ordinary Features

Student IDs, patient IDs, account numbers, and institution IDs may look numeric but usually function as identifiers.

They may still be needed for grouping, joins, longitudinal linkage, split design, and auditing. The question is whether they belong in the predictive feature matrix.

5.14 Datetime Features

Timestamps may be transformed into day of week, month, elapsed time, age at event, time since prior event, or academic term.

Datetime engineering must still pass the Chapter 4 availability test.

5.15 Feature Engineering: Representation as Modeling

For height h and weight w ,

$$BMI = \frac{w}{h^2},$$

with appropriate units.

An education study might construct

$$\text{credit completion ratio} = \frac{\text{credits completed}}{\text{credits attempted}}.$$

A Feature Is an Argument

An engineered feature says:

“I believe this representation may contain information relevant to the target.”

Feature engineering is therefore computational and conceptual.

5.16 Computer Vision Data: Representation Is Preprocessing

Images require the same disciplined thinking as tabular data, but their structure introduces additional choices.

An image may be represented as

$$I_i \in \mathbb{R}^{H \times W}$$

for grayscale data or

$$I_i \in \mathbb{R}^{H \times W \times C}$$

for multichannel data.

Common preparation decisions include:

- resizing and cropping;
- pixel-value scaling or normalization;
- color-space or channel handling;
- augmentation;
- image-quality screening;
- class-label verification;
- duplicate and near-duplicate detection;
- grouping related images from the same person, object, session, or source.

5.16.1 Flattening Changes the Representation

Classical algorithms often require a feature vector:

$$I_i \rightarrow x_i \in \mathbb{R}^p.$$

For an (8×8) grayscale image, flattening produces (p=64) pixel features.

```
from sklearn.datasets import load_digits

digits_cv = load_digits()

X_images_cv = digits_cv.images
X_flat_cv = digits_cv.data
y_cv = digits_cv.target

X_images_cv.shape, X_flat_cv.shape, y_cv.shape
```

```
((1797, 8, 8), (1797, 64), (1797,))
```

The flattened representation is convenient for many classical estimators, but it does not explicitly encode the two-dimensional neighborhood relationships visible in the original image.

5.16.2 Vision Leakage Can Be Subtle

Suppose several images come from the same patient, student, device, video, or acquisition session. Randomly splitting individual images can place highly related observations in both training and test sets.

The model may then appear to generalize while partly recognizing source-specific patterns.

Whenever images are naturally grouped, consider splitting by the higher-level unit:

Split by the unit that must generalize

Examples include patient, institution, camera, video, subject, or acquisition session.

5.16.3 Augmentation Belongs Inside the Training Process

Transformations such as flips, rotations, crops, or brightness changes can increase training diversity when substantively appropriate.

But augmentation must respect the target.

A horizontal flip may be harmless for some object-recognition problems and scientifically invalid for a task in which laterality matters.

A Vision Transformation Is Also an Assumption

Every crop, resize, normalization rule, augmentation, and color transformation states what visual information the model is allowed to treat as stable.

Computer-vision preprocessing is therefore not merely image cleanup. It is part of the model's representation of the world.

5.17 Interactions and Nonlinearity

An interaction can be written

$$x_3 = x_1 x_2.$$

Polynomial expansion may include

$$x, \quad x^2, \quad x^3.$$

Additional flexibility can reveal structure but also increase dimensionality and overfitting.

5.18 Outliers: Error, Rarity, or Signal?

An extreme value may be a data-entry error, unit mismatch, valid rare observation, subgroup member, or exactly the event we need to detect.

Outlier treatment should begin with interpretation, not automatic deletion.

5.19 ColumnTransformer: Different Columns, Different Transformations

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

numeric_features = ["age", "study_hours"]
```

```

categorical_features = ["program"]

numeric_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_pipeline, numeric_features),
        ("cat", categorical_pipeline, categorical_features)
    ]
)

preprocessor

```

```

ColumnTransformer(transformers=[('num',
                                Pipeline(steps=[('imputer',
                                                  SimpleImputer(strategy='median')),
                                                  ('scaler', StandardScaler())]),
                                ['age', 'study_hours']),
                                ('cat',
                                 Pipeline(steps=[('imputer',
                                                  SimpleImputer(strategy='most_frequent')),
                                                  ('onehot',
                                                  OneHotEncoder(handle_unknown='ignore'))]),
                                ['program'])])

```

The preprocessing logic exists, but training statistics have not yet been learned.

5.20 Pipeline: Preprocessing and Model as One Workflow

```

from sklearn.linear_model import LogisticRegression

```



```

model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(max_iter=1000))
])

model.fit(X_train, y_train)
model.score(X_test, y_test)

```

0.6

This becomes especially important during cross-validation because each training fold should learn its own preprocessing parameters. scikit-learn's estimator and pipeline ecosystem supports composable preprocessing and modeling workflows (Pedregosa et al. 2011).

Pipeline Principle

Anything that learns from data and affects the model should usually live inside the evaluation workflow.

5.21 Visual Experiment: Leakage Inflates Performance

```

from sklearn.datasets import make_classification
from sklearn.metrics import accuracy_score
from sklearn.linear_model import LogisticRegression
import matplotlib.pyplot as plt

X_syn, y_syn = make_classification(
    n_samples=1200,
    n_features=8,
    n_informative=4,
    class_sep=0.8,
    random_state=42
)

X_leaked = np.column_stack([X_syn, y_syn])

def evaluate(features):
    Xtr, Xte, ytr, yte = train_test_split(
        features, y_syn, test_size=0.3,

```

```

    random_state=42, stratify=y_syn
)
clf = LogisticRegression(max_iter=1000)
clf.fit(Xtr, ytr)
return accuracy_score(yte, clf.predict(Xte))

scores = [evaluate(X_syn), evaluate(X_leaked)]

plt.figure(figsize=(7, 4))
plt.bar(["Legitimate features", "Target leaked into X"], scores)
plt.ylabel("Test accuracy")
plt.ylim(0, 1.05)
plt.title("Leakage Can Make a Model Look Brilliant")
plt.show()

```

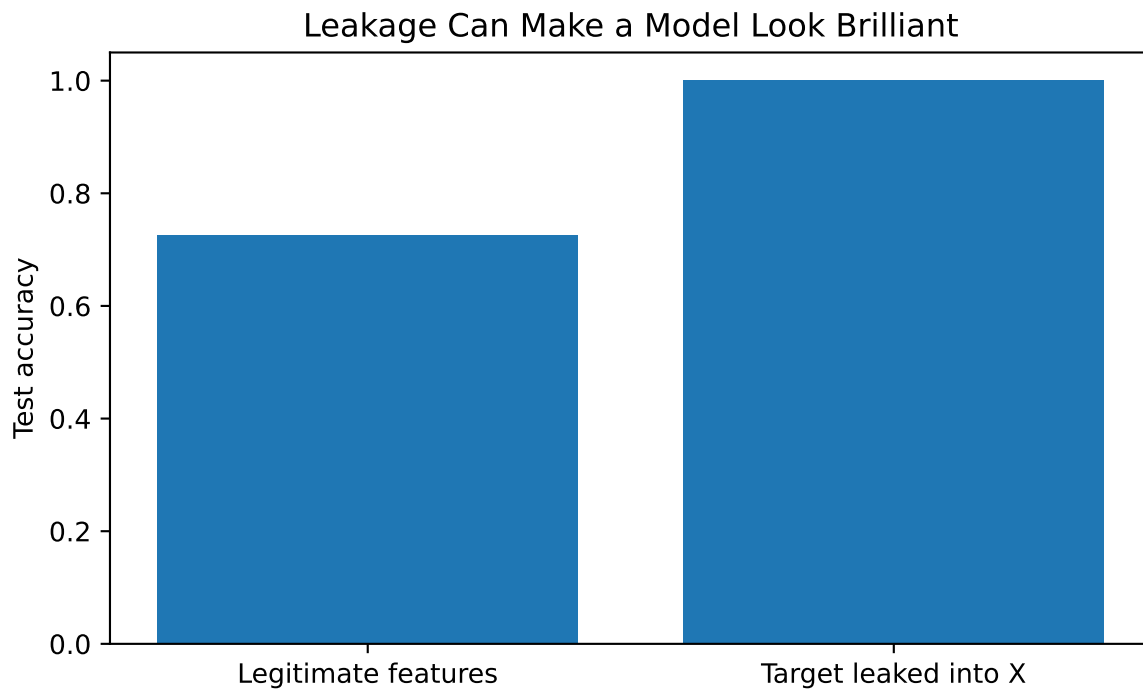


Figure 5.2: A leaked feature can create spectacular but meaningless test performance.

The leaked model is not more intelligent. The experiment has been corrupted.

5.22 Three Data Tracks

5.22.1 Health Track

Ask whether measurements were available at prediction time, whether special missing codes exist, whether units are consistent, whether a derived clinical feature partly defines the target, and whether complex survey design affects the intended inference.

5.22.2 Education Track

Ask whether variables align temporally with the outcome, whether lagged features are needed, whether definitions changed across years, whether IDs are used only for linkage/grouping, and whether rates have comparable denominator reliability.

5.22.3 Open ML Benchmark Track

Benchmark data let us deliberately alter scale, inject missingness, or add irrelevant features to study preprocessing behavior under controlled conditions.

5.23 Beyond the Algorithm: Data Are Made, Not Merely Found

The Dataset Is Already an Interpretation

Before the first model is fitted, choices have already determined what was measured, who was included, how categories were defined, which instruments were used, when measurements occurred, how missingness was encoded, and which observations survived collection.

Feature engineering adds another layer.

The model does not learn directly from the world. It learns from a **representation produced through measurement and transformation**.

This does not make quantitative analysis arbitrary. It makes documentation, domain knowledge, validation, and humility essential.

Ask:

What had to happen for this phenomenon to become this column?

5.24 AI Pair Programmer: Audit the Preprocessing

Ask an AI assistant to preprocess a mixed-type classification dataset.

Audit whether it:

- imputes before or after splitting;
- scales the complete dataset;
- encodes before cross-validation;
- silently drops missing rows;
- includes identifiers;
- uses post-outcome variables;
- handles unseen categories;
- documents transformations;
- places learned transformations inside a pipeline.

Do not judge the answer merely by whether the code runs.

5.25 Hands-On Lab: Build a Leakage-Safe Pipeline

Choose a dataset with numeric and categorical features.

1. document provenance and unit of analysis;
2. identify target and prediction time;
3. classify variables by analytical type;
4. identify IDs and leakage risks;
5. inspect missingness and unusual values;
6. split according to the Chapter 4 evaluation design;
7. build numeric and categorical preprocessing pipelines;
8. combine them with `ColumnTransformer`;
9. attach a simple estimator;
10. evaluate without independently preprocessing the test set;
11. inspect transformed feature names;
12. document each transformation and rationale.
13. identify one data-quality bottleneck that model tuning cannot directly repair;
14. propose a data-improvement intervention and explain how you would test whether it improves generalization.

5.25.1 Computer Vision Extension

Using `sklearn.datasets.load_digits`:

1. inspect the original image tensor and flattened feature matrix;
2. visualize examples from several classes;
3. document the unit of analysis and target;
4. identify an appropriate train/test split;
5. scale pixel features inside a leakage-safe pipeline;
6. fit a simple baseline classifier;
7. evaluate performance on unseen images;
8. inspect several misclassified images;
9. explain what information is lost or obscured when images are flattened;
10. describe how the evaluation design would need to change if multiple images came from the same person or acquisition session.

This is intentionally a **classical computer-vision baseline**. Later neural-network work can be compared against it rather than treating deep learning as the starting point.

5.26 Practitioner Track

Build robust preprocessing workflows that handle missing values, categorical variables, scaling, unknown categories, and repeated evaluation without manual intervention.

5.27 Researcher Track

Additionally justify preprocessing decisions in relation to measurement, sampling, timing, validity, reproducibility, and the scientific claim.

5.28 Research Studio: Build a Data Dictionary and Feature Ledger

Create a feature ledger containing:

Field	Description
Variable	source variable
Construct	what it represents
Unit/type	measurement scale
Availability time	when known
Missing codes	documented meanings

Field	Description
Transformation	proposed preprocessing
Derived feature	if applicable
Leakage risk	yes/no + rationale
Inclusion rationale	why it belongs
Provenance	authoritative source
Quality risk	plausible measurement/label/coverage problem
Quality evidence	how the risk will be assessed
Improvement action	what could improve the data rather than the model

Identify at least three candidate engineered features and explain the substantive theory behind each.

5.29 Professor's Challenge: The Perfect Preprocessing Notebook

A team reports excellent cross-validation performance:

```
df = pd.read_csv("data.csv")
df = df.dropna()

X = pd.get_dummies(df.drop(columns="target"))
X = StandardScaler().fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X, df["target"], random_state=42
)
```

They then cross-validate on `X_train`.

Critique complete-case deletion, encoding before splitting, scaling before splitting, preprocessing before cross-validation, possible IDs, feature availability, unseen categories, provenance, whether all variables should be scaled, and whether the split matches deployment.

Redesign the workflow with `ColumnTransformer` and `Pipeline`.

5.30 Knowledge Check

1. Why preserve raw data?
2. Why is `dropna()` not a complete strategy?
3. What is the evaluation boundary?
4. Why learn scaling parameters from training data?
5. How can preprocessing cause leakage?
6. When is one-hot encoding appropriate?
7. Why is an integer not necessarily quantitative?
8. Why exclude identifiers from the model matrix?
9. What is feature engineering?
10. Why can outlier removal destroy signal?
11. What does `ColumnTransformer` solve?
12. Why are pipelines important during cross-validation?
13. Why should engineered features pass the availability test?
14. What does it mean to say data are “made” rather than merely found?
15. What is a data-centric approach to machine learning?
16. Why can model complexity fail to repair a data-quality problem?
17. How does population coverage differ from simply having a large sample?
18. Why does poor performance not prove an outcome is inherently unpredictable?

5.31 Reproducibility Check

Record raw-data provenance, validation rules, exclusions, missing-value strategy, variable types, split strategy, random seed where applicable, feature lists, encoding, scaling, engineered features, fitted preprocessing objects, software dependencies, and code producing processed data.

5.32 Key Takeaways

- ML algorithms learn from representations, not directly from reality.
- Raw data should remain recoverable and transformations should be scripted.
- Cleaning decisions require substantive reasoning.
- Preprocessing learned from test data contaminates evaluation.
- Missingness should be understood before imputation or deletion.
- Variable meaning matters more than storage type.
- Feature engineering embeds domain assumptions.
- Outliers can be error, rarity, or signal.
- `ColumnTransformer` handles heterogeneous feature types.
- `Pipeline` keeps learned preprocessing inside the modeling workflow.
- Leakage can make a weak model appear extraordinary.

- A trustworthy feature should be meaningful, available at prediction time, reproducibly constructed, and evaluated without contamination.
- Diagnose data limitations before escalating algorithmic complexity.
- Label quality, measurement, timing, coverage, and provenance constrain achievable performance.
- Poor performance can reflect weak evidence or representation rather than an inherently unpredictable phenomenon.

5.33 Looking Ahead

With the prediction problem defined and the data pipeline constructed, the book now moves from workflow design to model families. Chapter 6 begins supervised learning with linear regression, using a deliberately transparent model to connect features, loss, residuals, generalization, regularization, and evidence about predictive performance.

Part III

Part III — Supervised Learning

6 Linear Regression: From Relationship to Prediction

6.1 Learning Objectives

By the end of this chapter, you should be able to:

- express simple and multiple linear regression mathematically;
- interpret intercepts and coefficients in context;
- explain ordinary least squares as loss minimization;
- connect residuals, MSE, RMSE, MAE, and R^2 to model behavior;
- fit regression models with scikit-learn and statsmodels;
- distinguish predictive modeling from statistical inference;
- diagnose nonlinearity, heteroscedasticity, influential observations, and multicollinearity;
- explain why association coefficients should not automatically be interpreted causally;
- use transformations and interactions when substantively justified;
- explain the bias–variance motivation for regularization;
- fit and compare OLS, Ridge, and Lasso within leakage-safe pipelines;
- evaluate regression using held-out data and appropriate baselines.
- recognize regression outputs within computer-vision workloads without confusing prediction with causal explanation.

6.2 Why This Matters

Linear regression is one of the most important models in statistics and machine learning. Its value is not that every relationship in the world is linear. Its value is that it gives us a transparent mathematical laboratory for understanding prediction, loss, coefficients, residuals, assumptions, generalization, and regularization.

Central Question

How can a linear model transform observed relationships into useful predictions—and what must be true before we trust its coefficients or predictions?

6.2.1 Connection to the Workflow

Chapters 4 and 5 established the problem specification and the data pipeline. Those choices now become part of the model itself: the target defines what is predicted, the features define what information is available, and the evaluation boundary determines what counts as honest evidence. Linear regression provides the first transparent setting in which to study how those decisions interact with model fitting and generalization.

6.3 The Simple Linear Model

For one predictor,

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i,$$

where β_0 is the intercept, β_1 the slope, and ϵ_i the unexplained component.

The fitted prediction is

$$\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_i.$$

The residual is

$$e_i = y_i - \hat{y}_i.$$

A slope of 2.4 means that a one-unit increase in x is associated with an expected 2.4-unit increase in the fitted outcome, holding nothing else constant in a one-predictor model. It does **not**, by itself, establish causation.

6.4 See the Line

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

rng = np.random.default_rng(42)
x = rng.uniform(0, 10, 80)
y = 4 + 2.5*x + rng.normal(0, 3, 80)
```

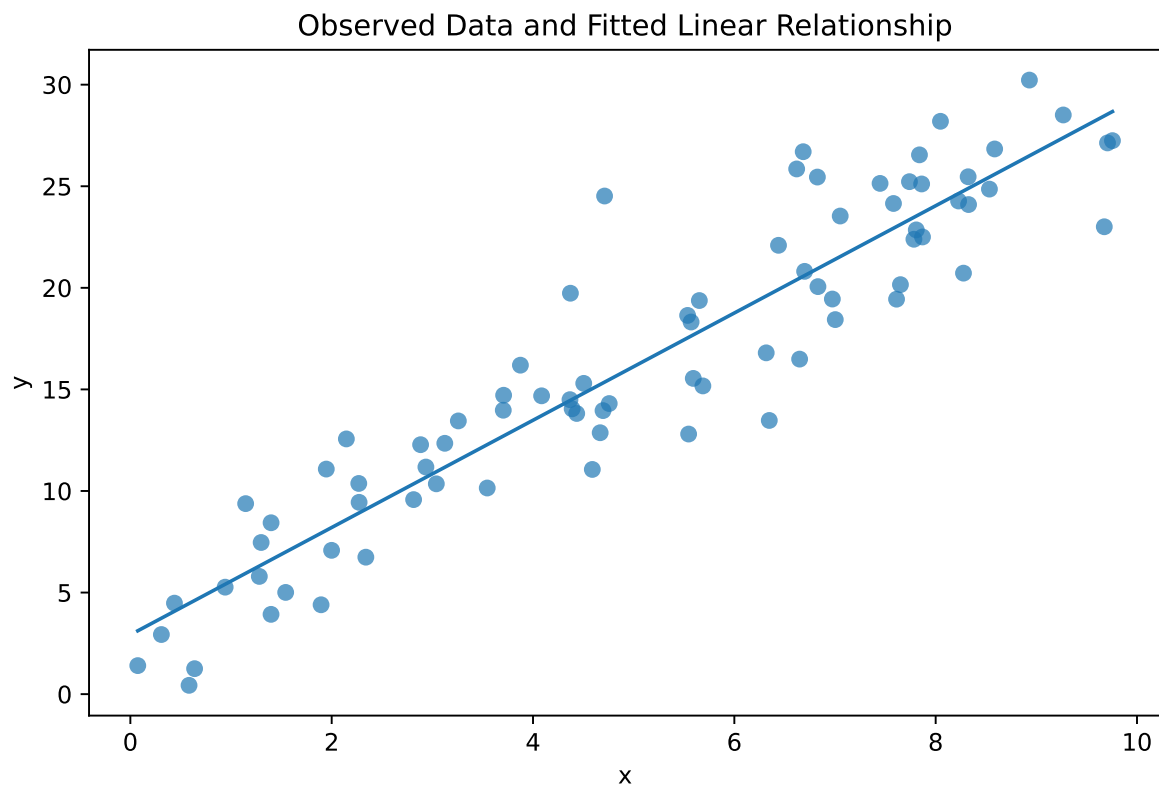
```

X = x.reshape(-1, 1)
model = LinearRegression().fit(X, y)
y_hat = model.predict(X)

plt.figure(figsize=(8, 5))
plt.scatter(x, y, alpha=0.7)
order = np.argsort(x)
plt.plot(x[order], y_hat[order])
plt.xlabel("x")
plt.ylabel("y")
plt.title("Observed Data and Fitted Linear Relationship")
plt.show()

model.intercept_, model.coef_[0]

```



6.5 Ordinary Least Squares

OLS chooses coefficients that minimize the residual sum of squares:

$$RSS(\beta_0, \beta_1) = \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2.$$

Equivalently, minimizing mean squared error gives

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

Squaring makes negative and positive residuals contribute positively and penalizes large errors strongly.

6.5.1 Why the Estimate Is a Minimizer

For simple regression,

$$\hat{\beta}_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2},$$

and

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}.$$

This reveals a connection to Chapter 3: the slope depends on how x and y vary together relative to the variation in x .

6.6 Matrix Form

For multiple predictors,

$$y = X\beta + \epsilon.$$

Under the usual full-rank conditions, the OLS solution can be written

$$\hat{\beta} = (X^\top X)^{-1} X^\top y.$$

This expression is conceptually useful, but numerical software generally does **not** need us to calculate an explicit matrix inverse. Stable numerical methods are preferred.

6.7 Multiple Linear Regression

With p predictors,

$$y_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip} + \epsilon_i.$$

A coefficient β_j represents the expected difference in the fitted outcome for a one-unit difference in x_j , **holding the other included predictors constant**, under the model specification.

That interpretation depends on units, coding, interactions, transformations, and which variables are included.

6.8 A Complete Predictive Workflow

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

data = load_diabetes(as_frame=True)
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

reg = LinearRegression()
reg.fit(X_train, y_train)
pred = reg.predict(X_test)

mae = mean_absolute_error(y_test, pred)
rmse = np.sqrt(mean_squared_error(y_test, pred))
r2 = r2_score(y_test, pred)

mae, rmse, r2
```

(41.54850685988061, np.float64(53.369566710321955), 0.4849058889476755)

6.9 Regression Metrics

6.9.1 Mean Absolute Error

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|.$$

MAE remains in the target's units and weights absolute errors linearly.

6.9.2 Root Mean Squared Error

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}.$$

RMSE is more sensitive to large errors.

6.9.3 Coefficient of Determination

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

R^2 compares squared error with a mean-prediction reference. On unseen data it can be negative.

No Metric Is “The” Truth

Metric choice expresses what kinds of error matter. A model can rank differently under MAE and RMSE.

6.10 Always Compare With a Baseline

```
from sklearn.dummy import DummyRegressor

baseline = DummyRegressor(strategy="mean")
baseline.fit(X_train, y_train)
baseline_pred = baseline.predict(X_test)

np.sqrt(mean_squared_error(y_test, baseline_pred))
```

```
np.float64(74.88122548938378)
```

A regression model should demonstrate value relative to a meaningful simple strategy.

6.11 Residuals: What the Model Failed to Explain

```
residuals = y_test - pred

plt.figure(figsize=(8, 5))
plt.scatter(pred, residuals, alpha=0.7)
plt.axhline(0, linestyle="--")
plt.xlabel("Fitted value")
plt.ylabel("Residual")
plt.title("Residuals vs. Fitted Values")
plt.show()
```

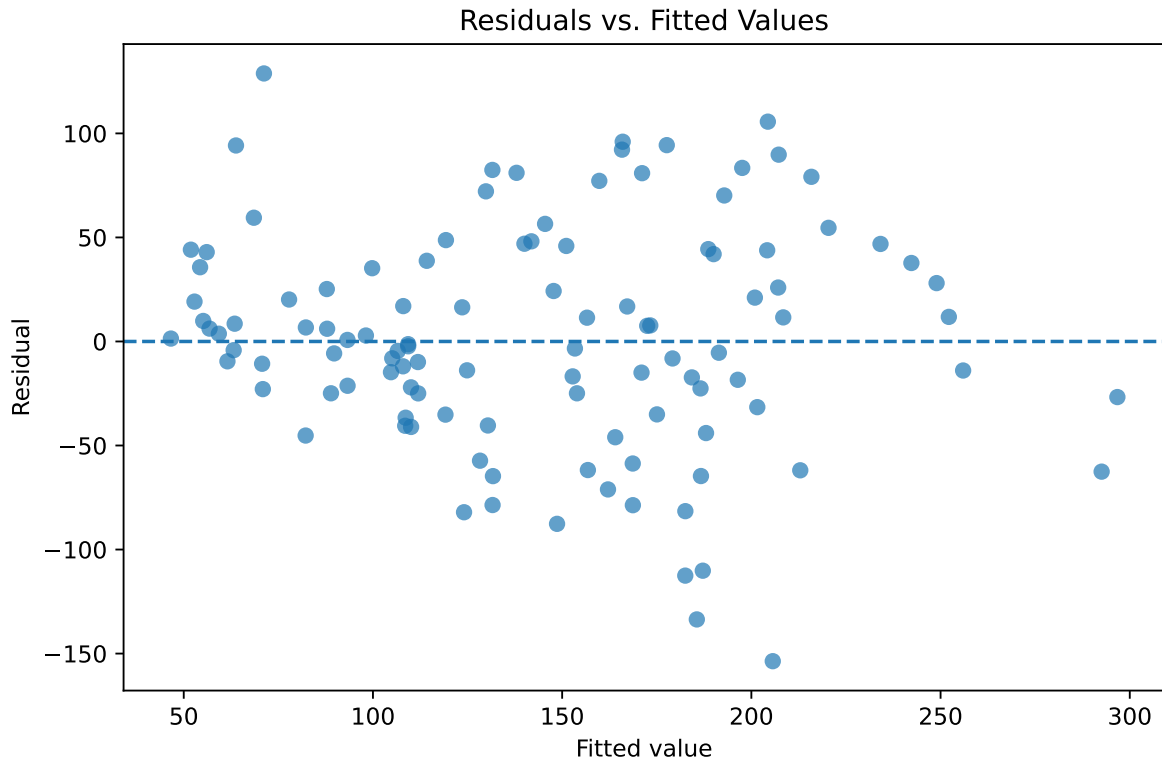



Figure 6.1: Residuals plotted against fitted values can reveal systematic model failure.

A useful residual plot should not show obvious systematic structure. Patterns can suggest nonlinearity, changing variance, omitted structure, unusual observations, or other specification problems.

6.12 Statistical Assumptions and Predictive Goals

Classical linear-model inference commonly considers assumptions involving linearity, error structure, independence, constant conditional variance, and distributional assumptions for particular inferential procedures.

Machine-learning prediction asks a related but different question:

Does the fitted model generalize well to unseen data drawn from the intended deployment population?

A violation may damage inference, prediction, or both—but not necessarily in identical ways.

6.13 Inference With statsmodels

scikit-learn emphasizes predictive estimators. When coefficient inference is central, statsmodels provides statistical-model estimation and diagnostic tools (statsmodels developers 2026).

```
import statsmodels.api as sm

X_sm = sm.add_constant(X_train)
ols = sm.OLS(y_train, X_sm).fit()
print(ols.summary().tables[1])
```

	coef	std err	t	P> t	[0.025	0.975]
const	151.6652	3.023	50.162	0.000	145.717	157.614
age	47.7497	71.642	0.667	0.506	-93.199	188.698
sex	-241.9909	70.762	-3.420	0.001	-381.209	-
102.773						
bmi	531.9711	79.049	6.730	0.000	376.450	687.492
bp	381.5629	75.058	5.084	0.000	233.894	529.232
s1	-918.5029	462.121	-1.988	0.048	-1827.682	-
9.324						
s2	508.2578	372.852	1.363	0.174	-225.294	1241.809
s3	116.9502	238.508	0.490	0.624	-352.292	586.192
s4	269.4923	190.628	1.414	0.158	-105.551	644.536
s5	695.8081	196.697	3.537	0.000	308.826	1082.790
s6	26.3246	76.855	0.343	0.732	-124.881	177.530

Confidence intervals and p-values answer inferential questions under model assumptions. They do not substitute for out-of-sample predictive evaluation.

Two Questions, Two Toolkits

Inference: What do the estimated relationships suggest, with uncertainty, under stated assumptions?

Prediction: How accurately does the fitted function generalize to unseen observations? A strong analysis may need one, the other, or both.

6.14 Same Model, Different Question

The distinction between explanatory and predictive modeling is methodological rather than merely terminological: the same model family can be evaluated differently depending on whether the goal is explanation or out-of-sample prediction (Shmueli 2010).

For the observed sample, the regression model can be written

$$y = X\beta + \epsilon,$$

can support predictive, inferential, or causal questions.

- **Predictive:** How accurately can (Y) be predicted for new observations?
- **Inferential:** What does the sample permit us to conclude about a population relationship?
- **Causal:** What would happen to (Y) if we intervened on (X)?

Prediction $\neq$ Inference $\neq$ Causal Effect

A statistically detectable coefficient may contribute little practical predictive value, while a highly predictive feature may not admit a causal interpretation.

6.14.1 Weak Predictive Gain Despite a Real Relationship

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

rng_sp = np.random.default_rng(42)
n_sp = 12000
x_sp = rng_sp.normal(size=n_sp)
y_sp = 0.10 * x_sp + rng_sp.normal(scale=1.0, size=n_sp)

X_sp = x_sp.reshape(-1, 1)

Xtr_sp, Xte_sp, ytr_sp, yte_sp = train_test_split(
    X_sp, y_sp, test_size=0.30, random_state=42
)
```

```

model_sp = LinearRegression().fit(Xtr_sp, ytr_sp)
pred_sp = model_sp.predict(Xte_sp)

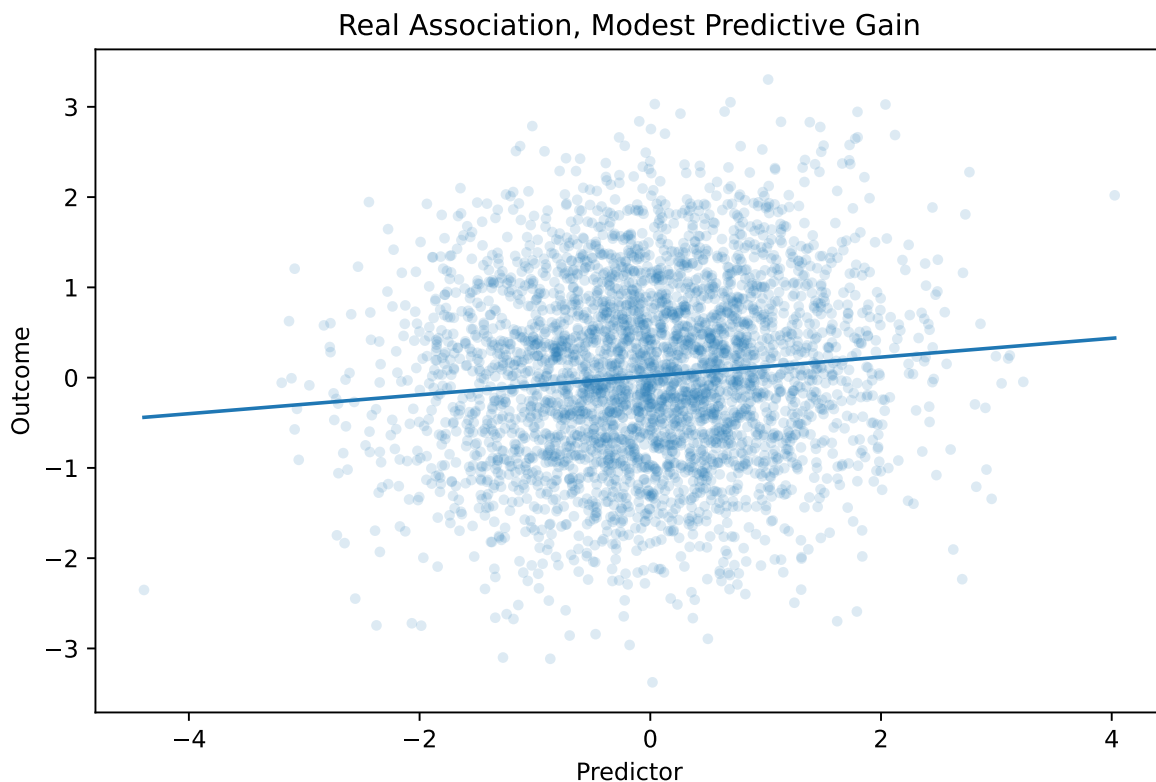
baseline_sp = np.repeat(ytr_sp.mean(), len(yte_sp))

rmse_model_sp = np.sqrt(mean_squared_error(yte_sp, pred_sp))
rmse_baseline_sp = np.sqrt(mean_squared_error(yte_sp, baseline_sp))

plt.figure(figsize=(8, 5))
plt.scatter(Xte_sp[:, 0], yte_sp, alpha=0.15, s=12)
x_sorted_sp = np.sort(Xte_sp[:, 0])
plt.plot(x_sorted_sp, model_sp.predict(x_sorted_sp.reshape(-1, 1)))
plt.xlabel("Predictor")
plt.ylabel("Outcome")
plt.title("Real Association, Modest Predictive Gain")
plt.show()

{
    "Estimated slope": model_sp.coef_[0],
    "Model RMSE": rmse_model_sp,
    "Mean-baseline RMSE": rmse_baseline_sp
}

```



(a) A real relationship can be estimated precisely in a large sample while adding only modest predictive value.

```
{'Estimated slope': np.float64(0.10441225848380067),
 'Model RMSE': np.float64(1.0005930586994922),
 'Mean-baseline RMSE': np.float64(1.005398804014632)}
```

(b)

Figure 6.2

Statistical Significance $\neq$ Predictive Importance

Likewise,

Low Prediction Error $\nRightarrow$ Causal Effect.

Research Reality Check

Before interpreting a regression model, state whether the goal is prediction, association/inference, or causal effect. Do not move between these goals simply because the same equation appears in all three settings.

6.15 Nonlinearity

A linear model can be linear in its parameters while using transformed features.

For example,

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \epsilon.$$

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import Pipeline

poly_model = Pipeline([
    ("poly", PolynomialFeatures(degree=2, include_bias=False)),
    ("reg", LinearRegression())
])
```

The purpose is not to add complexity automatically. It is to represent structure supported by the problem and validation evidence.

6.16 Interactions

Suppose

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_1 x_2 + \epsilon.$$

When $\beta_3 \neq 0$, the relationship involving x_1 depends on x_2 .

Interactions should have substantive meaning rather than being generated indiscriminately.

6.17 Multicollinearity

When predictors contain overlapping information, coefficient estimates can become unstable.

For two highly related predictors, the model may predict adequately while individual coefficients vary substantially across samples.

This is an important distinction:

Coefficient instability and predictive failure are not the same thing.

Multicollinearity motivates careful feature reasoning and, in predictive settings, regularization.

6.18 Ridge Regression

Ridge regression stabilizes coefficient estimation by adding an L_2 penalty, following the regularization approach introduced by Hoerl and Kennard (Hoerl and Kennard 1970):

$$\hat{\beta}^{ridge} = \arg \min_{\beta} \left[\sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p \beta_j^2 \right].$$

As λ increases, coefficients are increasingly shrunk toward zero.

6.19 Lasso Regression

Lasso uses an L_1 penalty and can perform both shrinkage and variable selection (Tibshirani 1996):

$$\hat{\beta}^{lasso} = \arg \min_{\beta} \left[\sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p |\beta_j| \right].$$

Lasso can shrink some coefficients exactly to zero.

In scikit-learn, regularized linear estimators are available through Ridge, Lasso, ElasticNet, and related estimators (scikit-learn developers 2026d).

6.20 Why Scaling Matters for Regularization

Penalty terms act directly on coefficient magnitudes. If features have very different scales, coefficients are not penalized comparably.

Use a pipeline:

```
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge

ridge_model = Pipeline([
    ("scale", StandardScaler()),
    ("ridge", Ridge(alpha=1.0))
])

ridge_model.fit(X_train, y_train)
ridge_pred = ridge_model.predict(X_test)
np.sqrt(mean_squared_error(y_test, ridge_pred))
```

```
np.float64(53.31823922016359)
```

The scaler learns only from training data.

6.21 Bias–Variance Intuition

A very flexible model may fit training data extremely well but respond strongly to sample-specific noise. A constrained model may introduce bias while reducing variance.

Regularization deliberately trades some fitting freedom for stability.

Conceptually,

$$\text{Expected Prediction Error} \approx \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}.$$

This decomposition is an intuition under particular conditions, not a universal observable partition for every fitted model.

6.22 Visual Experiment: Coefficient Shrinkage


```

from sklearn.linear_model import Ridge

alphas = np.logspace(-3, 3, 60)
coef_path = []

X_scaled = StandardScaler().fit_transform(X_train)

for alpha in alphas:
    ridge = Ridge(alpha=alpha)
    ridge.fit(X_scaled, y_train)
    coef_path.append(ridge.coef_)

coef_path = np.array(coef_path)

plt.figure(figsize=(8, 5))
for j in range(coef_path.shape[1]):
    plt.plot(alphas, coef_path[:, j])
plt.xscale("log")
plt.xlabel("Ridge alpha")
plt.ylabel("Coefficient")
plt.title("Regularization Shrinks Coefficients")
plt.show()

```

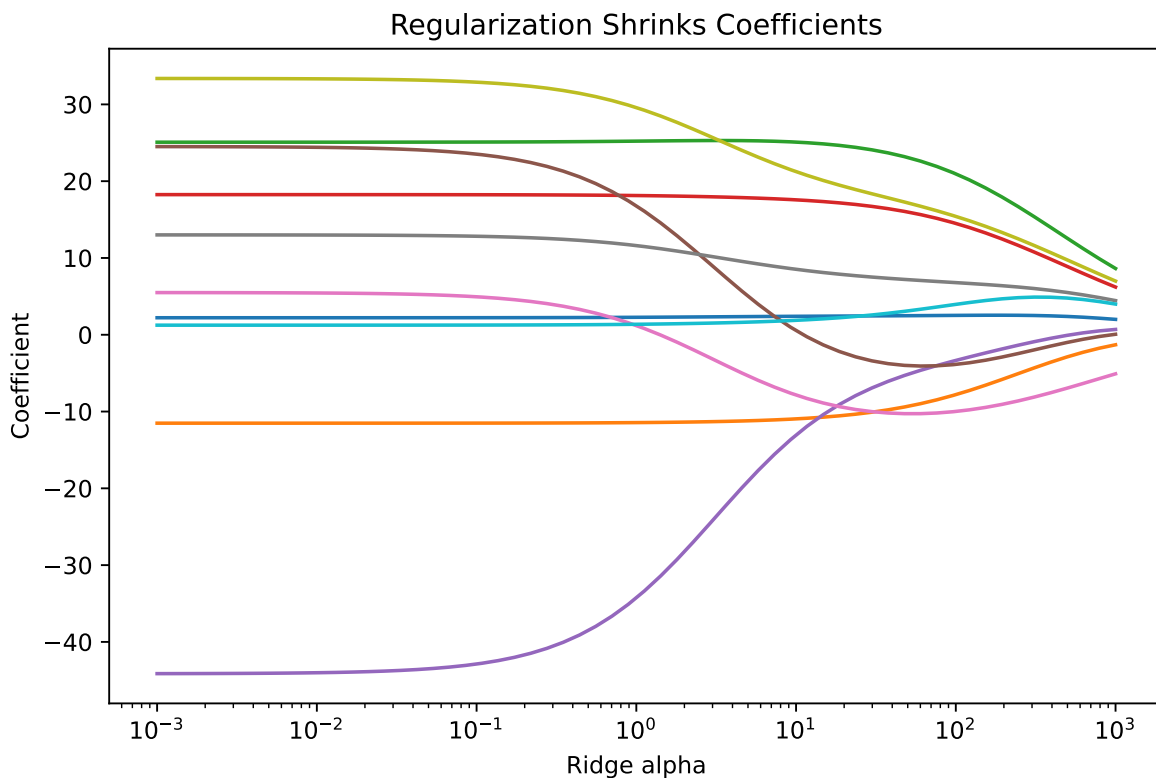


Figure 6.3: Ridge regularization progressively shrinks standardized coefficients.

For formal model selection, the scaling and hyperparameter search must remain inside the resampling workflow. We will deepen this in Chapter 16.

6.23 Coefficients Are Not Automatically Causes

Predictive association does not by itself identify an intervention effect; causal claims require assumptions and designs appropriate to causal inference (Hernán and Robins 2020).

Suppose a model finds

$$\hat{y} = 12 + 3.1(\text{study hours}) + 5.2(\text{attendance}).$$

It is tempting to say that increasing attendance by one unit **causes** a 5.2-unit outcome increase. Regression alone does not justify that conclusion.

Confounding, selection, measurement, reverse causation, and model specification all matter.

6.24 Computer Vision Connection: Regression From Images

Computer vision is not limited to classifying images. Many visual tasks contain continuous prediction components.

Examples include:

- estimating depth from an image;
- predicting object coordinates;
- estimating pose or orientation;
- predicting image-quality scores;
- estimating continuous physical measurements from visual data.

A visual regression problem can still be written conceptually as

$$\hat{y}_i = f_{\theta}(x_i),$$

where (x_i) is a numerical representation of image (i) and (y_i) is a continuous target.

The statistical cautions from this chapter still apply. A model that predicts a continuous quantity from pixels does not automatically explain the physical or biological mechanism that produced the target.

Vision Learning Thread

Later computer-vision examples will focus primarily on classification because the handwritten-digits dataset provides a compact reproducible benchmark. This regression connection is included to reinforce that **computer vision is a workload containing multiple prediction types**, not a synonym for image classification.

6.25 Three Tracks

6.25.1 Health Track

A regression problem might predict a continuous biomarker or physiological measurement. Researchers must distinguish predictive usefulness from etiological interpretation and consider whether survey design or complex sampling affects inferential goals.

6.25.2 Education Track

Institution-level regression might predict future retention or completion rates from prior-year characteristics. Temporal alignment is essential, and rates may have differing denominator reliability.

6.25.3 Open ML Benchmark Track

Benchmark regression datasets are useful for comparing OLS, Ridge, Lasso, nonlinear feature expansion, and sensitivity to scaling under controlled conditions.

6.26 Beyond the Algorithm: Is Simplicity a Limitation or a Strength?

The Intellectual Value of a Simple Model

A more complex model can be more accurate. It can also be harder to inspect, explain, challenge, and learn from.

Linear regression forces a strong claim: the outcome can be approximated through a weighted combination of represented features. That claim will often be incomplete. Its simplicity, however, makes assumptions visible.

A simple model can serve as:

- a baseline;
- a diagnostic instrument;
- a scientific summary;
- a communication tool;
- a test of whether complexity is actually necessary.

The relevant question is therefore not:

“Is the model simple?”

but:

“Is the model sufficiently accurate, appropriately specified, and useful for the purpose?”

Complexity should earn its place through evidence.

6.27 AI Pair Programmer

Ask an AI assistant to interpret coefficients from a fitted regression model.

Audit whether it:

- respects units;
- conditions interpretations on other included predictors;
- distinguishes association from causation;
- notices transformations and interactions;
- treats standardized coefficients correctly;
- confuses statistical significance with practical importance;
- reports training fit as evidence of generalization.

Then rewrite the interpretation.

6.28 Hands-On Lab: OLS to Regularization

Using a regression dataset:

1. complete the Chapter 4 Problem Framing Canvas;
2. complete the Chapter 5 Feature Ledger;
3. establish a `DummyRegressor` baseline;
4. fit OLS in a leakage-safe workflow;
5. calculate MAE, RMSE, and R^2 on held-out data;
6. inspect residuals;
7. fit Ridge and Lasso with scaling inside pipelines;
8. compare predictive performance;
9. compare coefficient behavior;
10. identify at least one possible nonlinearity or interaction;
11. distinguish predictive findings from inferential or causal claims;
12. document reproducibility decisions.

6.29 Practitioner Track

A practitioner should be able to fit, evaluate, diagnose, regularize, and communicate regression models while maintaining leakage-safe workflows.

6.30 Researcher Track

A researcher should additionally distinguish predictive from inferential aims, justify model specification, quantify uncertainty when appropriate, diagnose assumptions, and avoid causal overstatement.

6.31 Research Studio

Choose a health or education continuous outcome suitable for a public-data study.

Write:

1. research problem and contribution;
2. unit and target;
3. temporal framing;
4. candidate predictors with theory;
5. baseline;
6. OLS specification;
7. candidate interaction/nonlinearity;
8. regularized alternative;
9. validation strategy;
10. inferential versus predictive claims;
11. possible conference/journal contribution;
12. one claim the design cannot support.

6.32 Professor's Challenge: The Significant Model

A paper reports:

“Our regression model explains student success. All predictors with $p < .05$ are important causes. The model has $R^2 = .62$, demonstrating strong predictive accuracy.”

No held-out evaluation is reported. Predictors were selected after examining the full dataset. Several are highly correlated. Residual diagnostics are absent. One predictor is measured after the outcome period begins.

Critique:

- “explains”;
- causal language;
- significance versus importance;
- R^2 versus predictive performance;
- feature selection leakage;
- multicollinearity;
- temporal leakage;
- missing diagnostics;
- absence of a baseline and out-of-sample evaluation.

Redesign the analysis for both an **inferential objective** and a **predictive objective**.

6.33 Knowledge Check

1. What does a regression slope represent?
2. What does OLS minimize?
3. How are covariance and the simple-regression slope related?
4. Why can RMSE and MAE rank models differently?
5. What does a negative test-set R^2 imply?
6. Why inspect residuals?
7. How do inference and prediction differ?
8. Why can multicollinearity destabilize coefficients?
9. What does Ridge penalize?
10. What does Lasso penalize?
11. Why scale before regularized regression?
12. What is the bias–variance tradeoff?
13. Why is a coefficient not automatically causal?
14. When might a simple linear model be preferable to a more complex model?
15. How can a coefficient be statistically important but weak for prediction?
16. Why does strong predictive performance not establish causality?
17. Why should a predictive regression model be compared with a simple baseline?

6.34 Reproducibility Check

Record:

- problem specification;
- feature ledger;
- split/validation design;
- baseline;
- preprocessing pipeline;
- transformations and interactions;
- regression estimator and parameters;
- regularization hyperparameters;
- metrics;
- residual diagnostics;
- inferential procedures if used;
- random seeds;
- package versions;
- code generating every reported result.

6.35 Key Takeaways

- Linear regression is both a predictive model and a foundation for statistical reasoning.
- OLS minimizes squared residual error.
- Multiple-regression coefficients are conditional on the model specification.
- MAE, RMSE, and R^2 answer different performance questions.
- Residuals reveal structure the model failed to capture.
- Statistical inference and out-of-sample prediction are related but distinct goals.
- Multicollinearity can destabilize coefficients without necessarily destroying prediction.
- Ridge and Lasso constrain coefficient magnitude to improve stability/generalization.
- Scaling matters when regularization acts on coefficient magnitudes.
- Regression coefficients do not establish causation.
- Simple models remain valuable when their assumptions, performance, and purpose are appropriate.
- The same regression equation can serve different goals, but those goals require different evidence.
- Statistical significance does not imply predictive usefulness.
- Predictive performance does not establish causal effect.

7 Classification: From Probability to Decision

7.1 Learning Objectives

By the end of this chapter, you should be able to:

- distinguish probability estimation from classification decisions;
- explain why ordinary linear regression is generally unsuitable for binary outcomes;
- derive the logistic function from log-odds;
- interpret logistic-regression coefficients and odds ratios;
- construct and interpret confusion matrices;
- calculate accuracy, sensitivity, specificity, precision, and F1;
- explain how prevalence affects metric interpretation;
- use ROC and precision-recall curves;
- distinguish discrimination from calibration;
- explain how thresholds encode operational tradeoffs;
- compare classifiers using metrics aligned with intended use;
- extend classification reasoning to multiclass problems.
- build and evaluate a multiclass computer-vision baseline using image data.

7.2 Why This Matters

Classification is often described as predicting categories. But many classifiers first estimate

$$\hat{p}_{ik} = P(Y_i = k \mid x_i).$$

A decision rule then converts model output into a category or action.

Central Question

How do we move from estimating the probability of an outcome to making a defensible classification decision?

7.3 Why Not Ordinary Linear Regression?

For $Y \in \{0, 1\}$, a linear model can produce fitted values outside $[0, 1]$. Logistic regression instead models log-odds.

Let $p = P(Y = 1 \mid X = x)$. The odds are

$$\frac{p}{1-p},$$

and the logit is

$$\log\left(\frac{p}{1-p}\right).$$

Logistic regression assumes

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p.$$

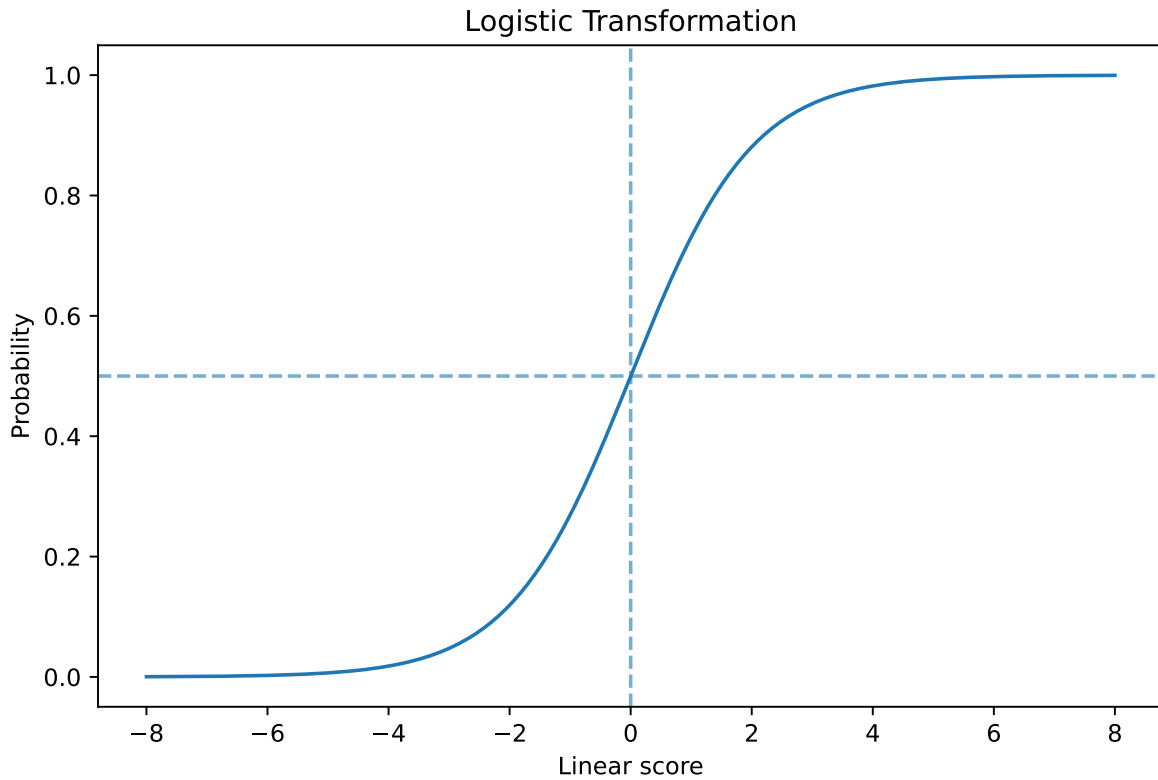
Therefore

$$p = \frac{1}{1 + \exp[-(\beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p)]}.$$

7.4 Visual Intuition: Logistic Function

```
import numpy as np
import matplotlib.pyplot as plt

z=np.linspace(-8,8,400)
p=1/(1+np.exp(-z))
plt.figure(figsize=(8,5))
plt.plot(z,p)
plt.axhline(.5,linestyle="--",alpha=.6)
plt.axvline(0,linestyle="--",alpha=.6)
plt.xlabel("Linear score")
plt.ylabel("Probability")
plt.title("Logistic Transformation")
plt.show()
```



7.5 Coefficients and Odds Ratios

β_j is a conditional change in log-odds. Exponentiation gives

$$OR_j = e^{\beta_j}.$$

Odds Ratio	Probability Difference
An odds ratio of 1.49 does not mean probability rises by 49 percentage points.	

7.6 Leakage-Safe Logistic Regression

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
```

```

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

data=load_breast_cancer(as_frame=True)
X=data.data
# Recode malignant as positive class 1.
y=(data.target==0).astype(int)

X_train,X_test,y_train,y_test=train_test_split(
    X,y,test_size=.25,random_state=42,stratify=y
)

model=Pipeline([
    ("scale",StandardScaler()),
    ("classifier",LogisticRegression(max_iter=2000))
])
model.fit(X_train,y_train)
prob=model.predict_proba(X_test)[:,:1]
pred=(prob>=.50).astype(int)

```

7.7 Probability Is Not Yet a Decision

$$\hat{y}_i = \begin{cases} 1, & \hat{p}_i \geq \tau \\ 0, & \hat{p}_i < \tau. \end{cases}$$

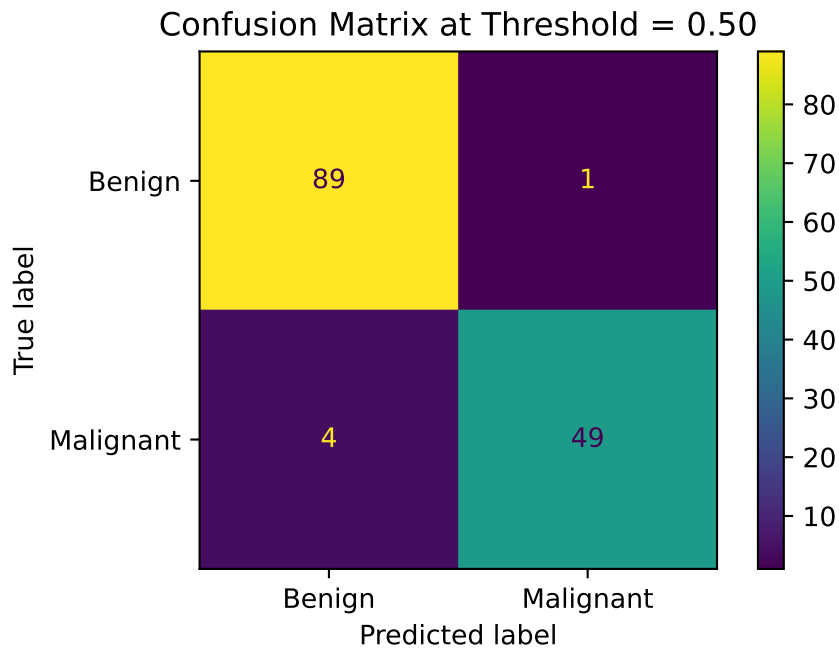
Thus,



7.8 Confusion Matrix

	Actual Positive	Actual Negative
Predicted Positive	TP	FP
Predicted Negative	FN	TN

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
cm=confusion_matrix(y_test,pred)
ConfusionMatrixDisplay(cm,display_labels=["Benign","Malignant"]).plot()
plt.title("Confusion Matrix at Threshold = 0.50")
plt.show()
```



7.9 Core Metrics

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Sensitivity = \frac{TP}{TP + FN}$$

$$Specificity = \frac{TN}{TN + FP}$$

$$Precision = \frac{TP}{TP + FP}$$

$$F_1 = 2 \frac{Precision \times Recall}{Precision + Recall}.$$

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
tn, fp, fn, tp = cm.ravel()
{
    "accuracy": accuracy_score(y_test, pred),
    "sensitivity": recall_score(y_test, pred),
    "specificity": tn / (tn + fp),
    "precision": precision_score(y_test, pred),
    "f1": f1_score(y_test, pred)
}
```

```
{'accuracy': 0.965034965034965,
 'sensitivity': 0.9245283018867925,
 'specificity': np.float64(0.9888888888888889),
 'precision': 0.98,
 'f1': 0.9514563106796117}
```

7.10 Prevalence Changes Meaning

A classifier predicting everyone negative achieves 99% accuracy when only 1% of observations are positive.

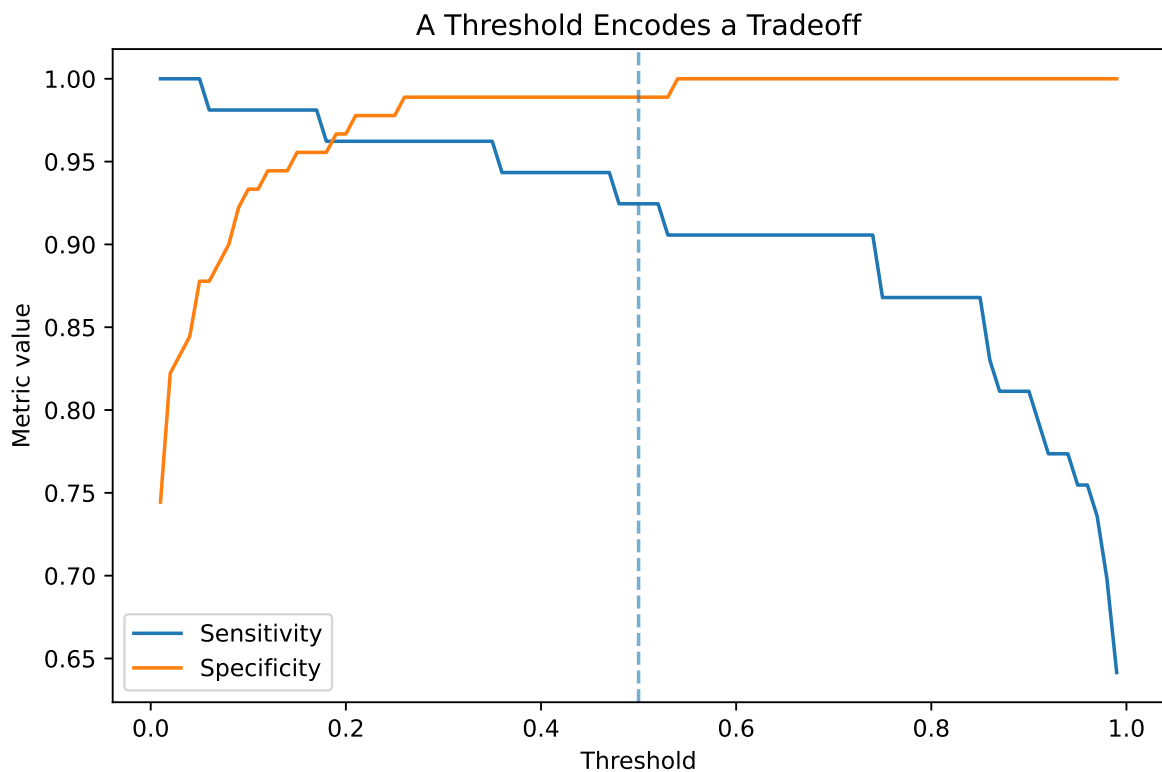
Metric Meaning Is Contextual

A metric is useful when it answers a question relevant to the problem—not merely because its value is large.

7.11 Thresholds Create Tradeoffs

```
threshold_grid = np.linspace(.01, .99, 99)
sens = []; spec = []
for t in threshold_grid:
    pr = (prob >= t).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_test, pr).ravel()
    sens.append(tp / (tp + fn))
    spec.append(tn / (tn + fp))
```

```
plt.figure(figsize=(8,5))
plt.plot(threshold_grid,sens,label="Sensitivity")
plt.plot(threshold_grid,spec,label="Specificity")
plt.axvline(.5,linestyle="--",alpha=.6)
plt.xlabel("Threshold")
plt.ylabel("Metric value")
plt.title("A Threshold Encodes a Tradeoff")
plt.legend()
plt.show()
```



There is no universally correct threshold.

7.12 ROC Analysis

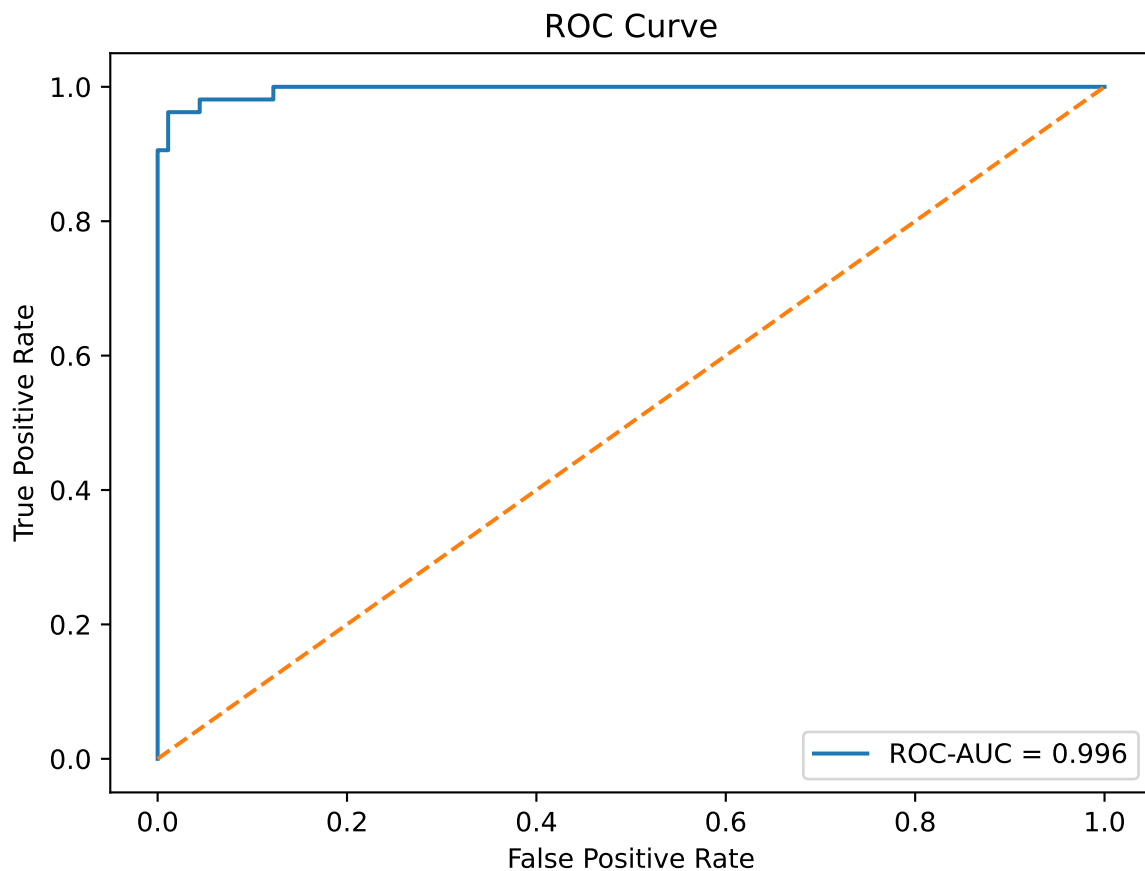
```
from sklearn.metrics import roc_curve,roc_auc_score
fpr,tpr,_=roc_curve(y_test,prob)
```

```

auc=roc_auc_score(y_test,prob)

plt.figure(figsize=(7,5))
plt.plot(fpr, tpr, label=f"ROC-AUC = {auc:.3f}")
plt.plot([0,1],[0,1],linestyle="--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()

```

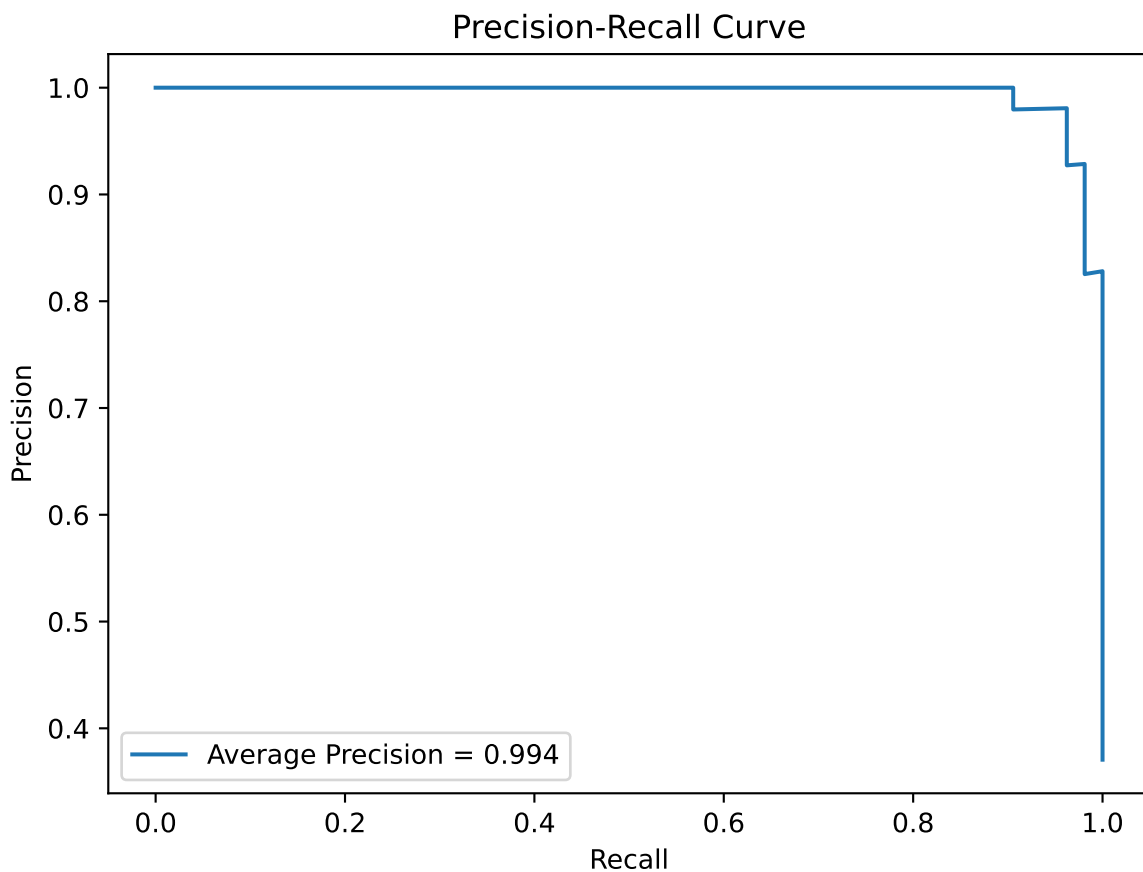


ROC-AUC summarizes discrimination/ranking across thresholds. It does not establish calibration or the quality of a particular decision threshold. ROC analysis provides a threshold-spanning view of the tradeoff between true-positive and false-positive rates (Fawcett 2006).

7.13 Precision-Recall Analysis

```
from sklearn.metrics import precision_recall_curve, average_precision_score
precision_vals, recall_vals, _ = precision_recall_curve(y_test, prob)
ap = average_precision_score(y_test, prob)

plt.figure(figsize=(7, 5))
plt.plot(recall_vals, precision_vals, label=f"Average Precision = {ap:.3f}")
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision-Recall Curve")
plt.legend()
plt.show()
```



Precision-recall analysis is often especially informative when the positive class is uncommon; under class imbalance, precision-recall curves can reveal performance differences that ROC

curves may obscure (Saito and Rehmsmeier 2015).

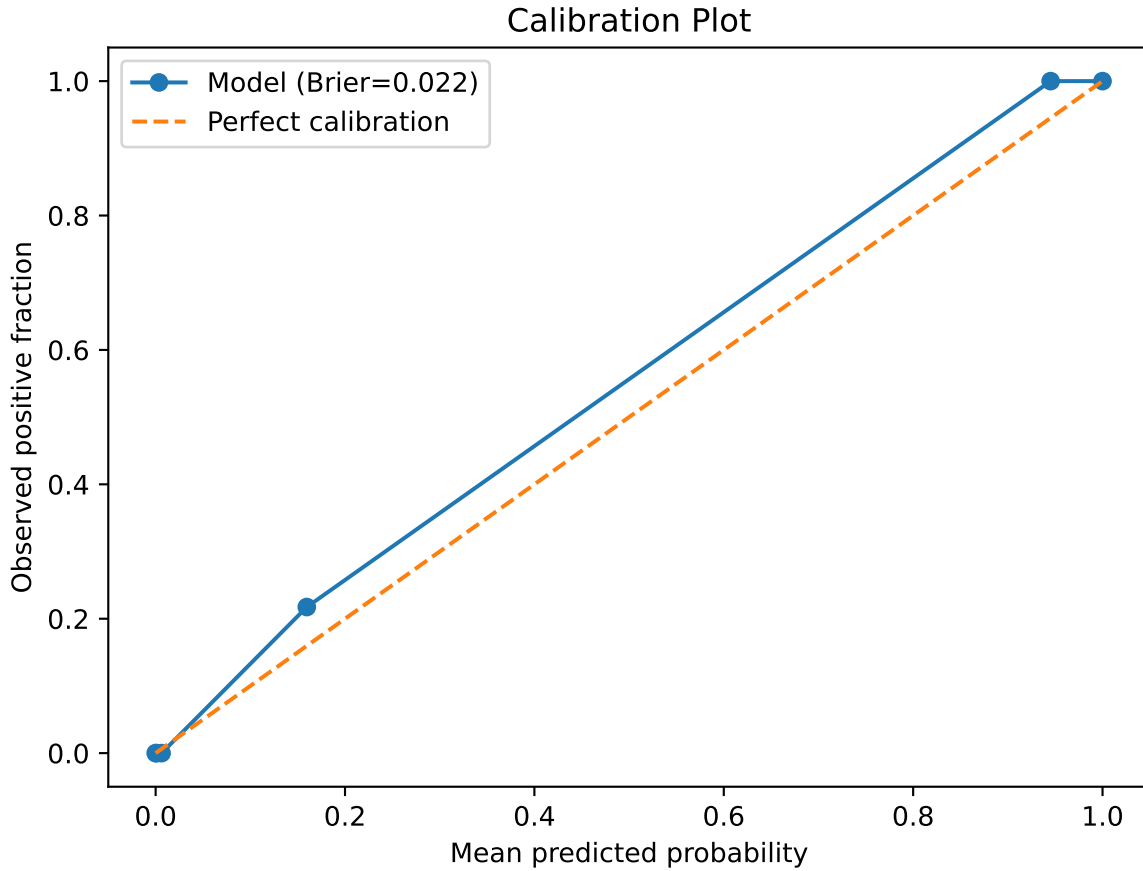
7.14 Discrimination Is Not Calibration

Discrimination concerns ranking. Calibration concerns whether predicted probabilities correspond to observed frequencies.

```
from sklearn.calibration import calibration_curve
from sklearn.metrics import brier_score_loss

frac, mean_pred = calibration_curve(y_test, prob, n_bins=6, strategy="quantile")
brier = brier_score_loss(y_test, prob)

plt.figure(figsize=(7, 5))
plt.plot(mean_pred, frac, marker="o", label=f"Model (Brier={brier:.3f})")
plt.plot([0, 1], [0, 1], linestyle="--", label="Perfect calibration")
plt.xlabel("Mean predicted probability")
plt.ylabel("Observed positive fraction")
plt.title("Calibration Plot")
plt.legend()
plt.show()
```



A model can discriminate well and still produce poorly calibrated probabilities. Probability calibration therefore deserves separate evaluation when predicted probabilities will support decisions (Niculescu-Mizil and Caruana 2005).

7.15 From Probability to Decision

A classifier may estimate

$$\hat{p}_i = P(Y_i = 1 \mid x_i),$$

but real systems eventually require actions:

Probability $\rightarrow$ Threshold $\rightarrow$ Decision $\rightarrow$ Consequence.

7.15.1 Why 0.50 Is Not Universal

The rule

$$\hat{Y} = \begin{cases} 1, & \hat{p} \geq 0.50 \\ 0, & \hat{p} < 0.50 \end{cases}$$

is a convention, not a law.

Thresholds should depend on false-positive and false-negative consequences, prevalence, intervention capacity, cost, benefit, and fairness.

If false positives cost (C_{FP}) and false negatives cost (C_{FN}),

$$\text{Cost}(t) = C_{\text{FP}}FP(t) + C_{\text{FN}}FN(t).$$

7.15.2 Threshold Choice Under Asymmetric Costs

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix

cost_fp = 1
cost_fn = 6

thresholds_dc = np.linspace(0.05, 0.95, 19)
rows_dc = []

for t_dc in thresholds_dc:
    pred_dc = (prob >= t_dc).astype(int)

    tn_dc, fp_dc, fn_dc, tp_dc = confusion_matrix(
        y_test, pred_dc, labels=[0, 1]
    ).ravel()

    rows_dc.append({
        "threshold": t_dc,
        "false_positives": fp_dc,
        "false_negatives": fn_dc,
        "cost": cost_fp * fp_dc + cost_fn * fn_dc
```

```

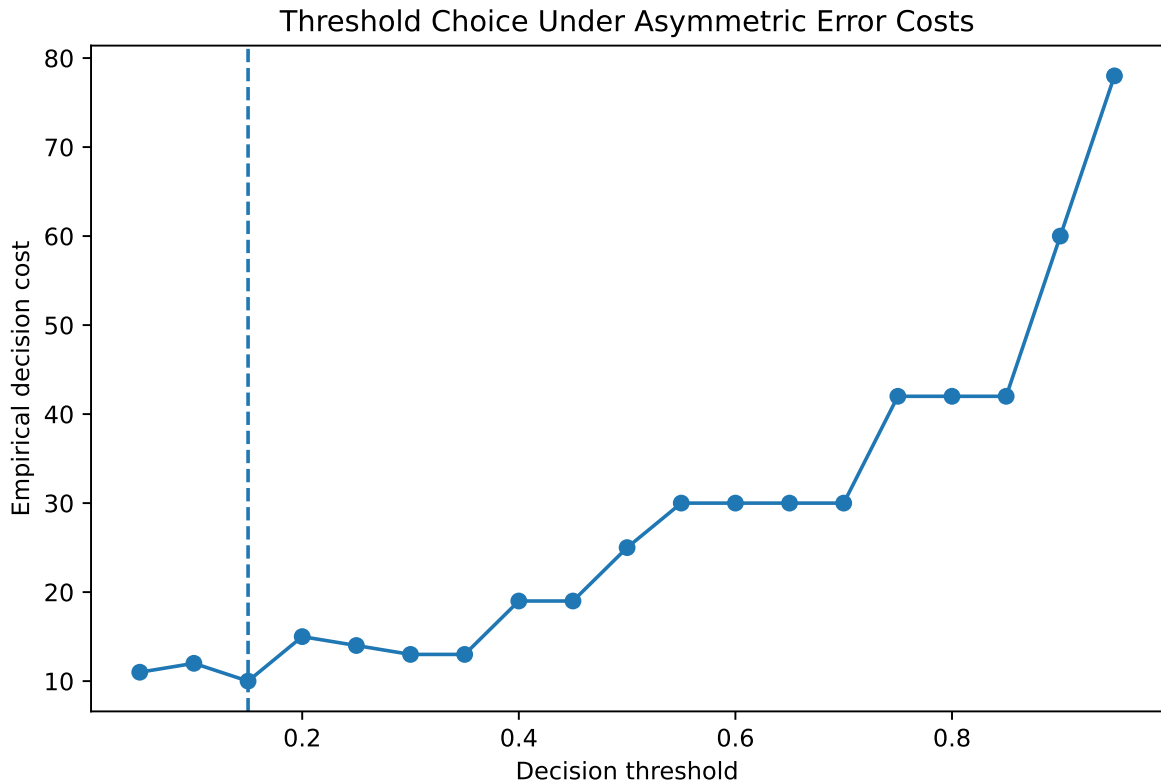
    })

cost_df = pd.DataFrame(rows_dc)
best_row_dc = cost_df.loc[cost_df["cost"].idxmin()]

plt.figure(figsize=(8, 5))
plt.plot(cost_df["threshold"], cost_df["cost"], marker="o")
plt.axvline(best_row_dc["threshold"], linestyle="--")
plt.xlabel("Decision threshold")
plt.ylabel("Empirical decision cost")
plt.title("Threshold Choice Under Asymmetric Error Costs")
plt.show()

best_row_dc

```



(a) The cost-minimizing threshold can differ from 0.50 when classification errors have asymmetric consequences.

```
threshold      0.15
false_positives 4.00
false_negatives 1.00
cost           10.00
Name: 2, dtype: float64
```

(b)

Figure 7.1

7.15.3 Computer Vision Learning Experience: Handwritten-Digit Classification

The handwritten-digits dataset provides a compact computer-vision workload in which each observation is an (8×8) grayscale image and the target is one of ten digit classes.

This is a **multiclass classification** problem:

$$Y_i \in \{0, 1, \dots, 9\}.$$

For image (i), the classical baseline used here represents the image as a flattened feature vector

$$x_i \in \mathbb{R}^{64}.$$

That representation makes the task compatible with logistic regression while giving us a baseline that later vision models can improve upon.

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

digits_cv = load_digits()

X_cv = digits_cv.data
y_cv = digits_cv.target
images_cv = digits_cv.images

fig, axes = plt.subplots(2, 5, figsize=(9, 4))
for ax, image_cv, label_cv in zip(
    axes.ravel(),
    images_cv[:10],
    y_cv[:10]
):
    ax.imshow(image_cv, cmap="gray")
    ax.set_title(f"Label: {label_cv}")
    ax.axis("off")

plt.tight_layout()
plt.show()
```

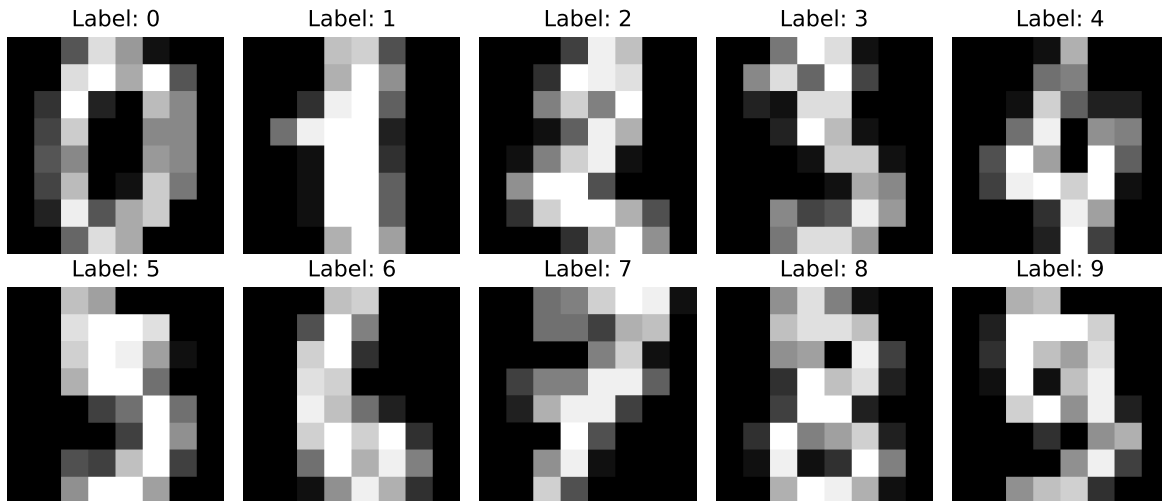


Figure 7.2: Examples from the handwritten-digits computer-vision workload.

```
X_train_cv, X_test_cv, y_train_cv, y_test_cv = train_test_split(
    X_cv,
    y_cv,
    test_size=0.25,
    random_state=42,
    stratify=y_cv
)

vision_logit = Pipeline([
    ("scale", StandardScaler()),
    ("model", LogisticRegression(max_iter=2000))
])

vision_logit.fit(
    X_train_cv,
    y_train_cv
)

pred_cv = vision_logit.predict(
    X_test_cv
)

accuracy_score(
    y_test_cv,
    pred_cv
)
```



```
)
```

```
0.9777777777777777
```

```
ConfusionMatrixDisplay.from_predictions(  
    y_test_cv,  
    pred_cv  
)  
plt.title("Handwritten-Digit Classification")  
plt.show()
```

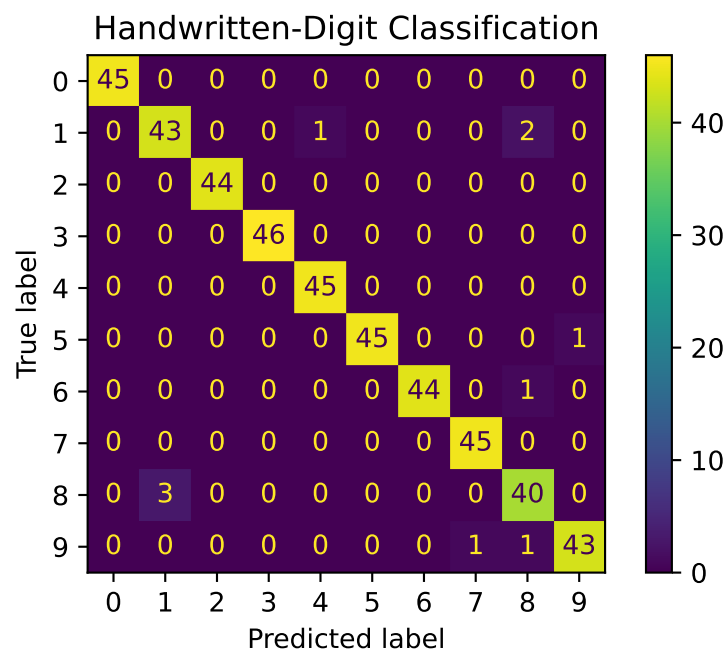


Figure 7.3: Confusion matrix for the logistic-regression computer-vision baseline.

7.15.4 Think Like a Data Scientist

Do not stop at overall accuracy.

Ask:

- Which digits are most often confused?
- Are the mistakes visually plausible?

- Does flattening discard spatial information that another representation could exploit?
- Would performance change on handwriting collected from a different population or device?
- What evaluation design would be required if several images came from the same person?

Research Reality Check

Strong performance on `load_digits` demonstrates performance on a small benchmark under a particular representation and split.

It does **not** establish that the model generalizes to every handwriting style, scanner, camera, demographic population, or image-acquisition process.

7.16 Three Tracks

Health: false negatives may delay diagnosis; false positives may trigger unnecessary testing.

Education: false positives may be acceptable for supportive outreach but harmful for punitive intervention.

Open ML: fraud thresholds balance missed fraud against blocked legitimate activity and investigator capacity.

Best Ranking Model $\neq$ Best Decision Policy

.

Research Reality Check

A high AUC does not establish deployment readiness. Decision quality also depends on calibration, threshold selection, error consequences, capacity constraints, subgroup behavior, and downstream action.

7.17 Threshold Selection as Decision Analysis

A simplified cost function is

$$C(\tau) = C_{FN}FN(\tau) + C_{FP}FP(\tau).$$

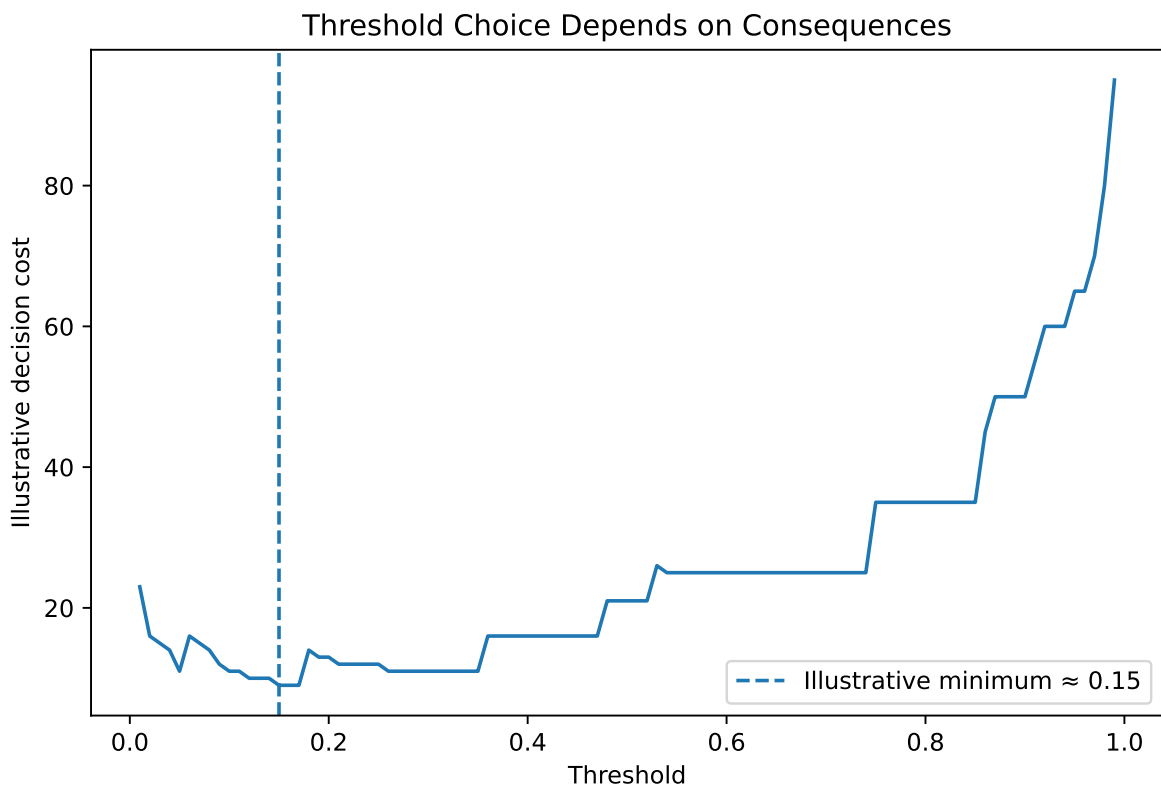
```

cost_fn=5
cost_fp=1
costs=[]
for t in threshold_grid:
    pr=(prob>=t).astype(int)
    tn,fp,fn,tp=confusion_matrix(y_test,pr).ravel()
    costs.append(cost_fn*fn+cost_fp*fp)

best_i=int(np.argmin(costs))
best_t=threshold_grid[best_i]

plt.figure(figsize=(8,5))
plt.plot(threshold_grid,costs)
plt.axvline(best_t,linestyle="--",label=f"Illustrative minimum {best_t:.2f}")
plt.xlabel("Threshold")
plt.ylabel("Illustrative decision cost")
plt.title("Threshold Choice Depends on Consequences")
plt.legend()
plt.show()

```



The cost values are illustrative. Real consequences require substantive justification.

7.18 Class Imbalance

Responses may include appropriate metrics, threshold adjustment, class weighting, resampling, additional data collection, or reconsidering the target.

Never Resample Before the Split

Oversampling or synthetic sampling must remain inside the training/resampling workflow.

7.19 Multiclass Classification

For $Y \in \{1, \dots, K\}$, models estimate probabilities across classes. Evaluation can use per-class metrics, confusion matrices, and macro or weighted averages.

Macro averaging gives each class equal weight. Weighted averaging weights classes by support.

7.20 Three Tracks

7.20.1 Health Track

Consider positive-outcome definition, prevalence, follow-up action, error consequences, calibration, subgroup performance, and the distinction between risk identification and treatment recommendation.

7.20.2 Education Track

Consider prediction timing, support following a flag, stigma or burden from false positives, missed support from false negatives, intervention capacity, and subgroup performance.

7.20.3 Open ML Benchmark Track

Manipulate scaling, prevalence, overlap, and thresholds to understand classifier behavior without equating benchmark performance with real-world usefulness.

7.21 Beyond the Algorithm: A Probability Is Not a Decision

The Space Between Prediction and Action

A predicted risk of 0.63 contains no instruction. It does not itself say to label, contact, restrict, intervene, or ignore.

Choosing $\tau = 0.50$ is also not neutral: it determines who experiences the consequences of the decision system.

What the model estimates $\neq$ What the organization decides.

Ask:

What decision rule transforms this probability into action, who chose that rule, and who bears the cost when it is wrong?

7.22 AI Pair Programmer

Ask AI to choose the “best” classifier from accuracy alone. Audit whether it asks about prevalence, intended use, error costs, threshold, calibration, capacity, subgroup performance, and uncertainty.

7.23 Hands-On Lab: Probability to Decision

1. Complete the Problem Framing Canvas.
2. Complete the Feature Ledger.
3. Establish a `DummyClassifier` baseline.
4. Build a leakage-safe logistic-regression pipeline.
5. Obtain predicted probabilities.
6. Evaluate thresholds .20, .50, and .80.
7. Calculate confusion matrices and core metrics.
8. Plot ROC and precision-recall curves.
9. Examine calibration.
10. Define an illustrative asymmetric cost structure.
11. Identify a candidate operational threshold.
12. Explain why it remains provisional.
13. Compare one additional classifier.
14. Distinguish ranking, probability, and decision quality.
15. Document unsupported causal claims.

7.24 Research Studio

Design a health, education, or benchmark classification study. Specify target, positive class, timing, expected prevalence, predictors, leakage risks, baseline, discrimination metrics, calibration, threshold strategy, asymmetric consequences, validation, scholarly contribution, and unsupported conclusions.

7.25 Professor's Challenge: Which Model Is Better?

Model A has higher accuracy and ROC-AUC but lower sensitivity and poor high-risk calibration. Model B has lower accuracy/AUC but substantially higher sensitivity and good calibration. The outcome is uncommon and false negatives are believed more consequential than additional follow-up.

Critique any claim that Model A is automatically superior. Address prevalence, discrimination, calibration, sensitivity, specificity, precision, threshold dependence, error consequences, metric uncertainty, threshold-selection leakage, external validation, subgroup performance, capacity, and deployment readiness.

7.26 Knowledge Check

1. Why is linear regression unsuitable for binary probability modeling?
2. What are odds?
3. What does the logit do?
4. How is a logistic coefficient interpreted?
5. Why is an odds ratio not a probability difference?
6. What converts probability into class?
7. What does sensitivity measure?
8. What does precision measure?
9. Why can accuracy mislead for rare outcomes?
10. What does ROC-AUC summarize?
11. Why use precision-recall analysis?
12. How do discrimination and calibration differ?
13. Why is .50 not automatically correct?
14. Why must resampling stay inside training?
15. What does macro averaging emphasize?
16. Why is a probability not a decision?
17. Why is 0.50 not a universal threshold?
18. How do asymmetric error costs affect threshold choice?
19. Why can models with similar AUCs produce different deployment decisions?

7.27 Reproducibility Check

Record positive-class definition, prevalence, feature ledger, preprocessing, validation design, baseline, classifier, probabilities, threshold-selection procedure, confusion matrices, discrimination metrics, calibration, cost assumptions, subgroup analyses, seeds, package versions, and all code.

Never select a threshold on the final test set and report performance on that same set as if the threshold were fixed in advance.

7.28 Key Takeaways

- Classification often begins with probability estimation.
- Logistic regression models log-odds as a linear function.
- Odds ratios and probability differences differ.
- Thresholds transform probabilities into classifications.
- Confusion matrices reveal error types.
- Accuracy can mislead under imbalance.
- ROC and precision-recall analysis reveal different aspects of ranking.
- Discrimination and calibration are distinct.
- Thresholds should reflect intended use and consequences.
- A highest-AUC model is not automatically the best decision system.
- A predicted probability does not contain an instruction for action.
- Classification probabilities become consequential only after a decision rule is applied.
- Thresholds should reflect consequences and constraints.
- Strong discrimination does not guarantee a good decision policy.

8 Decision Trees and Random Forests: Learning by Partitioning

8.1 Learning Objectives

By the end of this chapter, you should be able to:

- explain recursive partitioning as a supervised-learning strategy;
- distinguish root nodes, internal nodes, branches, and leaves;
- calculate and interpret Gini impurity and entropy;
- explain information gain and regression split criteria;
- fit classification and regression trees with scikit-learn;
- explain why unrestricted trees overfit;
- control tree complexity through depth, leaf-size, and pruning parameters;
- explain cost-complexity pruning;
- describe bootstrap aggregation (bagging);
- explain how Random Forests decorrelate trees through feature subsampling;
- interpret out-of-bag evaluation conceptually;
- distinguish impurity-based from permutation feature importance;
- explain why feature importance does not imply causal importance;
- use partial dependence cautiously as a model-behavior tool;
- compare single trees and forests using out-of-sample evidence;
- connect tree models to health, education, and benchmark research problems.
- interpret pixel-level feature importance cautiously in a computer-vision example.

8.2 Why This Matters

Linear and logistic regression represent outcomes through weighted combinations of features. Decision trees take a different approach: they repeatedly divide the feature space into regions.

A tree asks a sequence of questions:

$$\text{Is } x_j \leq s?$$

Each answer routes an observation toward another question or a terminal prediction.

This makes trees intuitive, nonlinear, and capable of discovering interactions without requiring us to specify every interaction in advance.

But that flexibility comes with a cost: a deep tree can memorize noise.

Central Question

How can recursive partitioning discover nonlinear structure while avoiding the temptation to memorize the training data?

8.3 From Linear Boundaries to Partitions

A logistic-regression classifier may separate classes using a linear boundary in the represented feature space.

A decision tree instead partitions the space into rectangles or regions.

For a split on feature x_j at threshold s ,

$$R_1(j, s) = \{x : x_j \leq s\},$$

$$R_2(j, s) = \{x : x_j > s\}.$$

The algorithm searches for a feature and threshold that improve the purity or predictive quality of the resulting regions.

8.4 Anatomy of a Tree

The modern classification-and-regression-tree framework is classically associated with CART (Breiman et al. 1984).

A decision tree contains:

- **root node** — the first split;
- **internal nodes** — subsequent decision rules;
- **branches** — outcomes of those rules;
- **leaves** — terminal predictions.

A path from root to leaf can be read as a conditional rule.

For example:

$$\text{if age} \leq 45 \quad \text{and} \quad \text{measure} > 12.7 \quad \Rightarrow \quad \hat{y} = 1.$$

A tree therefore represents prediction as a hierarchy of conditional partitions.

8.5 Classification Impurity

A classification tree needs a way to measure how mixed the classes are within a node.

8.5.1 Gini Impurity

For K classes,

$$Gini = 1 - \sum_{k=1}^K p_k^2,$$

where p_k is the proportion of observations in class k .

For a binary node with 50% positive and 50% negative,

$$Gini = 1 - (0.5^2 + 0.5^2) = 0.5.$$

A pure node has Gini impurity 0.

8.5.2 Entropy

Entropy is

$$H = - \sum_{k=1}^K p_k \log_2(p_k).$$

A pure node has entropy 0. Greater class mixture produces greater entropy.

8.6 Split Improvement

A candidate split can be evaluated through the weighted impurity after splitting:

$$I_{\text{split}} = \frac{n_L}{n} I_L + \frac{n_R}{n} I_R.$$

The impurity reduction is

$$\Delta I = I_{\text{parent}} - I_{\text{split}}.$$

The algorithm seeks splits that produce useful reductions.

Greedy Does Not Mean Globally Optimal

Standard tree construction is greedy: at each node, it chooses a locally favorable split. It does not generally search every possible complete tree and prove that the final tree is globally optimal.

8.7 Calculate Impurity

```
import numpy as np

def gini(proportions):
    p=np.asarray(proportions)
    return 1-np.sum(p**2)

def entropy(proportions):
    p=np.asarray(proportions)
    p=p[p>0]
    return -np.sum(p*np.log2(p))

gini([0.5,0.5]), entropy([0.5,0.5])
```

```
(np.float64(0.5), np.float64(1.0))
```

```
gini([0.9,0.1]), entropy([0.9,0.1])
```

```
(np.float64(0.17999999999999994), np.float64(0.4689955935892812))
```

Both measures recognize that a 90/10 node is purer than a 50/50 node.

8.8 Fit a Classification Tree

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

data=load_breast_cancer(as_frame=True)
X=data.data
y=(data.target==0).astype(int) # malignant = 1

X_train,X_test,y_train,y_test=train_test_split(
    X,y,test_size=.25,random_state=42,stratify=y
)

tree=DecisionTreeClassifier(
    max_depth=3,
    random_state=42
)
tree.fit(X_train,y_train)

tree.score(X_train,y_train), tree.score(X_test,y_test)
```

```
(0.9671361502347418, 0.8881118881118881)
```

Unlike logistic regression, ordinary decision trees generally do not require standardization because splits depend on ordering and thresholds rather than Euclidean scale.

8.9 Visualize the Tree

```
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

plt.figure(figsize=(16,8))
plot_tree(
    tree,
```

```

feature_names=X.columns,
class_names=["Benign", "Malignant"],
filled=False,
rounded=True,
fontsize=8
)
plt.title("A Shallow Classification Tree")
plt.show()

```

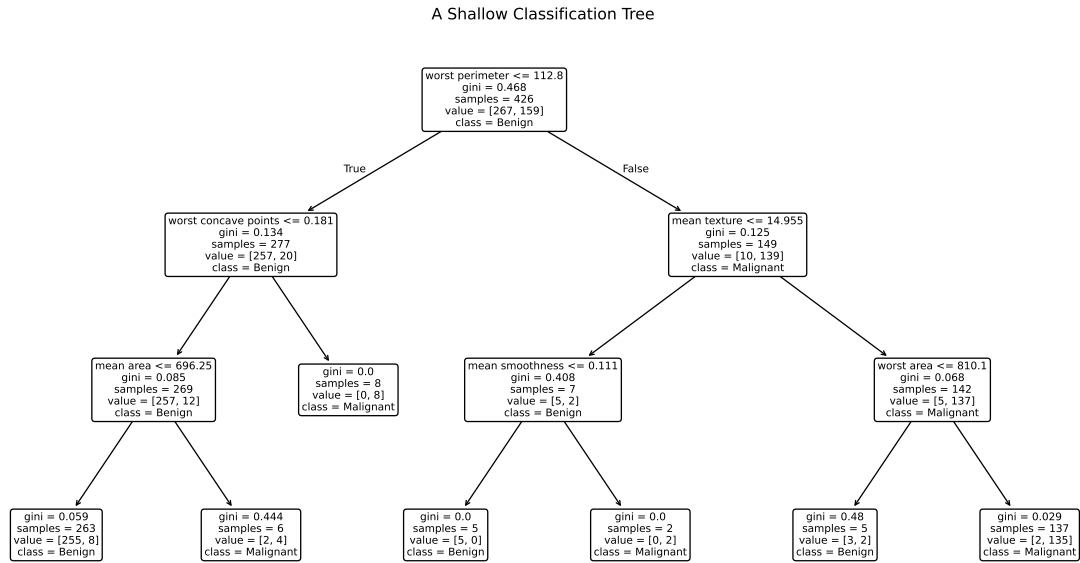


Figure 8.1: A shallow decision tree exposes the hierarchy of learned splits.

The tree is readable partly because we constrained its depth. Interpretability can disappear quickly as trees grow.

8.10 Regression Trees

Trees also predict continuous outcomes.

Within a terminal region R_m , a regression tree commonly predicts the mean:

$$\hat{y}_{R_m} = \frac{1}{|R_m|} \sum_{i: x_i \in R_m} y_i.$$

Splits can be chosen to reduce squared error:

$$\sum_{i:x_i \in R_1} (y_i - \bar{y}_{R_1})^2 + \sum_{i:x_i \in R_2} (y_i - \bar{y}_{R_2})^2.$$

The resulting function is piecewise constant.

8.11 Why Trees Overfit

An unrestricted tree can keep splitting until leaves contain very few observations.

Training error may fall dramatically:

$$\text{complexity} \uparrow \Rightarrow \text{training error} \downarrow,$$

but test error need not continue falling.

```
from sklearn.metrics import accuracy_score

depths=range(1,16)
train_scores=[]
test_scores=[]

for depth in depths:
    m=DecisionTreeClassifier(max_depth=depth,random_state=42)
    m.fit(X_train,y_train)
    train_scores.append(accuracy_score(y_train,m.predict(X_train)))
    test_scores.append(accuracy_score(y_test,m.predict(X_test)))

plt.figure(figsize=(8,5))
plt.plot(depths,train_scores,marker="o",label="Training accuracy")
plt.plot(depths,test_scores,marker="o",label="Test accuracy")
plt.xlabel("Maximum tree depth")
plt.ylabel("Accuracy")
plt.title("Tree Complexity and Generalization")
plt.legend()
plt.show()
```

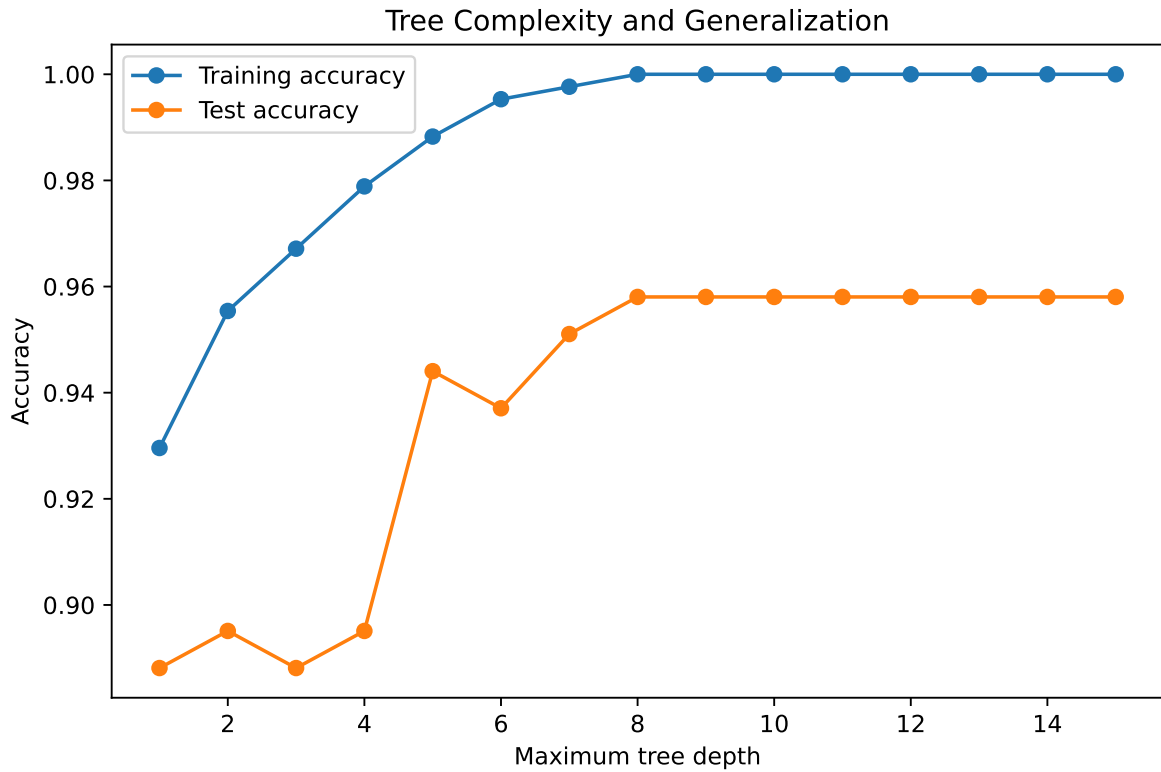


Figure 8.2: Increasing tree depth can reduce training error while eventually harming generalization.

This is a concrete bias–variance lesson: flexibility can reduce bias while increasing variance.

8.12 Controlling Tree Complexity

Important parameters include:

- `max_depth`;
- `min_samples_split`;
- `min_samples_leaf`;
- `max_leaf_nodes`;
- `ccp_alpha`.

These are not merely tuning knobs. They define how much structure the tree is allowed to learn.

8.13 Cost-Complexity Pruning

A large tree can be pruned by balancing fit against complexity.

A conceptual objective is

$$R_\alpha(T) = R(T) + \alpha|T|,$$

where $R(T)$ measures tree error, $|T|$ represents the number of terminal leaves, and α penalizes complexity.

Larger α favors smaller trees.

```
path=DecisionTreeClassifier(random_state=42).cost_complexity_pruning_path(  
    X_train,y_train  
)  
  
path.ccp_alphas[:5]
```

```
array([0.          , 0.00231856, 0.00233016, 0.00438185, 0.00459228])
```

Hyperparameter selection should occur within resampling rather than by repeatedly checking the final test set. Chapter 16 will formalize that workflow.

8.14 Instability: A Tree's Hidden Weakness

A small change in training observations can sometimes produce a substantially different tree.

This happens because early split choices affect every split below them.

Single trees can therefore have relatively high variance.

This weakness motivates ensemble methods.

8.15 Bagging: Many Unstable Models Can Become Stable Together

Bagging reduces variance by aggregating predictors fitted to bootstrap samples, an approach introduced by Breiman (Breiman 1996).

Bootstrap aggregation, or **bagging**, repeatedly samples training observations with replacement, fits a model to each bootstrap sample, and aggregates predictions.

For regression,

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{(b)}(x).$$

For classification, aggregation may use majority vote or averaged class probabilities.

The central idea is that averaging can reduce variance.

8.16 Interpretable Does Not Mean Stable

Decision trees are readable because their split structure can be visualized. But readability and stability are different properties.

Readable Model $\neq$ Stable Explanation.

Small sample changes can alter the root feature, threshold, and every downstream branch.

8.16.1 Bootstrap the Root Split

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier

rng_ts = np.random.default_rng(42)
root_features_ts = []

for _ in range(250):
    idx_ts = rng_ts.choice(len(X_train), size=len(X_train), replace=True)

    X_boot_ts = X_train.iloc[idx_ts] if hasattr(X_train, "iloc") else X_train[idx_ts]
```

```

y_boot_ts = y_train.iloc[idx_ts] if hasattr(y_train, "iloc") else np.asarray(y_train)[idx_ts]

tree_ts = DecisionTreeClassifier(
    max_depth=3,
    min_samples_leaf=8,
    random_state=42
).fit(X_boot_ts, y_boot_ts)

root_idx_ts = tree_ts.tree_.feature[0]

if hasattr(X_train, "columns"):
    root_name_ts = X_train.columns[root_idx_ts]
else:
    root_name_ts = f"feature_{root_idx_ts}"

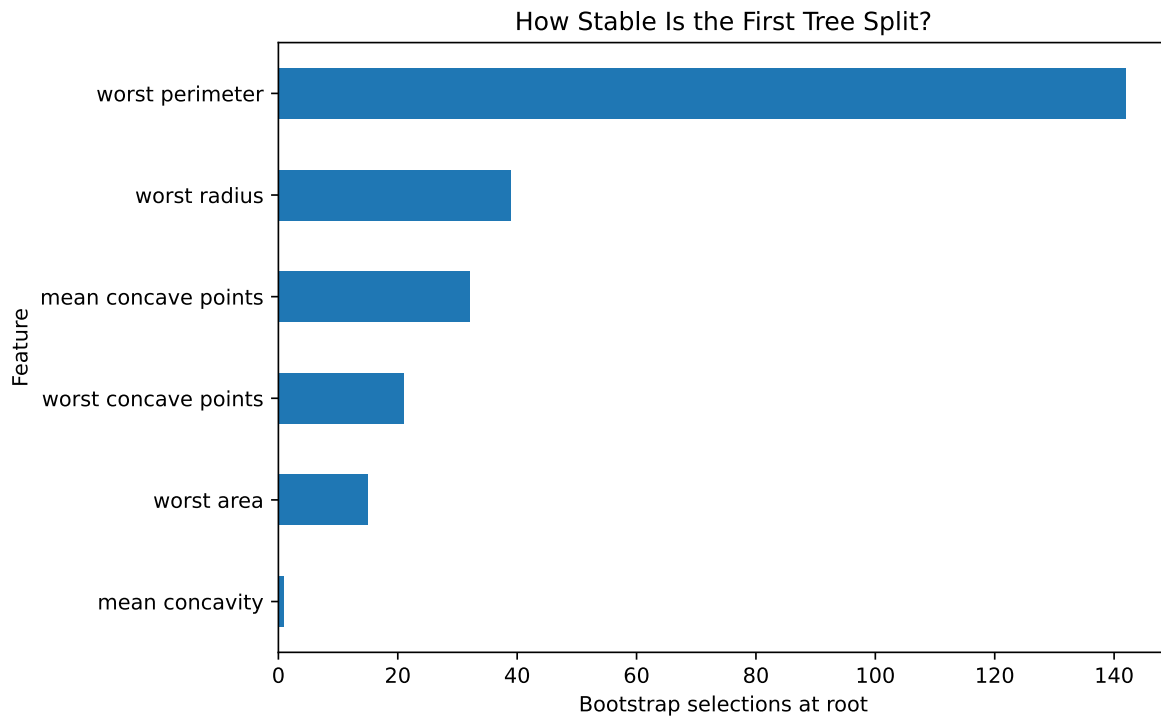
root_features_ts.append(root_name_ts)

root_counts_ts = pd.Series(root_features_ts).value_counts().head(10)

plt.figure(figsize=(8, 5))
root_counts_ts.sort_values().plot(kind="barh")
plt.xlabel("Bootstrap selections at root")
plt.ylabel("Feature")
plt.title("How Stable Is the First Tree Split?")
plt.tight_layout()
plt.show()

root_counts_ts

```



(a) Bootstrap resampling reveals whether a tree repeatedly selects the same feature at the root.

```
worst perimeter      142
worst radius         39
mean concave points  32
worst concave points 21
worst area           15
mean concavity        1
Name: count, dtype: int64
```

(b)

Figure 8.3

If several features frequently compete for the root, substantive interpretation should be cautious.

Random forests reduce variance by aggregating many trees, producing a tradeoff:

single-tree interpretability $\leftrightarrow$ ensemble stability and predictive strength.

Research Reality Check

A tree diagram is visually persuasive, but visual clarity is not evidence that the structure is unique, stable, causal, or reproducible.

An explanation should be evaluated, not merely displayed.

8.17 Random Forests

Random Forests extend bagging by adding **feature randomness**, reducing correlation among trees while retaining the variance-reduction benefits of aggregation (Breiman 2001).

At each candidate split, a tree considers only a random subset of predictors rather than all predictors.

Why?

If one predictor is overwhelmingly strong, ordinary bagged trees may repeatedly choose similar early splits. Their errors become correlated.

Random feature selection encourages diversity.

Conceptually,

Forest quality $\approx$ strong individual trees + low correlation among trees.

The exact tradeoff depends on the problem.

8.18 Fit a Random Forest

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score

forest=RandomForestClassifier(
    n_estimators=500,
    random_state=42,
    n_jobs=1
)

forest.fit(X_train,y_train)
```

```
forest_prob=forest.predict_proba(X_test)[: ,1]
forest_auc=roc_auc_score(y_test,forest_prob)

forest.score(X_test,y_test), forest_auc
```

```
(0.965034965034965, np.float64(0.9954926624737945))
```

Random Forest implementations and tree parameters are documented in scikit-learn’s ensemble and tree modules (scikit-learn developers 2026c, 2026a).

8.19 Why a Forest Can Generalize Better

Suppose individual tree errors have variance σ^2 and pairwise correlation ρ . A useful intuition for the variance of an average of B similarly distributed estimators is

$$Var(\bar{f}) \approx \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2.$$

As B grows, the second term shrinks. But highly correlated trees limit the benefit.

Random feature selection helps attack ρ .

8.20 Out-of-Bag Evaluation

Each bootstrap tree is trained on a sample drawn with replacement. Some training observations are omitted from that tree’s bootstrap sample.

These **out-of-bag (OOB)** observations can be predicted by trees that did not train on them.

```
forest_oob=RandomForestClassifier(
    n_estimators=500,
    oob_score=True,
    random_state=42,
    n_jobs=1
)
forest_oob.fit(X_train,y_train)
forest_oob.oob_score_
```

```
0.9553990610328639
```

OOB evaluation is useful, but it does not eliminate the need to think carefully about temporal, grouped, external, or final held-out validation.

8.21 Feature Importance: Important in What Sense?

Random Forests commonly expose impurity-based feature importance.

```
import pandas as pd

importance=pd.Series(
    forest.feature_importances_,
    index=X.columns
).sort_values(ascending=False)

importance.head(10)
```

```
worst perimeter      0.138356
worst area           0.128520
worst concave points 0.101787
mean concave points  0.094897
worst radius         0.093003
mean radius          0.060332
mean perimeter       0.050482
mean area            0.049681
mean concavity       0.044149
area error           0.035416
dtype: float64
```

This answers something like:

Which features contributed strongly to impurity reduction across this fitted forest?

It does **not** automatically answer:

- Which variable causes the outcome?
- Which variable should we intervene on?
- Which variable is scientifically most important?
- Which variable will remain important in another population?

8.22 Permutation Importance

Permutation importance asks how much predictive performance deteriorates when a feature's values are shuffled.

```
from sklearn.inspection import permutation_importance

perm=permutation_importance(
    forest,
    X_test,
    y_test,
    n_repeats=10,
    random_state=42,
    scoring="roc_auc"
)

perm_importance=pd.Series(
    perm.importances_mean,
    index=X.columns
).sort_values(ascending=False)

perm_importance.head(10)
```

```
worst area          0.009644
worst concave points 0.009193
worst perimeter      0.007243
worst radius         0.005545
mean concave points  0.004413
area error           0.002977
mean area            0.001719
worst texture        0.001363
mean texture         0.001038
worst concavity      0.000765
dtype: float64
```

This is closer to:

How much does this fitted model's predictive performance depend on this feature under this evaluation setup?

Even this is not causal importance. Correlated predictors can also complicate interpretation because information may be shared across features.

8.23 Compare Importance Measures

```
compare=pd.DataFrame({
    "Impurity importance": importance,
    "Permutation importance": perm_importance
}).dropna().head(10)

compare.sort_values("Permutation importance").plot(
    kind="barh",
    figsize=(9,6)
)
plt.xlabel("Importance")
plt.title("Two Meanings of Feature Importance")
plt.show()
```

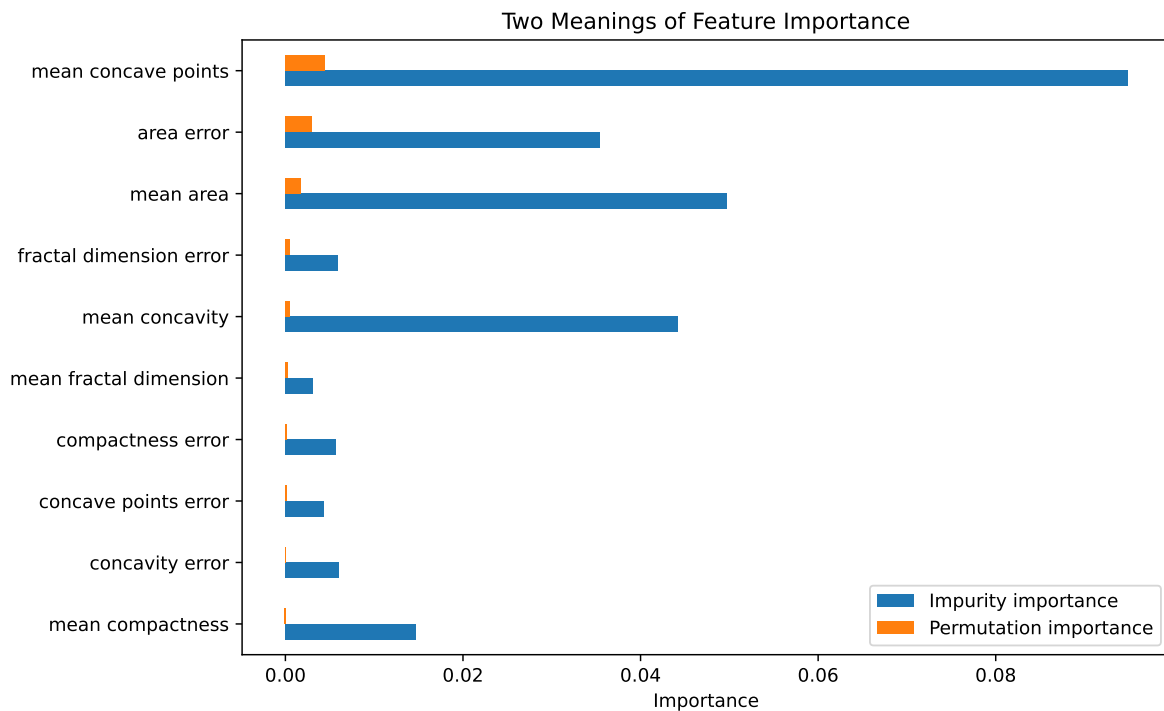


Figure 8.4: Impurity-based and permutation importance can tell different stories about model dependence.

Different importance methods answer different questions.

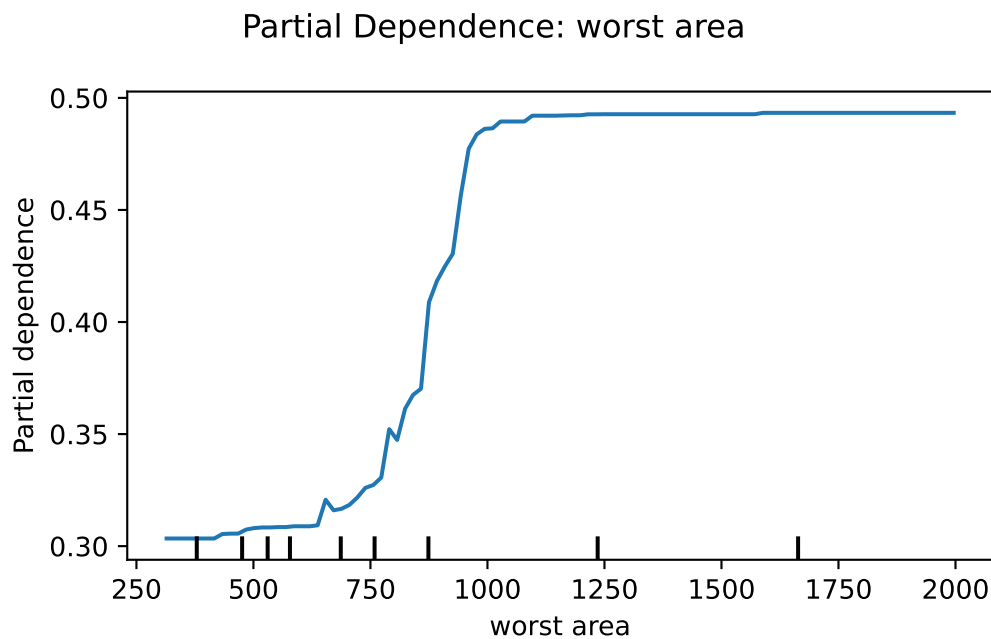
8.24 Partial Dependence: Model Behavior, Not Causal Effect

Partial dependence can summarize how model predictions change as a feature varies while averaging over the observed distribution of other features.

```
from sklearn.inspection import PartialDependenceDisplay

top_feature=perm_importance.index[0]

PartialDependenceDisplay.from_estimator(
    forest,
    X_test,
    [top_feature]
)
plt.suptitle(f"Partial Dependence: {top_feature}")
plt.tight_layout()
plt.show()
```



Interpret this as a description of the fitted model's behavior—not automatically as the effect of changing the feature in the real world.

8.25 Computer Vision Learning Experience: Pixel Importance Is Model Dependence

For flattened images, each pixel becomes a feature. Tree-based models can therefore assign feature importance to pixel locations.

Using the same handwritten-digits workload introduced in Chapter 7, a Random Forest can produce a (64)-element importance vector. Reshaping that vector to (8×8) makes the model's dependence visually inspectable.

```
import os
os.environ["LOKY_MAX_CPU_COUNT"] = "1"

from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
import matplotlib.pyplot as plt

digits_tree_cv = load_digits()

X_tree_cv = digits_tree_cv.data
y_tree_cv = digits_tree_cv.target

X_train_tree_cv, X_test_tree_cv, y_train_tree_cv, y_test_tree_cv = train_test_split(
    X_tree_cv,
    y_tree_cv,
    test_size=0.25,
    random_state=42,
    stratify=y_tree_cv
)

forest_tree_cv = RandomForestClassifier(
    n_estimators=300,
    random_state=42,
    n_jobs=1
)

forest_tree_cv.fit(
    X_train_tree_cv,
    y_train_tree_cv
)
```

```
pixel_importance_cv = forest_tree_cv.feature_importances_.reshape(
    8,
    8
)

plt.figure(figsize=(6, 5))
plt.imshow(
    pixel_importance_cv,
    cmap="viridis"
)
plt.colorbar(
    label="Impurity importance"
)
plt.title("Pixel Importance for Digit Classification")
plt.axis("off")
plt.show()
```

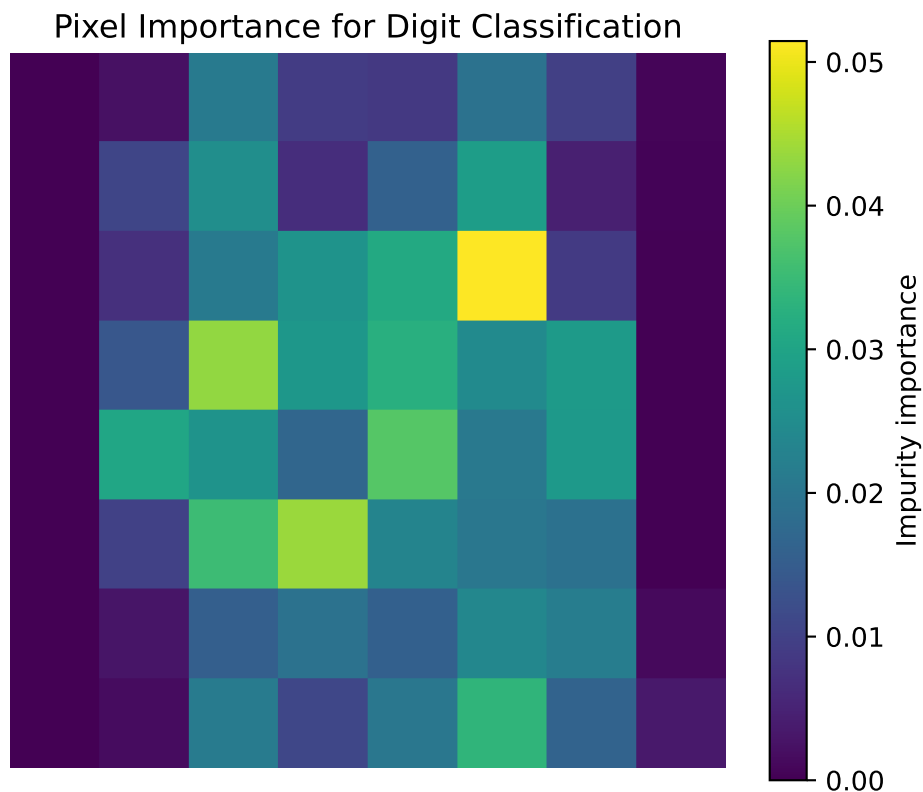


Figure 8.5: Random-Forest impurity importance reshaped into the original 8×8 image geometry.

The heatmap is useful, but it should not be interpreted as a causal map of which pixels “make” a digit what it is.

Importance is conditional on:

- this dataset;
- this representation;
- this fitted Random Forest;
- correlated pixel structure;
- the impurity-importance definition.

Visually Salient Importance $\neq$ Causal Visual Explanation

A useful extension is to compare this heatmap with permutation importance or with importance patterns from another fitted model.

8.26 Single Tree Versus Forest

Property	Single Tree	Random Forest
nonlinear relationships	yes	yes
interactions	automatic	automatic
scaling usually required	no	no
easy global visualization	often, if shallow	no
variance	often high	reduced by averaging
predictive strength	useful baseline	often stronger
individual decision path	easy to inspect	distributed across trees
feature importance	available	available, but interpret cautiously

A forest often gains predictive stability by sacrificing the simplicity of a single visible tree.

8.27 Three Tracks

8.27.1 Health Track

Tree models can discover nonlinear risk structure and interactions among measurements. But an “important” biomarker is not automatically a treatment target. Validate across relevant populations and inspect calibration and subgroup behavior when probabilities drive decisions.

8.27.2 Education Track

Trees can identify combinations of prior academic or institutional characteristics associated with retention or completion. Avoid turning predictive rules into deterministic labels about students. Prediction timing and intervention consequences remain central.

8.27.3 Open ML Benchmark Track

Benchmark datasets are ideal for comparing tree depth, pruning, instability, bagging, and Random Forests under controlled changes in noise and feature relevance.

8.28 Beyond the Algorithm: What Does “Important” Mean?

Importance Is Always Relative to a Question

Suppose a Random Forest reports that Feature A has the highest importance. It is tempting to conclude:

“Feature A is the most important determinant of the outcome.”

But importance can mean many things:

- frequently used for impurity reduction;
- strongly relied upon for prediction;
- useful because it proxies another variable;
- useful only in interactions;
- useful only in this sample;
- useful because of how the variable was measured.

None of these definitions automatically means **causal influence**.

A feature can be highly predictive precisely because it records the consequences of a process rather than its causes.

The deeper question is:

Important for what—prediction, explanation, intervention, measurement, or policy?

Machine learning becomes more intellectually rigorous when the word “importance” is never allowed to stand alone.

8.29 AI Pair Programmer: Explain My Forest

Ask an AI assistant to interpret the top five Random Forest feature importances.

Audit whether it:

- names the importance method;
- distinguishes predictive from causal importance;
- notices correlated features;
- acknowledges sample and model dependence;
- treats ranks as exact scientific truths;
- recommends interventions from predictive importance alone.

Then rewrite the interpretation.

8.30 Hands-On Lab: From One Tree to a Forest

Using a classification dataset:

1. complete the Problem Framing Canvas and Feature Ledger;
2. establish a baseline;
3. fit a shallow decision tree;
4. visualize the tree;
5. compare training and held-out performance;
6. increase tree depth and observe overfitting;
7. examine a cost-complexity pruning path;
8. fit a Random Forest;
9. compare tree and forest discrimination;
10. obtain OOB performance;
11. compare impurity and permutation importance;
12. inspect partial dependence for one feature;
13. explain why none of these results establishes causality;
14. document reproducibility decisions.

8.31 Practitioner Track

A practitioner should be able to control tree complexity, compare a single tree with a forest, use OOB information appropriately, and communicate feature importance without overclaiming.

8.32 Researcher Track

A researcher should additionally justify the validation design, examine model stability, treat importance measures as model-dependent quantities, investigate correlated predictors, and distinguish predictive discovery from explanatory or causal inference.

8.33 Research Studio

Design a health, education, or benchmark study comparing a linear/logistic model with tree-based models.

Specify:

1. substantive rationale;
2. target and timing;

3. baseline;
4. expected nonlinearities/interactions;
5. single-tree specification;
6. Random Forest specification;
7. validation strategy;
8. performance metrics;
9. stability analysis;
10. importance method;
11. interpretive limitations;
12. potential scholarly contribution.

8.34 Professor's Challenge: The Most Important Predictor

A research team fits a Random Forest and reports:

“Attendance is the most important cause of student retention because it has the highest Random Forest feature importance. Therefore, requiring more attendance will increase retention.”

The model has strong test-set AUC. Attendance is correlated with prior achievement, course engagement, and financial stability. Importance is based only on impurity reduction. No permutation analysis, uncertainty assessment, causal design, or external validation is reported.

Critique:

- the meaning of impurity importance;
- correlation and shared predictive information;
- predictive versus causal claims;
- intervention logic;
- model dependence;
- stability of the importance ranking;
- external validity;
- whether strong AUC strengthens the causal conclusion.

Design a stronger predictive analysis and explain what additional research design would be needed for a causal intervention claim.

8.35 Knowledge Check

1. What is recursive partitioning?
2. What does Gini impurity measure?
3. What does entropy measure?

4. What is impurity reduction?
5. Why is tree construction described as greedy?
6. How does a regression tree produce leaf predictions?
7. Why do deep trees overfit?
8. What does cost-complexity pruning do?
9. Why can a single tree be unstable?
10. What is bagging?
11. How does a Random Forest differ from ordinary bagging?
12. Why does reducing correlation among trees matter?
13. What are out-of-bag observations?
14. What does impurity-based importance measure?
15. What does permutation importance measure?
16. Why does feature importance not imply causation?
17. What does partial dependence describe?
18. Why might a forest outperform a single tree while being less globally interpretable?
19. Why can a small data change produce a different tree?
20. Why is an interpretable tree not automatically a stable explanation?
21. What interpretability/stability tradeoff does a random forest create?

8.36 Reproducibility Check

Record:

- problem specification;
- feature ledger;
- train/validation/test design;
- baseline;
- tree criterion;
- depth and leaf constraints;
- pruning parameters;
- forest tree count;
- feature-subsampling settings;
- bootstrap/OOB settings;
- random seeds;
- evaluation metrics;
- feature-importance method;
- permutation scoring metric and repeats;
- software versions;
- all code producing figures and tables.

8.37 Key Takeaways

- Trees learn nonlinear decision rules through recursive partitioning.
- Gini impurity and entropy quantify class mixture.
- Greedy splitting produces locally useful decisions but not guaranteed globally optimal trees.
- Deep trees can memorize training noise.
- Pruning and structural constraints manage complexity.
- Bagging reduces variance by averaging models trained on bootstrap samples.
- Random Forests add feature randomness to reduce correlation among trees.
- OOB predictions provide useful internal performance information.
- Impurity and permutation importance answer different questions.
- Feature importance is model-dependent and is not causal importance.
- Partial dependence describes fitted-model behavior, not intervention effects.
- Predictive complexity should be justified by improved generalization.
- Interpretability and stability are different properties.
- Tree structure can change under resampling.
- Ensembles can improve stability while reducing simple visual interpretability.

9 Support Vector Machines: Margins, Geometry, and Kernels

9.1 Learning Objectives

By the end of this chapter, you should be able to:

- express a linear decision boundary as a hyperplane;
- explain why the margin matters in support vector classification;
- identify support vectors and explain their special role;
- distinguish hard-margin and soft-margin classification;
- interpret slack variables and the regularization parameter C ;
- explain hinge loss conceptually and mathematically;
- explain why feature scaling is especially important for SVMs;
- fit linear and nonlinear SVM classifiers with leakage-safe pipelines;
- explain the kernel trick without explicitly constructing every transformed feature;
- distinguish linear, polynomial, and radial basis function kernels;
- interpret the role of γ in an RBF kernel;
- visualize nonlinear decision boundaries;
- compare SVMs with logistic regression and tree-based models;
- explain why probability estimates from an SVM require additional modeling;
- apply SVM reasoning to classification and regression problems;
- distinguish geometric separability from scientific explanation.
- evaluate an RBF-kernel SVM as a classical computer-vision baseline.

9.2 Why This Matters

Earlier chapters introduced two very different ways of learning.

Linear and logistic regression estimate weighted combinations of features. Decision trees recursively partition the feature space.

Support Vector Machines (SVMs) introduce a third perspective:

Find a boundary that separates observations while maximizing geometric separation from the nearest cases.

The observations closest to the boundary become especially important. These are the **support vectors**.

Even more strikingly, when a straight boundary is inadequate, kernels allow us to solve a nonlinear problem by working with similarities corresponding to a richer feature space.

Central Question

What if the difficulty of a classification problem depends not only on the algorithm, but on the geometry of the space in which we represent it?

9.3 Classification as Geometry

For two features, a linear decision boundary can be written

$$w_1x_1 + w_2x_2 + b = 0.$$

More generally,

$$w^\top x + b = 0.$$

The vector w determines the orientation of the hyperplane, while b shifts its location.

A classifier may use the sign of

$$f(x) = w^\top x + b$$

to assign classes.

For labels $y_i \in \{-1, +1\}$,

$$\hat{y}_i = \text{sign}(w^\top x_i + b).$$

9.4 Many Boundaries Can Separate the Same Data

Suppose a dataset is linearly separable. There may be many hyperplanes that classify every training observation correctly.

Which one should we choose?

SVMs prefer a boundary with a **large margin**: a wide region between the classes and the decision boundary.

For the canonical SVM scaling, the two supporting hyperplanes are

$$w^\top x + b = 1$$

and

$$w^\top x + b = -1.$$

The distance between them is

$$\frac{2}{\|w\|}.$$

Maximizing the margin is therefore equivalent to minimizing $\|w\|$.

9.5 Hard-Margin SVM

Support-vector machines formalize classification around maximum-margin separation, with the canonical support-vector network formulation developed by Cortes and Vapnik (Cortes and Vapnik 1995).

For perfectly separable data, the optimization can be written

$$\min_{w,b} \frac{1}{2} \|w\|^2$$

subject to

$$y_i(w^\top x_i + b) \geq 1 \quad \text{for all } i.$$

This demands perfect separation with the required margin.

Real datasets rarely behave so politely.

9.6 Support Vectors

The observations nearest the separating boundary determine the margin.

These observations are the **support vectors**.

Moving a distant observation slightly may not change the fitted boundary at all. Moving a support vector may change it substantially.

This is one of the most distinctive ideas in SVMs:

Not every training observation plays an equal geometric role in defining the final boundary.

9.7 Visual Experiment: Maximum-Margin Classification

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.svm import SVC

X_demo, y_demo = make_blobs(
    n_samples=70,
    centers=2,
    cluster_std=1.0,
    random_state=7
)

linear_svm = SVC(kernel="linear", C=1000)
linear_svm.fit(X_demo, y_demo)

plt.figure(figsize=(8, 6))
plt.scatter(
    X_demo[:, 0],
    X_demo[:, 1],
    c=y_demo,
    cmap="coolwarm",
    edgecolors="k"
)

ax = plt.gca()
xlim = ax.get_xlim()
```

```

ylim = ax.get_ylim()

xx = np.linspace(xlim[0], xlim[1], 200)
yy = np.linspace(ylim[0], ylim[1], 200)
YY, XX = np.meshgrid(yy, xx)
grid = np.c_[XX.ravel(), YY.ravel()]
Z = linear_svm.decision_function(grid).reshape(XX.shape)

ax.contour(
    XX, YY, Z,
    levels=[-1, 0, 1],
    linestyles=["--", "-", "--"]
)

ax.scatter(
    linear_svm.support_vectors[:, 0],
    linear_svm.support_vectors[:, 1],
    s=180,
    facecolors="none",
    edgecolors="black",
    linewidths=1.5,
    label="Support vectors"
)

plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Maximum-Margin Linear Classifier")
plt.legend()
plt.show()

```

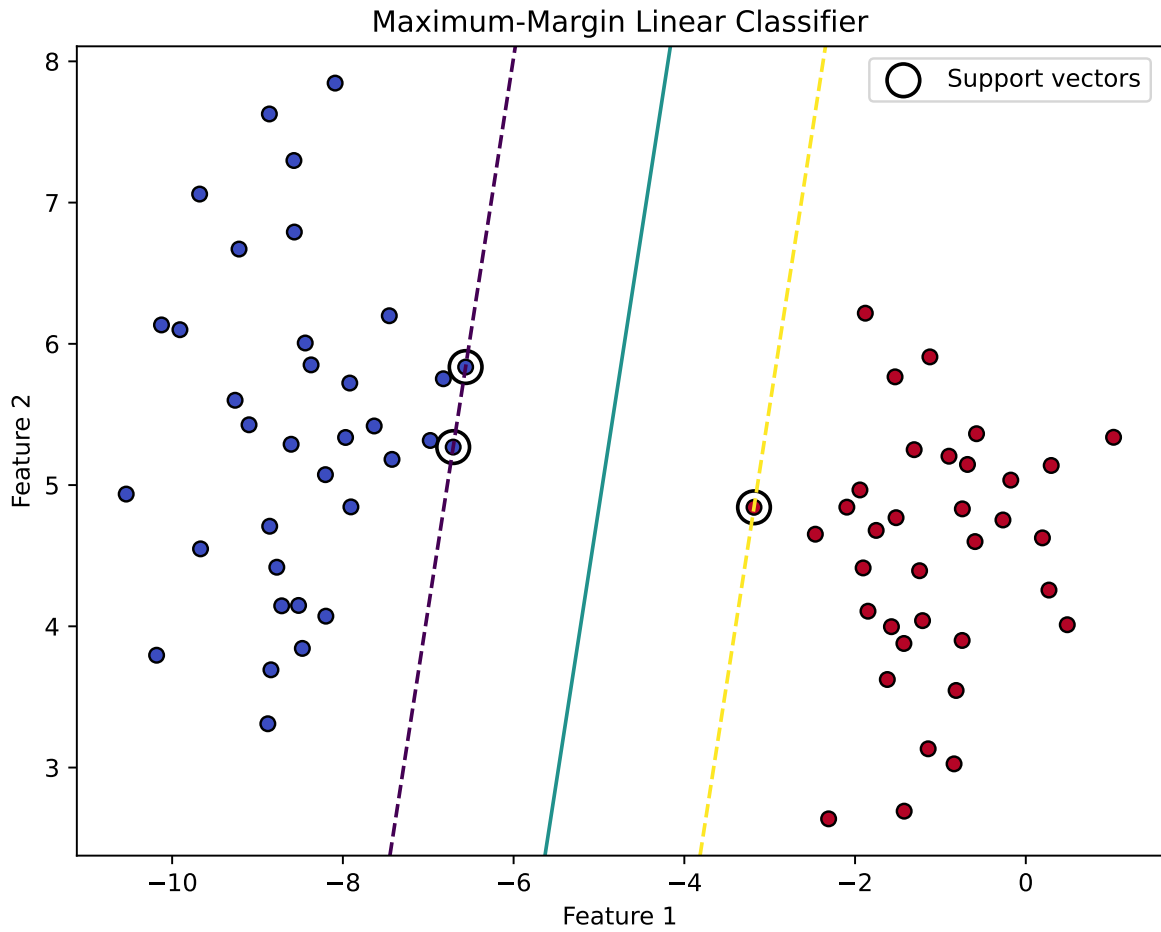


Figure 9.1: A linear SVM chooses a separating hyperplane whose margin is determined by support vectors.

9.8 Soft Margins: Allowing Imperfection

Perfect separation can be undesirable or impossible.

Soft-margin SVM introduces slack variables $\xi_i \geq 0$:

$$y_i(w^\top x_i + b) \geq 1 - \xi_i.$$

The optimization becomes

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i$$

subject to the soft-margin constraints.

The parameter C controls the penalty for margin violations.

9.8.1 Intuition for C

A **large** C penalizes violations strongly and pushes the model to fit training observations more aggressively.

A **small** C allows more violations in exchange for a wider, more regularized margin.

Thus,

$$C \uparrow \Rightarrow \text{less tolerance for violations,}$$

while

$$C \downarrow \Rightarrow \text{stronger regularization.}$$

The exact effect depends on the data and kernel.

9.9 Hinge Loss

Soft-margin classification can also be understood through hinge loss:

$$L_{\text{hinge}} = \max(0, 1 - yf(x)).$$

where

$$f(x) = w^\top x + b.$$

If an observation is correctly classified and sufficiently far beyond the margin, hinge loss is zero.

Observations inside the margin or on the wrong side incur loss.

```
margin = np.linspace(-2, 3, 400)
hinge = np.maximum(0, 1 - margin)

plt.figure(figsize=(8, 5))
plt.plot(margin, hinge)
plt.axvline(1, linestyle="--", alpha=0.6)
plt.xlabel("Signed margin: y f(x)")
plt.ylabel("Hinge loss")
plt.title("Hinge Loss")
plt.show()
```

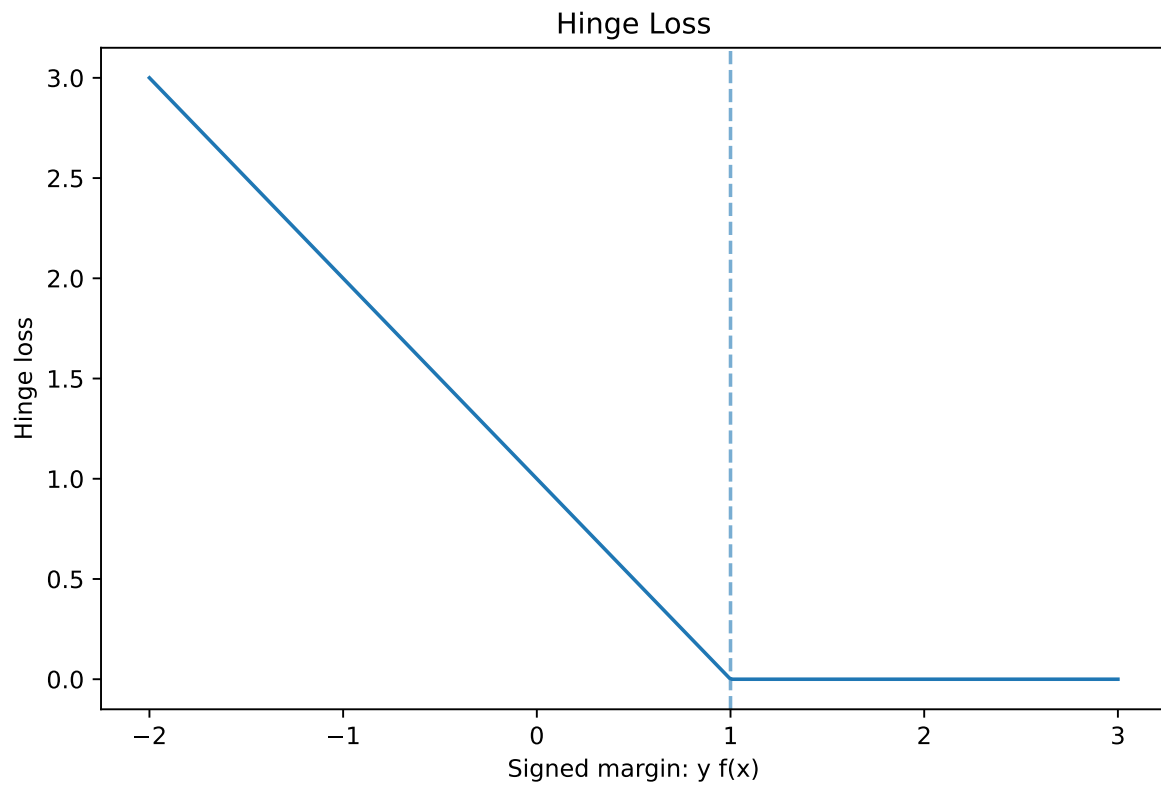


Figure 9.2: Hinge loss becomes zero once the signed margin reaches one.

9.10 Why Scaling Matters

SVM geometry depends on distances and inner products.

Suppose one feature ranges from 0 to 1 and another from 0 to 100,000. Without scaling, the larger-scale feature can dominate the geometry.

Standardization,

$$z_j = \frac{x_j - \mu_{j,\text{train}}}{\sigma_{j,\text{train}}},$$

places features on more comparable scales.

As in earlier chapters, scaling parameters must be learned from training data.

9.11 Leakage-Safe SVM Pipeline

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import roc_auc_score

data = load_breast_cancer(as_frame=True)
X = data.data
y = (data.target == 0).astype(int) # malignant = 1

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)

svm_linear = Pipeline([
    ("scale", StandardScaler()),
    ("svc", SVC(kernel="linear", C=1.0))
])

svm_linear.fit(X_train, y_train)

decision_scores = svm_linear.decision_function(X_test)
```

```
linear_auc = roc_auc_score(y_test, decision_scores)
```

```
linear_auc
```

```
np.float64(0.9928721174004193)
```

Notice that ROC-AUC can use the decision scores. We do not need calibrated probabilities merely to evaluate ranking.

9.12 The Decision Function Is Not a Probability

An SVM naturally produces a signed decision score:

$$f(x) = w^\top x + b$$

for the linear case.

The sign indicates the predicted side of the boundary, while magnitude relates to geometric position relative to the boundary under the model.

But

$$f(x) = 2.1$$

does **not** mean a 210% probability or a 0.21 probability.

If probability estimates are requested from SVC, additional probability estimation is performed and adds computational cost (scikit-learn developers 2026f).

Score	Probability
-------	-------------

Do not interpret `decision_function()` output as a probability.

When calibrated probabilities are required for decisions, evaluate their calibration explicitly.

9.13 When a Straight Boundary Is Not Enough

Consider a dataset in which one class forms an inner region and another surrounds it. No straight line can separate the classes well in the original two-dimensional space.

```
from sklearn.datasets import make_circles

X_circle, y_circle = make_circles(
    n_samples=400,
    factor=0.35,
    noise=0.08,
    random_state=42
)

plt.figure(figsize=(7, 6))
plt.scatter(
    X_circle[:, 0],
    X_circle[:, 1],
    c=y_circle,
    cmap="coolwarm",
    edgecolors="k",
    alpha=0.8
)
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("A Nonlinear Classification Problem")
plt.show()
```

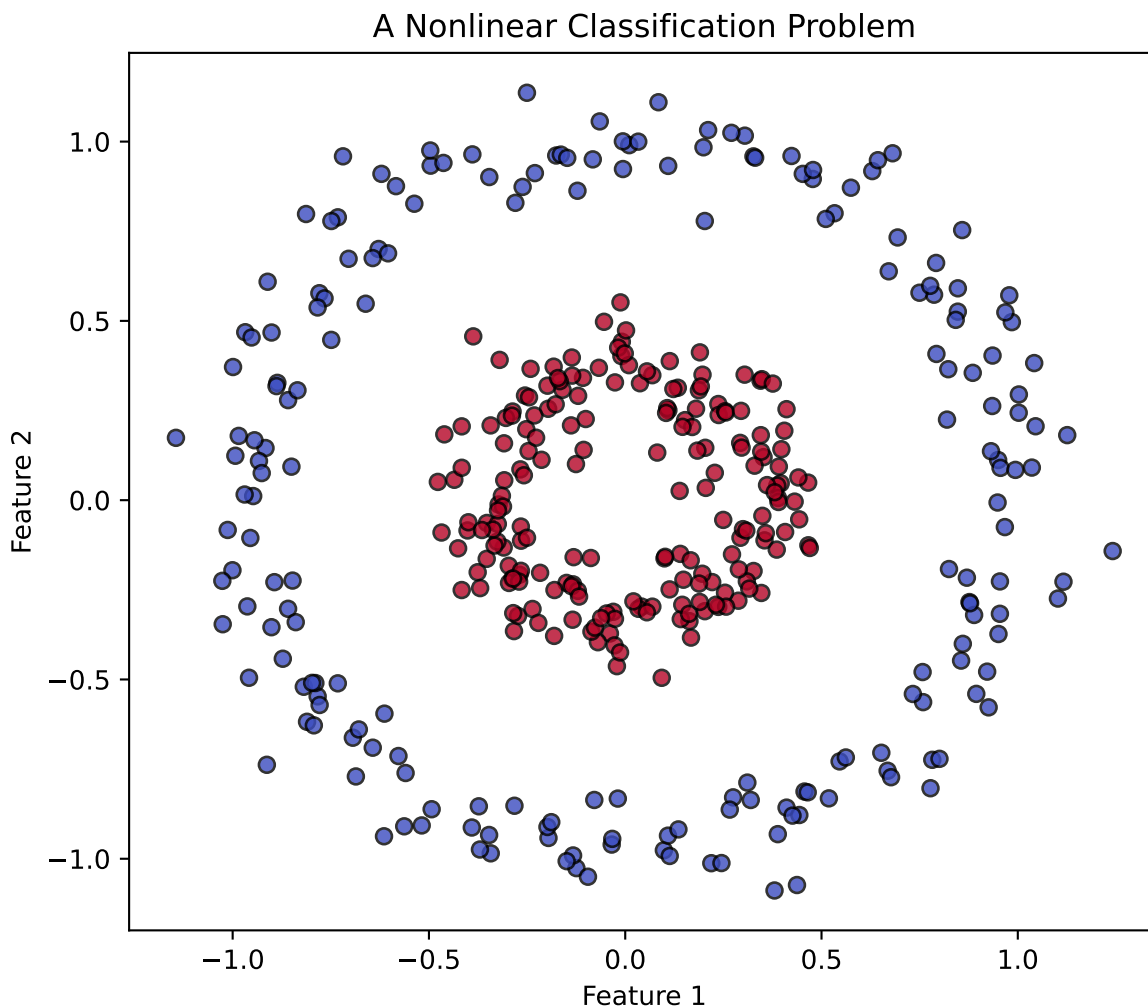


Figure 9.3: Some classification problems are not linearly separable in their original representation.

One possibility is to construct nonlinear features explicitly.

For example,

$$z = x_1^2 + x_2^2.$$

In the transformed representation, the classes may become much easier to separate.

This leads to one of the central ideas in machine learning: **representation can change difficulty**.

9.14 Feature Maps

Suppose we map an observation through

$$\phi(x).$$

A linear classifier in the transformed feature space has the form

$$f(x) = w^\top \phi(x) + b.$$

The transformed space may be much higher-dimensional than the original one.

Computing $\phi(x)$ explicitly can be expensive.

9.15 The Kernel Trick

SVM optimization depends heavily on inner products between observations.

A kernel function computes

$$K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$$

without requiring us to explicitly construct every transformed coordinate.

This is the **kernel trick**, a central idea in kernel methods that allows algorithms to operate through inner products in an implicit feature space (Schölkopf and Smola 2002).

It allows a linear separator in a transformed feature space to correspond to a nonlinear boundary in the original feature space.

The Important Idea

The kernel does not magically make the data “nonlinear.”

It changes the similarity representation used by the algorithm so that a linear solution in the induced feature space can represent nonlinear structure in the original space.

9.16 Representation Can Change the Difficulty of the Problem

A classification problem can appear difficult partly because of the coordinates in which it is expressed.

A transformation

$$x \rightarrow \phi(x)$$

can make structure easier to separate.

9.16.1 A Simple Feature Map

```
import numpy as np
import matplotlib.pyplot as plt

rng_rep = np.random.default_rng(42)

x_rep = np.linspace(-3, 3, 180)
y_rep = (np.abs(x_rep) > 1.4).astype(int)

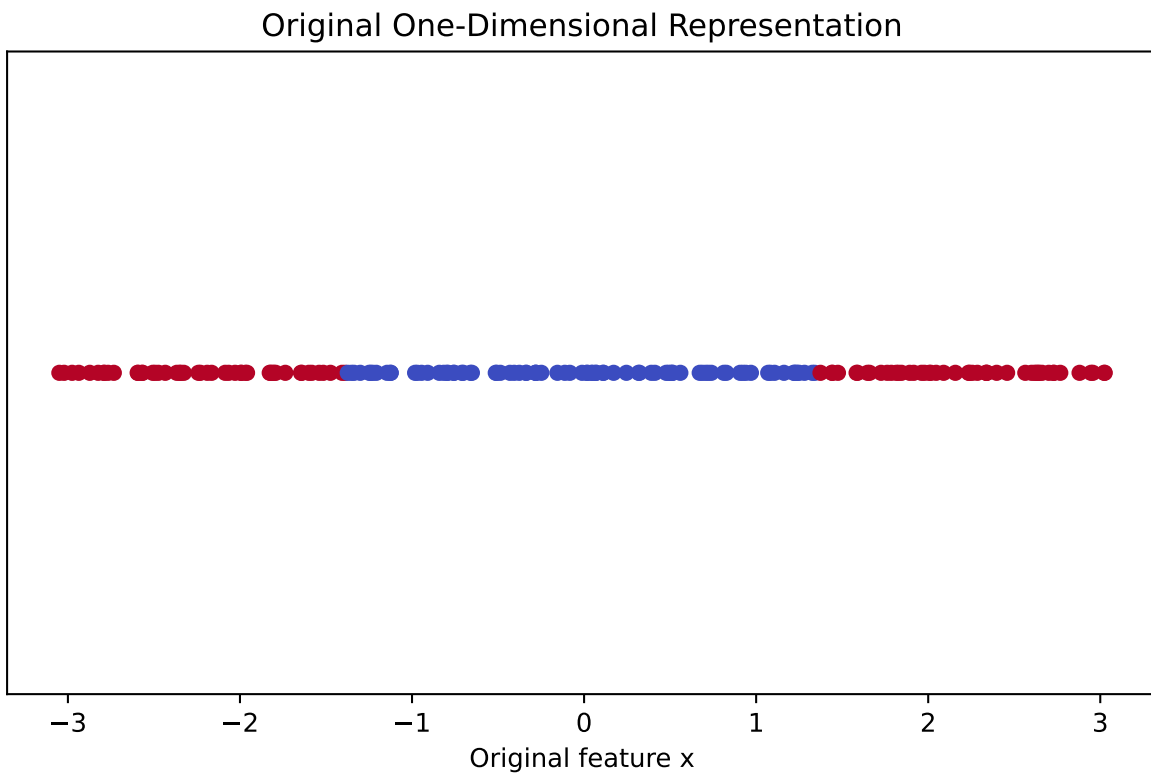
x_rep_noisy = x_rep + rng_rep.normal(scale=0.08, size=len(x_rep))

plt.figure(figsize=(8, 4.5))
plt.scatter(
    x_rep_noisy,
    np.zeros_like(x_rep_noisy),
    c=y_rep,
    cmap="coolwarm",
    s=28
)
plt.xlabel("Original feature x")
plt.yticks([])
plt.title("Original One-Dimensional Representation")
plt.show()

plt.figure(figsize=(8, 5))
plt.scatter(
    x_rep_noisy,
    x_rep_noisy**2,
    c=y_rep,
```



```
    cmap="coolwarm",  
    s=28  
)  
plt.xlabel("x")  
plt.ylabel(r"$x^2$")  
plt.title("Transformed Representation")  
plt.show()
```



(a) A nonlinear feature map can reveal a simpler geometry than the original representation.

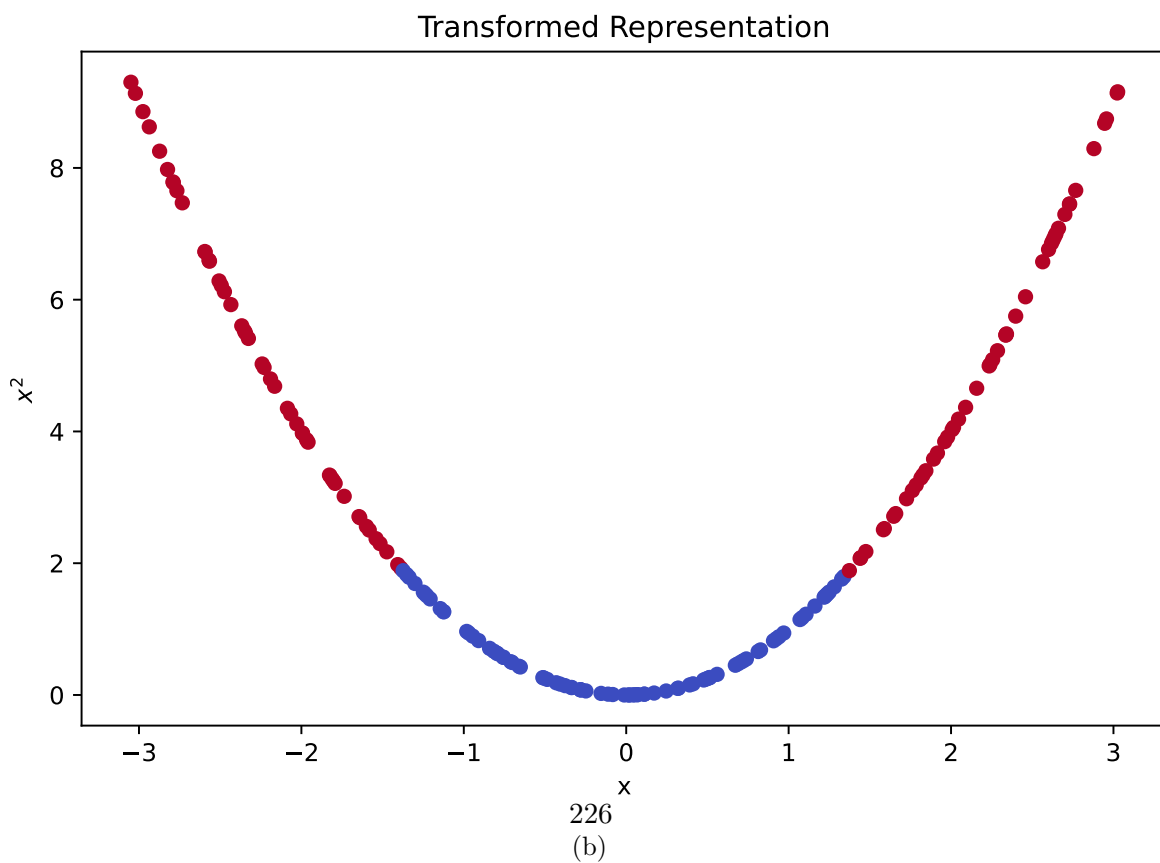


Figure 9.4

The observations did not change. The geometry available to the learner did.

9.16.2 Kernel as an Implicit Representation

A kernel computes

$$K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$$

without always constructing $\phi(x)$ explicitly.

Kernel choice is therefore not merely a tuning option. It is an assumption about similarity and geometry.

Kernel Choice = Assumption About Representation

9.16.3 Bridge to Representation Learning

Kernel methods motivate the broader progression

feature engineering $\rightarrow$ kernel-induced representation $\rightarrow$ learned representation.

Neural networks later learn intermediate representations:

$$x \rightarrow h^{(1)} \rightarrow h^{(2)} \rightarrow \dots \rightarrow h^{(L)}.$$

Beyond the Algorithm: Is Complexity in the World or in the Representation?

A boundary can look complicated in one coordinate system and simple in another.

A model never sees reality directly. It sees a representation.

Part of machine learning is therefore deciding **what geometry the model should be allowed to perceive**.

Research Reality Check

If one kernel strongly outperforms another, do not conclude only that it is “better.” Ask what the result suggests about the geometry and representation of the problem, and whether that advantage persists under resampling and external validation.

9.17 Common Kernels

9.17.1 Linear Kernel

$$K(x, z) = x^\top z.$$

Useful when a linear boundary is appropriate, especially in high-dimensional settings.

9.17.2 Polynomial Kernel

$$K(x, z) = (\gamma x^\top z + r)^d.$$

The degree d controls polynomial complexity.

9.17.3 Radial Basis Function Kernel

The RBF kernel is

$$K(x, z) = \exp(-\gamma \|x - z\|^2).$$

Nearby observations have similarity closer to 1; distant observations have similarity closer to 0.

9.18 Understanding γ

For the RBF kernel, γ controls how rapidly similarity decays with distance.

A **small** γ gives each observation a broad region of influence and often produces smoother boundaries.

A **large** γ gives each observation a narrow region of influence and can create highly flexible boundaries.

Conceptually,

$$\gamma \uparrow \Rightarrow \text{more local influence,}$$

while

$$\gamma \downarrow \Rightarrow \text{broader influence.}$$

The interaction between C and γ is crucial.

9.19 Visual Experiment: The Kernel Changes the Boundary

```
from sklearn.pipeline import make_pipeline

linear_circle = make_pipeline(
    StandardScaler(),
    SVC(kernel="linear", C=1.0)
)

rbf_circle = make_pipeline(
    StandardScaler(),
    SVC(kernel="rbf", C=1.0, gamma="scale")
)

linear_circle.fit(X_circle, y_circle)
rbf_circle.fit(X_circle, y_circle)

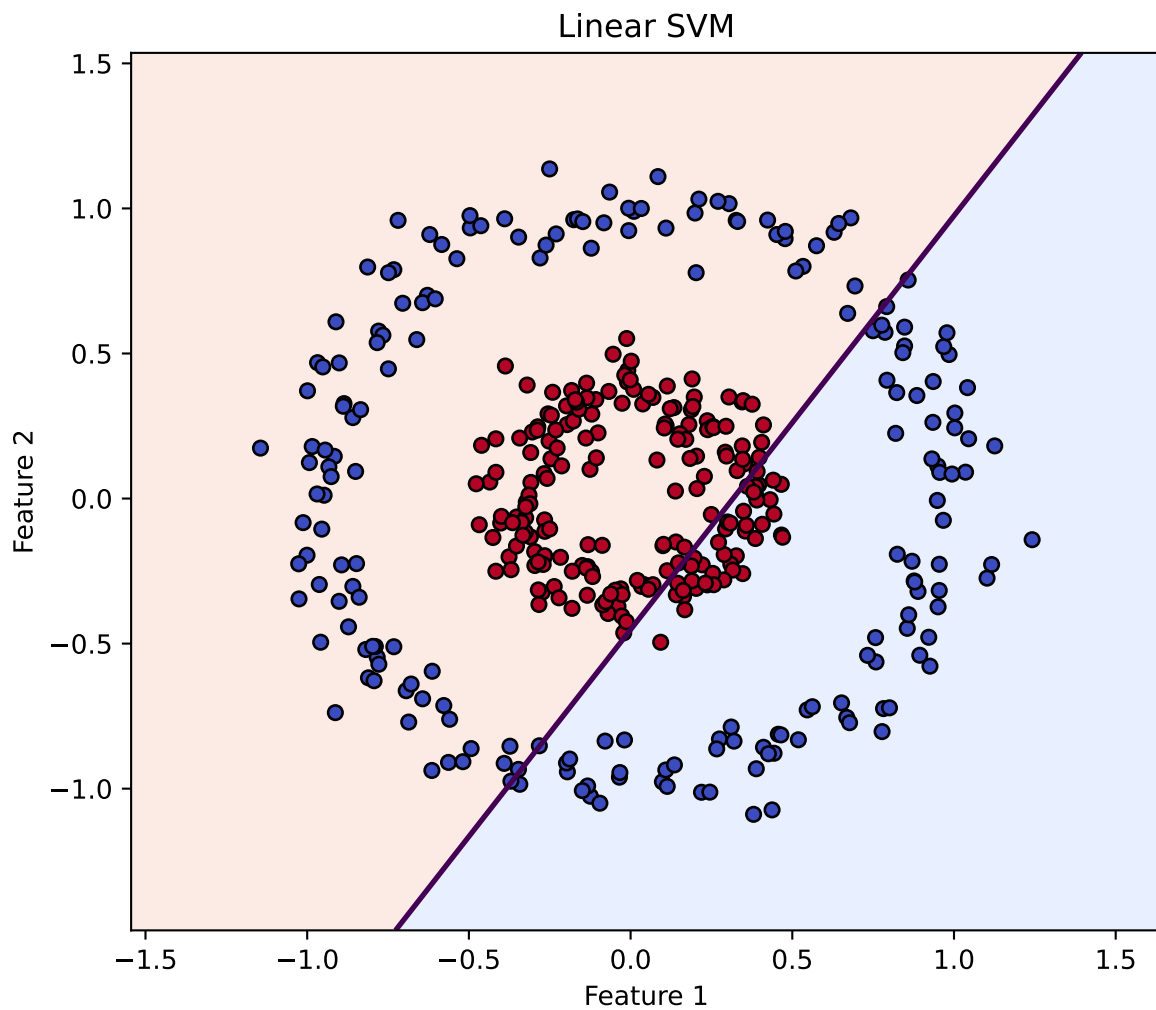
x1_min, x1_max = X_circle[:, 0].min() - 0.4, X_circle[:, 0].max() + 0.4
x2_min, x2_max = X_circle[:, 1].min() - 0.4, X_circle[:, 1].max() + 0.4

xx, yy = np.meshgrid(
    np.linspace(x1_min, x1_max, 350),
    np.linspace(x2_min, x2_max, 350)
)
grid = np.c_[xx.ravel(), yy.ravel()]

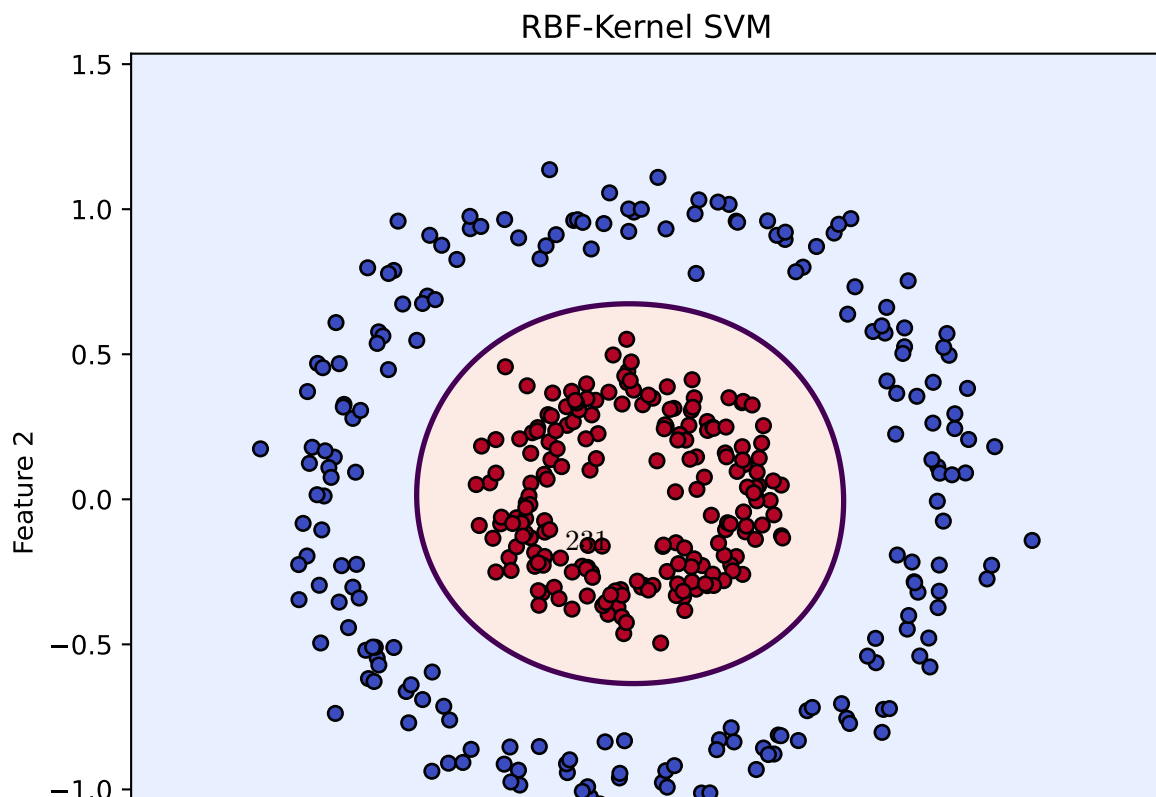
for fitted_model, title in [
    (linear_circle, "Linear SVM"),
    (rbf_circle, "RBF-Kernel SVM")
]:
    zz = fitted_model.decision_function(grid).reshape(xx.shape)

    plt.figure(figsize=(7, 6))
    plt.contourf(xx, yy, zz > 0, alpha=0.2, cmap="coolwarm")
    plt.contour(xx, yy, zz, levels=[0], linewidths=2)
    plt.scatter(
        X_circle[:, 0],
        X_circle[:, 1],
```

```
        c=y_circle,  
        cmap="coolwarm",  
        edgecolors="k",  
        s=30  
    )  
    plt.xlabel("Feature 1")  
    plt.ylabel("Feature 2")  
    plt.title(title)  
    plt.show()
```



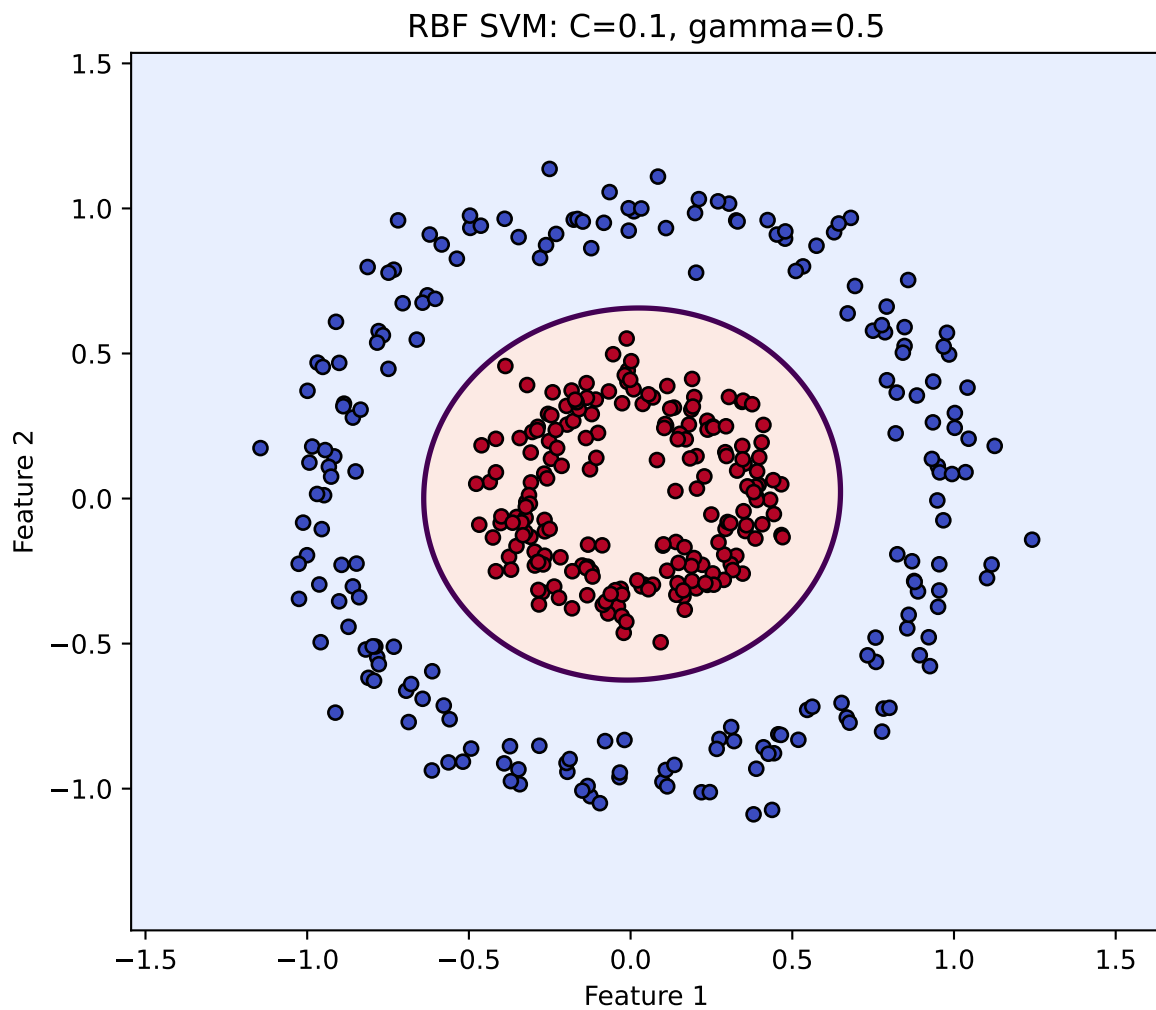
(a) A linear kernel cannot represent the circular structure, while an RBF kernel can create a nonlinear decision boundary.



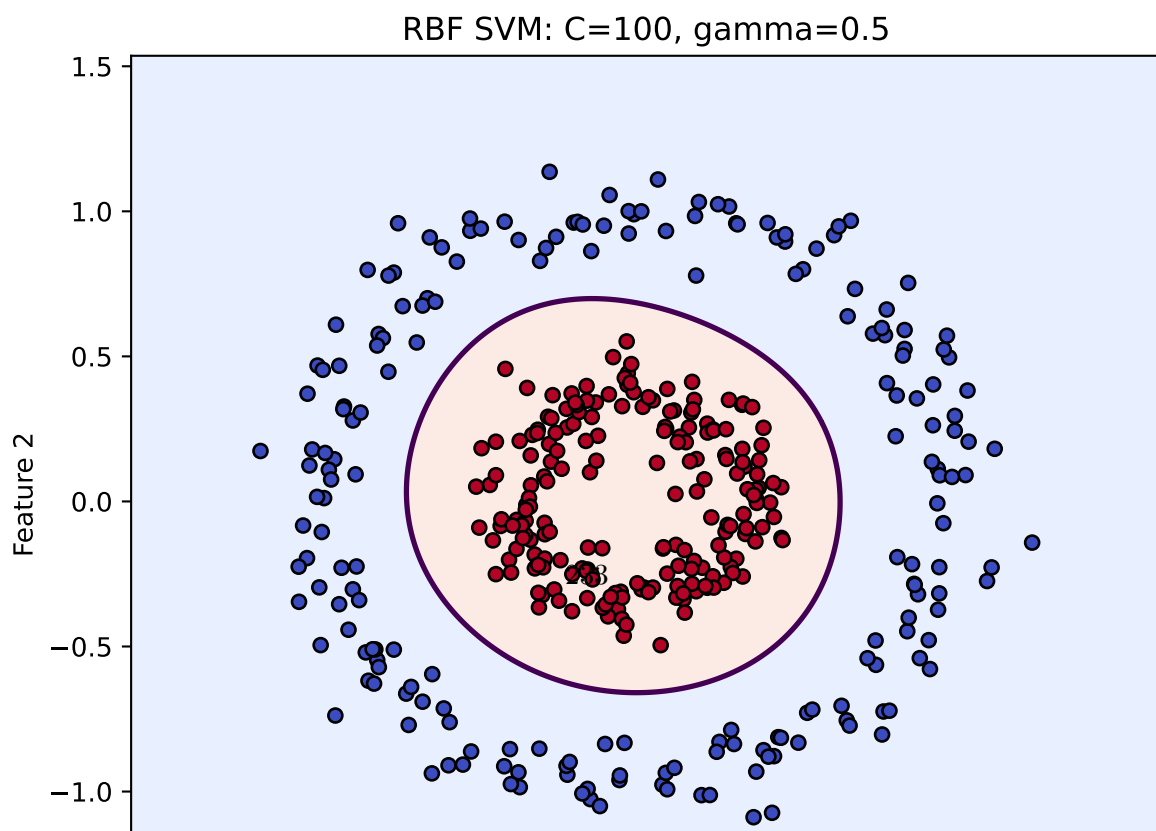
The RBF model does not prove that the underlying scientific mechanism is radial. It demonstrates that this similarity representation can fit the observed decision structure.

9.20 Visual Experiment: C and γ

```
settings = [  
    (0.1, 0.5),  
    (100, 0.5),  
    (1, 0.05),  
    (1, 20)  
]  
  
for C_value, gamma_value in settings:  
    m = make_pipeline(  
        StandardScaler(),  
        SVC(kernel="rbf", C=C_value, gamma=gamma_value)  
    )  
    m.fit(X_circle, y_circle)  
    zz = m.decision_function(grid).reshape(xx.shape)  
  
    plt.figure(figsize=(7, 6))  
    plt.contourf(xx, yy, zz > 0, alpha=0.2, cmap="coolwarm")  
    plt.contour(xx, yy, zz, levels=[0], linewidths=2)  
    plt.scatter(  
        X_circle[:, 0],  
        X_circle[:, 1],  
        c=y_circle,  
        cmap="coolwarm",  
        edgecolors="k",  
        s=28  
    )  
    plt.xlabel("Feature 1")  
    plt.ylabel("Feature 2")  
    plt.title(f"RBF SVM: C={C_value}, gamma={gamma_value}")  
    plt.show()
```

(a) C and γ jointly control the flexibility of an RBF SVM.



Do not choose these values by looking at final test performance repeatedly. Hyperparameter selection belongs inside the resampling workflow developed fully in Chapter 16.

9.21 Compare Logistic Regression, Linear SVM, and RBF SVM

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score

models = {
    "Logistic regression": Pipeline([
        ("scale", StandardScaler()),
        ("model", LogisticRegression(max_iter=2000))
    ]),
    "Linear SVM": Pipeline([
        ("scale", StandardScaler()),
        ("model", SVC(kernel="linear", C=1.0))
    ]),
    "RBF SVM": Pipeline([
        ("scale", StandardScaler()),
        ("model", SVC(kernel="rbf", C=1.0, gamma="scale"))
    ])
}

comparison = {}

for name, m in models.items():
    m.fit(X_train, y_train)

    if hasattr(m[-1], "predict_proba"):
        score = m.predict_proba(X_test)[:, 1]
    else:
        score = m.decision_function(X_test)

    comparison[name] = roc_auc_score(y_test, score)

comparison

{'Logistic regression': np.float64(0.9962264150943396),
 'Linear SVM': np.float64(0.9928721174004193),
 'RBF SVM': np.float64(0.9966457023060796)}
```

A model family should not be selected because it sounds more sophisticated. It should earn its complexity through appropriate validation and practical usefulness.

9.22 Computer Vision Learning Experience: An RBF SVM on Handwritten Digits

Support Vector Machines have a natural connection to computer vision because image representations can be high dimensional and class boundaries can be nonlinear.

We can revisit the handwritten-digits workload with an RBF-kernel SVM.

```
import os
os.environ["LOKY_MAX_CPU_COUNT"] = "1"

from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

digits_svm_cv = load_digits()

X_svm_cv = digits_svm_cv.data
y_svm_cv = digits_svm_cv.target

X_train_svm_cv, X_test_svm_cv, y_train_svm_cv, y_test_svm_cv = train_test_split(
    X_svm_cv,
    y_svm_cv,
    test_size=0.25,
    random_state=42,
    stratify=y_svm_cv
)

vision_svm = Pipeline([
    ("scale", StandardScaler()),
    (
        "model",
        SVC(
            kernel="rbf",
            C=3.0,
```

```

        gamma="scale"
    )
)
])

vision_svm.fit(
    X_train_svm_cv,
    y_train_svm_cv
)

pred_svm_cv = vision_svm.predict(
    X_test_svm_cv
)

accuracy_score(
    y_test_svm_cv,
    pred_svm_cv
)

```

0.98

9.22.1 What Did the Kernel Add?

The input remains a flattened (64)-pixel representation. The RBF kernel changes the geometry in which similarity is evaluated.

That makes this an important comparison with Chapter 7:

- Chapter 7 used a linear logistic-regression decision structure;
- this chapter allows a nonlinear kernel boundary;
- Chapter 13 will later introduce **learned** representations.

The learning progression is therefore:

Flattened Pixels → Linear Classifier → Kernel Geometry → Learned Visual Representation.

Research Reality Check

If the RBF SVM outperforms logistic regression, the evidence supports a difference in predictive performance under this benchmark and validation design.

It does not prove that the RBF representation captures the “true” geometry of handwritten

digits.

9.23 Computational Considerations

Kernel SVMs can become computationally demanding as the number of observations grows because training depends on relationships among training cases.

For large datasets, alternatives may include:

- linear SVM formulations;
- stochastic optimization;
- approximate kernel methods;
- tree ensembles;
- neural networks;
- other scalable estimators.

The right model depends on dimensionality, sample size, sparsity, deployment requirements, and validation evidence.

9.24 SVM for Regression

Support Vector Regression (SVR) extends margin-based reasoning to continuous outcomes.

Instead of demanding that predictions match every observation exactly, SVR can use an ϵ -insensitive region.

Conceptually, errors within an ϵ -tube are not penalized:

$$L_{\epsilon}(y, f(x)) = \max(0, |y - f(x)| - \epsilon).$$

This produces a regression formulation focused on deviations outside a tolerance band.

```
from sklearn.datasets import load_diabetes
from sklearn.svm import SVR
from sklearn.metrics import mean_squared_error

diabetes = load_diabetes(as_frame=True)
Xr = diabetes.data
yr = diabetes.target
```

```

Xr_train, Xr_test, yr_train, yr_test = train_test_split(
    Xr, yr, test_size=0.25, random_state=42
)

svr = Pipeline([
    ("scale", StandardScaler()),
    ("model", SVR(kernel="rbf", C=10, epsilon=0.1))
])

svr.fit(Xr_train, yr_train)
svr_pred = svr.predict(Xr_test)

np.sqrt(mean_squared_error(yr_test, svr_pred))

```

```
np.float64(51.91511845782987)
```

The values above are demonstrations, not tuned recommendations.

9.25 Three Tracks

9.25.1 Health Track

An SVM might classify a health outcome using many correlated measurements or high-dimensional features. Scaling, calibration, external validation, subgroup performance, and clinical action remain more important than the sophistication of the kernel.

9.25.2 Education Track

An SVM may identify complex boundaries among academic, engagement, and institutional features. A nonlinear boundary can improve prediction without yielding an easily interpretable explanation of why a student experiences an outcome.

9.25.3 Open ML Benchmark Track

Benchmark datasets allow controlled experiments with class overlap, nonlinear geometry, scaling, C , γ , kernels, and support-vector counts.

This track is especially useful for understanding **algorithmic geometry**.

9.26 Beyond the Algorithm: Change the Representation, Change the Problem

A Problem Can Look Different in Another Space

Two classes may appear hopelessly intertwined in the variables we initially observe.

A transformation may reveal structure.

This raises a broader intellectual question:

When we say a problem is difficult, are we describing the phenomenon—or our current representation of it?

Machine learning repeatedly teaches us that representation matters.

A useful coordinate system can make a complex pattern simple. A poor representation can make a simple phenomenon look complicated.

But representation also changes interpretation.

A high-dimensional kernel space may improve prediction while making the learned reasoning harder to explain in substantive terms.

So the goal is not:

“Transform until the model wins.”

The deeper goal is:

Choose representations that improve useful generalization while remaining defensible for the scientific or operational purpose.

The geometry we choose influences the patterns the algorithm is capable of seeing.

9.27 AI Pair Programmer: The Kernel Made It True

Ask an AI assistant to interpret an RBF SVM that substantially outperforms logistic regression.

Give it this claim:

“Because the RBF SVM has higher test AUC, the true relationship between the predictors and outcome must be nonlinear and the RBF kernel has discovered the underlying mechanism.”

Audit the response.

A strong critique should distinguish:

- predictive performance from mechanism;

- nonlinear predictive structure from causal structure;
- kernel representation from scientific theory;
- one test sample from external validity;
- hyperparameter selection from final evaluation;
- discrimination from calibration.

9.28 Hands-On Lab: From Hyperplane to Kernel

Using a classification dataset:

1. complete the Problem Framing Canvas;
2. complete the Feature Ledger;
3. establish a baseline;
4. fit logistic regression;
5. fit a linear SVM inside a scaling pipeline;
6. compare held-out discrimination;
7. inspect the number of support vectors;
8. fit an RBF SVM;
9. examine several C and γ combinations using training/resampling data;
10. compare linear and nonlinear decision behavior;
11. evaluate whether probabilities are actually needed;
12. if probabilities are used, inspect calibration;
13. compare SVM results with the tree/forest results from Chapter 8;
14. document computational cost;
15. state what the model cannot establish scientifically.

9.29 Practitioner Track

A practitioner should be able to build scaling-safe SVM pipelines, choose between linear and kernel formulations, understand C and γ , evaluate decision scores, and recognize computational constraints.

9.30 Researcher Track

A researcher should additionally justify the representation, separate predictive geometry from scientific mechanism, validate hyperparameters without test contamination, evaluate calibration when probabilities matter, and examine generalization across relevant populations.

9.31 Research Studio

Design a study in which SVMs are plausible because of high dimensionality or nonlinear structure.

Specify:

1. research or operational problem;
2. target and prediction time;
3. baseline;
4. feature representation;
5. reason scaling matters;
6. linear SVM;
7. candidate kernel;
8. C and γ search strategy;
9. validation design;
10. discrimination metric;
11. calibration requirement;
12. computational constraints;
13. comparison model;
14. interpretive limitations;
15. potential scholarly contribution.

9.32 Professor's Challenge: The Perfect Kernel

A research team tries 40 combinations of C and γ on the test set. They select the combination with the highest test AUC and report that AUC as final performance.

They then write:

“The RBF SVM significantly outperformed logistic regression, proving that the true data-generating process is nonlinear. The most difficult observations are the support vectors, which represent the most scientifically important cases.”

Critique:

- test-set tuning;
- optimistic performance estimation;
- the meaning of “significantly”;
- nonlinear prediction versus data-generating mechanism;
- support vectors versus scientific importance;
- scaling;
- calibration;

- uncertainty;
- external validation;
- computational reproducibility.

Redesign the experiment using nested or otherwise properly separated model selection and final evaluation.

9.33 Knowledge Check

1. What is a hyperplane?
2. What is the SVM margin?
3. Why does minimizing $\|w\|$ relate to maximizing margin?
4. What are support vectors?
5. Why is hard-margin classification often unrealistic?
6. What do slack variables represent?
7. What does C control?
8. What is hinge loss?
9. Why does feature scaling matter for SVMs?
10. Why is an SVM decision score not automatically a probability?
11. What is a feature map?
12. What does a kernel compute?
13. What is the kernel trick?
14. How does the RBF kernel use distance?
15. What does γ control?
16. Why can large γ create highly flexible boundaries?
17. What is SVR's ϵ -insensitive idea?
18. Why does better RBF performance not prove the true scientific mechanism is nonlinear?
19. How can nonlinear transformation simplify a classification problem?
20. What does a kernel compute conceptually?
21. Why is kernel choice a representational assumption?
22. How do kernels bridge classical feature engineering and neural representation learning?

9.34 Reproducibility Check

Record:

- problem specification;
- feature ledger;
- preprocessing/scaling pipeline;
- split and resampling design;

- baseline;
- kernel;
- C ;
- γ where applicable;
- polynomial degree where applicable;
- class weighting if used;
- probability-estimation method if used;
- calibration assessment;
- support-vector counts;
- evaluation metrics;
- computational environment;
- random seeds where applicable;
- package versions;
- all model-selection boundaries;
- code generating every result.

9.35 Key Takeaways

- SVMs frame classification as a geometric separation problem.
- The maximum-margin principle favors boundaries separated from nearby observations.
- Support vectors play a special role in defining the boundary.
- Soft margins balance margin width against violations.
- C controls the strength of the penalty for violations.
- Hinge loss penalizes observations that fail to achieve sufficient signed margin.
- Scaling is essential because SVM geometry depends on feature representation.
- Kernels allow nonlinear decision boundaries through implicit feature-space similarities.
- γ controls the locality of RBF influence.
- Decision scores are not automatically probabilities.
- Kernel SVMs can be computationally expensive on large datasets.
- SVR extends margin-based reasoning to continuous outcomes.
- Better nonlinear prediction does not establish a nonlinear causal mechanism.
- Changing the representation can change the apparent difficulty of a problem.
- Problem difficulty depends partly on representation.
- Kernels define similarity in implicit feature spaces.
- Kernel choice expresses an assumption about geometry.
- Kernel methods provide a bridge to learned representations.

10 Ensemble Learning: From Many Models to Stronger Predictions

10.1 Learning Objectives

By the end of this chapter, you should be able to:

- explain why combining models can improve predictive performance;
- distinguish bagging, boosting, voting, and stacking;
- connect bagging to variance reduction and Random Forests;
- explain AdaBoost as sequential reweighting of difficult observations;
- derive the central intuition of gradient boosting as functional optimization;
- explain learning rate, tree depth, and number of estimators as interacting complexity controls;
- fit AdaBoost, Gradient Boosting, and HistGradientBoosting models;
- compare ensemble methods using leakage-safe evaluation;
- explain why boosting can overfit or amplify problematic signal;
- distinguish hard voting from soft voting;
- explain stacking and the need for out-of-fold meta-features;
- compare ensemble performance with simpler baselines;
- evaluate ensemble predictions using discrimination, calibration, and task-relevant metrics;
- distinguish predictive gains from scientific explanation;
- reason about diversity, correlated errors, and collective failure.
- compare simple and ensemble models on a reproducible computer-vision benchmark without treating the leaderboard as the research contribution.

10.2 Why This Matters

Chapter 8 introduced one of machine learning’s most powerful ideas: a collection of unstable trees can become a strong predictor when their errors are averaged.

Ensemble learning generalizes that idea.

Instead of asking

Which single model should win?

we can ask

Can several models contribute different useful information to one prediction system?

Sometimes the answer is yes. But combining models is not automatically beneficial. If every model learns the same bias, uses the same leaked feature, or fails on the same subgroup, aggregation can make the system confidently wrong.

Central Question

When does combining imperfect learners create a stronger predictive system—and when does collective agreement merely reinforce shared error?

10.3 The Ensemble Principle

Suppose we have M regression models,

$$\hat{f}_1(x), \hat{f}_2(x), \dots, \hat{f}_M(x).$$

A simple ensemble average is

$$\hat{f}_{\text{ens}}(x) = \frac{1}{M} \sum_{m=1}^M \hat{f}_m(x).$$

For classification, we might average class probabilities or combine class votes.

The benefit depends partly on whether the models make **different errors**.

If every model makes exactly the same mistake, averaging does not remove it.

10.4 Four Major Ensemble Strategies

10.4.1 Bagging

Fit models largely independently on resampled data, then aggregate.

Primary intuition:

variance reduction.

10.4.2 Boosting

Fit learners sequentially so later learners focus on what the current ensemble has not handled well.

Primary intuition:

sequential error correction.

10.4.3 Voting

Combine predictions from different fitted model families.

Primary intuition:

consensus across models.

10.4.4 Stacking: Learn the Combination Rule

Train a meta-model to learn how to combine base-model predictions.

Primary intuition:

learn the combination rule.

10.5 Bagging Revisited

Bagging combines predictors trained on bootstrap samples to reduce variance (Breiman 1996). Random Forests extend this principle with randomized feature selection (Breiman 2001).

For bootstrap models $\hat{f}^{(1)}, \dots, \hat{f}^{(B)}$,

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{(b)}(x).$$

Bagging is especially useful for high-variance learners such as deep decision trees.

Chapter 8 showed that Random Forests extend bagging by introducing random feature subsets to reduce correlation among trees.

The ensemble lesson is broader:

Averaging helps most when component models are individually useful but not perfectly correlated.

10.6 Correlated Errors Limit the Wisdom of the Crowd

A useful variance intuition is

$$\text{Var}(\bar{f}) \approx \rho\sigma^2 + \frac{1-\rho}{M}\sigma^2,$$

where ρ represents average error correlation.

As M increases, the second term shrinks. The first does not.

If ρ is large, adding more similar models produces diminishing returns.

10.7 Boosting: Learn From What Is Still Wrong

Boosting does not fit all learners independently.

It builds an additive model:

$$F_M(x) = \sum_{m=1}^M \alpha_m h_m(x),$$

where $h_m(x)$ is a weak learner and α_m controls its contribution.

Each stage attempts to improve the current ensemble.

10.8 AdaBoost

AdaBoost increases attention to observations that previous learners classified incorrectly, building on the adaptive boosting framework developed by Freund and Schapire (Freund and Schapire 1997).

Begin with weights

$$w_i^{(1)} = \frac{1}{n}.$$

For learner m , compute weighted error

$$\epsilon_m = \frac{\sum_{i=1}^n w_i^{(m)} I(y_i \neq h_m(x_i))}{\sum_{i=1}^n w_i^{(m)}}.$$

A common binary-classification learner weight is

$$\alpha_m = \frac{1}{2} \log \left(\frac{1 - \epsilon_m}{\epsilon_m} \right).$$

Misclassified observations receive relatively greater emphasis for the next learner.

Conceptually,

missed cases $\rightarrow$ greater attention $\rightarrow$ next weak learner.

10.9 Fit AdaBoost

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import roc_auc_score

data=load_breast_cancer(as_frame=True)
X=data.data
y=(data.target==0).astype(int) # malignant = 1

X_train,X_test,y_train,y_test=train_test_split(
    X,y,test_size=.25,random_state=42,stratify=y
)

stump=DecisionTreeClassifier(max_depth=1,random_state=42)

ada=AdaBoostClassifier(
    estimator=stump,
    n_estimators=200,
    learning_rate=.05,
    random_state=42
)
```



```
ada.fit(X_train,y_train)
ada_prob=ada.predict_proba(X_test)[:,-1]
roc_auc_score(y_test,ada_prob)
```

```
np.float64(0.9943396226415094)
```

Weak learners can combine into a substantially more expressive decision function.

10.10 Why “Focus on Mistakes” Can Be Dangerous

Not every difficult observation represents useful structure.

A difficult case may be:

- mislabeled;
- corrupted;
- an extreme outlier;
- from a rare subgroup;
- structurally different from the training population.

Boosting can devote increasing attention to observations that should instead trigger data-quality or distributional investigation.

Hard Cases Are Not Automatically Informative Cases

An algorithm sees loss. A researcher must still ask **why** the loss is large.

10.11 Gradient Boosting: Optimization in Function Space

Gradient boosting provides a broader view, interpreting boosting as stage-wise optimization in function space (Friedman 2001).

We seek a predictive function $F(x)$ minimizing a loss:

$$\mathcal{L} = \sum_{i=1}^n L(y_i, F(x_i)).$$

Start with an initial model $F_0(x)$.

At stage m , compute pseudo-residuals:

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{m-1}}.$$

Fit a weak learner $h_m(x)$ to approximate these negative gradients.

Update:

$$F_m(x) = F_{m-1}(x) + \eta h_m(x),$$

where η is the learning rate.

This is why gradient boosting can be understood as **gradient descent over functions**.

10.12 Squared-Error Example

For

$$L(y, F) = \frac{1}{2}(y - F)^2,$$

the negative gradient is

$$-\frac{\partial L}{\partial F} = y - F.$$

So for squared-error regression, pseudo-residuals are ordinary residuals.

That gives a useful intuition:

Fit the next learner to what the current ensemble still gets wrong.

10.13 Visual Experiment: Sequential Residual Correction

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeRegressor

rng=np.random.default_rng(42)
x=np.linspace(0,6,120)
y=np.sin(x)+0.25*x+rng.normal(0,.15,len(x))
```

```

X_demo=x.reshape(-1,1)

prediction=np.full_like(y,y.mean())
learning_rate=.3

plt.figure(figsize=(8,5))
plt.scatter(x,y,s=18,alpha=.6,label="Observed")

for stage in range(1,5):
    residual=y-prediction
    learner=DecisionTreeRegressor(max_depth=2,random_state=stage)
    learner.fit(X_demo,residual)
    prediction += learning_rate*learner.predict(X_demo)

    if stage in [1,2,4]:
        plt.plot(x,prediction,label=f"After {stage} learner(s)")

plt.xlabel("x")
plt.ylabel("y")
plt.title("Boosting as Sequential Error Correction")
plt.legend()
plt.show()

```

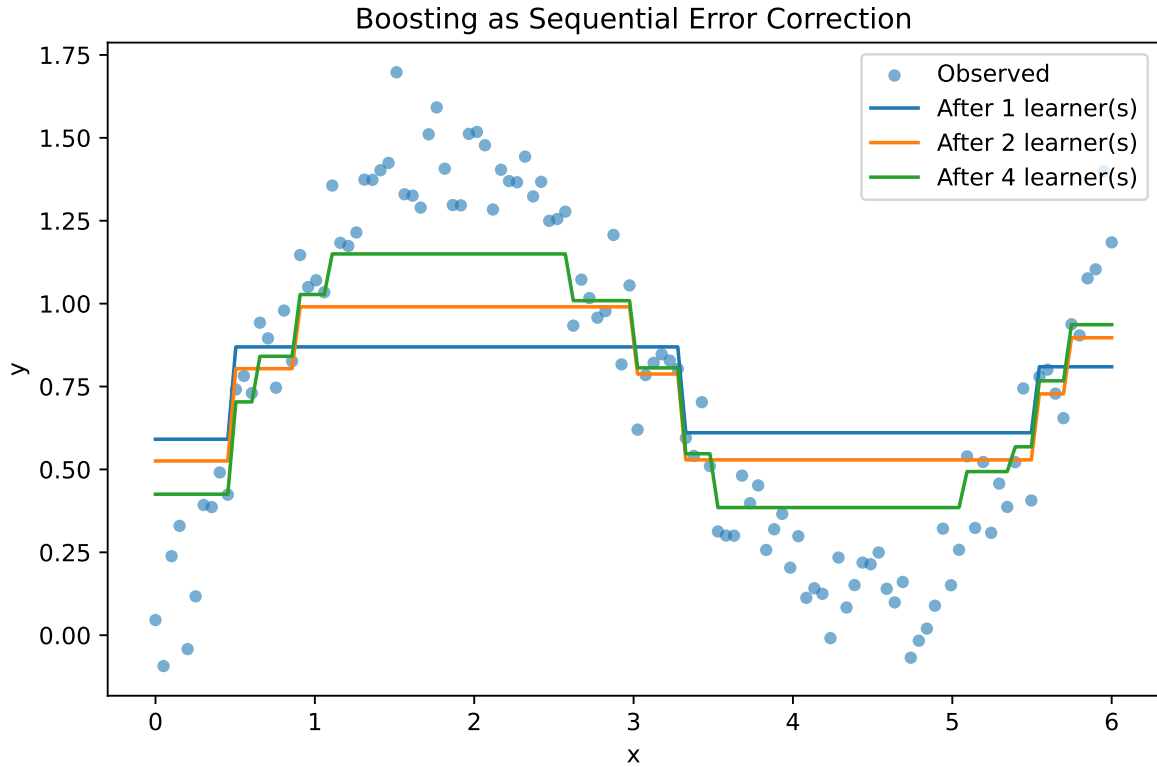


Figure 10.1: A sequence of shallow trees can progressively model residual structure.

10.14 Learning Rate and Number of Trees

Gradient boosting often involves a tradeoff between:

- number of boosting stages;
- learning rate;
- complexity of each weak learner.

A smaller learning rate makes each learner contribute less:

$$F_m = F_{m-1} + \eta h_m.$$

This often requires more learners.

There is no universal best combination. These parameters belong inside a validation/model-selection procedure.

10.15 Fit Gradient Boosting

```
from sklearn.ensemble import GradientBoostingClassifier

gb=GradientBoostingClassifier(
    n_estimators=200,
    learning_rate=.05,
    max_depth=2,
    random_state=42
)

gb.fit(X_train,y_train)
gb_prob=gb.predict_proba(X_test)[:,:1]
roc_auc_score(y_test,gb_prob)
```

```
np.float64(0.9943396226415094)
```

10.16 Modern Histogram-Based Gradient Boosting

Histogram-based methods discretize continuous feature values into bins, which can improve computational efficiency on larger datasets.

```
import os

# Prevent a Windows-specific joblib/loky physical-core detection warning.
# One core is sufficient for this small instructional example.
os.environ["LOKY_MAX_CPU_COUNT"] = "1"

from sklearn.ensemble import HistGradientBoostingClassifier

hgb = HistGradientBoostingClassifier(
    learning_rate=.08,
    max_iter=200,
    max_leaf_nodes=15,
    random_state=42
)

hgb.fit(X_train, y_train)
```

```
hgb_prob = hgb.predict_proba(X_test)[:, 1]

hgb_auc = roc_auc_score(y_test, hgb_prob)
hgb_auc
```

```
np.float64(0.9968553459119497)
```

scikit-learn provides AdaBoost, Gradient Boosting, histogram-based boosting, voting, and stacking estimators (scikit-learn developers 2026b).

10.17 The Modern Boosting Ecosystem

Gradient boosting is not a single implementation. Modern libraries share the core idea of sequentially improving an ensemble, but differ in how they construct trees, represent features, handle categories, regularize learning, and scale computation.

A useful conceptual map is:

Ecosystem	Distinguishing emphasis	Typical reason to encounter it
scikit-learn HistGradientBoosting	native scikit-learn workflow and histogram-based efficiency	strong general-purpose baseline inside sklearn pipelines
XGBoost (Chen and Guestrin 2016)	regularized boosting, mature ecosystem, broad deployment history	tabular prediction and established production/research workflows
LightGBM (Ke et al. 2017)	histogram methods and efficient leaf-wise growth	large tabular datasets and computational efficiency
CatBoost (Prokhorenkova et al. 2018)	ordered boosting and strong categorical-feature handling	datasets with important categorical predictors

These are not four unrelated algorithms. They are engineering and statistical variations on a shared boosting family.

10.17.1 What Actually Differs?

When comparing boosting systems, examine:

- tree-growth strategy;

- treatment of categorical variables;
- missing-value handling;
- regularization;
- sampling;
- histogram/binning strategy;
- computational scaling;
- GPU support;
- reproducibility controls;
- probability calibration;
- ecosystem and deployment constraints.

The practical question is therefore not:

Which boosting library is universally best?

but:

Which implementation best matches the data, validation design, computational constraints, and deployment environment?

Algorithm Family + Implementation Choices + Validation Design → Observed Performance
--

Research Reality Check

A paper that compares XGBoost, LightGBM, CatBoost, and another gradient booster has not necessarily compared four fundamentally different scientific ideas.

If the research contribution rests only on which library wins on one dataset, ask what generalizable knowledge the comparison produces.

10.18 Boosting and Overfitting

Boosting is powerful but not invulnerable.

Potential controls include:

- learning rate;
- number of stages;
- tree depth;
- leaf size;
- subsampling;
- regularization;

- early stopping where supported.

The central principle remains:

Complexity should be chosen using validation evidence, not final-test experimentation.

10.19 Compare Several Supervised Models

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC

candidate_models={
    "Logistic regression":Pipeline([
        ("scale",StandardScaler()),
        ("model",LogisticRegression(max_iter=2000))
    ]),
    "Random Forest":RandomForestClassifier(
        n_estimators=500,random_state=42,n_jobs=1
    ),
    "RBF SVM":Pipeline([
        ("scale",StandardScaler()),
        ("model",SVC(kernel="rbf",probability=True,random_state=42))
    ]),
    "AdaBoost":ada,
    "Gradient Boosting":gb,
    "Histogram GB":hgb
}

auc_results={}

for name,m in candidate_models.items():
    m.fit(X_train,y_train)
    prob_m=m.predict_proba(X_test)[:,-1]
    auc_results[name]=roc_auc_score(y_test,prob_m)

auc_results
```



```
{'Logistic regression': np.float64(0.9962264150943396),  
'Random Forest': np.float64(0.9954926624737945),  
'RBF SVM': np.float64(0.9966457023060796),  
'AdaBoost': np.float64(0.9943396226415094),  
'Gradient Boosting': np.float64(0.9943396226415094),  
'Histogram GB': np.float64(0.9968553459119497)}
```

This is a demonstration, not a proper final model-selection experiment. Chapter 16 will formalize repeated resampling, tuning, comparison uncertainty, and final evaluation.

10.20 Voting Ensembles

Suppose models produce class predictions

$$\hat{y}_1, \hat{y}_2, \dots, \hat{y}_M.$$

10.20.1 Hard Voting

Choose the class receiving the most votes.

10.20.2 Soft Voting

Average predicted class probabilities:

$$\hat{p}_{\text{ens}} = \sum_{m=1}^M w_m \hat{p}_m,$$

then classify using the resulting probability and decision rule.

Soft voting assumes the component probability outputs are meaningful enough to combine.

10.21 Fit a Soft-Voting Ensemble

```

from sklearn.ensemble import VotingClassifier

voting=VotingClassifier(
    estimators=[
        ("logit",Pipeline([
            ("scale",StandardScaler()),
            ("model",LogisticRegression(max_iter=2000))
        ])),
        ("rf",RandomForestClassifier(
            n_estimators=300,random_state=42,n_jobs=1
        )),
        ("gb",GradientBoostingClassifier(
            n_estimators=150,
            learning_rate=.05,
            max_depth=2,
            random_state=42
        ))
    ],
    voting="soft"
)

voting.fit(X_train,y_train)
voting_prob=voting.predict_proba(X_test)[:,-1]
roc_auc_score(y_test,voting_prob)

```

```
np.float64(0.9974842767295597)
```

A voting ensemble is useful only if its components add complementary information.

10.22 Stacking

Stacking uses predictions from base models as inputs to a meta-model, following the stacked-generalization framework introduced by Wolpert (Wolpert 1992).

Conceptually,

$$X \rightarrow \begin{bmatrix} \hat{f}_1(X) \\ \hat{f}_2(X) \\ \vdots \\ \hat{f}_M(X) \end{bmatrix} \rightarrow g(\cdot) \rightarrow \hat{y}.$$

The meta-model g learns how to combine base-model outputs.

10.22.1 The Leakage Danger

If base models predict the same observations they were trained on, the meta-model receives unrealistically optimistic features.

Proper stacking therefore uses **out-of-fold predictions** for training the meta-model.

10.23 Fit a Stacking Classifier

```
from sklearn.ensemble import StackingClassifier

base_estimators=[
    ("logit",Pipeline([
        ("scale",StandardScaler()),
        ("model",LogisticRegression(max_iter=2000))
    ])),
    ("rf",RandomForestClassifier(
        n_estimators=300,random_state=42,n_jobs=1
    )),
    ("gb",GradientBoostingClassifier(
        n_estimators=150,
        learning_rate=.05,
        max_depth=2,
        random_state=42
    ))
]

stack=StackingClassifier(
    estimators=base_estimators,
    final_estimator=LogisticRegression(max_iter=2000),
    cv=5,
    stack_method="predict_proba"
)

stack.fit(X_train,y_train)
stack_prob=stack.predict_proba(X_test)[:,-1]
roc_auc_score(y_test,stack_prob)
```

```
np.float64(0.9974842767295597)
```

The cross-validation parameter controls how predictions used to train the final estimator are generated.

10.24 Diversity: More Than Different Algorithm Names

An ensemble of logistic regression, Random Forest, and boosting sounds diverse.

But algorithm labels do not guarantee meaningful diversity.

Models may still share:

- the same leakage;
- the same target-definition error;
- the same missing population;
- the same measurement bias;
- the same proxy variables;
- the same distribution shift.

True ensemble diversity concerns **error structure**, not merely model branding.

10.25 Visual Experiment: When Averaging Helps

```
rng=np.random.default_rng(7)
n=1000
true=np.zeros(n)

shared=rng.normal(0,1,n)

independent_errors=[
    0.3*shared + rng.normal(0,1,n)
    for _ in range(20)
]

ensemble_sizes=range(1,21)
mse=[]

for m in ensemble_sizes:
    avg=np.mean(independent_errors[:m],axis=0)
```

```

mse.append(np.mean((true-avg)**2))

plt.figure(figsize=(8,5))
plt.plot(list(ensemble_sizes),mse,marker="o")
plt.xlabel("Number of component models")
plt.ylabel("Mean squared error")
plt.title("Averaging Partly Independent Errors")
plt.show()

```

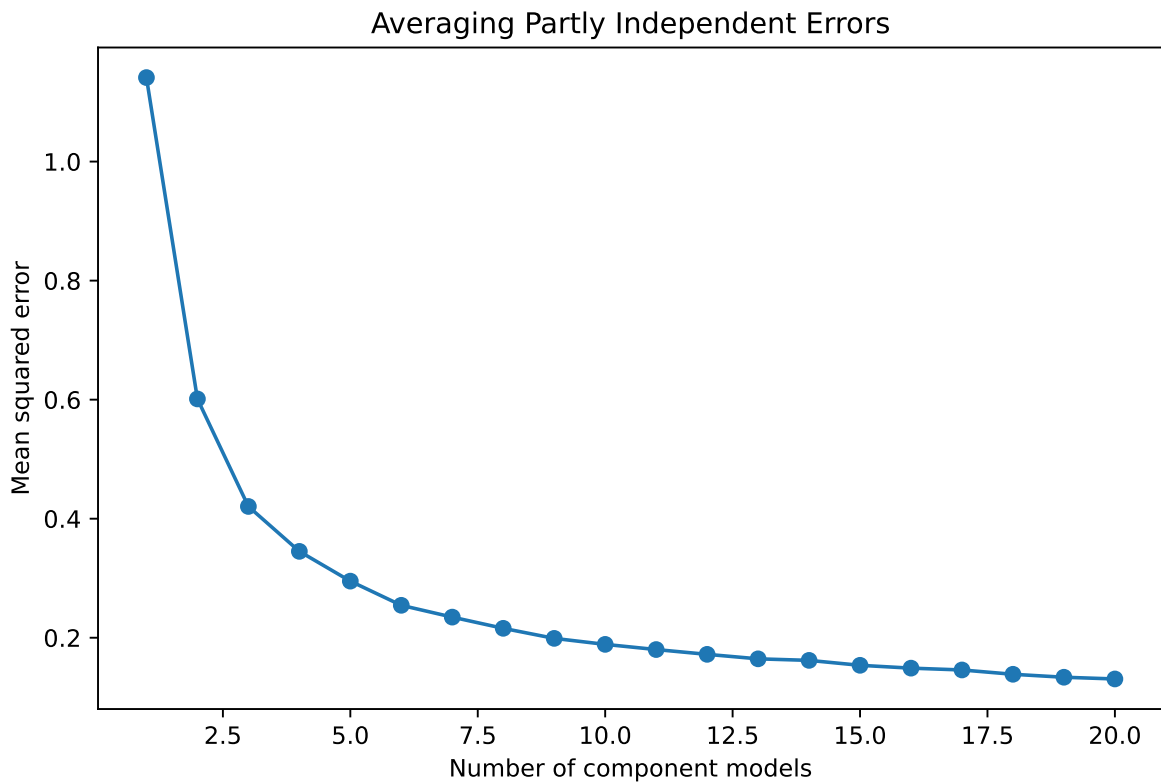


Figure 10.2: Averaging reduces error most effectively when component errors are not perfectly correlated.

The remaining shared component illustrates why correlated error does not disappear simply by adding models.

10.26 Calibration Still Matters

An ensemble can have excellent discrimination and poor calibration. Calibration should therefore be evaluated separately when probabilities matter (Niculescu-Mizil and Caruana 2005).

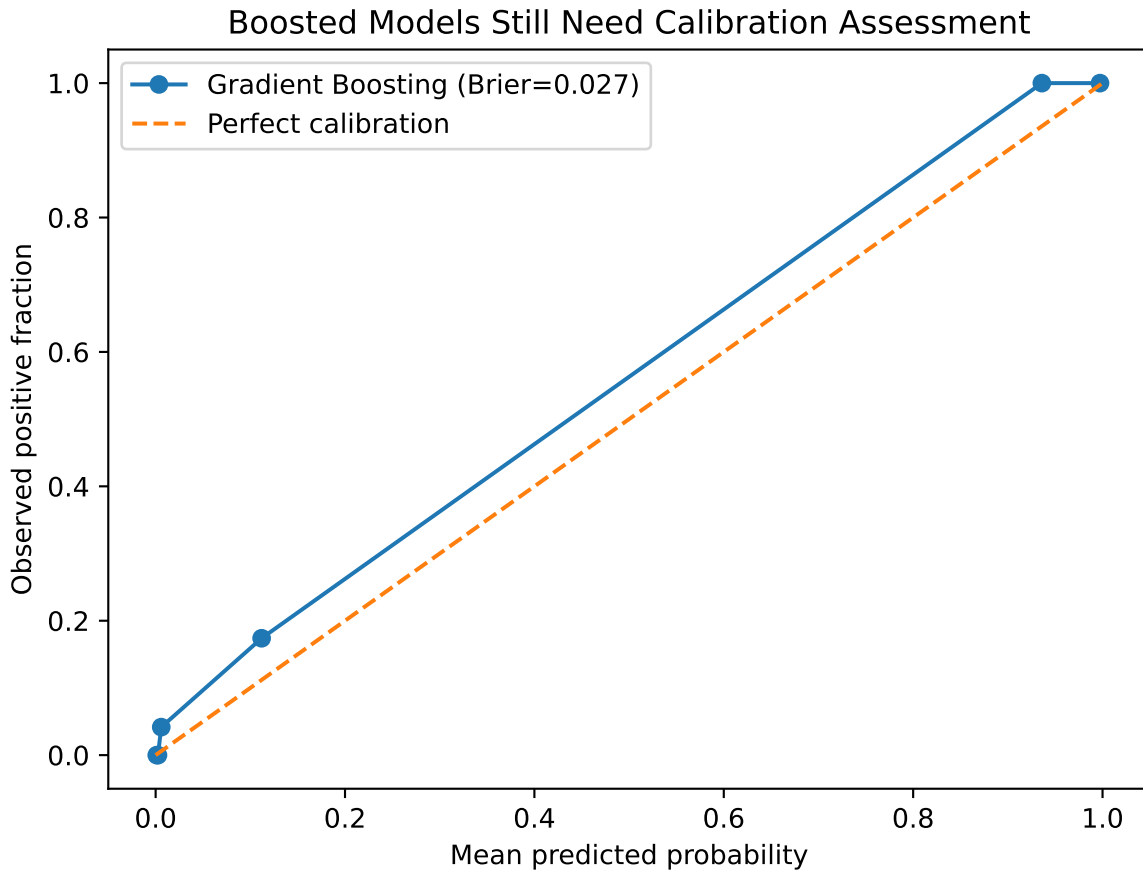
If probabilities drive decisions, evaluate calibration just as in Chapter 7.

```
from sklearn.calibration import calibration_curve
from sklearn.metrics import brier_score_loss

fraction_positive, mean_predicted = calibration_curve(
    y_test, gb_prob, n_bins=6, strategy="quantile"
)

brier = brier_score_loss(y_test, gb_prob)

plt.figure(figsize=(7, 5))
plt.plot(
    mean_predicted,
    fraction_positive,
    marker="o",
    label=f"Gradient Boosting (Brier={brier:.3f})"
)
plt.plot([0, 1], [0, 1], linestyle="--", label="Perfect calibration")
plt.xlabel("Mean predicted probability")
plt.ylabel("Observed positive fraction")
plt.title("Boosted Models Still Need Calibration Assessment")
plt.legend()
plt.show()
```



10.27 Feature Importance Returns—With the Same Warning

Tree-based boosting can provide feature-importance measures. Permutation importance can also be used.

But Chapter 8's principle remains:

Predictive importance $\neq$ causal importance.

A boosted ensemble can model complex associations exceptionally well while telling us little about what would happen under intervention.

10.28 Computer Vision Learning Experience: A Small Vision Model Comparison

The handwritten-digits workload now gives us a recurring benchmark across several chapters.

Rather than introducing a new dataset, we can ask a more mature question:

What do different model families contribute when the data, split, and evaluation metric are held constant?

The following comparison uses the same train/test split for a linear model, an RBF SVM, a Random Forest, and histogram-based gradient boosting.

```
import os
os.environ["LOKY_MAX_CPU_COUNT"] = "1"

from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier
from sklearn.metrics import accuracy_score
import pandas as pd

digits_ens_cv = load_digits()

X_ens_cv = digits_ens_cv.data
y_ens_cv = digits_ens_cv.target

X_train_ens_cv, X_test_ens_cv, y_train_ens_cv, y_test_ens_cv = train_test_split(
    X_ens_cv,
    y_ens_cv,
    test_size=0.25,
    random_state=42,
    stratify=y_ens_cv
)

models_ens_cv = {
    "Logistic regression": Pipeline([
        ("scale", StandardScaler()),
        ("model", LogisticRegression(max_iter=2000))
    ])
```



```

]),
"RBF SVM": Pipeline([
    ("scale", StandardScaler()),
    ("model", SVC(kernel="rbf", C=3.0, gamma="scale"))
]),
"Random Forest": RandomForestClassifier(
    n_estimators=300,
    random_state=42,
    n_jobs=1
),
"Histogram boosting": HistGradientBoostingClassifier(
    learning_rate=0.08,
    max_iter=180,
    random_state=42
)
}

vision_results = []

for model_name_cv, model_cv in models_ens_cv.items():
    model_cv.fit(
        X_train_ens_cv,
        y_train_ens_cv
    )

    pred_model_cv = model_cv.predict(
        X_test_ens_cv
    )

    vision_results.append({
        "Model": model_name_cv,
        "Accuracy": accuracy_score(
            y_test_ens_cv,
            pred_model_cv
        )
    })

vision_results_df = (
    pd.DataFrame(vision_results)
    .sort_values(
        "Accuracy",
        ascending=False
    )

```

```
)
)

vision_results_df
```

	Model	Accuracy
1	RBF SVM	0.980000
0	Logistic regression	0.977778
2	Random Forest	0.968889
3	Histogram boosting	0.962222

10.28.1 Do Not Turn the Table Into the Conclusion

The highest accuracy is not automatically the best scientific or operational choice.

Ask:

- How stable is the ranking under another split or repeated resampling?
- Which errors differ by digit class?
- How much more compute does the more complex model require?
- Are probability estimates needed?
- Which representation assumptions differ?
- Would the ranking survive visual domain shift?
- What new knowledge does the comparison produce?

Research Reality Check

“Model A achieved the highest accuracy on handwritten digits” is a result.

By itself, it is not yet a meaningful research contribution.

A publishable study needs a substantive or methodological question whose answer extends beyond the leaderboard.

10.29 Three Tracks

10.29.1 Health Track

Boosting can capture nonlinearities and interactions among health measurements. Evaluate calibration, external validity, subgroup performance, and whether difficult observations reflect rare but important cases or data-quality problems.

10.29.2 Education Track

An ensemble may improve early-warning predictions, but combining models does not resolve questions about intervention capacity, student labeling, temporal leakage, or whether predictive features are actionable.

10.29.3 Open ML Benchmark Track

Benchmark datasets allow controlled comparison of bagging, boosting, voting, and stacking while manipulating noise, class overlap, redundant features, and component-model diversity.

10.30 Beyond the Algorithm: When Does the Crowd Become Wise?

Agreement Is Not the Same as Truth

Ensemble learning resembles a broader idea: multiple imperfect judgments can sometimes outperform a single judgment.

But collective intelligence depends on conditions.

If members of the ensemble bring partly independent information, aggregation can cancel some individual errors.

If they all inherit the same flawed evidence, aggregation can reinforce the flaw.

A thousand models trained on leaked data do not create trustworthy evidence.

A hundred models trained on a systematically unrepresentative population do not create external validity.

Ten algorithms using the same proxy for an unmeasured construct do not create causal understanding.

The lesson extends beyond machine learning:

Diversity of voices matters only when it creates diversity of information and error.

The right question is not:

“How many models agree?”

It is:

“Why do they agree, where do their errors differ, and what assumptions do they share?”

10.31 AI Pair Programmer: Build Me the Ultimate Ensemble

Ask an AI assistant to combine every available classifier into one large ensemble.

Audit whether it asks:

- whether component errors are complementary;
- whether all models were evaluated on the same valid split;
- whether probabilities are calibrated;
- whether stacking uses out-of-fold predictions;
- whether added complexity improves performance meaningfully;
- whether latency or computational cost matters;
- whether shared leakage or bias exists;
- whether uncertainty in the performance difference is known.

Then redesign the ensemble to be simpler and evidence-driven.

10.32 Hands-On Lab: Bagging to Stacking

Using a classification dataset:

1. complete the Problem Framing Canvas;
2. complete the Feature Ledger;
3. establish a simple baseline;
4. fit logistic regression;
5. fit a single decision tree;
6. fit Random Forest;
7. fit AdaBoost;
8. fit Gradient Boosting;
9. compare discrimination;
10. inspect calibration for probability-producing models;
11. build a soft-voting ensemble;
12. build a stacking ensemble with out-of-fold meta-features;
13. compare performance and computational cost;
14. determine whether ensemble complexity earns its place;
15. identify shared failure modes;
16. document unsupported causal claims.

10.33 Practitioner Track

A practitioner should be able to choose among bagging, boosting, voting, and stacking; control ensemble complexity; prevent stacking leakage; justify added computational cost with validated gains; and treat library choice as an engineering and validation decision rather than a popularity contest.

10.34 Researcher Track

A researcher should additionally investigate uncertainty, calibration, stability, external validity, shared biases among component models, whether ensemble gains support prediction only or a broader scientific claim, distinguish comparison of algorithmic principles from comparison of software implementations; and document implementation versions, important hyperparameters, seeds, and evaluation design.

10.35 Research Studio

Design an ensemble study in health, education, or a benchmark domain.

Specify:

1. target and prediction time;
2. baseline;
3. candidate component models;
4. reason each model may contribute complementary structure;
5. bagging or boosting rationale;
6. voting or stacking rationale;
7. leakage-safe meta-learning strategy;
8. tuning strategy;
9. validation design;
10. discrimination and/or regression metrics;
11. calibration requirement;
12. computational constraints;
13. error-correlation analysis;
14. external validation;
15. interpretive limitations;
16. potential conference or journal contribution.

10.36 Professor's Challenge: The 12-Model Super Ensemble

A team trains 12 classifiers. Every model uses the same dataset and feature set. Hyperparameters are selected using the test set. The five highest-scoring models are stacked using their predictions on the same training observations used to fit them.

The resulting ensemble improves test AUC from .91 to .925.

The authors conclude:

“The ensemble’s agreement across diverse algorithms demonstrates robustness, eliminates model bias, and provides stronger evidence that the top-ranked features cause the outcome.”

Critique:

- test-set tuning;
- selection bias;
- stacking leakage;
- magnitude and uncertainty of the apparent improvement;
- algorithm diversity versus error diversity;
- shared data and measurement bias;
- calibration;
- external validity;
- causal claims;
- feature importance;
- computational complexity;
- whether .015 AUC improvement is operationally meaningful.

Redesign the experiment.

10.37 Knowledge Check

1. What is ensemble learning?
2. Why can averaging reduce variance?
3. Why does correlated error limit ensemble gains?
4. How does bagging differ from boosting?
5. How does AdaBoost respond to misclassified observations?
6. What does the AdaBoost learner weight represent conceptually?
7. What are pseudo-residuals in gradient boosting?
8. Why is gradient boosting described as optimization in function space?
9. What does learning rate control?
10. Why do learning rate and number of estimators interact?

11. How does hard voting differ from soft voting?
12. What assumption makes soft voting meaningful?
13. What is stacking?
14. Why must meta-model training use out-of-fold predictions?
15. Why do different algorithm names not guarantee useful diversity?
16. Why can boosting amplify data problems?
17. Why does ensemble performance not establish causal explanation?
18. What conditions make a “crowd” of models genuinely useful?
19. What ideas do modern boosting libraries share?
20. Why is comparing software packages not automatically equivalent to comparing fundamentally different algorithms?
21. What data and operational characteristics can make one boosting implementation more suitable than another?

10.38 Reproducibility Check

Record:

- problem specification;
- feature ledger;
- baseline;
- component estimators;
- preprocessing for each component;
- bootstrap settings;
- number of estimators;
- learning rates;
- tree complexity;
- subsampling settings;
- voting weights;
- stacking folds and meta-model;
- hyperparameter search boundaries;
- calibration assessment;
- evaluation metrics;
- random seeds;
- computational environment;
- training time where relevant;
- package versions;
- final test boundary;
- all code generating reported results.

10.39 Key Takeaways

- Ensembles combine multiple models to improve prediction or stability.
- Bagging primarily reduces variance through aggregation.
- Random Forests improve bagging by reducing correlation among trees.
- AdaBoost sequentially emphasizes difficult observations.
- Gradient boosting fits learners to negative loss gradients.
- Learning rate and ensemble size jointly control boosting behavior.
- Hard voting combines labels; soft voting combines probability estimates.
- Stacking learns a combination rule from out-of-fold base-model predictions.
- Algorithm diversity is not the same as error diversity.
- Shared leakage and shared bias survive aggregation.
- Ensembles still require calibration, validation, and external-validity analysis.
- More models do not automatically mean more trustworthy evidence.
- Ensemble complexity should earn its place through meaningful, reproducible improvement.
- Modern boosting systems share a common algorithmic lineage while differing in important implementation choices.
- Package-level performance comparisons require careful validation and should produce insight beyond a leaderboard.

10.40 Looking Ahead

Chapters 6–10 assumed that labeled outcomes were available for supervised learning. Chapter 11 removes that assumption and begins unsupervised learning, asking a different question: when no target label is supplied, what structure can the data reveal—and how do we determine whether that apparent structure is meaningful rather than an artifact of representation or method?

Part IV

Part IV — Unsupervised Learning

11 Clustering: Discovering Structure Without Labels

11.1 Learning Objectives

By the end of this chapter, you should be able to:

- explain how clustering differs from supervised classification;
- describe clustering as an exploratory modeling task rather than discovery of guaranteed “true” groups;
- explain why feature representation, scaling, and distance metrics define what similarity means;
- calculate and interpret Euclidean and Manhattan distance;
- derive the objective optimized by k -means clustering;
- explain the roles of centroids, assignment, and iterative updating;
- use the elbow method and silhouette analysis critically;
- explain why the number of clusters is not determined by a single diagnostic;
- fit and interpret hierarchical clustering;
- distinguish linkage strategies and interpret a dendrogram;
- explain density-based clustering using DBSCAN;
- interpret the roles of ε and `min_samples`;
- distinguish core, border, and noise observations;
- compare centroid-, hierarchy-, and density-based notions of structure;
- evaluate cluster stability and substantive usefulness;
- explain why clustering can create patterns even when meaningful groups do not exist;
- apply clustering to health, education, and open benchmark problems without turning descriptive segments into unsupported scientific categories.
- evaluate clustering of visual data as a representation-dependent exploratory task.

11.2 Why This Matters

Supervised learning begins with a target:

$$(X, Y) \longrightarrow \hat{f}.$$

In clustering, the labels disappear:

$$X \longrightarrow \text{structure?}$$

That question mark matters.

A clustering algorithm can always organize observations according to its mathematical definition of similarity. But the resulting groups are not automatically natural kinds, diagnoses, learning types, customer archetypes, or causal populations.

Central Question

If no labels tell us what the correct groups are, what does it mean for a clustering solution to be good?

11.2.1 From Supervised to Unsupervised Learning

Chapters 6–10 learned mappings from predictors to known outcomes. Clustering begins a different kind of inquiry. Without a supplied target, the algorithm must organize observations using patterns in the representation itself. This makes questions of scaling, distance, feature construction, validation, and interpretation even more central—not less.

11.3 From Prediction to Structure Discovery

In classification, an observed label gives us a criterion against which predictions can be evaluated.

In clustering, we usually do not have such an answer key.

Instead, we choose:

1. a set of observations;
2. a feature representation;
3. preprocessing;
4. a similarity or distance concept;
5. a clustering algorithm;
6. algorithm-specific parameters;
7. criteria for interpreting the resulting structure.

Thus clustering is not simply:

“Let the data speak.”

The analyst has already made consequential decisions about the language in which the data are allowed to speak.

11.4 What Does Similar Mean?

Suppose observations are represented by vectors

$$x_i = (x_{i1}, x_{i2}, \dots, x_{ip}).$$

A clustering method needs some notion of similarity or dissimilarity.

11.4.1 Euclidean Distance

$$d(x_i, x_j) = \sqrt{\sum_{k=1}^p (x_{ik} - x_{jk})^2}.$$

Euclidean distance corresponds to straight-line distance in feature space.

11.4.2 Manhattan Distance

$$d_1(x_i, x_j) = \sum_{k=1}^p |x_{ik} - x_{jk}|.$$

Different distance functions encode different geometric ideas about what it means for observations to be close.

11.5 Distance by Hand

```
import numpy as np

a=np.array([2,4])
b=np.array([5,8])

euclidean=np.sqrt(np.sum((a-b)**2))
manhattan=np.sum(np.abs(a-b))

euclidean, manhattan
```

```
(np.float64(5.0), np.int64(7))
```

11.6 Why Scaling Matters

Suppose one variable ranges from 0 to 5 and another from 0 to 100,000.

Under Euclidean distance, the second feature may dominate.

Standardization gives

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}.$$

But standardization is not philosophically neutral either. It changes the geometry by giving variables comparable variance-based scale.

Scaling Is a Modeling Decision

Ask not only “**Should I scale?**” but:

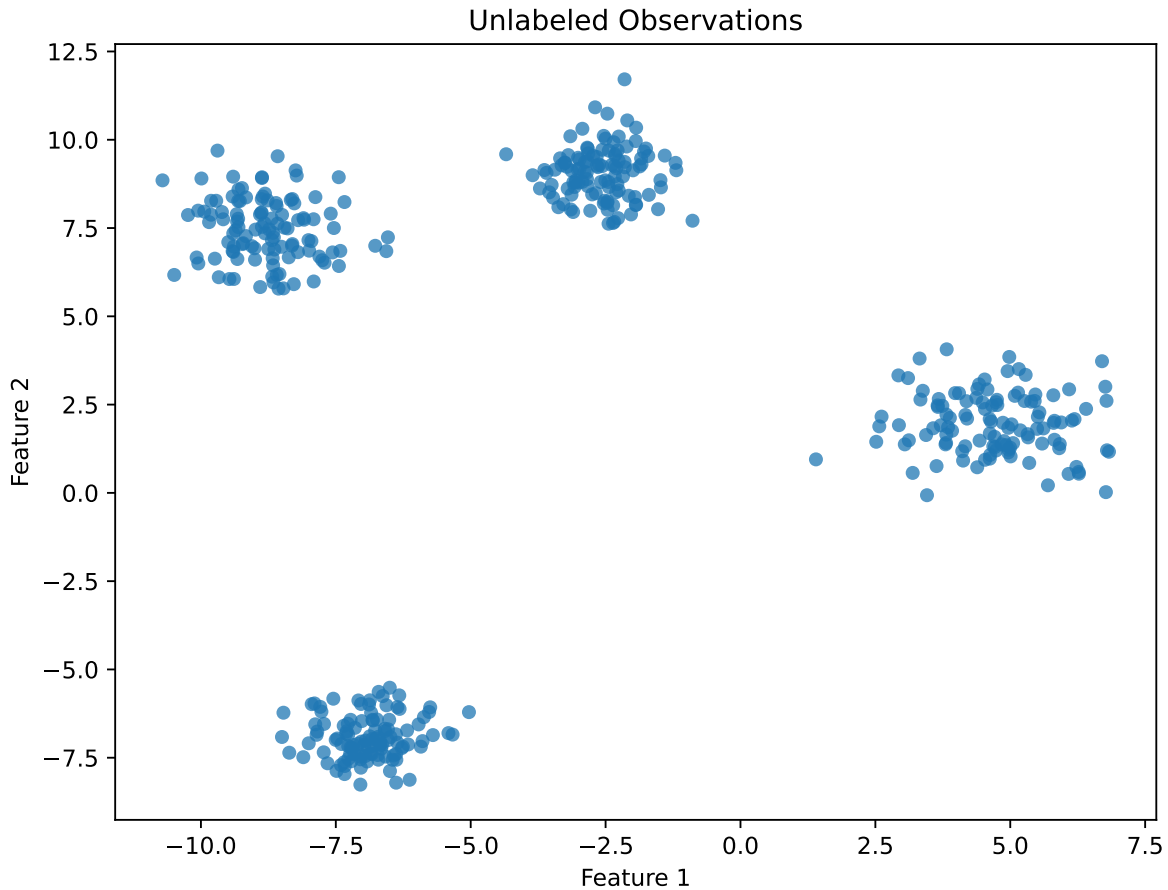
“**What definition of similarity do I want the resulting geometry to express?**”

11.7 A First Clustering Dataset

```
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt

X_demo, y_hidden=make_blobs(
    n_samples=450,
    centers=4,
    cluster_std=[0.7,1.0,0.6,0.9],
    random_state=42
)

plt.figure(figsize=(8,6))
plt.scatter(X_demo[:,0],X_demo[:,1],s=25,alpha=.75)
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Unlabeled Observations")
plt.show()
```



We retain `y_hidden` only because this is simulated instructional data. The clustering algorithm will not receive it.

11.8 Are Clusters Discovered or Constructed?

Clustering language often suggests that groups already exist in the data and the algorithm simply discovers them.

Sometimes that is a useful approximation.

But a clustering result depends on analytical choices such as

Features + Scaling + Distance + Algorithm + Hyperparameters → Clusters
--

Change those choices and the apparent groups may change.

11.8.1 The Same Observations, Different Representations

Consider observations measured on two variables with very different scales.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

rng_cr = np.random.default_rng(42)

group_a_cr = np.column_stack([
    rng_cr.normal(0, 0.6, 120),
    rng_cr.normal(20, 7, 120)
])

group_b_cr = np.column_stack([
    rng_cr.normal(3, 0.6, 120),
    rng_cr.normal(35, 7, 120)
])

X_cr = np.vstack([
    group_a_cr,
    group_b_cr
])

labels_raw_cr = KMeans(
    n_clusters=2,
    n_init=20,
    random_state=42
).fit_predict(X_cr)

X_scaled_cr = StandardScaler().fit_transform(X_cr)

labels_scaled_cr = KMeans(
    n_clusters=2,
    n_init=20,
    random_state=42
).fit_predict(X_scaled_cr)

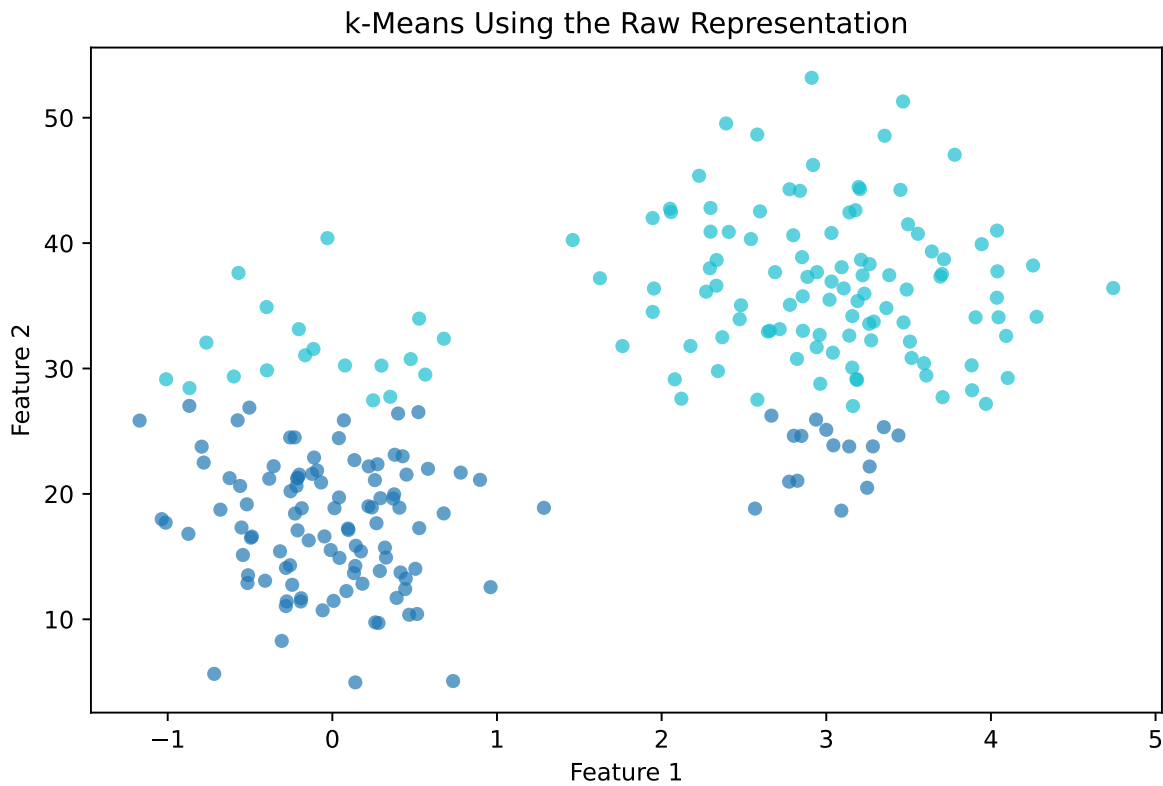
plt.figure(figsize=(8, 5))
plt.scatter(
    X_cr[:, 0],
```

```

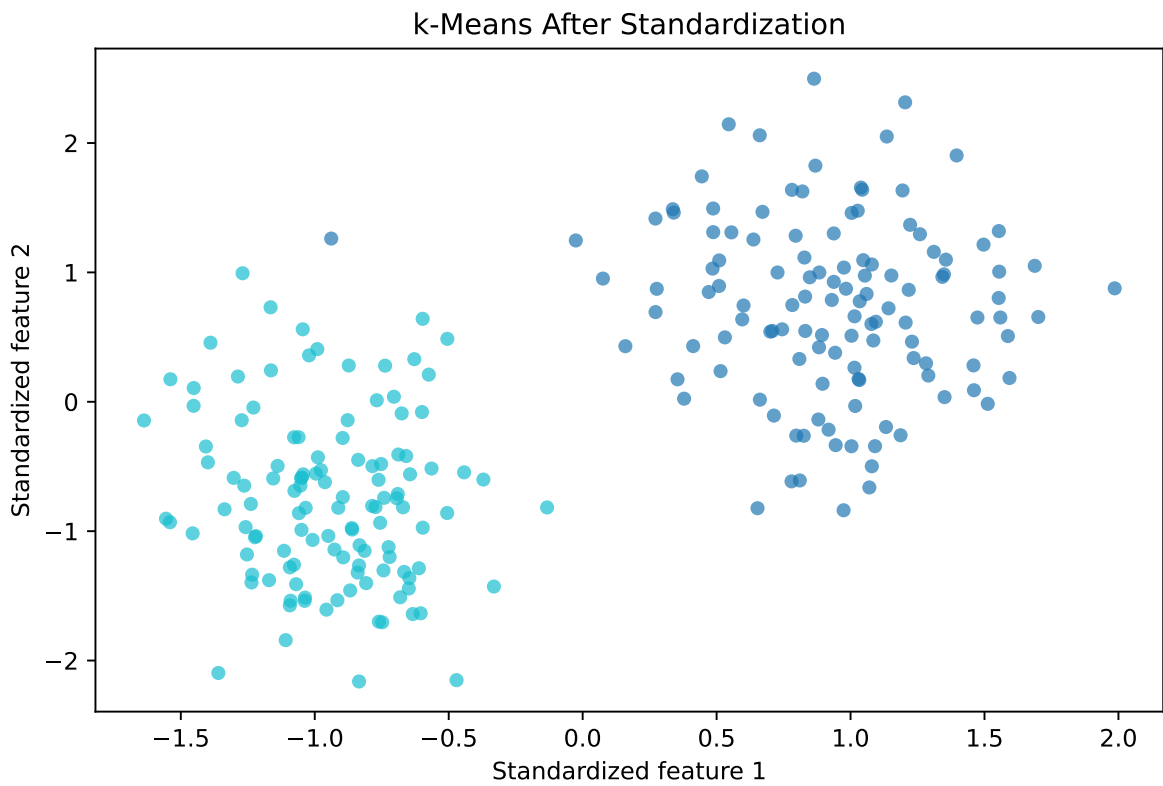
    X_cr[:, 1],
    c=labels_raw_cr,
    cmap="tab10",
    s=24,
    alpha=0.7
)
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("k-Means Using the Raw Representation")
plt.show()

plt.figure(figsize=(8, 5))
plt.scatter(
    X_scaled_cr[:, 0],
    X_scaled_cr[:, 1],
    c=labels_scaled_cr,
    cmap="tab10",
    s=24,
    alpha=0.7
)
plt.xlabel("Standardized feature 1")
plt.ylabel("Standardized feature 2")
plt.title("k-Means After Standardization")
plt.show()

```

(a) Clustering can change when feature scaling changes the geometry used by the algorithm.



281
(b)

Figure 11.1

The observations are the same. The geometry supplied to the algorithm is different.

11.8.2 Clusters Are Analytical Claims

A cluster should not automatically be interpreted as a naturally occurring type.

In health analytics, a cluster of patients may reflect measurement choices rather than distinct biological subtypes.

In education, student clusters may reflect institutional variables and scaling choices rather than fixed “types of learners.”

In Open ML, customer segments may change when behavioral, demographic, or transactional features are added or removed.

Therefore, cluster interpretation should include:

- stability under resampling;
- sensitivity to scaling;
- sensitivity to feature selection;
- comparison across algorithms;
- internal validity;
- external or substantive validity;
- domain interpretation.

Beyond the Algorithm: Categories Can Become Real Through Use

Once analysts name clusters, organizations may begin treating them as meaningful categories.

Labels such as “high-risk learner,” “complex patient,” or “premium customer” can influence interventions, resources, and expectations.

This creates an important responsibility: a computational grouping can become socially consequential even when its boundaries were analytically constructed.

Research Reality Check

Finding ($k=3$) clusters does not demonstrate that three natural populations exist.

A defensible claim is narrower:

Under this representation, distance measure, algorithm, and validation procedure, a three-cluster solution produced useful structure.

That is a stronger scientific statement because it makes the assumptions visible.

11.9 k -Means Clustering

k -means seeks K cluster centers, or centroids, following one of the foundational formulations of partition-based clustering (MacQueen 1967):

$$\mu_1, \mu_2, \dots, \mu_K.$$

It minimizes the within-cluster sum of squares:

$$J = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2.$$

This objective encodes a particular notion of a good cluster:

observations should be close to their assigned centroid under squared Euclidean distance.

11.10 The k -Means Algorithm

A simplified iteration is:

1. initialize K centroids;
2. assign each observation to its nearest centroid;
3. recompute each centroid from assigned observations;
4. repeat until assignments or centroids stabilize sufficiently.

The centroid for cluster C_k is

$$\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i.$$

11.11 Fit k -Means

```

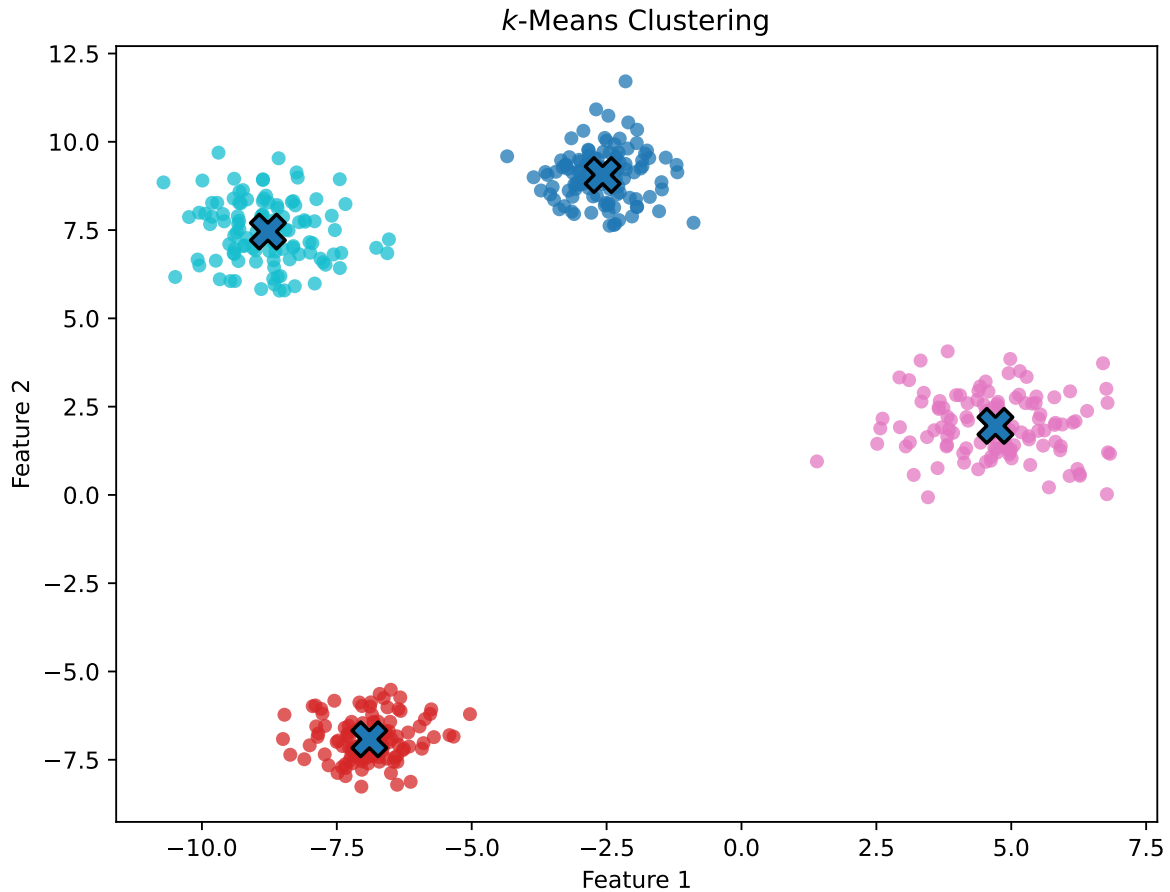
from sklearn.cluster import KMeans

kmeans=KMeans(
    n_clusters=4,
    n_init="auto",
    random_state=42
)

labels=kmeans.fit_predict(X_demo)
centers=kmeans.cluster_centers_

plt.figure(figsize=(8,6))
plt.scatter(
    X_demo[:,0],
    X_demo[:,1],
    c=labels,
    cmap="tab10",
    s=25,
    alpha=.75
)
plt.scatter(
    centers[:,0],
    centers[:,1],
    marker="X",
    s=220,
    edgecolors="black",
    linewidths=1.5
)
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("$k$-Means Clustering")
plt.show()

```



Cluster numbers such as 0, 1, 2, and 3 are arbitrary identifiers. Cluster 3 is not inherently “greater” than Cluster 1.

11.12 Initialization Matters

The k -means objective is nonconvex. Different initial centroids can lead to different local solutions.

Modern implementations use careful initialization strategies and multiple starts to improve robustness.

This is why reproducibility requires documenting:

- initialization strategy;
- number of initializations;
- random seed;
- software version.

11.13 How Many Clusters?

One of the most common mistakes in clustering is treating K as if the data contain a single objectively correct answer waiting to be revealed.

Usually, choosing K requires multiple forms of evidence.

11.14 Within-Cluster Sum of Squares and the Elbow

As K increases, within-cluster sum of squares cannot increase.

```
ks=range(1,11)
inertia=[]

for k in ks:
    m=KMeans(n_clusters=k,n_init="auto",random_state=42)
    m.fit(X_demo)
    inertia.append(m.inertia_)

plt.figure(figsize=(8,5))
plt.plot(list(ks),inertia,marker="o")
plt.xlabel("Number of clusters, K")
plt.ylabel("Within-cluster sum of squares")
plt.title("Elbow Analysis")
plt.show()
```

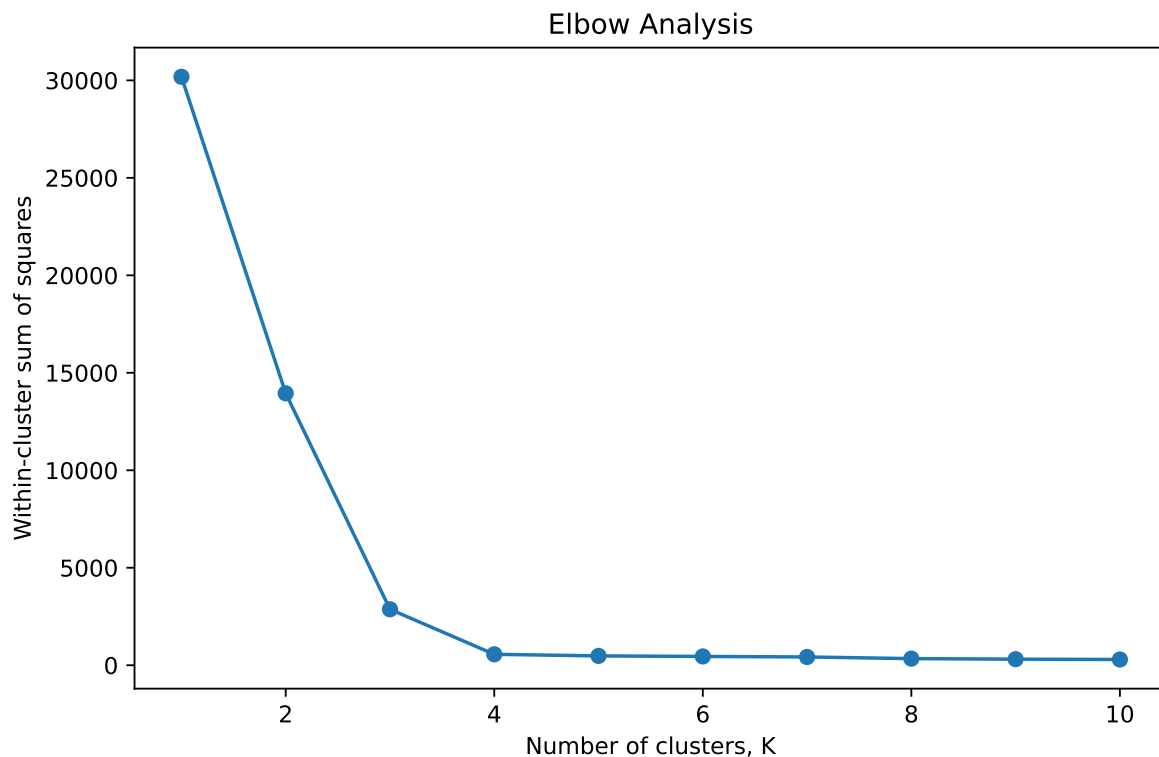


Figure 11.2: Within-cluster sum of squares decreases as more clusters are allowed; the analyst looks for diminishing returns rather than a mathematically guaranteed elbow.

The elbow method is a heuristic. Some datasets have no obvious elbow; analysts can disagree about where it occurs.

11.15 Silhouette Analysis

Silhouette analysis provides an internal diagnostic of how well an observation fits its assigned cluster relative to neighboring clusters (Rousseeuw 1987).

For observation i , let:

- $a(i)$ be its average dissimilarity to observations in its own cluster;
- $b(i)$ be the smallest average dissimilarity to another cluster.

The silhouette coefficient is

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}.$$

Values approach 1 when an observation is well matched to its cluster and separated from alternatives.

Values near 0 suggest boundary ambiguity.

Negative values suggest that another cluster may be closer on average.

```
from sklearn.metrics import silhouette_score

silhouette_results={}

for k in range(2,11):
    m=KMeans(n_clusters=k,n_init="auto",random_state=42)
    lab=m.fit_predict(X_demo)
    silhouette_results[k]=silhouette_score(X_demo,lab)

silhouette_results
```

```
{2: np.float64(0.6088044351869154),
 3: np.float64(0.7802566243321948),
 4: np.float64(0.8330144678792226),
 5: np.float64(0.7080931183466258),
 6: np.float64(0.6966266351282295),
 7: np.float64(0.5545070229283066),
 8: np.float64(0.4838105325521525),
 9: np.float64(0.33827775339131916),
10: np.float64(0.3376676801209995)}
```

```
plt.figure(figsize=(8,5))
plt.plot(
    list(silhouette_results.keys()),
    list(silhouette_results.values()),
    marker="o"
)
plt.xlabel("Number of clusters, K")
plt.ylabel("Mean silhouette score")
plt.title("Silhouette Analysis")
plt.show()
```

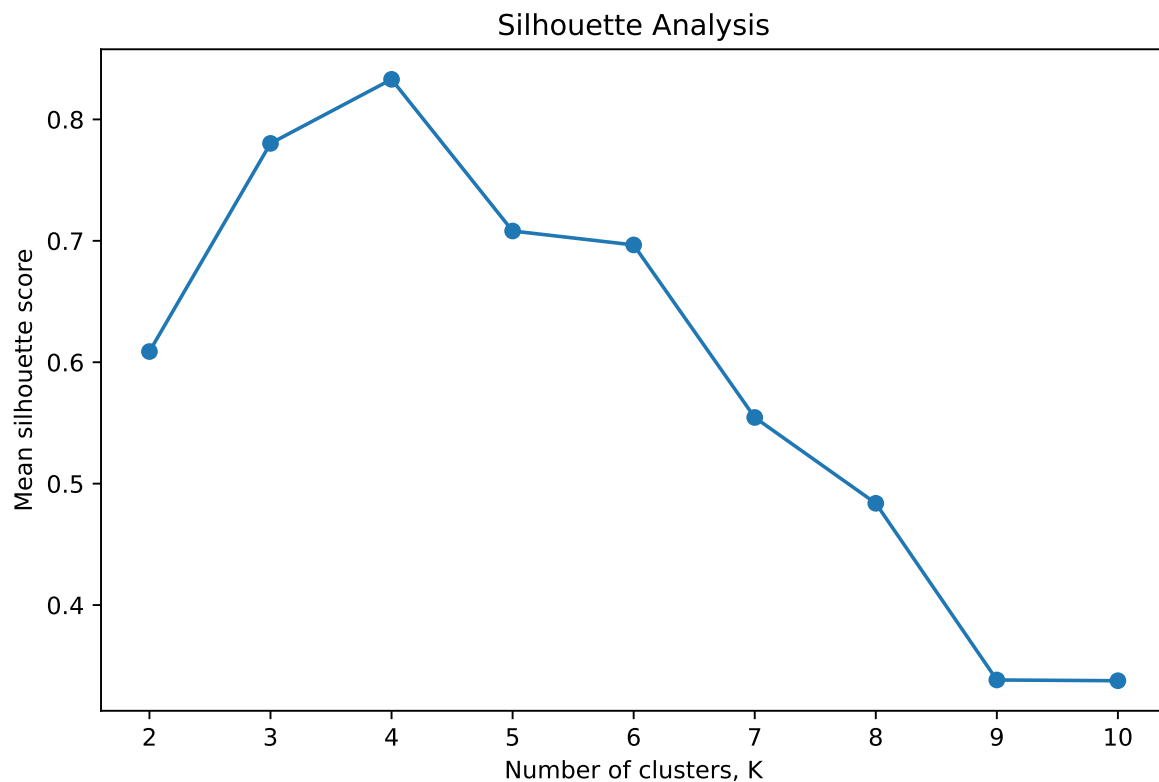



Figure 11.3: Silhouette scores provide one internal view of cohesion and separation across candidate values of K .

A high silhouette score does not prove that the clusters are substantively meaningful.

11.16 Internal Validation Is Not Scientific Validation

Internal criteria evaluate properties created by the same representation and distance structure used to construct the clusters.

A clustering solution may have excellent compactness and separation while being:

- unstable across samples;
- irrelevant to the substantive problem;
- driven by a nuisance variable;
- impossible to reproduce externally;
- misleading when interpreted as a real-world taxonomy.

Thus,

$$\boxed{\text{geometric separation}} \neq \boxed{\text{substantive validity}}.$$

11.17 Cluster Profiles

After clustering, analysts often summarize feature values by cluster.

```
import pandas as pd

demo_df=pd.DataFrame(
    X_demo,
    columns=["feature_1","feature_2"]
)
demo_df["cluster"]=labels

demo_df.groupby("cluster").mean()
```

	feature_1	feature_2
cluster		
0	-2.572844	9.058350
1	-6.896603	-6.920223
2	4.707056	1.955797
3	-8.777294	7.456918

Profiles describe the fitted partition.

They do not establish why the groups exist or whether the same groups will appear in another sample.

11.18 Hierarchical Clustering

Hierarchical clustering creates a nested sequence of groupings rather than requiring a single flat partition from the beginning.

11.18.1 Agglomerative Strategy

Agglomerative clustering begins with each observation as its own cluster and repeatedly merges clusters.

Conceptually,

$$n \text{ clusters} \rightarrow n - 1 \rightarrow n - 2 \rightarrow \cdots \rightarrow 1.$$

The critical question becomes:

How do we measure distance between clusters?

11.19 Linkage Strategies

11.19.1 Single Linkage

Distance between the closest pair of observations across two clusters.

11.19.2 Complete Linkage

Distance between the farthest pair.

11.19.3 Average Linkage

Average pairwise distance between observations across clusters.

11.19.4 Ward Linkage

Merges clusters to minimize the increase in within-cluster variance.

Different linkage definitions can produce very different structures.

11.20 Dendrogram

A dendrogram visualizes the hierarchy of merges.

```

from scipy.cluster.hierarchy import linkage,dendrogram

sample=X_demo[:80]

Z=linkage(sample,method="ward")

plt.figure(figsize=(12,6))
dendrogram(Z,no_labels=True)
plt.xlabel("Observations")
plt.ylabel("Merge distance")
plt.title("Hierarchical Clustering Dendrogram")
plt.show()

```

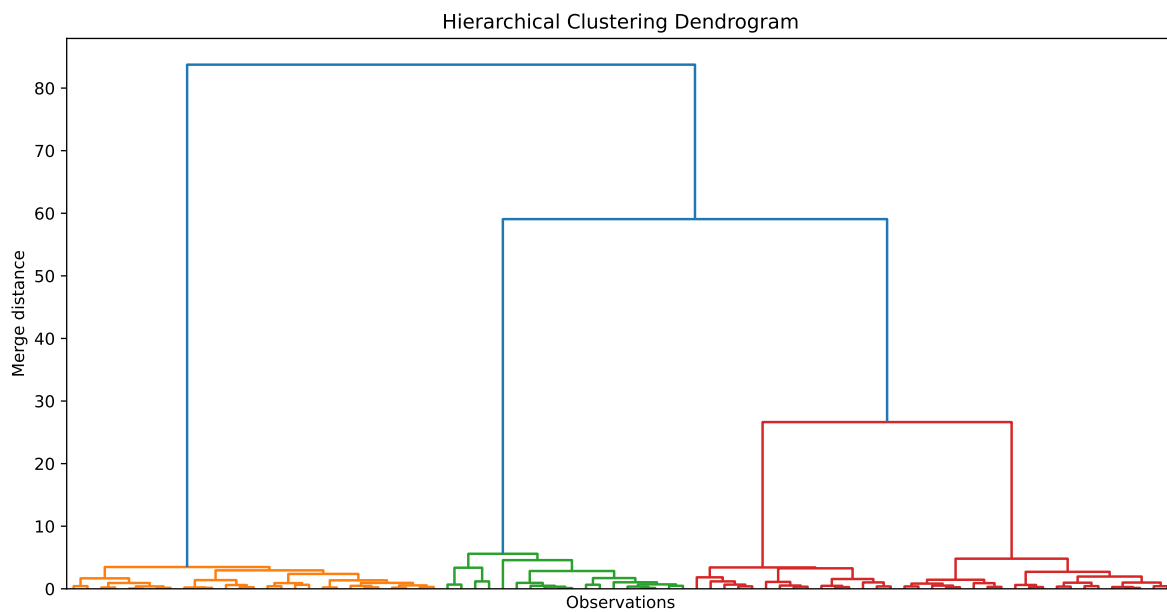


Figure 11.4: A dendrogram represents a hierarchy of merges; choosing where to cut the tree is an analytical decision.

A horizontal cut produces a particular number of clusters, but the dendrogram itself does not dictate one universally correct cut.

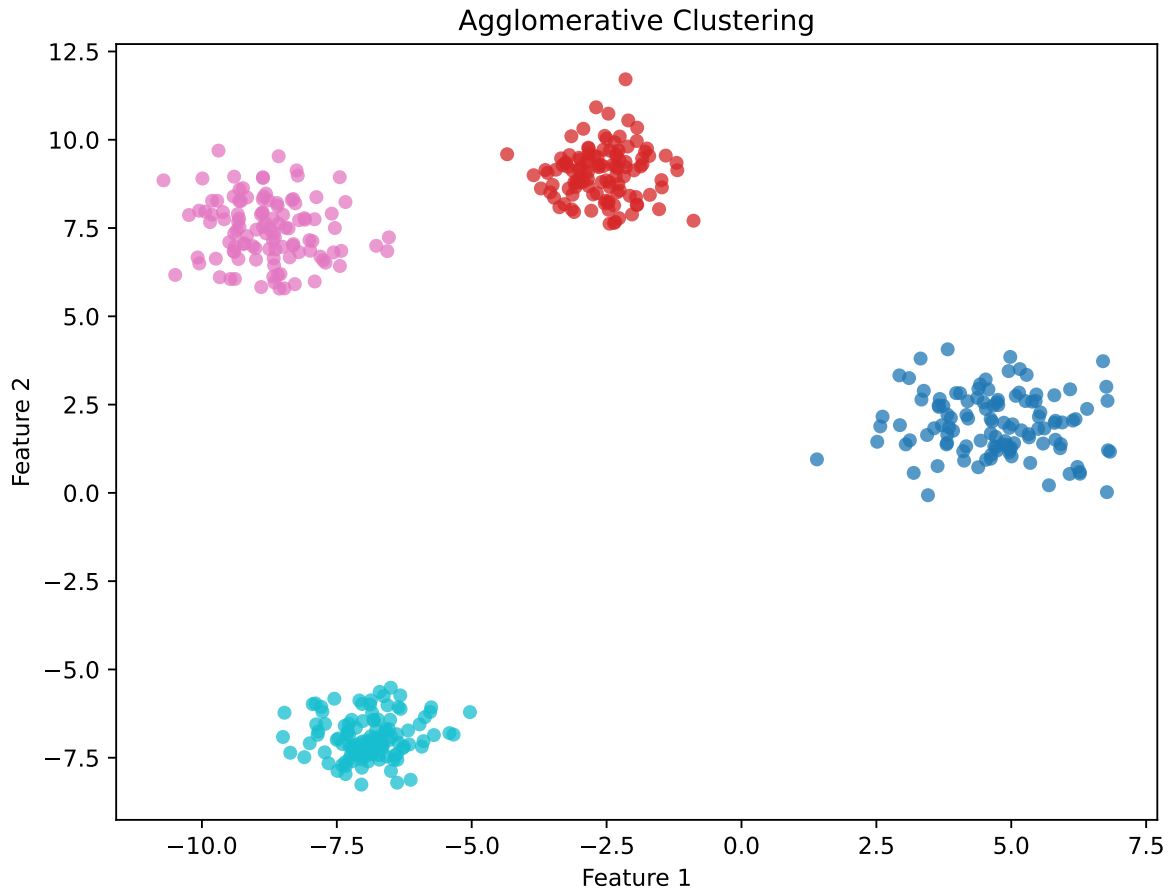
11.21 Agglomerative Clustering in scikit-learn

```
from sklearn.cluster import AgglomerativeClustering

agg=AgglomerativeClustering(
    n_clusters=4,
    linkage="ward"
)

agg_labels=agg.fit_predict(X_demo)

plt.figure(figsize=(8,6))
plt.scatter(
    X_demo[:,0],
    X_demo[:,1],
    c=agg_labels,
    cmap="tab10",
    s=25,
    alpha=.75
)
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Agglomerative Clustering")
plt.show()
```



11.22 A Different Definition of Structure: Density

k -means prefers compact centroid-centered clusters.

But what if clusters are curved or irregular?

Consider two interlocking moons.

```
from sklearn.datasets import make_moons

X_moons, _ = make_moons(
    n_samples=500,
    noise=.07,
    random_state=42
)
```

```
plt.figure(figsize=(8,6))
plt.scatter(X_moons[:,0],X_moons[:,1],s=22,alpha=.75)
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Unlabeled Nonconvex Structure")
plt.show()
```

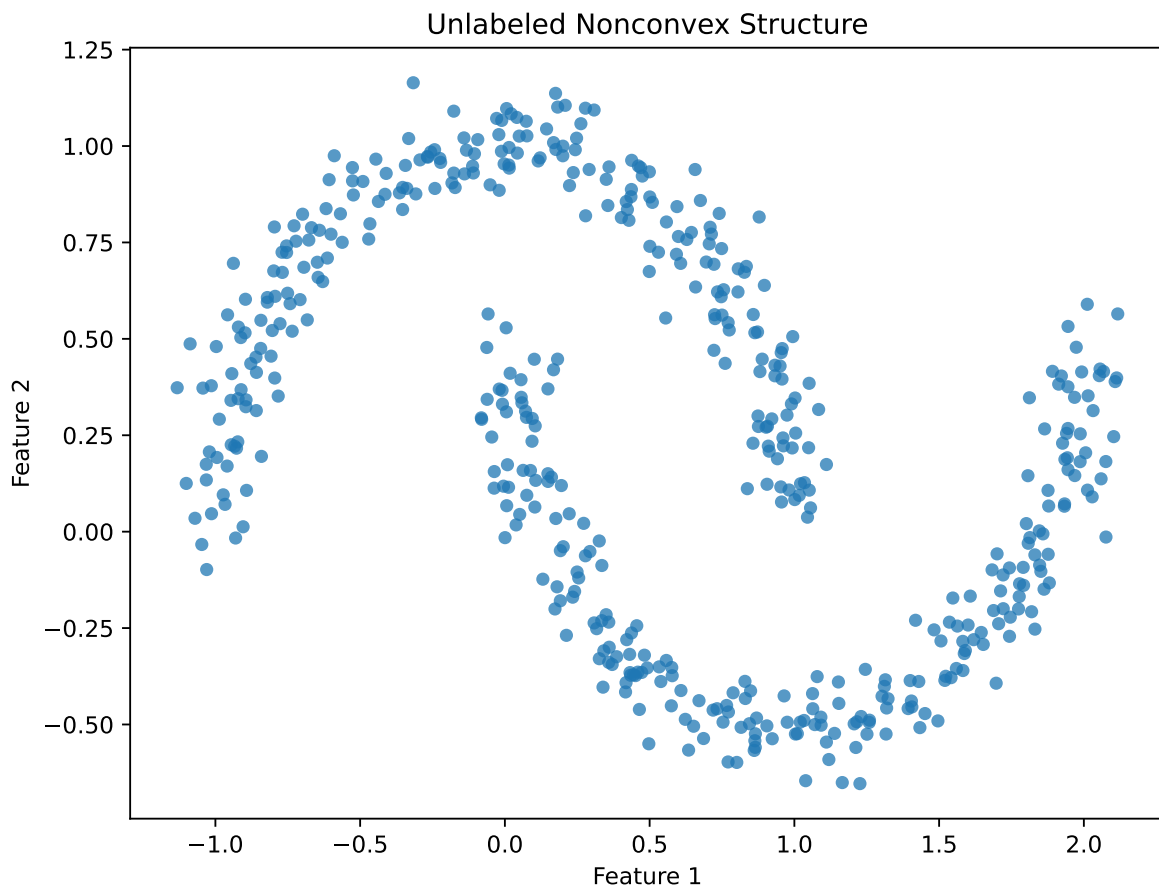


Figure 11.5: Nonconvex cluster structure challenges centroid-based clustering.

11.23 Why k -Means Can Struggle

```
moon_kmeans=KMeans(
    n_clusters=2,
```

```

        n_init="auto",
        random_state=42
    )

moon_k_labels=moon_kmeans.fit_predict(X_moons)

plt.figure(figsize=(8,6))
plt.scatter(
    X_moons[:,0],
    X_moons[:,1],
    c=moon_k_labels,
    cmap="tab10",
    s=22
)
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("$k$-Means on Two Moons")
plt.show()

```

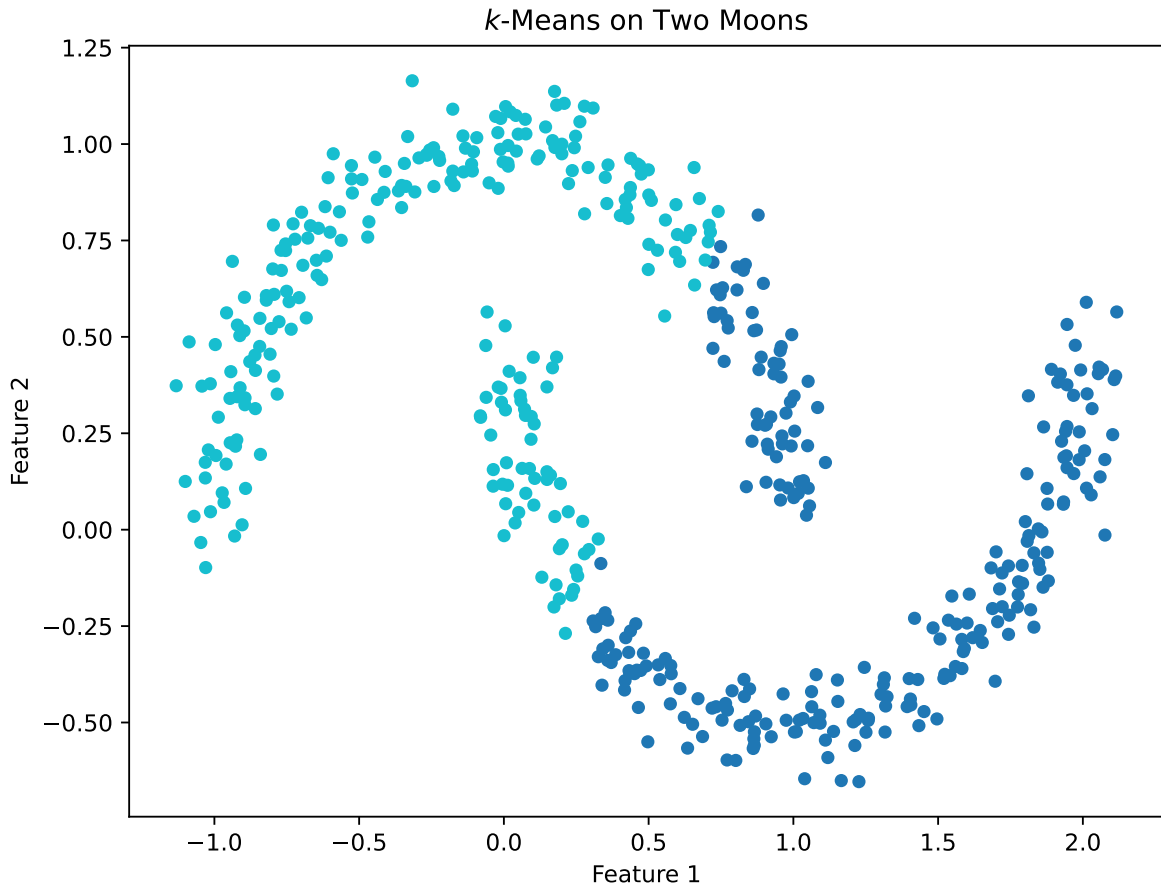



Figure 11.6: *k*-means imposes centroid-centered partitions that may not match nonconvex structure.

This is not a failure of optimization. It is a mismatch between the algorithm’s notion of cluster structure and the geometry of the data.

11.24 DBSCAN

DBSCAN—Density-Based Spatial Clustering of Applications with Noise—defines clusters through regions of sufficient local density.

Two important parameters are:

- ε (**eps**) — neighborhood radius;
- **min_samples** — minimum observations required for a sufficiently dense neighborhood.

For an observation x_i , define its ε -neighborhood:

$$N_\varepsilon(x_i) = \{x_j : d(x_i, x_j) \leq \varepsilon\}.$$

A **core point** has sufficiently many observations in this neighborhood.

A **border point** is reachable from a core region but does not itself meet the core density requirement.

A **noise point** is not assigned to a cluster under the fitted density rule.

11.25 Fit DBSCAN

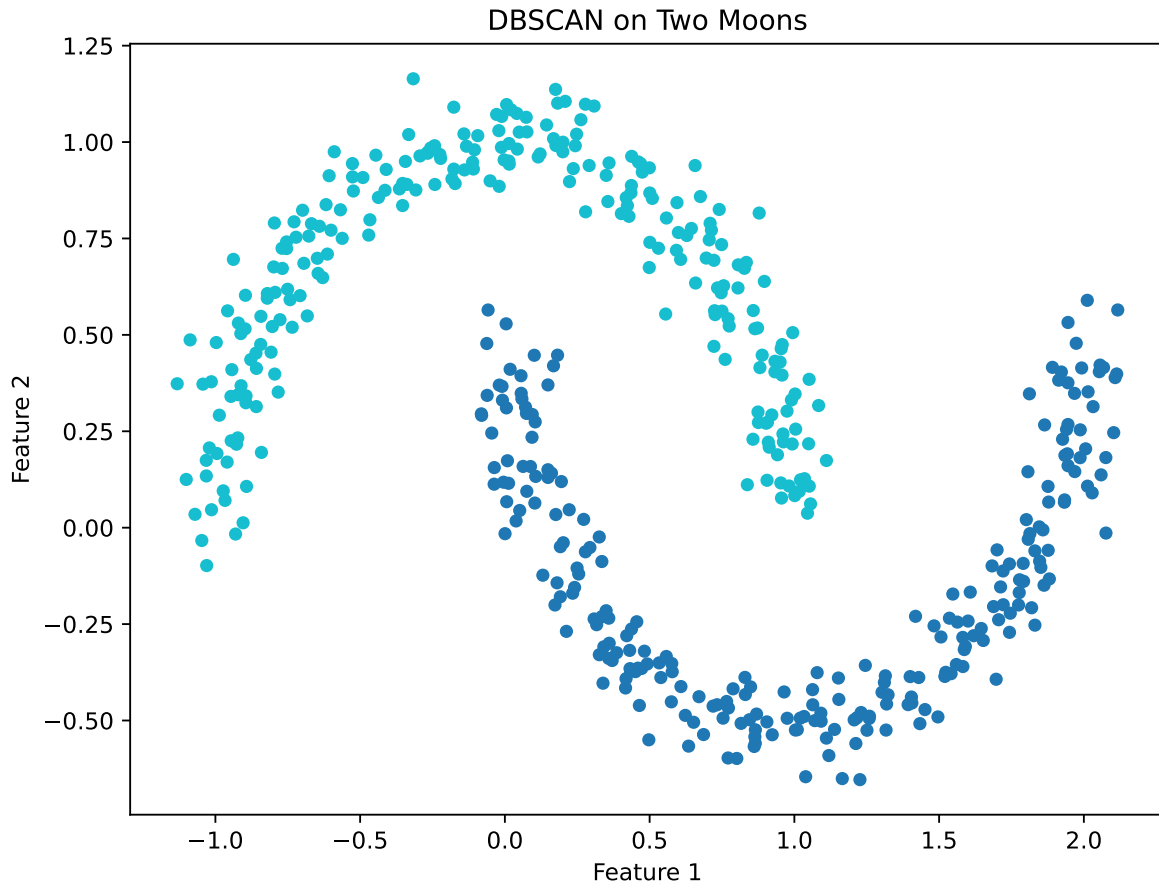
```
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler

X_moons_scaled=StandardScaler().fit_transform(X_moons)

dbscan=DBSCAN(
    eps=.25,
    min_samples=5
)

db_labels=dbscan.fit_predict(X_moons_scaled)

plt.figure(figsize=(8,6))
plt.scatter(
    X_moons[:,0],
    X_moons[:,1],
    c=db_labels,
    cmap="tab10",
    s=22
)
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("DBSCAN on Two Moons")
plt.show()
```



DBSCAN can discover irregularly shaped clusters and explicitly label some observations as noise.

But it is sensitive to the density scale implied by `eps`, `min_samples`, preprocessing, and the data distribution.

11.26 Compare Three Notions of a Cluster

Method	A cluster is approximately...	Key assumptions/choices
<i>k</i> -means	observations near a centroid	<i>K</i> , squared Euclidean geometry
hierarchical	observations joined within a merge hierarchy	distance, linkage, cut
DBSCAN	a connected region of sufficient density	<code>eps</code> , <code>min_samples</code> , distance

These algorithms are not merely alternative software commands.

They encode different answers to:

What kind of structure counts as a group?

11.27 Cluster Stability

If small changes in the data produce completely different clusters, interpretation should be cautious.

A practical stability investigation might:

1. resample observations;
2. refit clustering;
3. align resulting partitions where possible;
4. compare pairwise co-membership or adjusted agreement;
5. inspect whether substantive profiles persist.

For two partitions with known correspondence, the Adjusted Rand Index (ARI) is one possible agreement measure:

```
from sklearn.metrics import adjusted_rand_score

rng=np.random.default_rng(42)
indices=rng.choice(
    len(X_demo),
    size=int(.9*len(X_demo)),
    replace=False
)

m_full=KMeans(n_clusters=4,n_init="auto",random_state=42).fit(X_demo)
m_subset=KMeans(n_clusters=4,n_init="auto",random_state=42).fit(X_demo[indices])

full_labels_on_subset=m_full.predict(X_demo[indices])
subset_labels=m_subset.labels_

adjusted_rand_score(full_labels_on_subset,subset_labels)
```

1.0

This demonstration is intentionally simple. Rigorous cluster stability can require more careful resampling and label-alignment strategies.

11.28 External Variables and Cluster Usefulness

Sometimes variables not used to construct clusters can help assess whether a partition has external relevance.

For example, after clustering on pre-intervention measurements, researchers might compare clusters on a later outcome.

But caution is essential:

- post hoc comparisons can inflate false discoveries;
- external association does not validate every interpretation;
- using an outcome both to construct and “validate” clusters is circular;
- association with an external variable does not make clusters causal entities.

11.29 Can Clustering Find Groups That Are Not Really There?

Yes.

If you ask k -means for $K = 4$, it will return four clusters even when the underlying distribution is continuous and contains no natural four-group structure.

```
rng=np.random.default_rng(10)
continuous=rng.multivariate_normal(
    mean=[0,0],
    cov=[[1,.75],[.75,1]],
    size=500
)

forced=KMeans(
    n_clusters=4,
    n_init="auto",
    random_state=42
).fit_predict(continuous)

plt.figure(figsize=(8,6))
plt.scatter(
    continuous[:,0],
    continuous[:,1],
    c=forced,
    cmap="tab10",
    s=22
)
```

```
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Four Clusters Imposed on Continuous Structure")
plt.show()
```

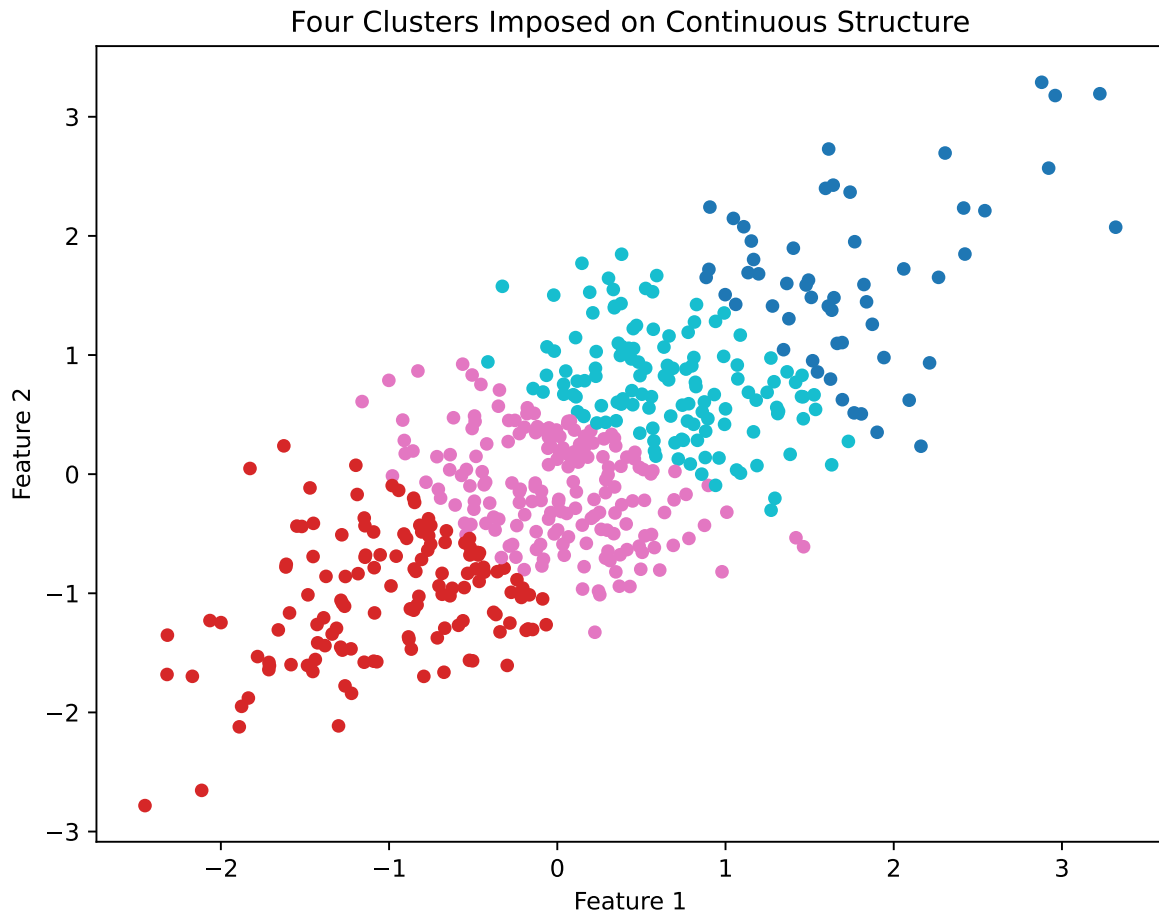


Figure 11.7: A clustering algorithm can partition continuous structure even when discrete natural groups are not evident.

The colors are real outputs of the algorithm.

That does not mean four natural populations existed before we requested them.

11.30 Computer Vision Learning Experience: Clustering Images Without Labels

Computer vision also contains unsupervised workloads. Suppose image labels are unavailable and we ask whether visually similar observations form useful groups.

Using handwritten digits, we can cluster flattened images without supplying digit labels to (k)-means. Known labels are retained only for **post hoc evaluation**, not fitting.

```
import os
os.environ["LOKY_MAX_CPU_COUNT"] = "1"

from sklearn.datasets import load_digits
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import adjusted_rand_score

digits_cluster_cv = load_digits()

X_cluster_cv = digits_cluster_cv.data
y_cluster_cv = digits_cluster_cv.target

X_cluster_scaled_cv = StandardScaler().fit_transform(X_cluster_cv)

kmeans_digits_cv = KMeans(
    n_clusters=10,
    n_init=20,
    random_state=42
)

clusters_digits_cv = kmeans_digits_cv.fit_predict(X_cluster_scaled_cv)

adjusted_rand_score(y_cluster_cv, clusters_digits_cv)
```

0.4779623470372548

The adjusted Rand index compares the discovered partition with known labels while accounting for agreement expected by chance. Cluster numbers do not need to match digit numbers.

A modest score would not necessarily mean clustering “failed.” (k)-means optimizes geometric compactness, not semantic digit identity.

Research Reality Check

If image clusters resemble known classes, that is evidence that the chosen representation contains class-related structure. It does not establish that the clusters are natural visual kinds or that the same structure will appear under another representation.

11.31 Three Tracks

11.31.1 Health Track

Potential applications include exploratory phenotyping, behavioral profiles, or patterns of health measurements. Avoid turning clusters into diagnoses without clinical validation. Consider measurement scale, missingness, stability, external replication, and whether continuous variation is being artificially discretized.

11.31.2 Education Track

Clustering may explore patterns of engagement, course-taking, or academic behavior. Avoid labels such as “at-risk learner type” unless the evidence supports a stable and useful construct. Student segments can become self-fulfilling if descriptive clusters are treated as identities.

11.31.3 Open ML Benchmark Track

Simulated blobs, moons, circles, and varying-density datasets allow students to test which definitions of structure are favored by k -means, hierarchical methods, and DBSCAN.

11.32 Beyond the Algorithm: Do Clusters Exist Before We Name Them?

Discovery or Construction?

Clustering is often described as “discovering natural groups in the data.”

Sometimes meaningful group structure may indeed exist.

But every clustering analysis also contains construction:

- which people or objects were included;
- which variables represented them;
- how those variables were measured;
- how missing values were handled;
- how features were scaled;
- what distance meant;
- which algorithm was chosen;
- which parameters were set;
- where the analyst decided one cluster ended and another began.

This does not make clustering arbitrary.

It makes clustering **model-dependent**.

A useful philosophical distinction is:

$$\boxed{\text{pattern in data}} \neq \boxed{\text{natural category in the world}}.$$

The responsible analyst therefore asks:

Are we discovering a stable structure that matters—or constructing a convenient partition that helps us think?

Both can be useful.

They are not the same claim.

11.33 AI Pair Programmer: Name These Clusters

Give an AI assistant a table of cluster means and ask it to name each cluster.

It may quickly produce labels such as:

- “high achievers”;
- “disengaged students”;
- “healthy lifestyle group”;
- “high-risk patients.”

Audit those labels.

Ask:

1. Which features actually support the label?
2. Does the label introduce moral or causal meaning not present in the data?
3. Are differences large enough to justify distinct names?
4. Are clusters stable?
5. Would the labels remain appropriate in another sample?
6. Could continuous variation be masquerading as categories?

Rewrite the labels descriptively before using interpretive names.

11.34 Hands-On Lab: What Counts as a Cluster?

Using a health, education, or benchmark dataset:

1. define the exploratory purpose;
2. construct a Feature Ledger;
3. identify variables that should not enter the clustering;
4. examine distributions and missingness;
5. justify scaling;
6. define the distance concept;
7. fit k -means for several values of K ;
8. inspect inertia and silhouette scores;
9. profile candidate clusters;
10. fit hierarchical clustering;
11. compare at least two linkage strategies;
12. fit DBSCAN where appropriate;
13. compare the different notions of structure;
14. conduct a stability check;
15. use an external variable cautiously if available;
16. write a substantive interpretation;
17. write a competing interpretation in which the structure is continuous rather than categorical;
18. document unsupported claims.

11.35 Practitioner Track

A practitioner should be able to choose clustering methods based on the desired notion of similarity, scale features appropriately, use internal diagnostics without treating them as proof, and communicate clusters as operational segments rather than unquestioned truths.

11.36 Researcher Track

A researcher should additionally examine stability, external replication, sensitivity to preprocessing and algorithm choice, uncertainty in cluster interpretation, and whether discrete groups are scientifically defensible relative to continuous alternatives.

11.37 Research Studio

Design a clustering study suitable for a conference or journal paper.

Specify:

1. substantive research question;
2. population and sampling frame;
3. feature-selection rationale;
4. variables intentionally excluded;
5. preprocessing;
6. similarity/distance definition;
7. candidate clustering algorithms;
8. parameter-selection strategy;
9. internal validation;
10. stability assessment;
11. external validation variables;
12. replication strategy;
13. cluster-labeling protocol;
14. sensitivity analyses;
15. continuous alternative explanation;
16. ethical implications of assigning cluster labels;
17. potential scholarly contribution.

11.38 Professor's Challenge: Four Types of Students

A university clusters 20,000 students using attendance, LMS activity, GPA, financial holds, advising contacts, and course withdrawals.

The analyst runs k -means with $K = 4$ because administrators requested “four student types.” The four resulting groups are labeled:

1. Thriving Students
2. Quiet Achievers
3. Disengaged Students

4. High-Risk Students

The mean silhouette score is 0.41.

Administrators propose automatically assigning different advising interventions based on cluster membership.

Critique:

- the predetermined choice of K ;
- scaling and feature weighting;
- the meaning of Euclidean distance;
- use of financial variables;
- whether the labels are descriptive or judgmental;
- silhouette interpretation;
- cluster stability;
- continuous versus categorical structure;
- temporal ordering;
- external validation;
- subgroup composition;
- intervention consequences;
- whether clustering is sufficient evidence for differential treatment.

Design a stronger exploratory study before deployment.

11.39 Knowledge Check

1. How does clustering differ from classification?
2. What does a distance metric encode?
3. What is Euclidean distance?
4. Why can scaling change a clustering solution?
5. What objective does k -means minimize?
6. What is a centroid?
7. Why can k -means produce different solutions across initializations?
8. Why does inertia always decline as K increases?
9. What does the silhouette coefficient measure?
10. Why does a high silhouette score not establish substantive validity?
11. What is agglomerative clustering?
12. How do linkage methods differ?
13. What does a dendrogram represent?
14. What does DBSCAN mean by a dense region?
15. What roles do `eps` and `min_samples` play?
16. What is a noise point?

17. Why might k -means fail on two interlocking moons?
18. Why can an algorithm return clusters when no natural categories exist?
19. What is cluster stability?
20. Why should cluster labels be treated cautiously?
21. Why can feature scaling change a clustering solution?
22. In what sense can clusters be partly constructed rather than merely discovered?
23. What evidence would strengthen a claim that a cluster solution is substantively meaningful?

11.40 Reproducibility Check

Record:

- research/exploratory purpose;
- analytic sample;
- feature inclusion/exclusion;
- missing-data handling;
- transformations;
- scaling;
- distance metric;
- clustering algorithm;
- K or density parameters;
- initialization;
- number of starts;
- random seeds;
- linkage method;
- internal metrics;
- stability procedure;
- external validation variables;
- cluster-labeling procedure;
- software/package versions;
- all code producing tables and figures.

11.41 Key Takeaways

- Clustering seeks structure without observed target labels.
- Similarity is defined through representation, scaling, and distance.
- k -means minimizes within-cluster squared distance to centroids.
- The elbow method and silhouette score are diagnostics, not truth detectors.
- Hierarchical clustering represents nested relationships among observations.

- Linkage determines what “close clusters” means.
- DBSCAN defines structure through local density and can identify noise.
- Different algorithms embody different definitions of a cluster.
- Internal separation does not establish substantive validity.
- Stability and external replication strengthen cluster interpretations.
- Clustering can impose discrete groups on continuous variation.
- Cluster labels can introduce meaning not contained in the algorithm.
- A useful partition and a natural category are different scientific claims.
- Clusters depend on representation, distance, algorithm, and tuning choices.
- Cluster labels should not automatically be treated as natural categories.
- Stability and substantive validity matter alongside internal clustering metrics.

12 Dimensionality Reduction: Compressing Data Without Losing the Story

12.1 Learning Objectives

By the end of this chapter, you should be able to:

- explain the curse of dimensionality and why high-dimensional data create statistical and computational challenges;
- distinguish feature selection from feature extraction;
- explain dimensionality reduction as a change of representation;
- calculate variance, covariance, and standardized features;
- explain eigenvectors and eigenvalues in the context of Principal Component Analysis (PCA);
- derive the PCA optimization objective conceptually;
- interpret principal-component scores, loadings, and explained variance;
- distinguish explained variance from predictive or substantive importance;
- use scree plots and cumulative explained variance critically;
- reconstruct data from a reduced PCA representation and interpret reconstruction error;
- explain why scaling can radically change PCA;
- use PCA inside a leakage-safe machine-learning pipeline;
- compare PCA for visualization with PCA for predictive modeling;
- explain the purpose and limitations of t-SNE;
- explain the neighborhood-preserving intuition of UMAP;
- recognize why two-dimensional embeddings can create visually compelling but misleading apparent clusters;
- compare PCA, t-SNE, and UMAP based on purpose rather than visual appeal;
- apply dimensionality reduction responsibly in health, education, and research workflows.
- use dimensionality reduction to explore visual representations without turning embeddings into unsupported class claims.

12.2 Why This Matters

Modern datasets can contain hundreds, thousands, or even millions of features.

A document can be represented by thousands of terms. An image can contain millions of pixel values. Genomic data may contain tens of thousands of measurements. Learning-management systems can generate hundreds of behavioral indicators.

More variables can contain more information.

But more dimensions can also create:

- redundancy;
- noise;
- multicollinearity;
- computational burden;
- unstable models;
- difficult visualization;
- distance concentration;
- interpretive overload.

Dimensionality reduction asks whether we can construct a smaller representation

$$X \in \mathbb{R}^{n \times p} \longrightarrow Z \in \mathbb{R}^{n \times k}, \quad k \ll p,$$

while retaining information useful for our purpose.

Central Question

When we compress a high-dimensional world into fewer dimensions, what information are we preserving—and what are we choosing to forget?

12.3 Feature Selection Versus Feature Extraction

These approaches reduce dimensionality differently.

12.3.1 Feature Selection

Choose a subset of the original variables:

$$(x_1, x_2, \dots, x_p) \rightarrow (x_2, x_5, x_9).$$

The retained variables preserve their original meaning.

12.3.2 Feature Extraction

Construct new variables from combinations of the originals:

$$(x_1, x_2, \dots, x_p) \rightarrow (z_1, z_2, \dots, z_k).$$

PCA is a feature-extraction method.

The new dimensions are not usually identical to familiar substantive variables.

12.4 Why High Dimensions Become Difficult

Imagine uniformly distributed points in increasingly high-dimensional spaces.

As dimensionality grows, data become sparse relative to the volume of the space.

Distance-based intuition can also weaken because nearest and farthest observations may become less distinct under some high-dimensional distributions.

This family of problems is often called the **curse of dimensionality**.

It does not mean that high-dimensional data are unusable. It means that methods, sample sizes, representations, and regularization must respond to the geometry.

12.5 A Representation Problem

Suppose three variables measure related aspects of academic engagement:

- LMS logins;
- assignment activity;
- discussion participation.

They may share substantial information.

Could a smaller set of latent numerical directions summarize much of their variation?

PCA answers this geometrically.

12.6 Principal Component Analysis

Principal component analysis has roots in Pearson's geometric formulation and Hotelling's statistical development of principal components (Pearson 1901; Hotelling 1933).

12.6.1 Begin With Variance

For a variable X with mean $\bar{x}$, sample variance is

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2.$$

Variance describes spread around the mean.

PCA seeks directions through multivariate space along which projected observations have large variance.

12.6.2 Covariance

For variables X_j and X_k ,

$$\text{Cov}(X_j, X_k) = \frac{1}{n-1} \sum_{i=1}^n (x_{ij} - \bar{x}_j)(x_{ik} - \bar{x}_k).$$

Positive covariance indicates that the variables tend to increase together.

Negative covariance indicates opposing movement.

The covariance matrix is

$$\Sigma = \begin{bmatrix} \text{Var}(X_1) & \text{Cov}(X_1, X_2) & \cdots \\ \text{Cov}(X_2, X_1) & \text{Var}(X_2) & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}.$$

12.6.3 Covariance by Hand

```
import numpy as np

X_small = np.array([
    [2.0, 3.0],
    [3.0, 5.0],
    [4.0, 4.0],
    [5.0, 7.0],
    [6.0, 8.0]
])
```

```
np.cov(X_small, rowvar=False)
```

```
array([[2.5, 3. ],  
       [3. , 4.3]])
```

12.6.4 PCA as a Rotation of the Coordinate System

Consider centered observations

$$x_i - \bar{x}.$$

PCA finds a unit vector w_1 such that the projected values

$$z_{i1} = w_1^\top (x_i - \bar{x})$$

have maximum variance.

The first principal component solves

$$\max_{\|w_1\|=1} \text{Var}(Xw_1).$$

The second component finds the direction of greatest remaining variance subject to being orthogonal to the first.

Thus,

$$PC_1 \perp PC_2 \perp \dots \perp PC_p.$$

PCA does not simply delete dimensions.

It rotates the coordinate system and then allows us to retain selected directions.

12.6.5 Eigenvectors and Eigenvalues

For covariance matrix Σ , an eigenvector v satisfies

$$\Sigma v = \lambda v.$$

Here:

- v gives a principal direction;
- λ gives the variance associated with that direction.

Sorting eigenvalues from largest to smallest gives the principal components in decreasing order of explained variance.

If

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p,$$

then the explained-variance ratio of component j is

$$EVR_j = \frac{\lambda_j}{\sum_{k=1}^p \lambda_k}.$$

12.6.6 A Geometric PCA Example

```
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

rng = np.random.default_rng(42)

cov = np.array([
    [3.0, 2.4],
    [2.4, 2.2]
])

X_geom = rng.multivariate_normal(
    mean=[0, 0],
    cov=cov,
    size=300
)
```

```

pca_geom = PCA(n_components=2)
pca_geom.fit(X_geom)

plt.figure(figsize=(8, 6))
plt.scatter(
    X_geom[:, 0],
    X_geom[:, 1],
    alpha=.45,
    s=22
)

mean = pca_geom.mean_

for component, variance in zip(
    pca_geom.components_,
    pca_geom.explained_variance_
):
    endpoint = mean + component * np.sqrt(variance) * 2
    plt.annotate(
        "",
        xy=endpoint,
        xytext=mean,
        arrowprops=dict(arrowstyle="->", linewidth=2)
    )

plt.xlabel("Original Feature 1")
plt.ylabel("Original Feature 2")
plt.title("Principal Directions of Variation")
plt.axis("equal")
plt.show()

```

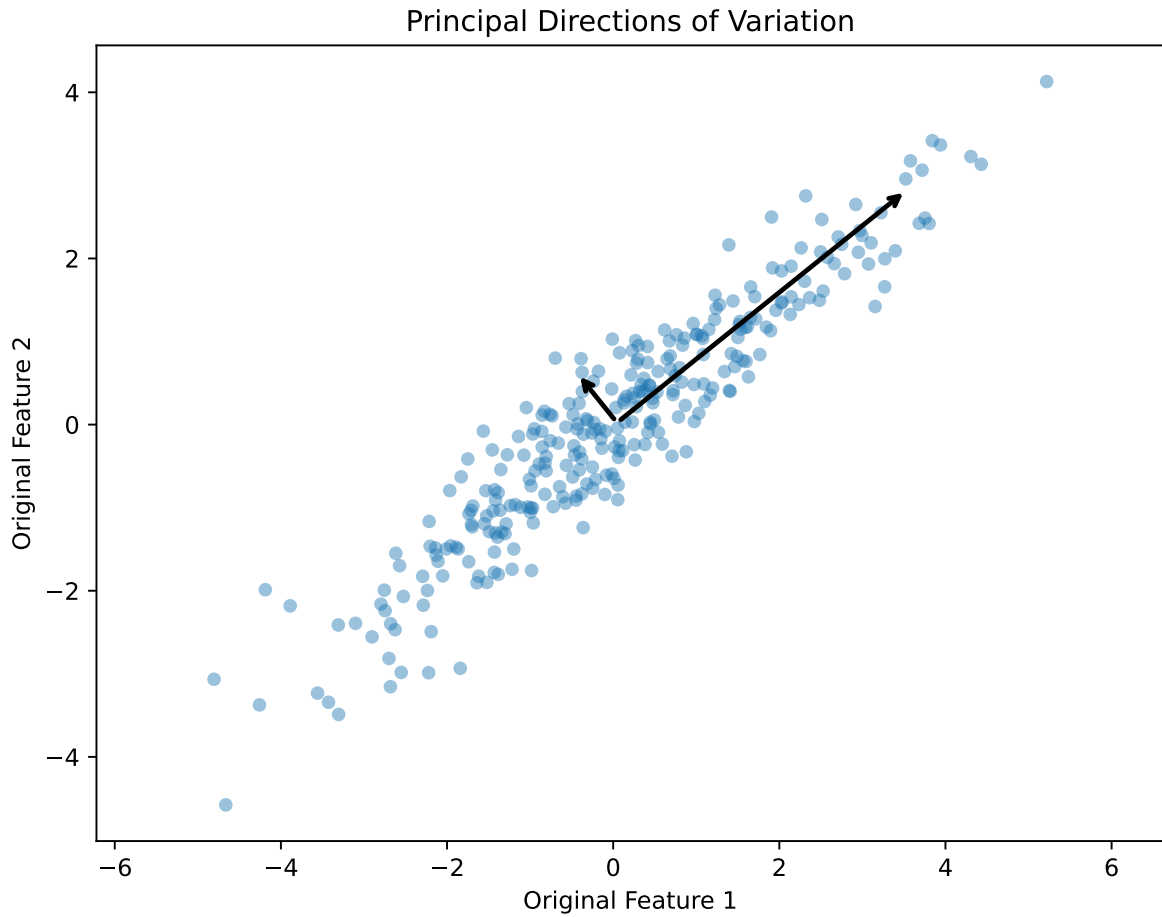


Figure 12.1: PCA identifies orthogonal directions aligned with decreasing variance in the data.

The arrows show directions of variation, not causal effects.

12.6.7 Principal Component Scores

Once the directions are estimated, observations can be transformed:

$$Z = X_c W,$$

where X_c is centered data and columns of W contain selected principal directions.

Each row of Z contains the **component scores** for an observation.

```
Z_geom = pca_geom.transform(X_geom)
```

```
Z_geom[:5]
```

```
array([[ -7.22602736e-01, -4.31764942e-01],  
       [-1.74352843e+00,  3.50756872e-01],  
       [ 4.33914973e+00, -4.05706324e-01],  
       [-3.33703033e-01, -1.25759373e-01],  
       [-3.54877773e-03, -3.36444936e-01]])
```

12.6.8 Loadings

The component coefficients describe how original variables contribute to each principal direction.

In scikit-learn, `components_` contains the principal axes.

```
pca_geom.components_
```

```
array([[ 0.77976061,  0.62607778],  
       [-0.62607778,  0.77976061]])
```

The sign of a principal component is arbitrary: multiplying an eigenvector by -1 describes the same axis.

Therefore, do not attach meaning to component sign without considering the full loading pattern.

12.6.9 Scaling Can Change the Answer

PCA based on the covariance matrix is sensitive to measurement scale.

Consider a variable measured in dollars and another measured on a 1-to-5 scale. The variable with much larger numerical variance may dominate the first components.

For many applications, we therefore standardize:

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}.$$

PCA on standardized variables is closely related to performing PCA on a correlation matrix.

But scaling should be justified rather than automatic.

12.6.10 PCA on a Real Dataset

```
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler

cancer = load_breast_cancer(as_frame=True)
X = cancer.data
y = cancer.target

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

pca = PCA()
X_pca = pca.fit_transform(X_scaled)

pca.explained_variance_ratio_[0:10]
```

```
array([0.44272026, 0.18971182, 0.09393163, 0.06602135, 0.05495768,
       0.04024522, 0.02250734, 0.01588724, 0.01389649, 0.01168978])
```

This full-data transformation is used here for **exploratory demonstration**. When PCA is part of predictive modeling, preprocessing must be learned only from training data.

12.6.11 Scree Plot

```
components = np.arange(
    1,
    len(pca.explained_variance_ratio_) + 1
)

plt.figure(figsize=(9, 5))
plt.plot(
    components,
    pca.explained_variance_ratio_,
    marker="o"
)
plt.xlabel("Principal component")
plt.ylabel("Explained variance ratio")
plt.title("PCA Scree Plot")
plt.show()
```

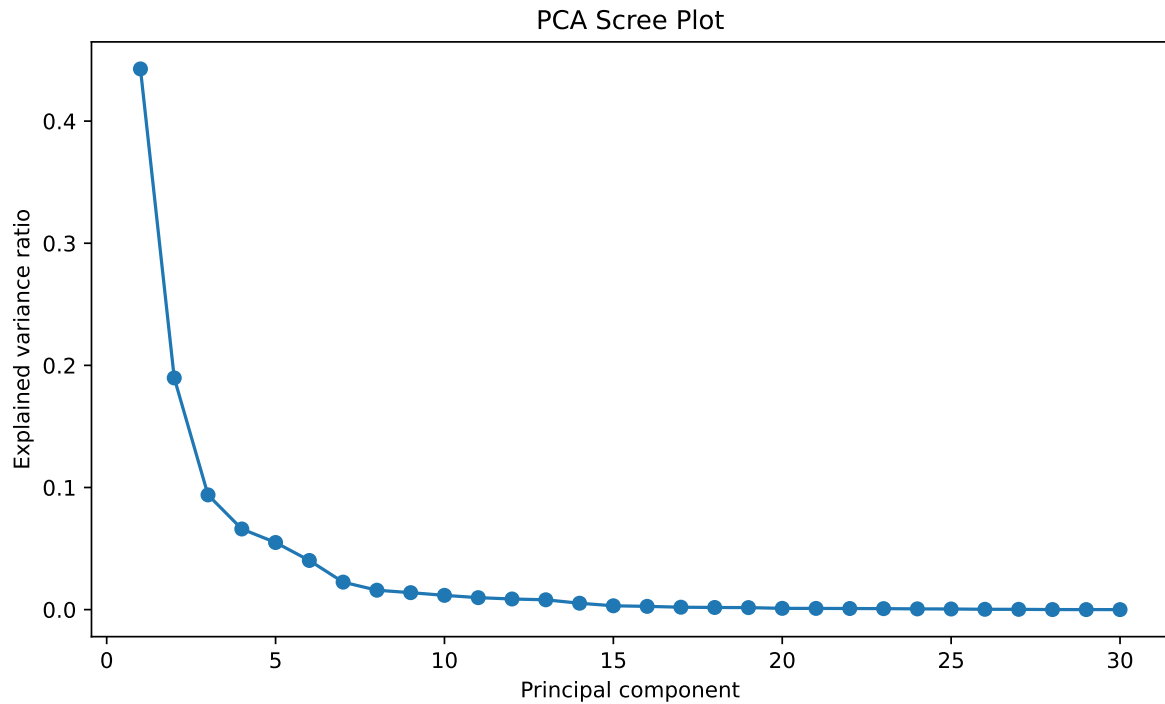



Figure 12.2: A scree plot displays the proportion of variance associated with successive principal components.

A scree plot can help identify diminishing returns, but it does not provide a universally correct number of components.

12.6.12 Cumulative Explained Variance

```
cumulative = np.cumsum(
    pca.explained_variance_ratio_
)

plt.figure(figsize=(9, 5))
plt.plot(
    components,
    cumulative,
    marker="o"
)
plt.axhline(.90, linestyle="--")
```

```
plt.axhline(.95, linestyle="--")
plt.xlabel("Number of principal components")
plt.ylabel("Cumulative explained variance")
plt.title("Cumulative Explained Variance")
plt.show()
```

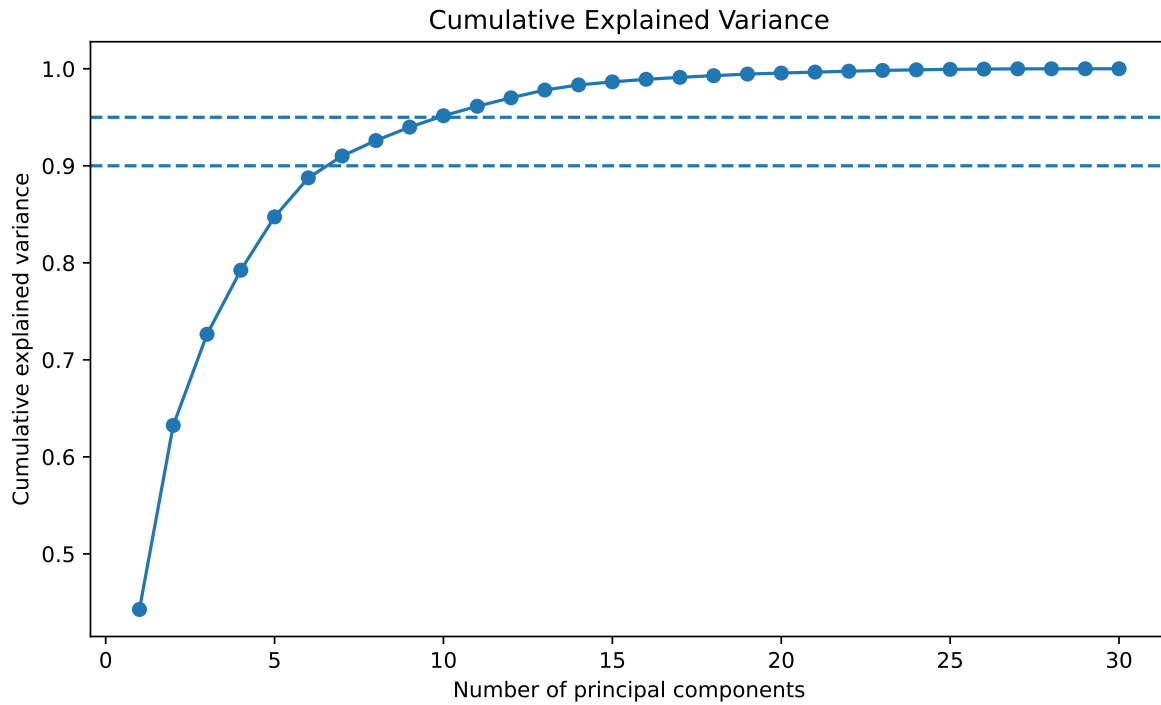


Figure 12.3: Cumulative explained variance shows how much total variance is retained as components are added.

Thresholds such as 90% or 95% are conventions, not laws.

The right amount of retained variance depends on purpose.

12.6.13 Explained Variance Is Not Importance

A principal component explaining substantial variance captures a direction along which predictors vary.

It does not necessarily capture:

- the features most predictive of an outcome;

- the variables most scientifically important;
- the variables most actionable;
- causal mechanisms.

A low-variance direction can sometimes contain strong predictive signal.

Thus,

$$\boxed{\text{explained predictor variance}} \neq \boxed{\text{outcome relevance}}.$$

12.6.14 Visualizing Observations With PCA

```
plt.figure(figsize=(8, 6))
plt.scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    c=y,
    cmap="coolwarm",
    alpha=.65,
    s=25
)
plt.xlabel(
    f"PC1 ({pca.explained_variance_ratio_[0]:.1%})"
)
plt.ylabel(
    f"PC2 ({pca.explained_variance_ratio_[1]:.1%})"
)
plt.title("PCA Projection")
plt.show()
```

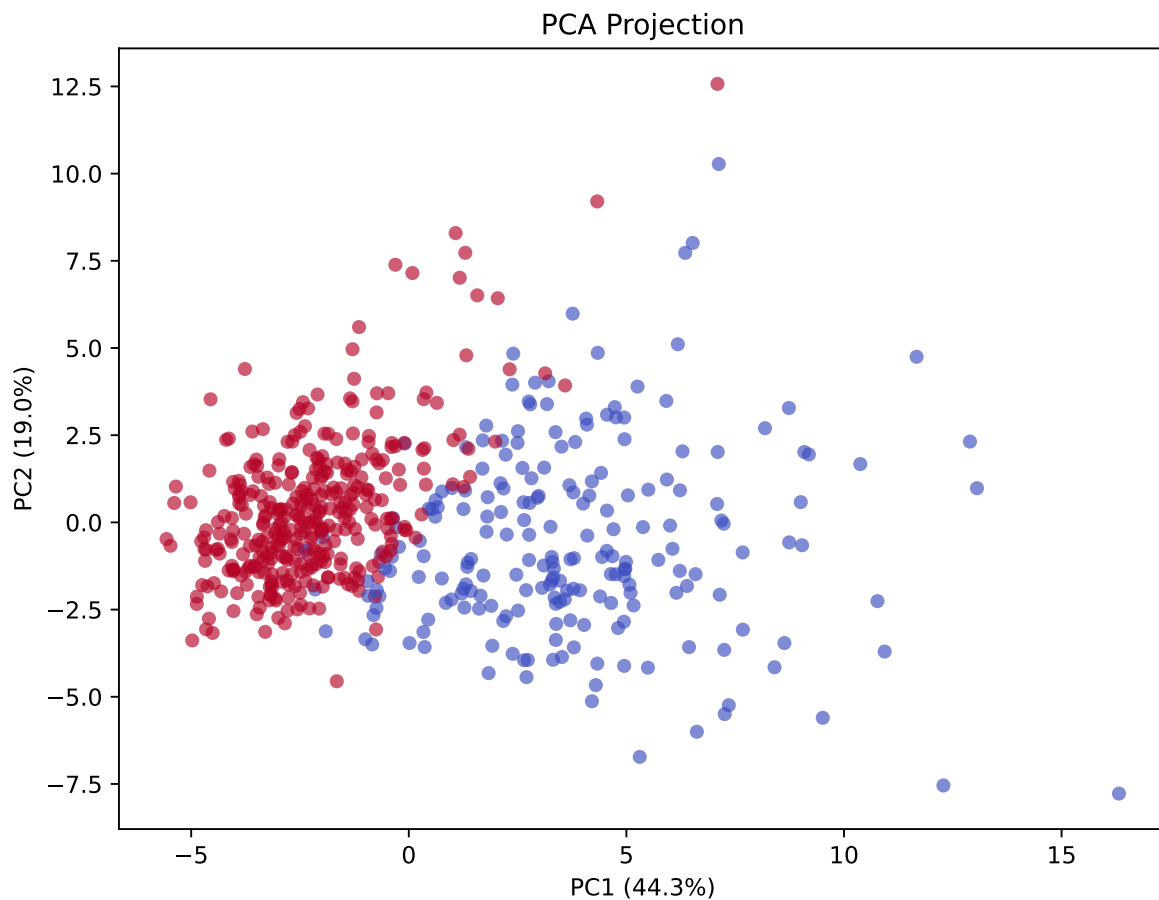


Figure 12.4: The first two principal components provide a two-dimensional projection of the standardized feature space.

The class labels are used only for coloring after the unsupervised PCA transformation.

PCA itself did not use y .

12.6.15 Interpreting Loadings

```
import pandas as pd

loading_df = pd.DataFrame(
    pca.components_[:3].T,
    index=X.columns,
    columns=["PC1", "PC2", "PC3"]
)
```

```
)

loading_df.loc[
    loading_df["PC1"].abs().sort_values(
        ascending=False
    ).index
].head(10)
```

	PC1	PC2	PC3
mean concave points	0.260854	-0.034768	-0.025564
mean concavity	0.258400	0.060165	0.002734
worst concave points	0.250886	-0.008257	-0.170344
mean compactness	0.239285	0.151892	-0.074092
worst perimeter	0.236640	-0.199878	-0.048547
worst concavity	0.228768	0.097964	-0.173057
worst radius	0.227997	-0.219866	-0.047507
mean perimeter	0.227537	-0.215181	-0.009314
worst area	0.224871	-0.219352	-0.011902
mean area	0.220995	-0.231077	0.028700

A component may combine many correlated measurements.

Interpretation should consider:

- magnitude;
- sign pattern;
- measurement meaning;
- whether variables were standardized;
- whether the component is stable across samples.

12.6.16 Reconstruction

If we retain only $k < p$ components,

$$Z_k = X_c W_k.$$

An approximation to the original centered data is

$$\hat{X}_c = Z_k W_k^\top.$$

Information outside the retained subspace is lost.

```
pca_5 = PCA(n_components=5)
Z_5 = pca_5.fit_transform(X_scaled)

X_reconstructed = pca_5.inverse_transform(Z_5)

reconstruction_mse = np.mean(
    (X_scaled - X_reconstructed) ** 2
)

reconstruction_mse
```

```
np.float64(0.15265725683192763)
```

This gives dimensionality reduction a concrete interpretation:

compression trades representational detail for simplicity.

12.6.17 Reconstruction Error Across Dimensions

```
component_counts = range(1, 16)
errors = []

for k in component_counts:
    pca_k = PCA(n_components=k)
    z_k = pca_k.fit_transform(X_scaled)
    x_hat = pca_k.inverse_transform(z_k)

    errors.append(
        np.mean((X_scaled - x_hat) ** 2)
    )

plt.figure(figsize=(8, 5))
plt.plot(
    list(component_counts),
    errors,
    marker="o"
)
plt.xlabel("Number of retained components")
```

```
plt.ylabel("Reconstruction MSE")
plt.title("Compression Versus Reconstruction")
plt.show()
```

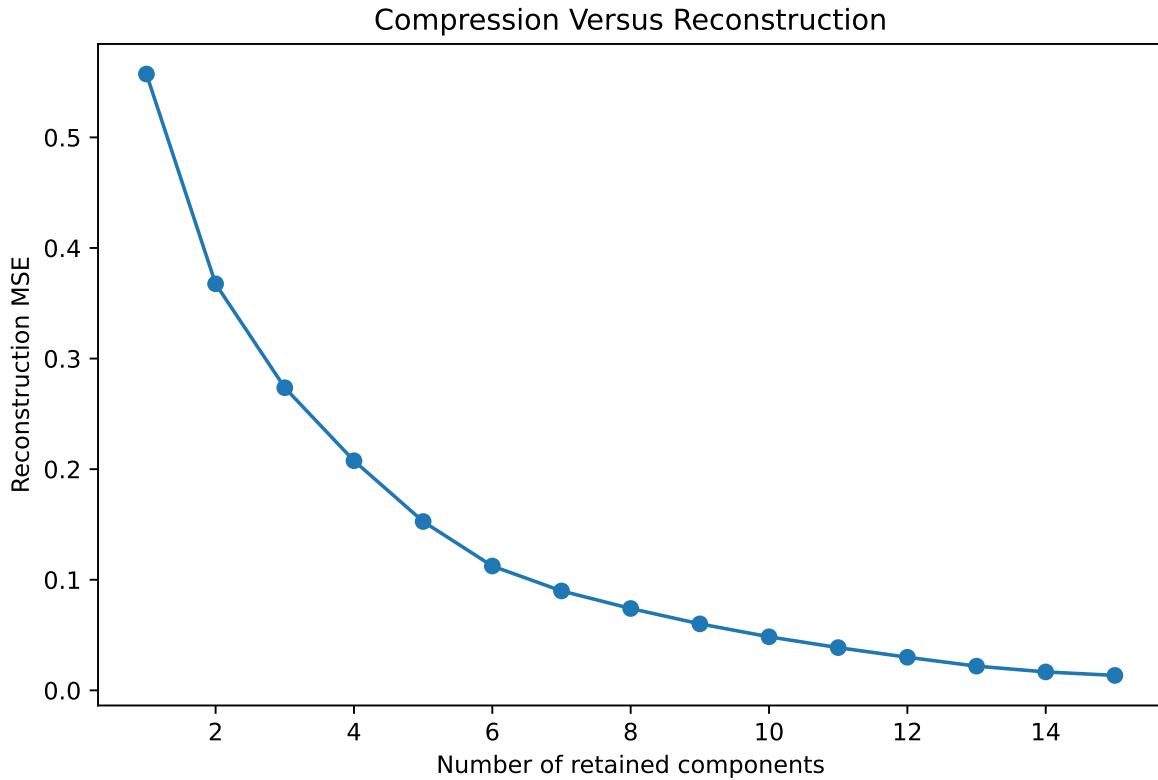


Figure 12.5: Reconstruction error decreases as more principal components are retained.

12.6.18 PCA in Predictive Modeling

If PCA is part of a supervised-learning workflow, both scaling and PCA must be fitted inside the training pipeline.

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score

X_train, X_test, y_train, y_test = train_test_split(
    X,
```

```

    y,
    test_size=.25,
    random_state=42,
    stratify=y
)

pca_logit = Pipeline([
    ("scale", StandardScaler()),
    ("pca", PCA(n_components=.95)),
    ("model", LogisticRegression(max_iter=2000))
])

pca_logit.fit(X_train, y_train)

pca_prob = pca_logit.predict_proba(
    X_test
)[: , 1]

roc_auc_score(y_test, pca_prob)

```

```
np.float64(0.9972746331236897)
```

The test data did not influence the scaling parameters or principal directions.

12.6.19 Does PCA Improve Prediction?

Not necessarily.

Compare a logistic model with and without PCA.

```

plain_logit = Pipeline([
    ("scale", StandardScaler()),
    ("model", LogisticRegression(max_iter=2000))
])

plain_logit.fit(X_train, y_train)

plain_prob = plain_logit.predict_proba(
    X_test
)[: , 1]

```



```
{
    "Logistic regression":
        roc_auc_score(y_test, plain_prob),
    "PCA + logistic regression":
        roc_auc_score(y_test, pca_prob)
}
```

```
{'Logistic regression': np.float64(0.9976939203354298),
 'PCA + logistic regression': np.float64(0.9972746331236897)}
```

PCA can improve computational efficiency, reduce collinearity, or regularize a representation.

It can also discard predictive information.

It must earn its place empirically.

12.7 Dimensionality Reduction Is an Information Choice

Dimensionality reduction creates a lower-dimensional representation

$$X \in \mathbb{R}^{n \times p} \rightarrow Z \in \mathbb{R}^{n \times k}, \quad k < p.$$

The purpose may be visualization, compression, denoising, feature extraction, or computational efficiency.

But reducing dimension means deciding what information should be preserved.

12.7.1 PCA Preserves Variance, Not Meaning

PCA identifies directions that capture large amounts of variance.

That is mathematically useful, but high variance is not synonymous with substantive importance.

A low-variance feature could still be:

- clinically important;
- highly predictive for a minority group;
- essential for identifying rare events;
- scientifically meaningful;
- important for fairness analysis.

Thus,

Variance Preserved $\neq$ Meaning Preserved

12.7.2 Reconstruction Error Makes the Tradeoff Visible

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

X_pca_ic = StandardScaler().fit_transform(X)

max_components_ic = min(
    X_pca_ic.shape[1],
    12
)

components_ic = range(
    1,
    max_components_ic + 1
)

errors_ic = []

for k_ic in components_ic:
    pca_ic = PCA(
        n_components=k_ic
    )

    Z_ic = pca_ic.fit_transform(
        X_pca_ic
    )

    X_reconstructed_ic = pca_ic.inverse_transform(
        Z_ic
    )

    mse_ic = np.mean(
        (X_pca_ic - X_reconstructed_ic) ** 2
```

```

    )

    errors_ic.append(
        mse_ic
    )

plt.figure(figsize=(8, 5))
plt.plot(
    list(components_ic),
    errors_ic,
    marker="o"
)
plt.xlabel("Number of retained components")
plt.ylabel("Mean reconstruction error")
plt.title("Compression Versus Reconstruction")
plt.show()

```

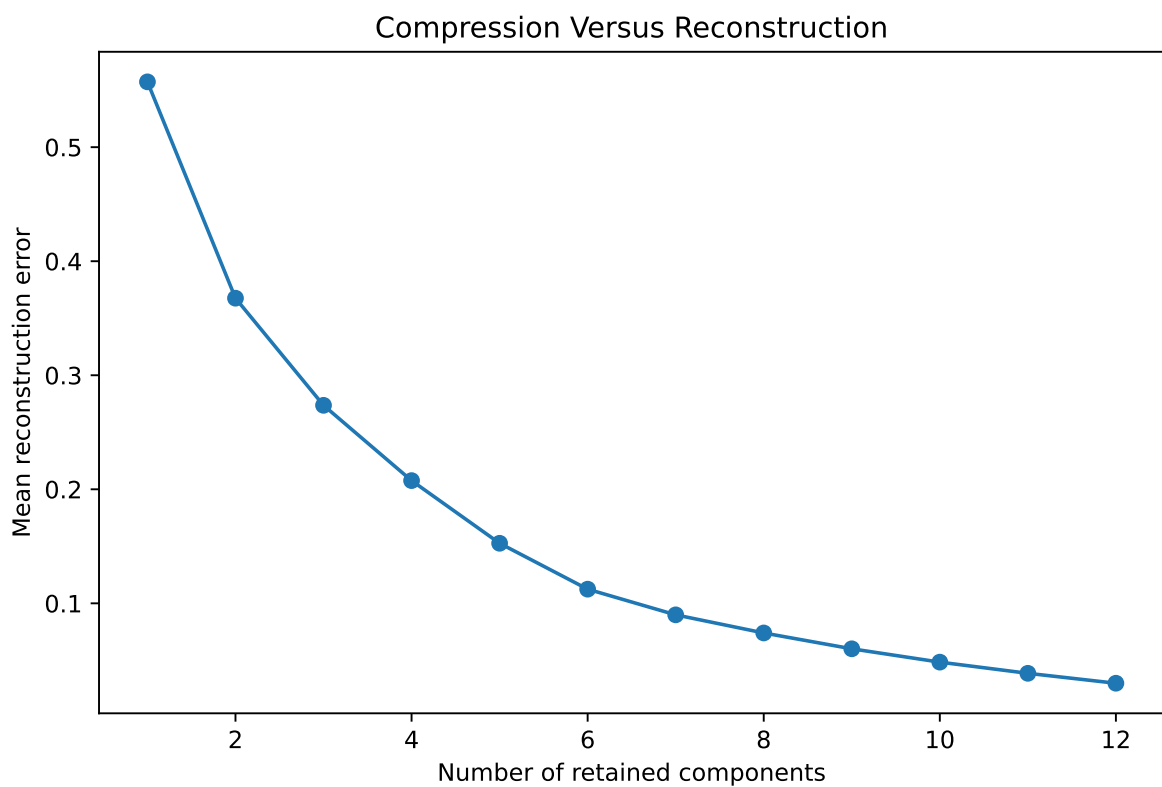


Figure 12.6: Retaining more principal components reduces reconstruction error, illustrating the tradeoff between compression and retained information.

12.7.3 Visualization Is Not the Data

Nonlinear methods such as t-SNE can produce compelling two-dimensional embeddings.

But:

Two-Dimensional Embedding $\neq$ The Original Data Geometry

For t-SNE in particular:

- local neighborhoods are often more meaningful than global distances;
- apparent gaps should not automatically be interpreted as true population separation;
- cluster size can be visually misleading;
- different seeds or hyperparameters can change the embedding.

A visually persuasive map is therefore an analytical representation, not direct observation of an underlying truth.

12.8 Nonlinear Embeddings

PCA is linear.

Many high-dimensional datasets may contain nonlinear structure.

Methods such as t-SNE and UMAP are especially popular for producing low-dimensional visualizations.

Their beautiful plots require careful interpretation.

12.8.1 t-SNE

t-distributed Stochastic Neighbor Embedding (t-SNE) seeks a low-dimensional representation that preserves local neighborhood relationships probabilistically (Maaten and Hinton 2008).

At a high level:

1. convert high-dimensional pairwise relationships into neighborhood probabilities;
2. create corresponding low-dimensional relationships;
3. optimize the embedding so these neighborhood structures are similar.

A common objective is based on Kullback-Leibler divergence:

$$KL(P\|Q) = \sum_{i \neq j} p_{ij} \log \left(\frac{p_{ij}}{q_{ij}} \right).$$

The goal is not to preserve every original distance.

12.8.2 t-SNE Example

```
#| warning: false

from sklearn.manifold import TSNE

X_scaled_tsne = StandardScaler().fit_transform(X)

tsne = TSNE(
    n_components=2,
    perplexity=30,
    init="pca",
    learning_rate="auto",
    random_state=42
)

X_tsne = tsne.fit_transform(X_scaled_tsne)

plt.figure(figsize=(8, 6))
plt.scatter(
    X_tsne[:, 0],
    X_tsne[:, 1],
    c=y,
    cmap="coolwarm",
    alpha=.65,
    s=25
)
plt.xlabel("t-SNE dimension 1")
plt.ylabel("t-SNE dimension 2")
plt.title("t-SNE Embedding")
plt.show()
```

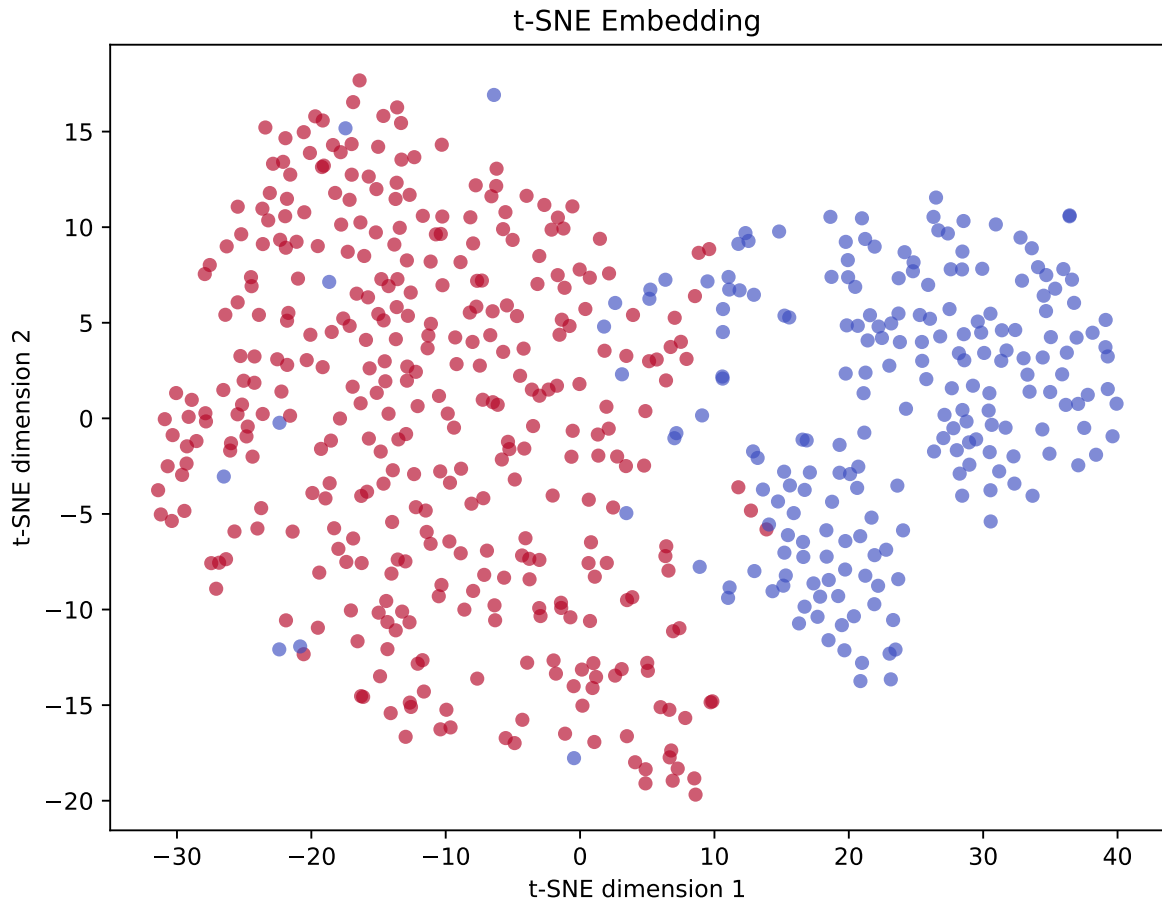


Figure 12.7: t-SNE can reveal local neighborhood structure, but distances and apparent gaps in the two-dimensional embedding require cautious interpretation.

The axes do not have the same direct interpretation as PCA axes.

12.8.3 Perplexity

Perplexity influences the effective neighborhood scale considered by t-SNE.

Different perplexity values can produce different visual structures.

Therefore, a responsible t-SNE analysis should examine sensitivity rather than presenting one attractive embedding as definitive.

12.8.4 t-SNE Is Primarily a Visualization Tool

A common misuse is:

1. run t-SNE;
2. see islands;
3. declare that natural clusters have been discovered.

That conclusion does not follow automatically.

The embedding itself was optimized to represent neighborhood relationships in a low-dimensional display.

Apparent visual clusters may depend on:

- preprocessing;
- perplexity;
- initialization;
- random seed;
- sample composition;
- other optimization settings.

12.8.5 UMAP

Uniform Manifold Approximation and Projection (UMAP) is another nonlinear dimensionality-reduction method widely used for visualization and representation learning.

Conceptually, UMAP attempts to construct a graph representing local high-dimensional relationships and optimize a lower-dimensional graph-like representation (McInnes, Healy, and Melville 2018; UMAP Developers 2026).

Two widely discussed parameters are:

- `n_neighbors` — neighborhood scale;
- `min_dist` — how tightly observations may be packed in the embedding.

UMAP is provided by the optional `umap-learn` package rather than core scikit-learn.

Optional Modern Extension

The core chapter does not require `umap-learn`, keeping the main book environment lightweight.

Students who install it can run:

```

from umap import UMAP

umap_model = UMAP(
    n_neighbors=15,
    min_dist=0.1,
    n_components=2,
    random_state=42
)

X_umap = umap_model.fit_transform(X_scaled)

```

As with t-SNE, a visually separated embedding is not by itself evidence of natural categories.

12.9 PCA, t-SNE, or UMAP?

Method	Main strength	Global geometry	Interpretability	Typical role
PCA	linear compression	relatively strong	comparatively high	compression, preprocessing, visualization
t-SNE	local visualization	weak	low	exploratory visualization
UMAP	local/manifold representation	partial	low	visualization, nonlinear representation

No method dominates for every purpose.

The correct question is not:

Which one makes the prettiest plot?

It is:

Which representation preserves the information relevant to the analytical task?

12.10 A Visualization Can Manufacture Confidence

Two-dimensional plots are cognitively powerful.

Humans readily see:

- islands;
- boundaries;
- gradients;
- outliers;
- apparent categories.

But a 2D embedding is a transformation of a much larger space.

Visual clarity can exceed evidential clarity.

This is especially dangerous when an embedding is colored by known outcomes after the fact.

12.11 Stress-Test the Representation

A responsible dimensionality-reduction workflow should ask:

- Does the structure persist under different preprocessing?
- Does it persist under different seeds?
- Does it persist across plausible parameter values?
- Does it replicate in another sample?
- Is the pattern also visible through other methods?
- Does the low-dimensional representation preserve what the application needs?
- Are conclusions based on visualization alone?

12.12 Computer Vision Learning Experience: Projecting Image Space

An (8×8) grayscale digit contains (64) pixel values. PCA can compress this visual feature space into fewer dimensions.

```
from sklearn.datasets import load_digits
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
```

```

digits_pca_cv = load_digits()

X_pca_cv = StandardScaler().fit_transform(digits_pca_cv.data)
y_pca_cv = digits_pca_cv.target

pca_digits_cv = PCA(n_components=2)
Z_pca_cv = pca_digits_cv.fit_transform(X_pca_cv)

plt.figure(figsize=(8, 6))
plt.scatter(
    Z_pca_cv[:, 0],
    Z_pca_cv[:, 1],
    c=y_pca_cv,
    cmap="tab10",
    s=20,
    alpha=0.65
)
plt.xlabel("Principal component 1")
plt.ylabel("Principal component 2")
plt.title("PCA Projection of Handwritten Digits")
plt.colorbar(label="Digit label")
plt.show()

```

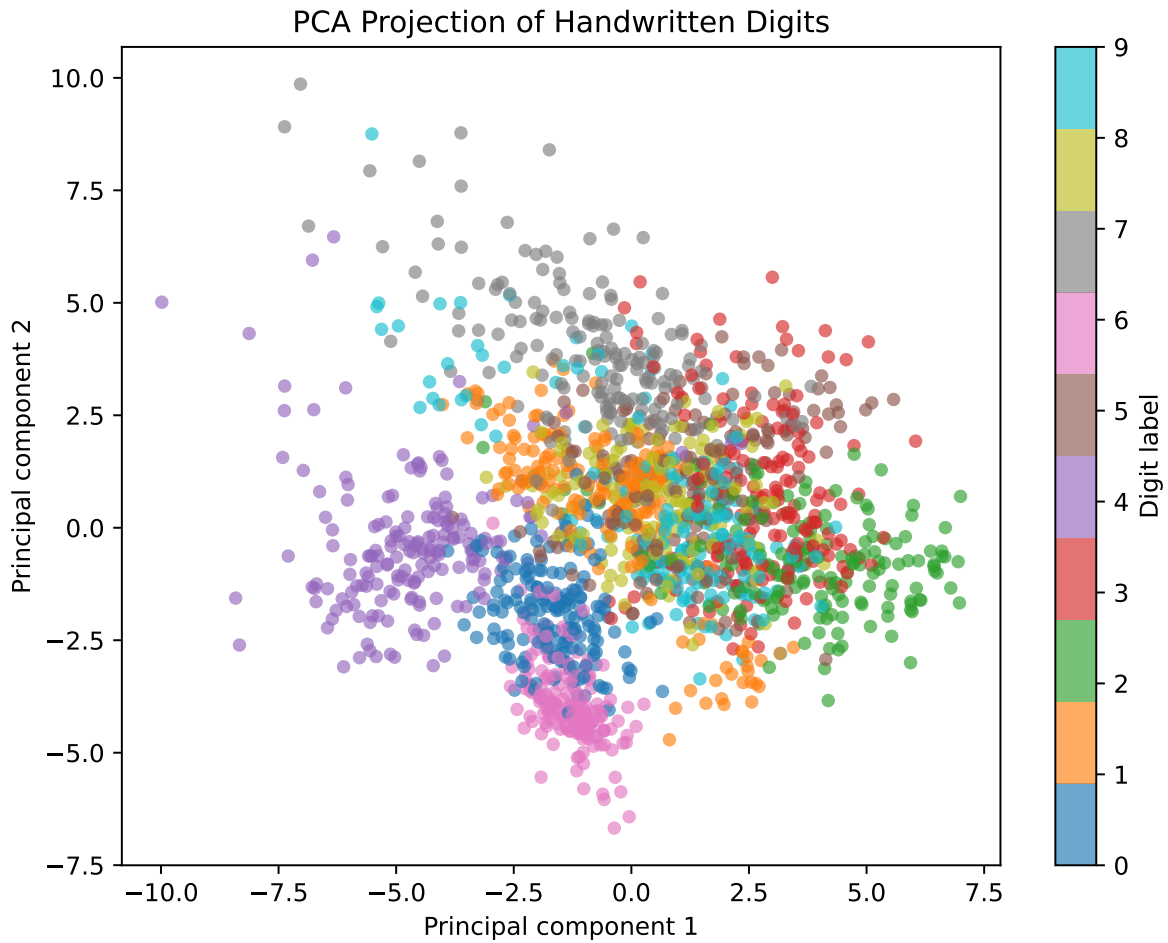


Figure 12.8: Two-dimensional PCA projection of handwritten-digit images. Color uses known labels only to help interpret the projection.

The colors were not used by PCA. They are added afterward to ask whether the unsupervised representation aligns with known semantic classes.

Visible Separation in an Embedding $\neq$ Perfect Class Separability

Compare this PCA representation with the t-SNE example already in the chapter. Ask which structure each method preserves, which patterns remain ambiguous, how sensitive the display is to analytical choices, and whether conclusions would survive a change in image source.

The goal is not to select the prettiest plot. The goal is to understand what each representation preserves.

12.13 Three Tracks

12.13.1 Health Track

PCA can summarize correlated biomarkers or physiological measurements. Nonlinear embeddings can visualize patient heterogeneity. But a visual patient subgroup is not a diagnosis. External clinical validity, stability, calibration of downstream models, and population transportability remain necessary.

Health: compression may obscure rare but clinically important signals.

12.13.2 Education Track

Dimensionality reduction can summarize correlated engagement indicators, survey scales, or learning behaviors. A two-dimensional student map can support exploration, but spatial proximity does not automatically imply equivalent learning needs or justify intervention.

Education: components explaining large overall variance may not preserve information relevant to smaller student populations.

12.13.3 Open ML Benchmark Track

High-dimensional benchmark datasets allow controlled comparison of retained variance, reconstruction, predictive performance, neighborhood preservation, runtime, and sensitivity to embedding parameters.

Open ML: visualization can help exploration, but apparent clusters should be validated in the original feature space or with independent evidence.

Research Reality Check

A two-dimensional embedding that shows three visually separated clouds does not, by itself, establish three meaningful clusters.

Dimensionality reduction and clustering answer different questions. Do not turn a visualization into a population taxonomy without additional evidence.

12.14 Beyond the Algorithm: What Must Be Forgotten?

Compression Is a Choice About Memory

Dimensionality reduction is often described as removing unnecessary information. But the algorithm does not know what humanity considers unnecessary. PCA preserves directions of variance. t-SNE emphasizes local neighborhoods. UMAP encodes another geometric view of neighborhood structure. Each method remembers some relationships more faithfully than others. This suggests a broader principle:

Every representation is simultaneously a way of seeing and a way of not seeing.

A map leaves out details so that geography becomes comprehensible.
A statistical model leaves out variation so that structure becomes estimable.
A scientific theory leaves out parts of reality so that explanation becomes possible.
The intellectual question is therefore not whether we simplify.
We must.
The question is:
Did we discard information that mattered for the claim we eventually made?

12.15 AI Pair Programmer: I Found Three Patient Types

Give an AI assistant a two-dimensional t-SNE plot containing three visually separated regions.
Tell it:

“The t-SNE plot clearly proves that our population consists of three biologically distinct patient types.”

Audit the response.

A strong critique should ask about:

- preprocessing;
- dimensionality;
- perplexity;
- random seed;
- visual versus quantitative evidence;
- stability;

- clustering performed independently of the embedding;
- external biological validation;
- replication;
- continuous alternatives;
- whether t-SNE preserved relevant global relationships.

Rewrite the claim into defensible scientific language.

12.16 Hands-On Lab: Compress, Visualize, Predict

Using a health, education, or benchmark dataset:

1. define the analytical purpose;
2. complete the Feature Ledger;
3. examine feature scales and correlations;
4. standardize where justified;
5. fit PCA;
6. inspect eigenvalue/explained-variance behavior;
7. create scree and cumulative-variance plots;
8. inspect component loadings;
9. reconstruct the data using several values of k ;
10. quantify reconstruction error;
11. create a two-component PCA visualization;
12. fit a supervised baseline without PCA;
13. fit the same model with leakage-safe PCA;
14. compare predictive performance and dimensionality;
15. create a t-SNE visualization;
16. vary at least one t-SNE parameter;
17. optionally create a UMAP visualization;
18. compare the stories told by the representations;
19. document what each representation cannot establish.

12.17 Practitioner Track

A practitioner should be able to use PCA for compression and preprocessing, select dimensionality based on task requirements, build leakage-safe pipelines, and treat nonlinear embeddings primarily as exploratory representations.

12.18 Researcher Track

A researcher should additionally examine loading stability, sensitivity to scaling, reconstruction, parameter sensitivity, replication, downstream inferential consequences, and whether visual structures support substantive claims.

12.19 Research Studio

Design a dimensionality-reduction study suitable for a conference or journal paper.

Specify:

1. substantive problem;
2. high-dimensional feature source;
3. unit of analysis;
4. preprocessing;
5. scaling rationale;
6. PCA or alternative method;
7. dimensionality-selection strategy;
8. loading interpretation;
9. reconstruction assessment;
10. downstream prediction task if applicable;
11. leakage-safe pipeline;
12. nonlinear visualization strategy;
13. sensitivity analyses;
14. stability assessment;
15. external validation;
16. interpretive limitations;
17. reproducibility plan;
18. potential scholarly contribution.

12.20 Professor's Challenge: The Two-Dimensional Truth

Researchers collect 150 behavioral variables from 8,000 university students.

They standardize the variables, run t-SNE once using default settings, and obtain four visually separated islands.

They name the islands:

1. Deep Learners
2. Strategic Learners

3. Passive Learners
4. At-Risk Learners

They report:

“Machine learning discovered four naturally occurring student learning styles.”

The university proposes assigning students to different curricula based on the visualization.

Critique:

- representation choices;
- standardization;
- use of t-SNE;
- parameter sensitivity;
- stochasticity;
- visual cluster inference;
- absence of an independent clustering analysis;
- cluster stability;
- labeling;
- construct validity;
- continuous alternatives;
- external validation;
- intervention evidence;
- ethical consequences.

Design a stronger research program before any curricular assignment.

12.21 Knowledge Check

1. What is dimensionality reduction?
2. How does feature extraction differ from feature selection?
3. What is the curse of dimensionality?
4. What does covariance measure?
5. What does PCA optimize?
6. What is an eigenvector in PCA?
7. What does its eigenvalue represent?
8. Why are principal components orthogonal?
9. What is a component score?
10. What are PCA loadings?
11. Why can scaling radically change PCA?
12. What does explained-variance ratio measure?
13. Why is explained variance not equivalent to predictive importance?

14. What is reconstruction error?
15. Why must PCA be inside a predictive pipeline?
16. What does t-SNE attempt to preserve?
17. What does perplexity influence?
18. Why should distances between t-SNE islands be interpreted cautiously?
19. What is UMAP used for?
20. Why can a compelling 2D visualization support an unjustified scientific claim?
21. Why does preserving variance not guarantee preserving substantive meaning?
22. What does reconstruction error tell us about dimensionality reduction?
23. Why should distances and gaps in t-SNE plots be interpreted cautiously?

12.22 Reproducibility Check

Record:

- analytical purpose;
- sample;
- feature inclusion/exclusion;
- missing-data treatment;
- transformations;
- scaling;
- covariance/correlation choice;
- PCA component count;
- explained-variance criterion;
- loading calculations;
- reconstruction analysis;
- pipeline placement;
- predictive validation design;
- t-SNE parameters;
- UMAP parameters if used;
- random seeds;
- sensitivity analyses;
- software/package versions;
- code producing every figure and table.

12.23 Key Takeaways

- Dimensionality reduction constructs a smaller representation of high-dimensional data.
- Feature selection retains original variables; feature extraction creates new dimensions.
- PCA identifies orthogonal directions of decreasing variance.

- Eigenvectors define principal directions and eigenvalues quantify their variance.
- Scaling changes PCA geometry and must be justified.
- Explained variance does not equal predictive, causal, or substantive importance.
- Reconstruction makes information loss measurable.
- PCA used for prediction must be fitted within the training pipeline.
- t-SNE emphasizes local neighborhood visualization rather than faithful preservation of all distances.
- UMAP offers a modern nonlinear representation based on neighborhood structure.
- Attractive embeddings can create unjustified confidence in apparent categories.
- Representation sensitivity and external validation matter.
- Compression is not merely technical: it determines what information remains available for subsequent reasoning.
- Dimensionality reduction chooses what information to preserve and what to discard.
- PCA preserves variance rather than substantive meaning.
- Low-dimensional visualizations are representations, not the original geometry.

12.24 Looking Ahead

Dimensionality reduction makes representation explicit: what a model can learn depends on what information its representation preserves. Chapter 13 extends that idea from constructed low-dimensional representations to **learned representations**, showing how neural networks build increasingly useful internal features through layers, nonlinear transformations, and optimization.

Part V

Part V — Modern Machine Learning

13 Neural Networks: Learning Representations Through Layers

13.1 Learning Objectives

By the end of this chapter, you should be able to:

- connect neural networks to linear and logistic models;
- describe a neuron using weights, bias, and an activation function;
- explain why nonlinear activation functions are necessary;
- calculate forward propagation through a small network;
- distinguish sigmoid, tanh, ReLU, and softmax;
- connect output layers to appropriate loss functions;
- explain gradient descent and learning rate;
- use the chain rule to explain backpropagation;
- distinguish epochs, batches, parameters, and hyperparameters;
- explain initialization, regularization, dropout, and early stopping;
- build and evaluate an MLP with scikit-learn;
- explain learned representations;
- recognize when a neural network is unnecessary.
- train a compact neural-network computer-vision model and connect it to learned visual representations.

13.2 Why This Matters

A linear model learns

$$\hat{y} = w^\top x + b.$$

A neural network composes familiar operations:

weighted sums $\rightarrow$ nonlinear transformations $\rightarrow$ loss $\rightarrow$ optimization .

Hidden layers can construct new representations useful for prediction.

Central Question

When a model learns its own representations, have we removed human judgment—or moved human judgment into the architecture, data, objective, and evaluation?

13.2.1 From Representation to Representation Learning

Chapter 12 showed that changing a representation can reveal, compress, or distort structure in data. Neural networks take the next step: rather than relying only on representations chosen in advance, they can learn intermediate representations jointly with the predictive task. This connection is central to understanding why deep learning became influential in vision, language, and other high-dimensional domains.

13.3 From Logistic Regression to a Neuron

Logistic regression computes

$$z = w^\top x + b$$

and

$$\hat{p} = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

A neuron generalizes this pattern:

$$z = \sum_{j=1}^p w_j x_j + b, \quad a = \phi(z).$$

13.4 A Neuron by Hand

```
import numpy as np
x=np.array([0.8,-0.4,1.2])
w=np.array([0.5,-0.7,0.3])
b=0.1
z=w@x+b
z
```

```
np.float64(1.1400000000000001)
```

```
sigmoid=lambda z: 1/(1+np.exp(-z))
sigmoid(z)
```

```
np.float64(0.7576796390037049)
```

13.5 Hidden Layers and Representation Learning

A central strength of neural networks is their ability to learn intermediate representations rather than rely only on manually specified features (Bengio, Courville, and Vincent 2013; I. Goodfellow, Bengio, and Courville 2016).

A hidden layer computes

$$h = \phi(W^{(1)}x + b^{(1)}),$$

followed by an output layer

$$\hat{y} = g(W^{(2)}h + b^{(2)}).$$

The hidden vector h is a learned representation of the original input.

13.6 Why Nonlinearity Is Essential

Without nonlinear activation,

$$h = W_1x + b_1$$

and

$$y = W_2 h + b_2$$

collapse to

$$y = (W_2 W_1) x + (W_2 b_1 + b_2),$$

which is still linear.

Depth without nonlinearity collapses back into linearity.

13.7 Activation Functions

13.7.1 Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

13.7.2 Hyperbolic Tangent

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}.$$

13.7.3 ReLU

$$\text{ReLU}(z) = \max(0, z).$$

13.7.4 Softmax

For multiclass logits,

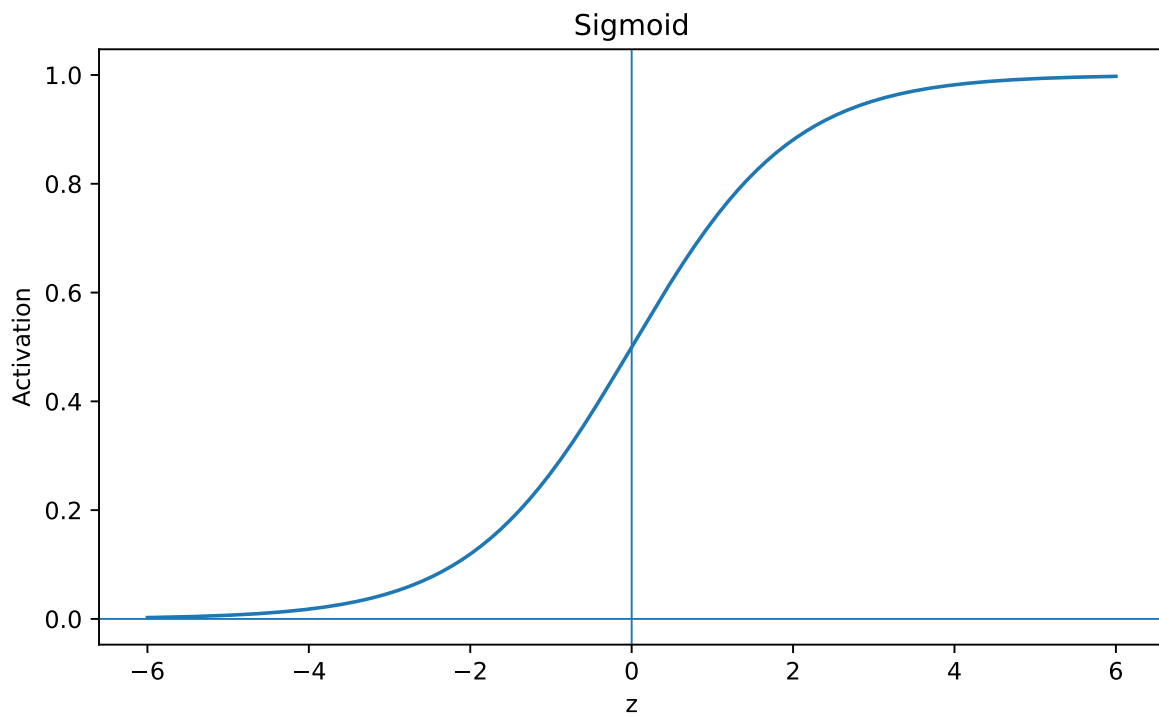
$$P(Y = k \mid x) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}.$$

13.8 Visualize Common Activations

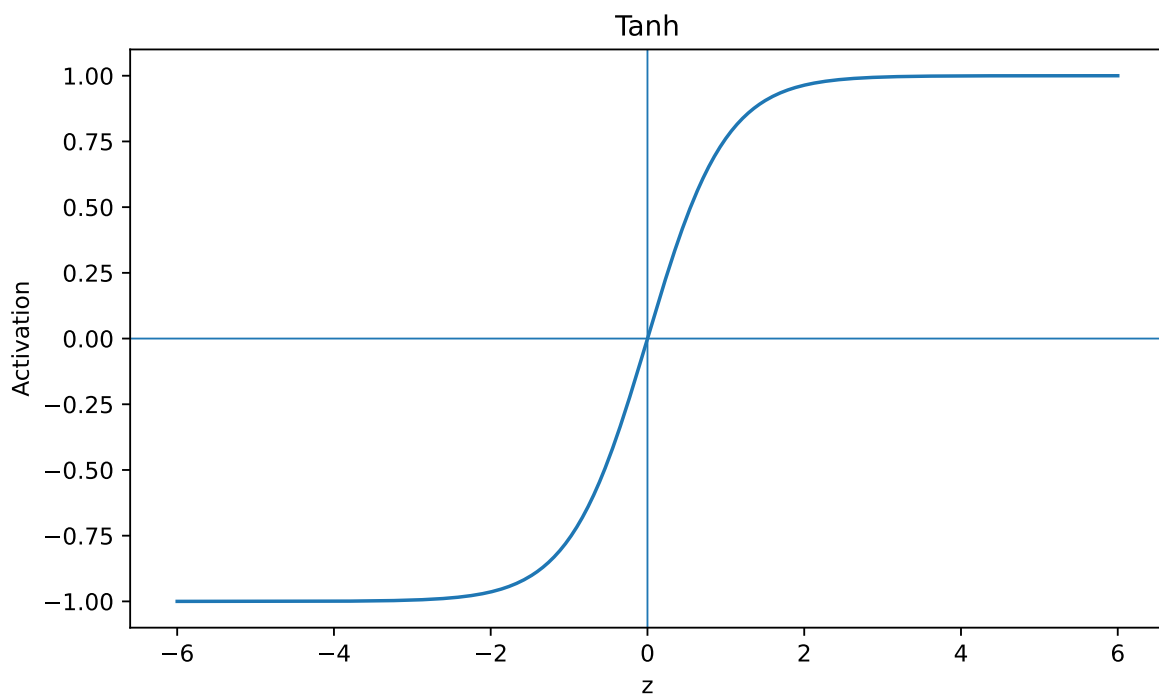
```

import matplotlib.pyplot as plt
z_grid=np.linspace(-6,6,500)
functions={
    "Sigmoid":1/(1+np.exp(-z_grid)),
    "Tanh":np.tanh(z_grid),
    "ReLU":np.maximum(0,z_grid)
}
for name,values in functions.items():
    plt.figure(figsize=(8,4.5))
    plt.plot(z_grid,values)
    plt.axhline(0,linewidth=.8)
    plt.axvline(0,linewidth=.8)
    plt.xlabel("z")
    plt.ylabel("Activation")
    plt.title(name)
    plt.show()

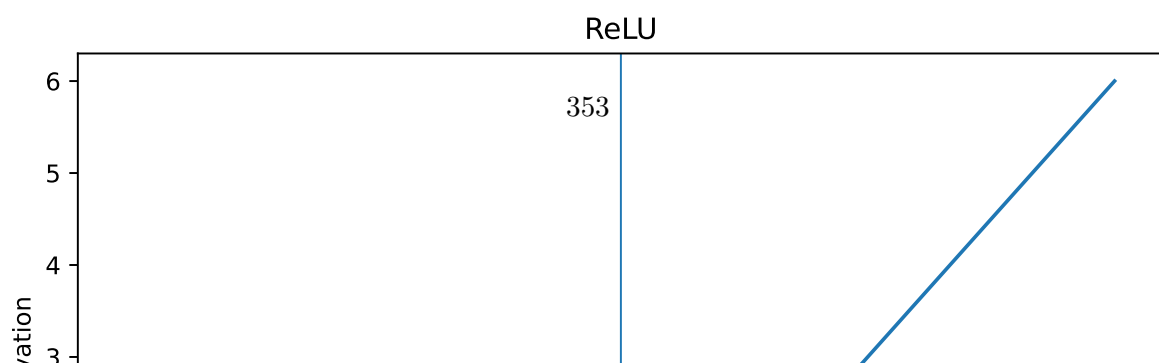
```

(a) Common neural-network activation functions.



(b)



13.9 Forward Propagation by Hand

```
x=np.array([0.6,-1.2])
W1=np.array([[0.4,-0.3],[0.7,0.2],[-0.5,0.8]])
b1=np.array([0.1,-0.2,0.05])
W2=np.array([[0.6,-0.4,0.9]])
b2=np.array([0.15])

z1=W1@x+b1
h=np.maximum(0,z1)
z2=W2@h+b2
p=sigmoid(z2)
z1,h,z2,p
```

```
(array([ 0.7 , -0.02, -1.21]),
 array([0.7, 0. , 0. ]),
 array([0.57]),
 array([0.63876318]))
```

Every value is inspectable. A neural network is a computational graph, not magic.

13.10 Neural Networks as Representation Learning

A neural network is more than a flexible nonlinear prediction function.

Its hidden layers can learn intermediate representations of the input.

For a feedforward network,

$$h^{(1)} = g(W^{(1)}x + b^{(1)}),$$

and later layers transform earlier representations:

$$h^{(\ell)} = g(W^{(\ell)}h^{(\ell-1)} + b^{(\ell)}).$$

The network therefore constructs a sequence:

$$x \rightarrow h^{(1)} \rightarrow h^{(2)} \rightarrow \dots \rightarrow h^{(L)} \rightarrow \hat{y}$$

13.10.1 From Feature Engineering to Learned Features

Earlier machine-learning methods often depend heavily on features designed by analysts.

Support vector machines showed that changing representation can change the geometry of a problem:

$$x \rightarrow \phi(x).$$

Neural networks extend this idea by learning useful transformations from data.

A useful conceptual progression is:

hand-crafted features $\rightarrow$ kernel-induced features $\rightarrow$ learned representations.

This is one reason deep learning became especially powerful for images, language, audio, and other high-dimensional data.

13.10.2 What Might a Hidden Representation Learn?

Imagine a model trained on text.

The raw input may begin as tokens or embeddings.

Intermediate layers may encode increasingly useful patterns related to:

- syntax;
- semantic similarity;
- topic;
- context;
- task-relevant distinctions.

For images, early layers may respond to simple local patterns while later representations can support increasingly abstract visual distinctions.

These interpretations should be treated cautiously: internal representations are distributed, context-dependent, and not guaranteed to correspond neatly to human concepts.

13.10.3 Learned Does Not Mean Understood

A network can learn a representation that improves prediction without providing a human-readable explanation for why that representation works.

Thus:

Useful Representation $\neq$ Human-Interpretable Representation

This creates a recurring machine-learning tradeoff.

Models can learn increasingly powerful internal representations while becoming harder to explain using simple human concepts.

13.10.4 Representation Quality Depends on the Objective

A network learns features useful for the objective it is trained to optimize.

If the objective rewards a shortcut, the learned representation may encode the shortcut.

If training data contain systematic biases, representations can preserve or amplify them.

If deployment differs from training, representations may no longer organize the world in a useful way.

Beyond the Algorithm: What Does It Mean for a Machine to “Represent” Something?

A hidden layer does not necessarily contain a miniature human concept such as “health,” “learning,” or “trust.”

Representation in machine learning is operational: internal states organize information in ways that support the optimization objective.

The philosophical question is therefore not simply:

What does this neuron mean?

but:

What distinctions has the system learned to preserve because they help it perform the task?

Research Reality Check

Strong neural-network performance does not prove that the model learned the intended concept.

It may have learned proxies, artifacts, shortcuts, or site-specific signals. When the substantive interpretation matters, test the representation rather than inferring its meaning from predictive performance alone.

13.10.5 Bridge Forward

This idea prepares the reader for later chapters:

- NLP uses learned representations of language;
- generative AI relies on large-scale learned representations;
- model evaluation asks whether those representations generalize;
- responsible ML asks what unwanted structure they encode;
- reproducible ML asks whether the evidence supporting these claims can be reconstructed.

13.11 Architecture

An architecture

$$30 \rightarrow 64 \rightarrow 32 \rightarrow 1$$

contains 30 inputs, two hidden layers, and one output.

Layer sizes are **hyperparameters**. Learned weights and biases are **parameters**.

13.12 Loss Functions Give the Network a Goal

For regression,

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

For binary classification,

$$L = -\frac{1}{n} \sum_{i=1}^n [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)].$$

The loss tells optimization what kinds of errors to reduce.

13.13 Gradient Descent

Let all parameters be θ . Training seeks

$$\theta^* = \arg \min_{\theta} L(\theta).$$

Gradient descent updates

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(\theta_t).$$

The learning rate η controls update size.

13.14 Gradient Descent Demonstration

For

$$L(w) = (w - 3)^2, \quad \frac{dL}{dw} = 2(w - 3),$$

we can watch optimization reduce loss.

```
w_value=-4.0
eta=.15
history=[]
for step in range(18):
    loss=(w_value-3)**2
    history.append((step,w_value,loss))
    gradient=2*(w_value-3)
    w_value-=eta*gradient

steps=[r[0] for r in history]
losses=[r[2] for r in history]

plt.figure(figsize=(8,5))
plt.plot(steps,losses,marker="o")
plt.xlabel("Optimization step")
plt.ylabel("Loss")
plt.title("Gradient Descent Reduces Loss")
plt.show()
```

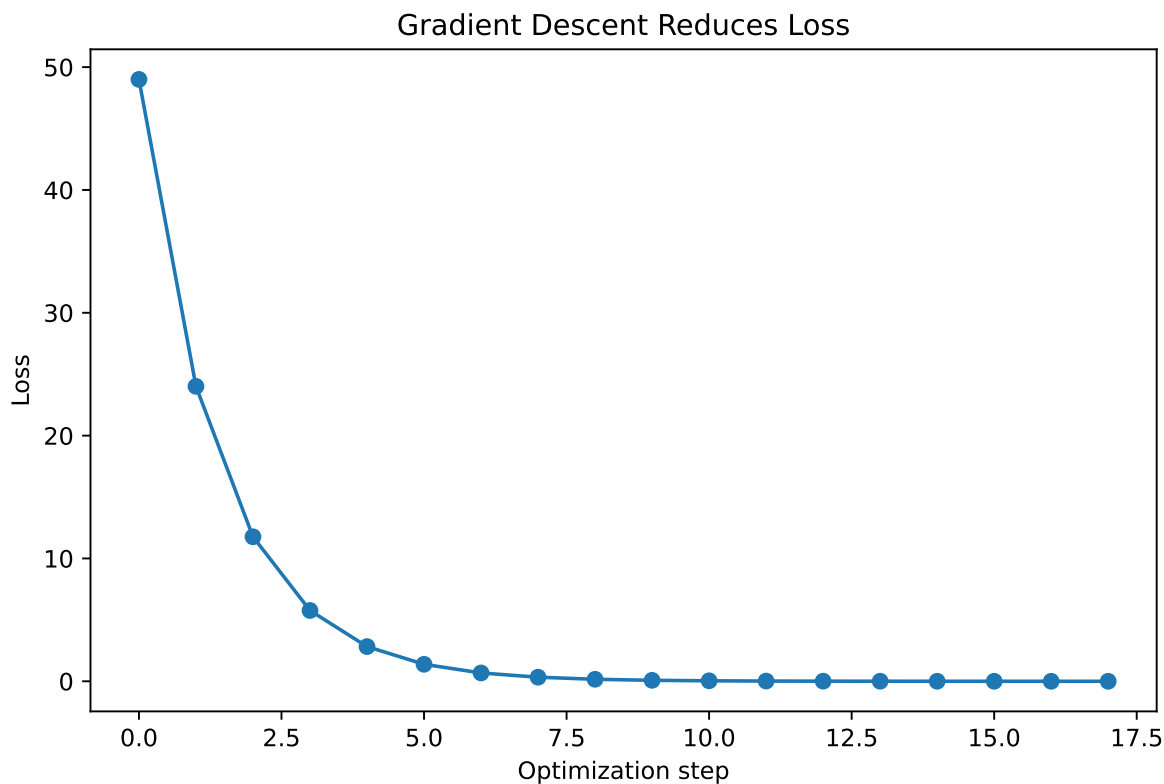


Figure 13.2: Gradient descent iteratively moves toward a lower-loss region.

13.15 The Chain Rule

If

$$L = L(a), \quad a = \phi(z), \quad z = wx + b,$$

then

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial z} \frac{\partial z}{\partial w}.$$

This allows sensitivity to propagate backward through composed operations.

13.16 Backpropagation

Backpropagation is an efficient application of the chain rule for computing gradients through layered networks; its modern neural-network formulation was established by Rumelhart, Hinton, and Williams (Rumelhart, Hinton, and Williams 1986):

forward pass $\rightarrow$ loss $\rightarrow$ backward gradients $\rightarrow$ parameter update .

For sigmoid output with binary cross-entropy,

$$\frac{\partial L}{\partial z} = \hat{p} - y,$$

so

$$\frac{\partial L}{\partial w} = (\hat{p} - y)x$$

and

$$\frac{\partial L}{\partial b} = \hat{p} - y.$$

13.17 Numerical Gradient Check

```
def single_loss(w,x,y,b=0):
    p=1/(1+np.exp(-(w*x+b)))
    return -(y*np.log(p)+(1-y)*np.log(1-p))

x_val=.8
y_val=1
w_val=.4
eps=1e-5

p_val=1/(1+np.exp(-(w_val*x_val)))
analytical=(p_val-y_val)*x_val
numerical=(
    single_loss(w_val+eps,x_val,y_val)
    -single_loss(w_val-eps,x_val,y_val)
)/(2*eps)
```


analytical, numerical

```
(np.float64(-0.3365405982810004), np.float64(-0.3365405982724834))
```

13.18 Batches and Epochs

Batch gradient descent uses all training observations per update.

Stochastic gradient descent uses one.

Mini-batch gradient descent uses a subset.

An **epoch** is one pass through the training dataset.

More epochs do not automatically mean better generalization.

13.19 Vanishing and Exploding Gradients

Backpropagation repeatedly multiplies derivatives through layers.

Products of small derivatives can approach zero:

$$\prod_{\ell=1}^L \frac{\partial h^{(\ell)}}{\partial h^{(\ell-1)}} \rightarrow 0.$$

Very large gradients can instead destabilize training.

13.20 Initialization

Initialization affects how signals and gradients propagate through a network; principled initialization schemes help avoid poorly scaled activations and gradients (Glorot and Bengio 2010).
and Regularization

Random initialization breaks symmetry among neurons. Initialization scale also affects optimization.

L2 regularization adds a penalty:

$$L_{reg} = L + \lambda \sum_j w_j^2.$$

Dropout randomly suppresses activations during training, providing a widely used regularization strategy for reducing co-adaptation and overfitting in neural networks (Srivastava et al. 2014).

Early stopping halts training when validation performance stops improving meaningfully.

13.21 Training and Validation Curves

```
epochs=np.arange(1,81)
train_loss=.9*np.exp(-epochs/25)+.08
validation_loss=.8*np.exp(-epochs/20)+.13+.00008*(epochs-42).clip(min=0)**2

plt.figure(figsize=(8,5))
plt.plot(epochs,train_loss,label="Training loss")
plt.plot(epochs,validation_loss,label="Validation loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training Versus Validation Loss")
plt.legend()
plt.show()
```

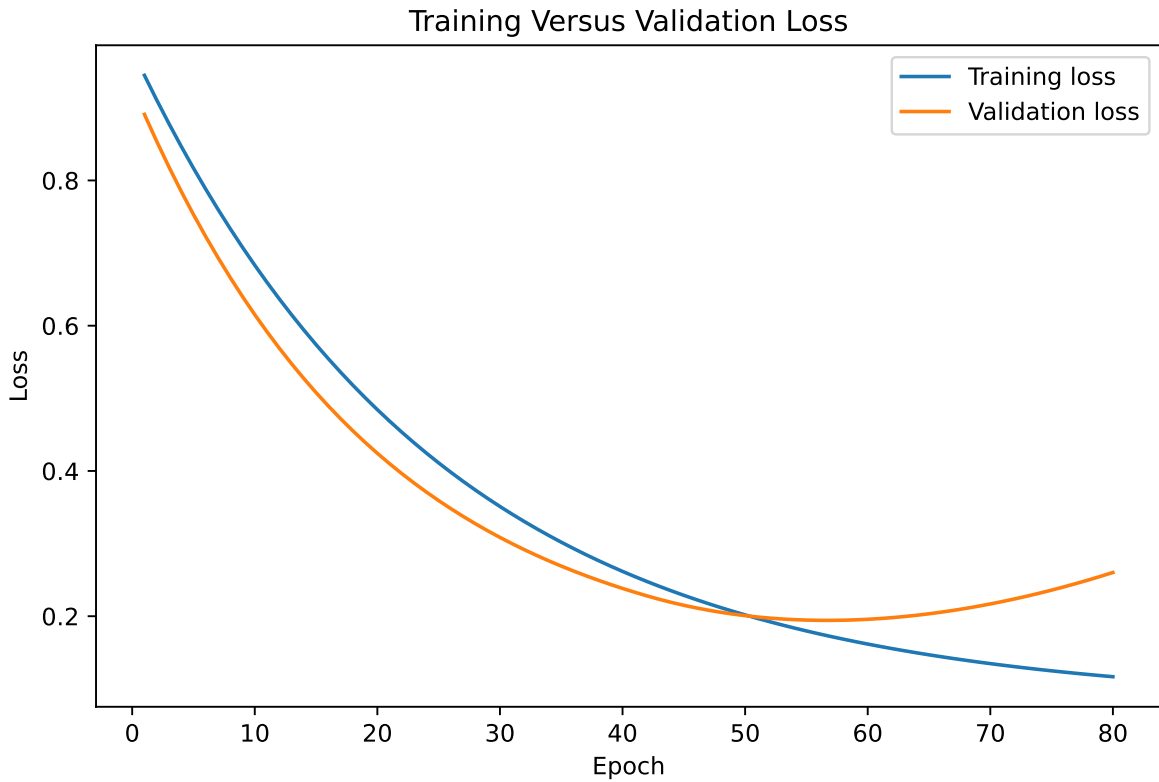


Figure 13.3: Conceptual learning curves illustrate how validation loss can worsen while training loss continues to decline.

These curves are simulated for instruction.

13.22 Build an MLP With scikit-learn

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import roc_auc_score

data=load_breast_cancer(as_frame=True)
X=data.data
y=(data.target==0).astype(int)
```

```

X_train,X_test,y_train,y_test=train_test_split(
    X,y,test_size=.25,random_state=42,stratify=y
)

mlp=Pipeline([
    ("scale",StandardScaler()),
    ("model",MLPClassifier(
        hidden_layer_sizes=(64,32),
        activation="relu",
        solver="adam",
        alpha=.001,
        learning_rate_init=.001,
        max_iter=1000,
        early_stopping=True,
        validation_fraction=.15,
        n_iter_no_change=20,
        random_state=42
    ))
])

mlp.fit(X_train,y_train)
mlp_prob=mlp.predict_proba(X_test)[: ,1]
roc_auc_score(y_test,mlp_prob)

```

```
np.float64(0.9972746331236897)
```

13.23 Inspect the Learned Architecture

```

fitted_mlp=mlp.named_steps["model"]

{
    "Layers":fitted_mlp.n_layers_,
    "Outputs":fitted_mlp.n_outputs_,
    "Iterations":fitted_mlp.n_iter_,
    "Final training loss":fitted_mlp.loss_
}

```

```
{'Layers': 4,
```

```
'Outputs': 1,  
'Iterations': 63,  
'Final training loss': np.float64(0.038450243636669365)}
```

```
[weights.shape for weights in fitted_mlp.coefs_]
```

```
[(30, 64), (64, 32), (32, 1)]
```

13.24 Compare With a Simpler Baseline

```
from sklearn.linear_model import LogisticRegression  
  
logit=Pipeline([  
    ("scale",StandardScaler()),  
    ("model",LogisticRegression(max_iter=2000))  
)  
  
logit.fit(X_train,y_train)  
logit_prob=logit.predict_proba(X_test)[:,-1]  
  
{  
    "Logistic regression":roc_auc_score(y_test,logit_prob),  
    "Neural network":roc_auc_score(y_test,mlp_prob)  
}
```

```
{'Logistic regression': np.float64(0.9962264150943396),  
 'Neural network': np.float64(0.9972746331236897)}
```

A neural network should earn its added complexity.

13.25 Calibration

```
from sklearn.calibration import calibration_curve  
from sklearn.metrics import brier_score_loss  
  
frac_pos,mean_pred=calibration_curve(  

```

```

    y_test,mlp_prob,n_bins=6,strategy="quantile"
)
brier=brier_score_loss(y_test,mlp_prob)

plt.figure(figsize=(7,5))
plt.plot(mean_pred,frac_pos,marker="o",label=f"MLP (Brier={brier:.3f})")
plt.plot([0,1],[0,1],linestyle="--",label="Perfect calibration")
plt.xlabel("Mean predicted probability")
plt.ylabel("Observed positive fraction")
plt.title("MLP Calibration")
plt.legend()
plt.show()

```

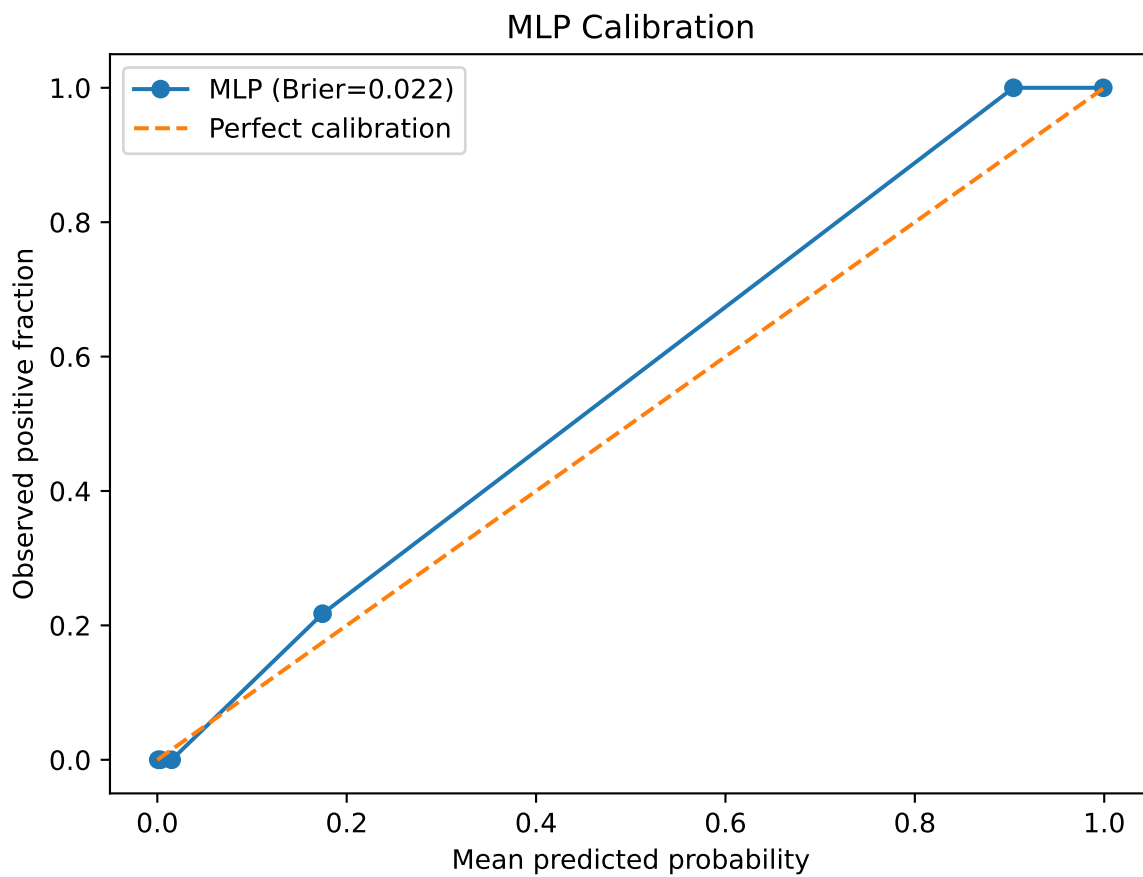


Figure 13.4: Neural-network probabilities require calibration assessment when interpreted as risks.

13.26 Learned Representations

PCA constructs representations that preserve predictor variance under linear constraints.

A supervised neural network learns representations optimized for a task-specific loss:

PCA: representation for variance

versus

neural network: representation for predictive loss.

13.27 Representation Learning Does Not Eliminate Feature Engineering

Humans still choose what data are collected, the target, sampling frame, architecture, loss, optimization, evaluation metrics, stopping rules, and deployment criteria.

Representation learning relocates some feature construction from explicit human design to optimization.

13.28 Universal Approximation—Carefully Interpreted

Approximation capacity does not imply that:

- optimization will find the desired function;
- finite data are sufficient;
- generalization is guaranteed;
- the learned relationship is causal;
- deeper is always better.

Capacity and successful learning are different questions.

13.29 Neural Networks for Regression

```

from sklearn.datasets import load_diabetes
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_squared_error

diabetes=load_diabetes(as_frame=True)

Xr_train,Xr_test,yr_train,yr_test=train_test_split(
    diabetes.data,diabetes.target,test_size=.25,random_state=42
)

mlp_reg=Pipeline([
    ("scale",StandardScaler()),
    ("model",MLPRegressor(
        hidden_layer_sizes=(64,32),
        activation="relu",
        alpha=.001,
        learning_rate_init=.001,
        max_iter=1500,
        early_stopping=True,
        random_state=42
    ))
])

mlp_reg.fit(Xr_train,yr_train)
reg_pred=mlp_reg.predict(Xr_test)
np.sqrt(mean_squared_error(yr_test,reg_pred))

```

```
np.float64(56.33537572573954)
```

13.30 When Should We Use Deep Learning?

Neural networks are especially influential for images, audio, natural language, video, high-dimensional signals, and multimodal data.

For small and medium tabular datasets, simpler models and boosted trees can be highly competitive.

Use neural networks when representational capacity addresses a real property of the problem—not because “deep learning” sounds advanced.

13.31 Deep Learning Ecosystem

The core chapter uses scikit-learn to preserve the book's lightweight environment.

Modern research commonly uses frameworks such as PyTorch, JAX, and TensorFlow/Keras.

Optional Advanced Path

A companion lab can later rebuild the network in PyTorch so students explicitly write the architecture, forward pass, loss, optimizer, training loop, backpropagation, and parameter update.

13.32 Computer Vision Learning Experience: From Pixels to a Neural Classifier

The recurring handwritten-digits workload now reaches an important transition.

Earlier chapters treated each (8×8) image as a (64)-feature vector and used logistic regression, trees, ensembles, and kernel methods. A neural network can learn nonlinear intermediate representations from those pixel features.

```
import os
os.environ["LOKY_MAX_CPU_COUNT"] = "1"

from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

digits_nn_cv = load_digits()

X_nn_cv = digits_nn_cv.data
y_nn_cv = digits_nn_cv.target

X_train_nn_cv, X_test_nn_cv, y_train_nn_cv, y_test_nn_cv = train_test_split(
    X_nn_cv,
    y_nn_cv,
    test_size=0.25,
    random_state=42,
```

```

        stratify=y_nn_cv
    )

vision_mlp = Pipeline([
    ("scale", StandardScaler()),
    ("model", MLPClassifier(
        hidden_layer_sizes=(64, 32),
        activation="relu",
        max_iter=600,
        random_state=42,
        early_stopping=True
    ))
])

vision_mlp.fit(X_train_nn_cv, y_train_nn_cv)
pred_nn_cv = vision_mlp.predict(X_test_nn_cv)

accuracy_score(y_test_nn_cv, pred_nn_cv)

```

0.96

This is a neural computer-vision model, but it is **not yet a convolutional neural network**. Flattening removes the image's explicit two-dimensional structure before learning begins.

13.32.1 Why Convolution Changed Computer Vision

Convolutional neural networks (CNNs) exploit spatial structure more directly. Local filters can detect patterns such as edges and textures across image locations; influential early CNN systems demonstrated how gradient-based learning could learn hierarchical visual features for document recognition (LeCun et al. 1998).

Image → Local Features → Higher-Level Features → Prediction.

Modern vision architectures also include vision transformers and multimodal systems. The architectural details are beyond this chapter's core scope, but the conceptual lesson is central:

Architecture Encodes Assumptions About Useful Representation

Beyond the Algorithm: Seeing Is Not Merely Classifying Pixels

A model that classifies an image accurately has demonstrated task performance, not human-like seeing. The distinction between **successful representation** and **human understanding** remains essential.

The vision thread now connects image arrays, flattened pixels, classical models, kernel geometry, learned representations, and spatial architectures.

13.33 Three Tracks

13.33.1 Health Track

Neural networks can model nonlinear relationships in physiological signals, imaging, longitudinal measurements, and multimodal data. Research-grade work still requires calibration, external validation, subgroup analysis, leakage prevention, and meaningful baselines.

13.33.2 Education Track

Applications include multimodal learning analytics, sequence modeling, text analysis, and behavioral traces. Predictive accuracy must not convert opaque representations into deterministic labels about student ability or potential.

13.33.3 Open ML Benchmark Track

Benchmarks support controlled experiments with width, depth, learning rate, regularization, convergence, and comparison against simpler models.

13.34 Beyond the Algorithm: Where Did Human Judgment Go?

Automation Does Not Remove Choice

Representation learning can make feature construction less visible.
It is tempting to say:

“The network discovered the features objectively.”

But someone determined what could be measured, defined the target, selected observations, chose the loss, architecture, and evaluation criteria.

The network optimized within that designed world.

Automation often changes the location of human judgment rather than eliminating it.

The responsible question is:

Which human and institutional choices shaped what the system was capable of learning?

13.35 AI Pair Programmer: Make It Deeper

Ask an AI assistant:

“My neural network has two hidden layers and 95% validation accuracy. Make it much more powerful by adding as many layers as possible.”

Audit whether it asks about sample size, modality, baseline, train/validation gap, test isolation, calibration, learning curves, regularization, and whether added capacity is justified.

Then request the **simplest architecture that could answer the problem well**.

13.36 Hands-On Lab: From Neuron to Network

1. Define the target and prediction time.
2. Establish a non-neural baseline.
3. Standardize features in a pipeline.
4. Build a one-hidden-layer MLP.
5. Inspect convergence.
6. Build a two-hidden-layer MLP.
7. Vary width and regularization.
8. Compare learning rates.
9. Use early stopping.
10. Compare task-relevant metrics.
11. Assess calibration where probabilities matter.
12. Compare computational cost.
13. Examine relevant subgroups.
14. Determine whether added complexity earns its place.
15. Document unsupported claims.

13.37 Practitioner Track

A practitioner should understand architecture, activations, losses, optimization, scaling, regularization, early stopping, and evaluation well enough to diagnose an MLP without treating it as magic.

13.38 Researcher Track

A researcher should additionally justify architecture and loss, separate tuning from final evaluation, quantify uncertainty, examine calibration and subgroup behavior, compare against strong baselines, document compute, and avoid causal interpretations without supporting designs.

13.39 Research Studio

Design a neural-network study specifying the problem, modality, target, sample size, baselines, architecture, activations, loss, optimizer, learning rate, batch size, regularization, early stopping, tuning design, final test boundary, calibration, subgroup analysis, external validation, compute plan, interpretive limits, and scholarly contribution.

13.40 Professor's Challenge: Deep Learning Must Be Better

A university has 3,200 student records with 45 tabular predictors.

Researchers fit a six-hidden-layer network with more than 500,000 trainable parameters. After trying dozens of architectures, they select the model with the highest test-set accuracy.

The network achieves 88.2% accuracy. Logistic regression achieves 87.9%.

They conclude:

“Deep learning discovered complex latent learning patterns inaccessible to traditional statistical models and therefore provides a fundamentally superior understanding of student success.”

Critique model capacity, architecture rationale, test reuse, the 0.3-point difference, calibration, class imbalance, baseline comparison, representation claims, prediction versus understanding, subgroup performance, reproducibility, compute, and external validity.

Redesign the experiment.

13.41 Knowledge Check

1. How is a neuron related to logistic regression?
2. What do weights and bias do?
3. Why are nonlinear activations necessary?
4. What is forward propagation?
5. How do sigmoid and ReLU differ?
6. When is softmax commonly used?
7. What role does a loss function play?
8. What is gradient descent?
9. What does learning rate control?
10. What is the chain rule?
11. What is backpropagation?
12. What is an epoch?
13. How do batch, stochastic, and mini-batch learning differ?
14. What are vanishing gradients?
15. Why does initialization matter?
16. What is L2 regularization?
17. What is dropout?
18. What is early stopping?
19. Why compare neural networks with simpler baselines?
20. What is representation learning?
21. How does it differ from PCA?
22. Why does representation learning not eliminate human judgment?
23. Why does approximation capacity not guarantee successful learning?
24. When might a neural network be unnecessary?
25. How does representation learning differ from hand-crafted feature engineering?
26. How do SVM kernels provide a conceptual bridge to neural networks?
27. Why does a useful learned representation not necessarily have a simple human interpretation?
28. How can the training objective shape what a network learns to represent?

13.42 Reproducibility Check

Record the question, data version, sample, target, prediction time, train/validation/test design, preprocessing, architecture, activations, loss, optimizer, learning rate, batch size, epochs, initialization, regularization, early stopping, hyperparameter search, seeds, baselines, metrics, calibration, subgroup analyses, training time, hardware, software versions, saved configuration, and code.

13.43 Key Takeaways

- Neural networks compose weighted sums and nonlinear transformations.
- Logistic regression provides a bridge to understanding a neuron.
- Without nonlinear activations, stacked linear layers remain linear.
- Hidden layers learn intermediate representations.
- Loss functions define the optimization objective.
- Gradient descent uses local gradients to update parameters.
- Backpropagation efficiently applies the chain rule.
- Architecture, learning rate, initialization, and regularization influence optimization.
- Neural-network probabilities still require calibration assessment.
- Learned representations optimize an objective, not scientific meaning.
- High capacity does not guarantee better generalization.
- Strong simpler baselines remain essential.
- Automation relocates human judgment; it does not eliminate it.
- Hidden layers learn intermediate representations that support prediction.
- Neural networks extend the idea that changing representation can simplify a learning problem.
- Predictive usefulness does not guarantee human interpretability or conceptual validity.

14 Natural Language Processing: From Words to Meaningful Representations

14.1 Learning Objectives

By the end of this chapter, you should be able to:

- explain why human language creates unusual machine-learning challenges;
- define corpus, document, token, vocabulary, n-gram, and document-term matrix;
- distinguish tokenization from normalization;
- explain the tradeoffs of stop-word removal, stemming, and lemmatization;
- construct bag-of-words and TF-IDF representations;
- derive and interpret term frequency and inverse document frequency;
- explain sparsity and high dimensionality in text data;
- use cosine similarity to compare document vectors;
- build leakage-safe text-classification pipelines;
- evaluate text classifiers with metrics appropriate to the task;
- inspect model coefficients to understand lexical signals;
- distinguish lexical association from semantic understanding;
- explain distributed word embeddings conceptually;
- connect embeddings to representation learning from Chapter 13;
- explain contextual embeddings and why a word can require different representations in different sentences;
- derive the intuition behind self-attention;
- explain queries, keys, values, and attention weights;
- describe the role of positional information in transformer models;
- distinguish encoder-style, decoder-style, and sequence-to-sequence transformer objectives at a high level;
- recognize data leakage, spurious lexical cues, subgroup harms, and construct-validity problems in NLP;
- design reproducible NLP studies using health, education, or public benchmark text.
- distinguish language-only NLP from multimodal learning that jointly represents text and visual information.

14.2 Why Language Is Different

Most earlier chapters assumed that observations were already represented as columns of numbers.

Language does not arrive that way.

A student writes:

“I understand the material, but I still feel lost when I try to apply it.”

A patient writes:

“The medication helped the pain, but I have been exhausted since Tuesday.”

A scientific abstract contains arguments, terminology, uncertainty, relationships, and context.

Before a machine-learning algorithm can operate on these texts, we must construct a numerical representation.

The NLP pipeline therefore begins with a profound design problem:

$$\text{language} \longrightarrow \text{representation} \longrightarrow \text{model}.$$

Central Question

When language becomes numbers, which aspects of meaning survive the transformation—and which disappear?

14.3 The Unit of Analysis

Before tokenizing anything, identify the analytical unit.

Possible units include:

- an entire document;
- a paragraph;
- a sentence;
- an utterance;
- a social-media post;
- a clinical note;
- a discussion-board contribution;
- a token.

The unit must align with the research question.

Predicting a document-level label from sentence fragments changes the problem.

14.4 Core Vocabulary

A **corpus** is a collection of text documents.

A **document** is one analytical text unit.

A **token** is a unit produced by tokenization.

A **vocabulary** is the set of distinct tokens/features represented by the model.

An **n-gram** is a contiguous sequence of n tokens.

Examples:

- unigram: machine
- bigram: machine learning
- trigram: machine learning model

14.5 Tokenization

Tokenization divides text into units.

A simple whitespace tokenizer might turn:

```
Machine learning changes research.
```

into:

```
Machine  
learning  
changes  
research.
```

But real tokenization must confront:

- punctuation;
- contractions;
- hyphenation;
- URLs;

- emojis;
- hashtags;
- numbers;
- abbreviations;
- multilingual text;
- domain terminology.

Modern subword tokenizers go further by representing words through reusable pieces.

Tokenization is therefore not clerical preprocessing. It defines the symbols available to the model.

14.6 A Simple Tokenization Demonstration

```
import re

text = """
Machine learning can support research,
but representation choices matter.
"""

tokens = re.findall(
    r"\b[\w'-]+\b",
    text.lower()
)

tokens
```

```
['machine',
 'learning',
 'can',
 'support',
 'research',
 'but',
 'representation',
 'choices',
 'matter']
```

This tokenizer is intentionally simple. Production NLP systems often require more sophisticated rules or learned subword tokenizers.

14.7 Text Normalization

Common transformations include:

- lowercasing;
- punctuation handling;
- number normalization;
- spelling normalization;
- stop-word removal;
- stemming;
- lemmatization.

Each transformation changes the information available to the model.

For example, removing the word **not** could reverse sentiment.

Aggressive normalization can improve statistical efficiency while destroying useful meaning.

14.8 Stop Words

Stop words are frequently occurring function words such as **the**, **and**, or **is**.

Removing them can reduce dimensionality in some lexical models.

But consider:

“The treatment was effective.”

versus

“The treatment was **not** effective.”

A generic stop-word list that removes negation could be disastrous.

Domain context matters.

14.9 Stemming and Lemmatization

Stemming applies heuristic rules to reduce words to stems.

Lemmatization attempts to map inflected words to linguistically meaningful base forms.

For example:

$$\{\text{learn, learned, learning}\} \rightarrow \text{learn}$$

may be useful in some applications.

But modern contextual models often operate without traditional stemming or lemmatization.

Preprocessing should follow the representation and task.

14.10 Bag-of-Words Foundations

14.11 From Documents to Counts

Suppose our corpus contains:

1. machine learning supports research
2. research supports learning
3. machine learning requires data

A bag-of-words representation records token counts while largely ignoring order.

If the vocabulary is

$$V = \{\text{machine, learning, supports, research, requires, data}\},$$

each document becomes a numerical vector.

14.12 CountVectorizer

```

from sklearn.feature_extraction.text import CountVectorizer
import pandas as pd

documents = [
    "machine learning supports research",
    "research supports learning",
    "machine learning requires data"
]

count_vectorizer = CountVectorizer()
X_count = count_vectorizer.fit_transform(
    documents
)

count_df = pd.DataFrame(
    X_count.toarray(),
    columns=count_vectorizer.get_feature_names_out()
)

count_df

```

	data	learning	machine	requires	research	supports
0	0	1	1	0	1	1
1	0	1	0	0	1	1
2	1	1	1	1	0	0

This is a **document-term matrix**.

Rows represent documents.

Columns represent vocabulary features.

14.13 Sparsity

Real vocabularies can contain tens or hundreds of thousands of features.

Any individual document uses only a small fraction of them.

The resulting matrix is sparse.

If

$$X \in \mathbb{R}^{n \times p},$$

p may be enormous while most entries equal zero.

Sparse matrix representations avoid storing every zero explicitly.

This is one reason we should generally avoid converting large text matrices with `.toarray()`.

We did so above only because the teaching example is tiny.

14.14 What Bag of Words Forgets

Compare:

`student understands model`

and

`model understands student`

A unigram bag-of-words representation can treat them identically.

Thus:

$$\boxed{\text{same token counts}} \not\Rightarrow \boxed{\text{same meaning}}.$$

14.15 N-Grams Restore Some Local Order

```
bigram_vectorizer = CountVectorizer(
    ngram_range=(1, 2)
)

X_bigram = bigram_vectorizer.fit_transform(
    documents
)

bigram_vectorizer.get_feature_names_out()
```

```
array(['data', 'learning', 'learning requires', 'learning supports',  
      'machine', 'machine learning', 'requires', 'requires data',  
      'research', 'research supports', 'supports', 'supports learning',  
      'supports research'], dtype=object)
```

Bigrams can preserve phrases such as `machine learning`.

But vocabulary size grows rapidly.

This creates a tradeoff between richer local context and dimensionality/sparsity.

14.16 TF-IDF

14.17 Why Counts Are Not Enough

A term appearing frequently in every document may be less discriminating than a term concentrated in a smaller subset.

TF-IDF combines term-specific evidence with corpus-level rarity, reflecting a long information-retrieval tradition of term weighting (Salton and Buckley 1988):

- **term frequency** — importance within a document;
- **inverse document frequency** — rarity across documents.

14.18 Term Frequency

A simple term frequency is

$$TF(t, d) = \text{count of term } t \text{ in document } d.$$

Variants normalize by document length or use sublinear scaling.

14.19 Inverse Document Frequency

A common smoothed form is

$$IDF(t) = \log \left(\frac{1 + n}{1 + df(t)} \right) + 1,$$

where:

- n is the number of documents;
- $df(t)$ is the number of documents containing term t .

Terms appearing in fewer documents receive greater IDF weight.

14.20 TF-IDF Weight

$$TFIDF(t, d) = TF(t, d) \times IDF(t).$$

The exact implementation can vary, so reproducible research should document the software and parameterization.

14.21 TF-IDF in Python

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer()

X_tfidf = tfidf.fit_transform(
    documents
)

tfidf_df = pd.DataFrame(
    X_tfidf.toarray(),
    columns=tfidf.get_feature_names_out()
)

tfidf_df.round(3)
```

	data	learning	machine	requires	research	supports
0	0.000	0.409	0.527	0.000	0.527	0.527
1	0.000	0.481	0.000	0.000	0.620	0.620
2	0.584	0.345	0.445	0.584	0.000	0.000

14.22 Cosine Similarity

Text vectors are often compared using cosine similarity.

For vectors x and y ,

$$\cos(\theta) = \frac{x^\top y}{\|x\|_2 \|y\|_2}.$$

Cosine similarity focuses on orientation rather than raw vector magnitude.

14.23 Document Similarity

```
from sklearn.metrics.pairwise import cosine_similarity

similarities = cosine_similarity(
    X_tfidf
)

pd.DataFrame(
    similarities,
    index=["Doc 1", "Doc 2", "Doc 3"],
    columns=["Doc 1", "Doc 2", "Doc 3"]
).round(3)
```

	Doc 1	Doc 2	Doc 3
Doc 1	1.000	0.850	0.375
Doc 2	0.850	1.000	0.166
Doc 3	0.375	0.166	1.000

A high cosine similarity means the documents have similar vector directions under the chosen representation.

It does not prove that humans would judge the documents semantically equivalent.

14.24 Text Classification

14.25 A Public Benchmark

For an executable classification example, we use the scikit-learn 20 Newsgroups dataset.

Because it is downloaded from the internet when first used, the main book should avoid requiring the download during every render.

Instead, we will create a small synthetic corpus for guaranteed rendering and provide the public benchmark workflow as a research extension.

14.26 A Reproducible Teaching Corpus

```
texts = [  
    "the course examples were clear and helpful",  
    "I understood the lecture and enjoyed the project",  
    "the assignment instructions were confusing",  
    "I could not understand the coding exercise",  
    "excellent explanations and useful examples",  
    "the project helped me apply machine learning",  
    "the lecture moved too quickly and was unclear",  
    "I struggled with the assignment requirements",  
    "the examples made the concept easy to understand",  
    "the instructions were vague and frustrating",  
    "helpful feedback improved my final project",  
    "I was confused by the explanation of the model",  
    "the hands on exercise was valuable",  
    "the activity did not explain the concept well",  
    "clear examples helped me understand regression",  
    "I could not follow the instructions for the lab",  
    "the project was engaging and practical",  
    "the lesson was difficult to follow",  
    "the coding demonstration was very useful",  
    "the assignment description was unclear"  
]
```

```
labels = [
    1,1,0,0,1,
    1,0,0,1,0,
    1,0,1,0,1,
    0,1,0,1,0
]
```

Here 1 indicates broadly positive/clear feedback and 0 indicates broadly negative/confusing feedback.

This corpus is for workflow demonstration, not substantive research.

14.27 Leakage-Safe NLP Pipeline

The vectorizer must learn vocabulary and IDF weights from the training data only.

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

X_train, X_test, y_train, y_test = train_test_split(
    texts,
    labels,
    test_size=.30,
    random_state=42,
    stratify=labels
)

text_model = Pipeline([
    (
        "tfidf",
        TfidfVectorizer(
            ngram_range=(1, 2),
            min_df=1
        )
    ),
    (
        "model",
        LogisticRegression(
            max_iter=2000
        )
    )
])
```

```

    )
])

text_model.fit(
    X_train,
    y_train
)

text_prob = text_model.predict_proba(
    X_test
)[: , 1]

text_pred = text_model.predict(
    X_test
)

```

This is the text equivalent of the leakage-safe pipelines developed earlier.

14.28 Evaluate the Classifier

```

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score
)

{
    "Accuracy": accuracy_score(
        y_test,
        text_pred
    ),
    "Precision": precision_score(
        y_test,
        text_pred
    ),
    "Recall": recall_score(
        y_test,

```

```

        text_pred
    ),
    "F1": f1_score(
        y_test,
        text_pred
    ),
    "ROC-AUC": roc_auc_score(
        y_test,
        text_prob
    )
}

```

```

{'Accuracy': 0.6666666666666666,
 'Precision': 1.0,
 'Recall': 0.3333333333333333,
 'F1': 0.5,
 'ROC-AUC': np.float64(0.5555555555555556)}

```

With a tiny synthetic corpus, these metrics are unstable.

That instability is itself instructive: impressive percentages from small text samples should not be overinterpreted.

14.29 Inspect Lexical Signals

Linear models provide a useful window into which features are associated with predictions.

```

feature_names = (
    text_model
    .named_steps["tfidf"]
    .get_feature_names_out()
)

coefficients = (
    text_model
    .named_steps["model"]
    .coef_[0]
)

coef_df = pd.DataFrame({

```

```

    "feature": feature_names,
    "coefficient": coefficients
})

coef_df.sort_values(
    "coefficient",
    ascending=False
).head(10)

```

	feature	coefficient
85	project	0.347821
103	the project	0.292112
36	examples	0.275115
12	clear	0.209737
53	helpful	0.207823
52	helped me	0.204508
51	helped	0.204508
71	me	0.204508
0	and	0.187348
33	engaging and	0.137968

And features associated with the opposite class:

```

coef_df.sort_values(
    "coefficient"
).head(10)

```

	feature	coefficient
7	assignment	-0.247968
92	the assignment	-0.247968
115	was	-0.229688
109	unclear	-0.223887
119	was unclear	-0.223887
57	instructions	-0.216702
23	could not	-0.205611
22	could	-0.205611
79	not	-0.205611
47	follow	-0.205443

A coefficient indicates predictive association under this fitted model.

It does not establish that a word caused the outcome or that the model “understands” the concept represented by the word.

14.30 Spurious Lexical Cues

Suppose all essays receiving low scores happened to contain a particular course code because one instructor used a different prompt.

A classifier might learn the course code rather than writing quality.

Text models are especially vulnerable to shortcuts involving:

- source;
- author;
- template;
- timestamp;
- institution;
- formatting;
- demographic proxies;
- label artifacts.

High predictive accuracy can coexist with invalid measurement.

14.31 Distributed Representations

14.32 From Sparse Counts to Dense Vectors

Bag-of-words and TF-IDF create sparse vectors whose dimensions correspond to vocabulary features.

Distributed embeddings instead represent tokens using dense vectors:

$$w \longrightarrow e_w \in \mathbb{R}^d,$$

where typically

$$d \ll |V|.$$

Words appearing in similar linguistic contexts can acquire nearby representations.

14.33 The Distributional Idea

A foundational intuition is:

words used in similar contexts often have related meanings.

Rather than manually defining semantic categories, representation learning can infer statistical regularities from large corpora.

This changes the representation from:

word identity

toward:

position in a learned semantic space.

14.34 One-Hot Versus Dense Representation

For a vocabulary of size V , a one-hot vector contains one 1 and $V - 1$ zeros.

For example:

$$\text{data} \rightarrow [0, 0, 1, 0, \dots, 0].$$

The one-hot representation does not encode similarity directly.

An embedding might instead be:

$$\text{data} \rightarrow [0.18, -0.42, 0.73, \dots].$$

The coordinates are learned rather than manually assigned.

14.35 Word2Vec Intuition

Word2Vec helped popularize efficient learning of distributed word representations from large corpora (Mikolov et al. 2013). Two classic Word2Vec objectives are:

- Continuous Bag of Words (CBOW): predict a target word from surrounding context;
- Skip-Gram: predict surrounding context from a target word.

The training task is instrumental.

The learned embedding matrix becomes the useful artifact.

This is an important pattern in modern machine learning:

A model can learn a representation through a prediction task different from the downstream task we ultimately care about.

14.36 Static Embeddings Have a Limitation

Consider:

“She deposited money at the **bank**.”

and

“They sat on the **bank** of the river.”

A static embedding assigns one vector to **bank**.

But meaning depends on context.

We therefore want:

$$e(\text{bank} \mid \text{sentence 1}) \neq e(\text{bank} \mid \text{sentence 2}).$$

This motivates **contextual embeddings**.

14.37 Attention

14.38 Why Context Requires Interaction

To represent a word contextually, the model needs a mechanism through which tokens can influence one another.

Attention provides such a mechanism.

For each token representation, a model constructs:

- a **query**;
- a **key**;
- a **value**.

For matrix X :

$$Q = XW_Q,$$

$$K = XW_K,$$

$$V = XW_V.$$

14.39 Scaled Dot-Product Attention

Attention is commonly written as

$$Attention(Q, K, V) = softmax \left(\frac{QK^\top}{\sqrt{d_k}} \right) V.$$

This compact equation is central to modern NLP.

Let's unpack it.

14.40 Step 1: Compare Queries and Keys

The matrix

$$QK^\top$$

contains compatibility scores between token queries and token keys.

A large dot product suggests a stronger relationship under the learned projections.

14.41 Step 2: Scale

Divide by

$$\sqrt{d_k}.$$

Without scaling, dot products can become large as dimensionality increases, pushing softmax toward extreme values and making optimization less stable.

14.42 Step 3: Softmax

For each query, softmax converts scores into weights:

$$\alpha_{ij} = \frac{\exp(s_{ij})}{\sum_m \exp(s_{im})}.$$

The weights sum to one across the attended positions.

14.43 Step 4: Weighted Values

The contextual output for token i is

$$h_i = \sum_j \alpha_{ij} v_j.$$

A token's representation is therefore updated using information from other tokens.

14.44 Attention by Hand

```
tokens = [
    "student",
    "understood",
    "the",
    "model"
]

scores = np.array([
    [2.2, 1.0, 0.2, 0.8],
    [0.7, 2.1, 0.1, 1.8],
    [0.4, 0.5, 1.4, 0.6],
    [0.9, 1.7, 0.2, 2.3]
])

def row_softmax(matrix):
    shifted = matrix - matrix.max(
        axis=1,
        keepdims=True
    )
    exp_values = np.exp(shifted)
    return exp_values / exp_values.sum(
        axis=1,
        keepdims=True
    )

attention_weights = row_softmax(
    scores
)

plt.figure(figsize=(7, 5))
plt.imshow(
    attention_weights,
    aspect="auto"
)
plt.xticks(
    range(len(tokens)),
    tokens,
    rotation=30
)
```

```
plt.yticks(
    range(len(tokens)),
    tokens
)
plt.xlabel("Key token")
plt.ylabel("Query token")
plt.title("Toy Attention Weights")
plt.colorbar(label="Attention weight")
plt.show()
```

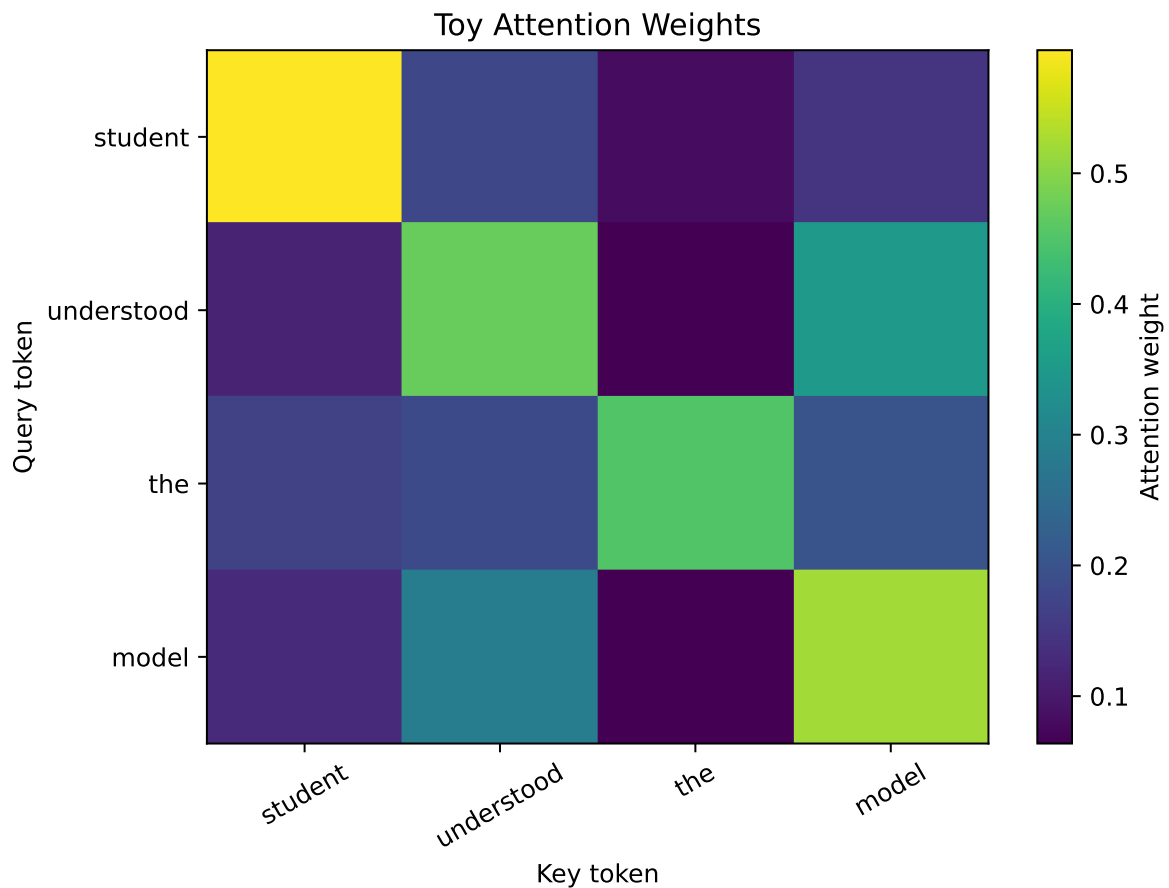


Figure 14.1: A toy attention matrix shows how each token can assign different weights to other tokens.

These weights are constructed for teaching rather than learned from a trained language model.

14.45 Attention Is Not Automatically Explanation

An attention weight tells us something about a model's internal computation.

It should not automatically be interpreted as:

- human reasoning;
- causal importance;
- psychological attention;
- a complete explanation of a prediction.

Model internals and scientific explanations are different objects.

14.46 Transformers

14.47 The Transformer Idea

The transformer architecture builds representations using attention rather than relying exclusively on recurrence (Vaswani et al. 2017).

A simplified transformer block includes:

token representations $\rightarrow$ multi-head attention $\rightarrow$ feed-forward network $\rightarrow$ updated representations.

Residual connections and normalization support optimization.

14.48 Multi-Head Attention

Instead of computing one attention pattern, transformers can compute multiple heads:

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V).$$

The heads are combined:

$$MultiHead(Q, K, V) = Concat(head_1, \dots, head_h)W^O.$$

Different heads can learn different interaction patterns.

We should be cautious about assigning simple linguistic meanings to individual heads without evidence.

14.49 Positional Information

Self-attention alone does not inherently encode token order in the same way recurrence does.

Transformers therefore incorporate positional information.

Conceptually:

$$\text{input representation} = \text{token embedding} + \text{position representation}.$$

Modern architectures implement positional information in several ways.

14.50 Encoder-Style Models

Encoder-style transformers are especially suited to constructing contextual representations from an input sequence.

Typical applications include:

- classification;
- token labeling;
- semantic similarity;
- information extraction.

BERT is a historically important example of this family (Devlin et al. 2019).

14.51 Decoder-Style Models

Decoder-style transformers predict tokens autoregressively:

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_1, \dots, x_{t-1}).$$

This next-token-prediction structure forms a foundation for many modern generative language models.

We develop that much more deeply in Chapter 15.

14.52 Encoder-Decoder Models

Sequence-to-sequence transformers can use an encoder to represent an input and a decoder to generate an output.

Applications include:

- translation;
- summarization;
- structured generation.

14.53 From NLP to Foundation Models

The progression across this chapter can now be seen as a history of increasingly expressive representations:

counts $\rightarrow$ TF-IDF $\rightarrow$ static embeddings $\rightarrow$ contextual embeddings $\rightarrow$ transformers.

Each step preserves different information and introduces new assumptions, computational costs, and opportunities.

14.54 Research-Grade NLP

14.55 Split by the Real Generalization Boundary

Randomly splitting documents is not always appropriate.

Suppose several notes come from the same patient, essays from the same student, or posts from the same author.

If documents from one person occur in both train and test sets, performance may partly reflect recognition of the person rather than generalization to new people.

Possible split boundaries include:

- person;
- institution;
- course;
- instructor;
- time;
- geographic site;

- publication venue.

The split should emulate the deployment or scientific generalization claim.

14.56 Duplicate and Near-Duplicate Text

Copied templates, boilerplate, repeated prompts, quoted material, and duplicated documents can inflate performance.

Text research should inspect overlap before modeling.

14.57 Temporal Leakage

Language changes over time.

A model may exploit information unavailable at the intended prediction time.

Examples include:

- later clinical diagnoses appearing in notes;
- instructor feedback written after grading;
- references to final outcomes;
- retrospective summaries.

Define the prediction timestamp before selecting text.

14.58 Construct Validity

Suppose a researcher predicts “student engagement” from discussion posts.

What exactly is the label?

Attendance?

Instructor rating?

Post frequency?

Self-report?

Course completion?

The sophistication of the language model cannot repair a weak construct definition.

14.59 Fairness and Language Variation

Language reflects:

- culture;
- dialect;
- educational experience;
- profession;
- age;
- geography;
- disability-related communication patterns;
- multilingual backgrounds.

A text model may interpret legitimate language variation as predictive signal.

Responsible evaluation should examine relevant subgroups and avoid treating one linguistic norm as inherently superior.

14.60 From NLP to Multimodal Learning

Natural-language processing focuses on language, while computer vision focuses on visual information. Many modern systems operate across both.

A multimodal system may learn from paired observations such as image-text pairs; contrastive vision-language systems such as CLIP demonstrate how aligned representations can be learned from large collections of such pairs (Radford et al. 2021).

For example, it may learn from

(image, text),

or combine several modalities:

$$x_i = (x_i^{\text{text}}, x_i^{\text{image}}, x_i^{\text{audio}}).$$

Applications include image captioning, text-to-image retrieval, visual question answering, medical image-text analysis, and interpreting diagrams together with written explanations.

14.60.1 Shared Spaces and Cross-Modal Alignment

One strategy is to learn representations in which related text and images are close in a shared embedding space. Conceptually,

$$f_{\text{text}}(x_i^{\text{text}}) \approx f_{\text{image}}(x_i^{\text{image}})$$

when the observations describe related content.

This bridges dimensionality reduction and representation learning with the language representations developed in this chapter.

Research Reality Check

A multimodal model that aligns an image with a caption has learned useful statistical correspondence. That does not establish that the system interprets either modality as a human does, nor that its representations are free from biases inherited from image-text data.

14.60.2 Learning Experience: Design a Multimodal Study

Choose one book track and specify the unit of analysis, text modality, visual modality, analytical objective, representation of each modality, alignment strategy, one leakage risk, one construct-validity concern, one domain-shift concern, and the evidence required before making a substantive claim.

14.61 Three Tracks

14.61.1 Health Track

Potential sources include public health narratives, biomedical abstracts, clinical text where access and governance permit, patient-generated text, and public datasets such as PubMed-derived corpora.

Research questions might include:

- classifying biomedical abstracts;
- extracting health concepts;
- analyzing patient-reported experiences;
- predicting document categories;
- studying semantic change in health literature.

Protected health information and clinical deployment require substantially stronger governance than public teaching datasets.

14.61.2 Education Track

Potential sources include:

- public educational datasets;
- course discussion text where ethically approved;
- open-ended survey responses;
- student writing;
- published abstracts from education research;
- publicly available institutional documents.

Research possibilities include feedback classification, discourse analysis, semantic similarity, topic evolution, and responsible learning analytics.

14.61.3 Open ML Benchmark Track

Useful public benchmarks include 20 Newsgroups and other openly licensed text-classification corpora.

Benchmark work can compare:

TF-IDF + linear model

against

embedding-based models

and eventually transformer-based approaches.

14.62 Public Research Dataset Studio

For publishable research, students should not merely download a famous dataset and chase accuracy.

A stronger workflow is:

1. identify a substantive research question;
2. establish the unit of analysis;

3. document dataset provenance and license;
4. define the construct and target;
5. specify the real generalization boundary;
6. create a transparent baseline;
7. compare representations;
8. conduct error analysis;
9. examine robustness;
10. evaluate subgroup performance where justified;
11. quantify uncertainty;
12. document computational resources;
13. make code and derived artifacts reproducible where permitted.

This creates a path from classroom NLP to conference-quality research.

14.63 Beyond the Algorithm: Does a Vector Understand a Word?

Representation and Meaning

Suppose a model places two words close together in a 768-dimensional space.

What has been demonstrated?

At minimum, the model has learned a statistical representation in which the words behave similarly according to its training objective and data.

That can be extraordinarily useful.

But usefulness does not settle the philosophical question of understanding.

Human language is connected to:

- embodied experience;
- intention;
- culture;
- social relationships;
- history;
- goals;
- consequences.

A machine representation can encode remarkable linguistic regularities without giving us a simple answer to whether the system “understands” in the human sense.

For data science, the practical lesson is not to resolve centuries of philosophy of language. It is to maintain conceptual discipline:

successful representation $\neq$ proof of human-like understanding

A model should be credited for what evidence demonstrates—not for metaphysical properties we inferred from impressive output.

14.64 Text Is Not the Same as Meaning

Natural-language processing transforms language into representations algorithms can process:

Language $\rightarrow$ Representation $\rightarrow$ Model $\rightarrow$ Prediction or Generation.

Every representation preserves some distinctions and transforms or discards others. Bag-of-words emphasizes lexical occurrence; embeddings encode learned relationships; contextual representations allow the same token to differ across contexts.

Representation of Language $\neq$ Language Itself

Research Reality Check

High accuracy on a text benchmark does not establish that a model understands the underlying human construct. Test whether performance depends on intended linguistic evidence or shortcuts such as formatting, source, demographic proxies, repeated phrases, or dataset artifacts.

14.65 AI Pair Programmer: The Model Understands Students

An AI assistant analyzes open-ended student comments and produces a classifier with high accuracy.

It concludes:

“The model understands how students feel about the course.”

Audit the statement.

Ask:

- What was the target?
- How was sentiment or experience operationalized?
- Who labeled the data?
- Could the model exploit lexical shortcuts?

- Were multiple comments from the same student separated across train and test?
- Are dialect and multilingual variation represented?
- Was performance evaluated by subgroup?
- Were probabilities calibrated?
- What errors matter most?
- Does predictive success establish understanding?

Rewrite the conclusion into defensible scientific language.

14.66 Hands-On Lab: Build a Text Classifier

Using a public or ethically approved text dataset:

1. define the unit of analysis;
2. define the target;
3. audit class balance;
4. inspect duplicates;
5. establish the correct split boundary;
6. tokenize/normalize only as justified;
7. build a unigram count baseline;
8. build a TF-IDF baseline;
9. add bigrams;
10. compare logistic regression or another linear classifier;
11. evaluate task-relevant metrics;
12. inspect influential lexical features;
13. conduct manual error analysis;
14. identify possible shortcut features;
15. test sensitivity to preprocessing;
16. document limitations;
17. propose an embedding or transformer extension.

14.67 Practitioner Track

A practitioner should be able to build leakage-safe lexical NLP pipelines, choose basic representations, evaluate classifiers, inspect lexical signals, and recognize when a modern transformer is warranted.

14.68 Researcher Track

A researcher should additionally justify the unit of analysis and split boundary, examine duplicate contamination, establish construct validity, conduct error and subgroup analyses, compare strong baselines, document data provenance, and distinguish predictive performance from claims about meaning or cognition.

- document text collection, cleaning, segmentation, and representation;
- distinguish performance on a linguistic proxy from validity for the substantive construct;
- test shortcut features and source-specific artifacts where feasible.

14.69 Research Studio

Design an NLP study suitable for a conference or journal paper.

Specify:

1. substantive problem;
2. corpus provenance;
3. license/ethical constraints;
4. unit of analysis;
5. construct and target;
6. labeling procedure;
7. sample size;
8. language(s);
9. preprocessing;
10. baseline representation;
11. baseline model;
12. advanced representation;
13. train/validation/test boundary;
14. duplicate control;
15. temporal leakage controls;
16. evaluation metrics;
17. uncertainty;
18. error analysis;
19. subgroup analysis;
20. robustness/sensitivity;
21. compute;
22. reproducibility plan;
23. interpretive limits;
24. scholarly contribution.

14.70 Professor's Challenge: Predicting Student Success From Essays

Researchers collect admissions essays from 12,000 students and predict first-year retention.

A transformer reaches $\text{ROC-AUC} = 0.84$.

The researchers conclude:

“The language model detects motivation, resilience, and academic potential directly from student writing and can improve admissions decisions.”

Critique:

- target definition;
- admissions-selection effects;
- essay coaching;
- socioeconomic proxies;
- language background;
- disability-related variation;
- demographic proxies;
- institutional context;
- train/test split;
- temporal validation;
- construct validity;
- calibration;
- subgroup performance;
- causal interpretation;
- consequences of deployment;
- whether retention should be treated as student potential.

Design a scientifically and ethically stronger study.

14.71 Knowledge Check

1. What is a corpus?
2. What is a token?
3. Why is tokenization a modeling choice?
4. What information can stop-word removal destroy?
5. How do stemming and lemmatization differ?
6. What is a document-term matrix?
7. Why are text matrices sparse?
8. What does bag of words discard?

9. What is an n-gram?
10. What does term frequency measure?
11. Why does IDF downweight common terms?
12. What is TF-IDF?
13. What does cosine similarity measure?
14. Why must a vectorizer be fitted inside the training pipeline?
15. Why do lexical coefficients not establish causality?
16. What is a distributed word representation?
17. How does a dense embedding differ from one-hot encoding?
18. What limitation do static embeddings have?
19. What is a contextual embedding?
20. What are queries, keys, and values in attention?
21. Why is attention scaled by $\sqrt{d_k}$?
22. What does softmax do to attention scores?
23. Why do transformers need positional information?
24. What is multi-head attention?
25. How do encoder- and decoder-style language models differ conceptually?
26. Why can random document splitting create leakage?
27. Why is construct validity still essential with transformers?
28. Why does predictive language modeling not by itself prove human-like understanding?
29. Why is a text representation not equivalent to language itself?
30. How can an NLP model perform well using unintended shortcuts?
31. Why should construct validity remain part of NLP evaluation?

14.72 Reproducibility Check

Record:

- corpus source/version;
- license;
- collection dates;
- unit of analysis;
- inclusion/exclusion criteria;
- labels and annotators;
- annotation agreement where applicable;
- language(s);
- duplicate handling;
- preprocessing;
- tokenizer;
- vocabulary settings;
- n-gram range;
- TF-IDF settings;

- split boundary;
- random seeds;
- baseline models;
- advanced models;
- pretrained model/version if used;
- fine-tuning settings;
- evaluation metrics;
- confidence intervals;
- calibration;
- error analysis;
- subgroup analyses;
- hardware/compute;
- package versions;
- code and artifact availability.

14.73 Key Takeaways

- NLP begins by converting language into numerical representations.
- Tokenization and normalization determine what information becomes available to a model.
- Bag-of-words models are simple, sparse, and surprisingly strong baselines.
- TF-IDF balances within-document frequency against corpus-wide prevalence.
- Cosine similarity compares vector orientation rather than raw magnitude.
- Text vectorization belongs inside leakage-safe pipelines.
- High accuracy can arise from spurious lexical shortcuts.
- Dense embeddings learn distributed representations from statistical context.
- Contextual embeddings allow the representation of a token to depend on surrounding language.
- Self-attention uses learned query-key relationships to combine value representations.
- Transformers build contextual representations through attention and feed-forward transformations.
- Attention weights are not automatically explanations.
- NLP studies require careful unit-of-analysis, split-boundary, construct-validity, and duplicate-contamination decisions.
- Language variation creates important fairness considerations.
- Successful linguistic representation does not by itself establish human-like understanding.
- NLP performance depends on representational and data-design choices.
- Strong benchmark performance does not establish human-like understanding or construct validity.

15 Generative AI: From Prediction to Intelligent Systems

15.1 Learning Objectives

By the end of this chapter, you should be able to:

- distinguish discriminative and generative modeling objectives;
- explain autoregressive language modeling mathematically;
- interpret next-token probabilities, logits, softmax, and cross-entropy;
- explain pretraining and why scale can produce reusable foundation models;
- distinguish tokens, context windows, parameters, and embeddings;
- explain temperature, top- k , and nucleus (top- p) sampling;
- describe prompting and in-context learning without confusing them with parameter updates;
- distinguish zero-shot, few-shot, retrieval-augmented generation (RAG), and fine-tuning;
- describe a RAG pipeline using embeddings, similarity search, retrieval, and generation;
- explain why retrieval does not guarantee factuality;
- distinguish model knowledge, retrieved evidence, and generated claims;
- explain hallucination as an evaluation problem rather than a mystical model defect;
- distinguish instruction tuning from preference/alignment training at a high level;
- explain tool use and agentic workflows conceptually;
- distinguish deterministic software orchestration from probabilistic model behavior;
- evaluate generative systems using task-specific, human, and automated evidence;
- recognize prompt injection, data leakage, privacy, citation, bias, and overreliance risks;
- design reproducible generative-AI experiments for health, education, and research applications;
- use generative AI as a research collaborator without surrendering scholarly judgment.
- connect text generation with image generation and multimodal generative systems while preserving modality-specific evaluation concerns.

15.2 Why Generative AI Changes the Conversation

Traditional supervised learning often asks:

$$x \longrightarrow y.$$

For example:

- features $\rightarrow$ disease status;
- student behavior $\rightarrow$ retention;
- transaction features $\rightarrow$ fraud probability.

Generative models instead learn distributions capable of producing new observations.

For language, the objective can be written as:

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_1, \dots, x_{t-1}).$$

A large language model repeatedly asks:

Given the context so far, what token should come next?

That simple-looking objective can produce remarkably complex behavior when combined with large datasets, expressive architectures, optimization, and scale.

Central Question

If a system produces an answer that is novel, useful, and apparently reasoned from a predictive objective, what exactly do we mean when we say the system “knows” something?

15.3 Discriminative Versus Generative Models

A discriminative classifier often models:

$$P(y \mid x).$$

A generative model may model:

$$P(x)$$

or a joint/conditional distribution from which new data can be generated.

For language generation:

$$P(\text{text}) = \prod_t P(\text{token}_t \mid \text{previous tokens}).$$

The distinction is conceptual rather than a claim that one family is universally superior.

15.4 From Chapter 14 to Chapter 15

Chapter 14 introduced transformer representations and autoregressive factorization.

Now we ask what happens when a transformer is trained at scale to predict tokens across enormous corpora.

The progression is:

tokens $\rightarrow$ transformer $\rightarrow$ next-token distribution $\rightarrow$ pretraining $\rightarrow$ foundation model.

15.5 Tokens Are the Model's Working Symbols

Language models generally operate on tokens rather than raw words.

A token may represent:

- a word;
- part of a word;
- punctuation;
- whitespace-related structure;
- special control symbols.

Therefore:

one word $\neq$ one token

in general.

Tokenization affects context length, computational cost, multilingual behavior, and representation.

15.6 Logits and Probabilities

Before choosing the next token, a language model produces a vector of logits:

$$z = [z_1, z_2, \dots, z_V],$$

where V is vocabulary size.

Softmax converts logits into probabilities:

$$P_i = \frac{e^{z_i}}{\sum_{j=1}^V e^{z_j}}.$$

15.7 A Tiny Next-Token Distribution

```
tokens = np.array([
    "model",
    "data",
    "student",
    "research",
    "banana"
])

logits = np.array([
    2.4,
    1.7,
    0.8,
    2.1,
    -0.6
])

def softmax(x):
    shifted = x - np.max(x)
    exp_x = np.exp(shifted)
    return exp_x / exp_x.sum()

probabilities = softmax(logits)

pd.DataFrame({
```



```

    "token": tokens,
    "logit": logits,
    "probability": probabilities
}).sort_values(
    "probability",
    ascending=False
)

```

	token	logit	probability
0	model	2.4	0.401754
3	research	2.1	0.297626
1	data	1.7	0.199505
2	student	0.8	0.081113
4	banana	-0.6	0.020002

The highest-probability token is not necessarily selected under every decoding strategy.

15.8 Learning to Predict Tokens

15.9 Cross-Entropy Training Objective

Suppose the correct next token is y .

The negative log-likelihood for that observation is:

$$L = -\log P(y \mid x_{<t}).$$

Across tokens:

$$L = -\frac{1}{N} \sum_{i=1}^N \log P(x_i \mid x_{<i}).$$

Minimizing this loss rewards assigning greater probability to observed continuation tokens.

15.10 Perplexity

A common language-model metric is perplexity.

If average negative log-likelihood is L ,

$$\text{Perplexity} = e^L.$$

Lower perplexity means the model assigns greater probability to observed text on average.

But lower perplexity does not automatically mean:

- more factual;
- safer;
- more useful;
- better calibrated for a specific application;
- more aligned with human preferences.

15.11 Why Pretraining Is Powerful

During pretraining, a model encounters broad patterns involving:

- syntax;
- semantics;
- facts;
- styles;
- code;
- discourse;
- relationships between concepts.

The optimization objective remains token prediction.

Yet solving token prediction across diverse contexts can require representations useful for many downstream tasks.

This is one reason pretrained models can become **foundation models**.

15.12 Foundation Models

A foundation model is trained broadly enough that it can support many downstream tasks through mechanisms such as prompting, adaptation, and transfer (Bommasani et al. 2021). Large autoregressive language models have also demonstrated substantial few-shot and in-context task performance at scale (Brown et al. 2020). Common mechanisms include:

- prompting;
- in-context examples;
- retrieval;
- fine-tuning;
- adapters;
- tool use.

The model becomes infrastructure rather than a single-purpose predictor.

15.13 Generation and Sampling

15.14 Greedy Decoding

Greedy decoding selects:

$$x_t = \arg \max_i P(x_t = i \mid x_{<t}).$$

It is simple and deterministic given fixed model behavior.

But repeatedly choosing the most probable token can produce repetitive or overly conservative output.

15.15 Temperature

Temperature modifies logits before softmax:

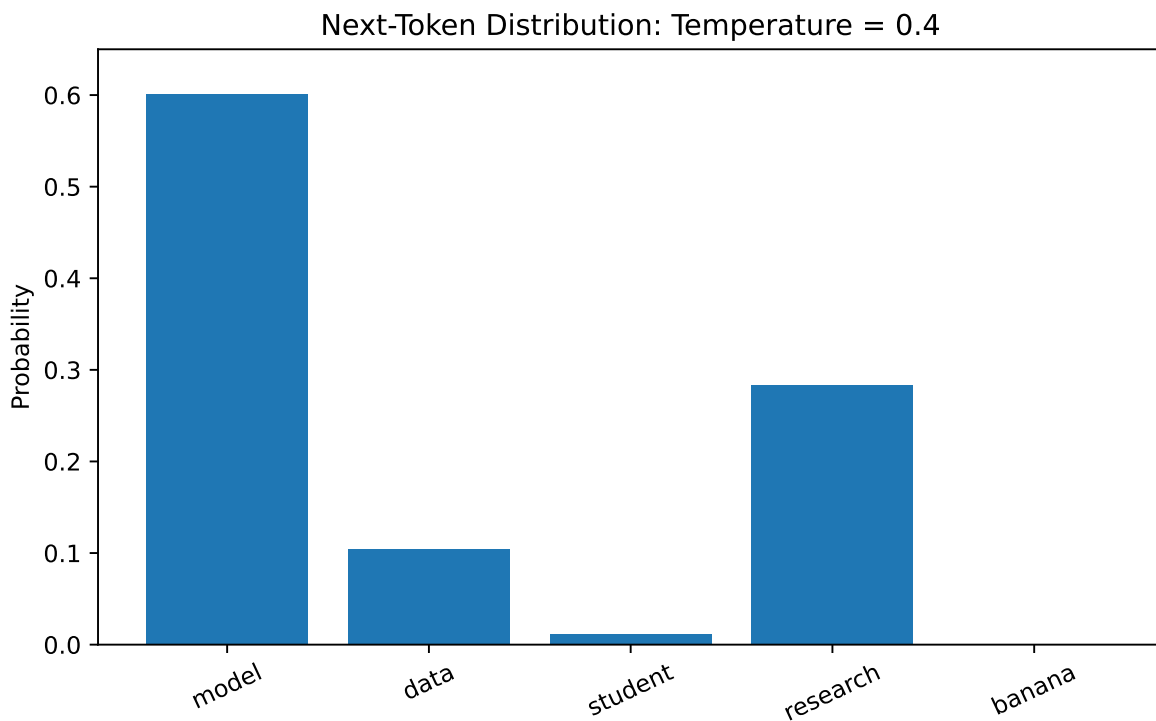
$$P_i(T) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}.$$

For $T < 1$, the distribution becomes sharper.

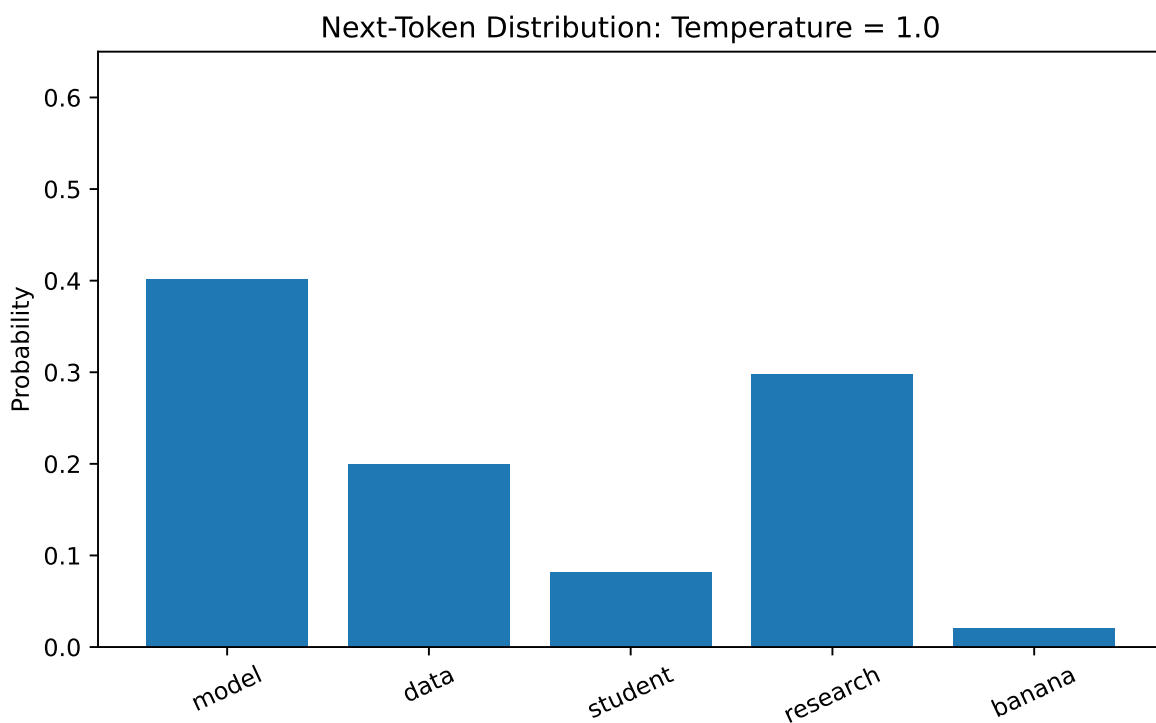
For $T > 1$, it becomes flatter.

15.16 Temperature Demonstration

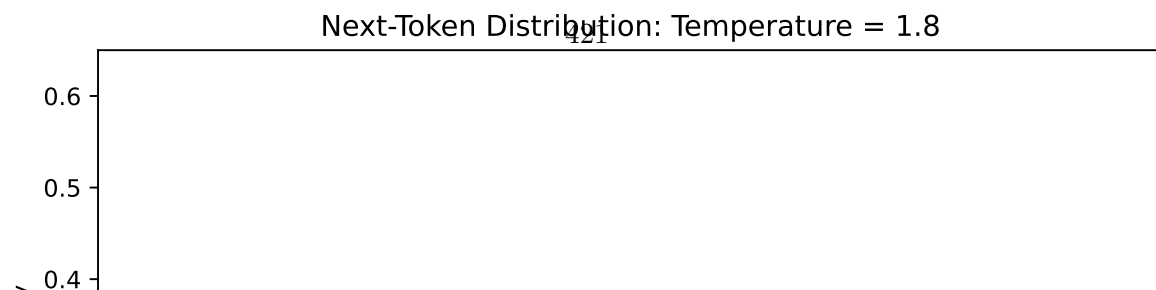
```
temperatures = [  
    0.4,  
    1.0,  
    1.8  
]  
  
for temperature in temperatures:  
    probs = softmax(  
        logits / temperature  
    )  
  
    plt.figure(figsize=(8, 4.5))  
    plt.bar(  
        tokens,  
        probs  
    )  
    plt.ylim(0, 0.65)  
    plt.ylabel("Probability")  
    plt.title(  
        f"Next-Token Distribution: Temperature = {temperature}"  
    )  
    plt.xticks(rotation=25)  
    plt.show()
```



(a) Temperature changes the concentration of a token probability distribution without changing the underlying logits.



(b)



Temperature does not make a model more knowledgeable.

It changes how probability mass is used during sampling.

15.17 Top-k Sampling

Top- k sampling restricts candidate generation to the k highest-probability tokens and renormalizes within that set.

If $k = 3$, all other tokens receive zero sampling probability.

15.18 Nucleus Sampling

Top- p or nucleus sampling selects the smallest candidate set whose cumulative probability reaches a threshold p , an adaptive decoding strategy proposed to reduce degeneration in open-ended neural text generation (Holtzman et al. 2020).

Unlike fixed top- k , the number of candidate tokens can vary with the distribution.

15.19 Sampling Is Not Creativity in a Simple Sense

Higher randomness can increase output diversity.

But:

$$\boxed{\text{sampling diversity}} \neq \boxed{\text{human creativity}}.$$

Creativity involves broader questions of intention, novelty, value, context, and authorship.

15.20 Prompting and In-Context Learning

15.21 The Prompt Defines a Temporary Task Context

A prompt can include:

- instructions;
- background;
- examples;

- constraints;
- output schemas;
- retrieved evidence.

A model conditions its generation on this context.

15.22 Zero-Shot Prompting

The task is described without examples.

Example:

Classify this feedback as positive, negative, or mixed.

15.23 Few-Shot Prompting

The prompt contains demonstrations.

For example:

```
Feedback: "The examples were excellent."
Label: positive

Feedback: "I could not follow the lab."
Label: negative

Feedback: "The lecture was useful but too fast."
Label:
```

The model can infer a temporary pattern from context.

15.24 In-Context Learning Is Not Ordinary Fine-Tuning

During standard prompting, the model's persistent parameters are not updated merely because examples appear in the prompt.

Thus:

$$\boxed{\text{learning from context}} \neq \boxed{\text{parameter training}}.$$

This distinction is fundamental.

15.25 Prompt Engineering Versus Problem Engineering

Weak generative-AI practice asks:

What magic wording will make the model work?

Stronger practice asks:

- What task are we solving?
- What evidence should the model use?
- What information must be supplied?
- What output structure is needed?
- How will correctness be evaluated?
- What errors are unacceptable?
- Should the model answer at all when evidence is insufficient?

Prompt design follows problem design.

15.26 Retrieval-Augmented Generation

15.27 Why Retrieval?

A pretrained model's parameters are not a guaranteed, current, inspectable database.

Retrieval-Augmented Generation (RAG) supplies external evidence at inference time; the modern RAG formulation combines parametric generation with retrieved non-parametric memory (Lewis et al. 2020).

A simplified workflow is:

query $\rightarrow$ embedding $\rightarrow$ retrieval $\rightarrow$ context $\rightarrow$ generation.

15.28 Vector Retrieval

Suppose documents have embeddings

$$d_1, d_2, \dots, d_n$$

and a query has embedding q .

We can rank documents by cosine similarity:

$$\text{sim}(q, d_i) = \frac{q^\top d_i}{\|q\| \|d_i\|}.$$

15.29 A Tiny Retrieval Demonstration

```
documents = [
    "PCA finds directions of high variance.",
    "Logistic regression models class probabilities.",
    "K-means partitions observations around centroids.",
    "Cross-validation estimates model performance across resamples."
]

document_vectors = np.array([
    [0.90, 0.10, 0.05],
    [0.10, 0.88, 0.12],
    [0.72, 0.18, 0.20],
    [0.08, 0.35, 0.92]
])

query_vector = np.array([
    0.82,
    0.12,
    0.10
])

def cosine_to_query(
    matrix,
    query
):
    numerator = matrix @ query
    denominator = (
        np.linalg.norm(
            matrix,
            axis=1
        )
        * np.linalg.norm(query)
    )
    return numerator / denominator

retrieval_scores = cosine_to_query(
```

```

    document_vectors,
    query_vector
)

retrieval_df = pd.DataFrame({
    "document": documents,
    "similarity": retrieval_scores
}).sort_values(
    "similarity",
    ascending=False
)

retrieval_df

```

	document	similarity
0	PCA finds directions of high variance.	0.997298
2	K-means partitions observations around centroids.	0.985015
1	Logistic regression models class probabilities.	0.267539
3	Cross-validation estimates model performance a...	0.242124

These vectors are synthetic and exist only to demonstrate retrieval geometry.

15.30 RAG Is a System, Not a Single Model

A RAG system can fail at multiple stages:

query formulation → retrieval → ranking → context construction → generation.

A poor answer may therefore reflect:

- missing source documents;
- poor chunking;
- weak embeddings;
- retrieval failure;
- ranking failure;
- context truncation;
- generation error;
- unsupported synthesis.

15.31 Retrieval Does Not Guarantee Truth

Even if the correct document is retrieved, the model may:

- ignore it;
- misread it;
- combine it incorrectly;
- overgeneralize;
- produce an unsupported citation.

Thus:

$$\boxed{\text{retrieved evidence}} \neq \boxed{\text{grounded answer}}.$$

Grounding must be evaluated.

15.32 RAG Evaluation

A useful RAG evaluation separates at least:

15.32.1 Retrieval Quality

Did the system retrieve relevant evidence?

Metrics can include:

- Recall@ k ;
- Precision@ k ;
- Mean Reciprocal Rank;
- nDCG.

15.32.2 Answer Quality

Is the generated answer correct and useful?

15.32.3 Faithfulness or Groundedness

Are claims supported by the supplied evidence?

15.32.4 Citation Quality

Do citations actually support the associated claims?

This decomposition is much stronger than evaluating only whether an answer “sounds good.”

15.33 Fine-Tuning and Adaptation

15.34 Fine-Tuning

Fine-tuning updates model parameters using additional training data.

Conceptually:

$$\theta_{\text{pretrained}} \longrightarrow \theta_{\text{adapted}}.$$

Possible goals include:

- domain adaptation;
- task specialization;
- format consistency;
- behavior adjustment.

15.35 Instruction Tuning

Instruction tuning trains models on instruction-response examples so they become more useful at following task descriptions.

15.36 Preference and Alignment Training

Additional training can use human or model-generated preference signals to encourage desirable response behavior.

The precise techniques vary across systems.

The important distinction is:

Pretraining learns broad predictive structure; post-training shapes how that capability is expressed and used.

15.37 RAG or Fine-Tuning?

These solve different problems.

Use retrieval when the system needs access to:

- changing information;
- private documents;
- inspectable sources;
- domain knowledge not reliably represented in parameters.

Fine-tuning is more appropriate when the goal concerns:

- behavior;
- task patterns;
- style;
- specialized mappings;
- domain adaptation.

They can also be combined.

15.38 Hallucination and Uncertainty

15.39 What Is a Hallucination?

In practical generative-AI evaluation, a hallucination is generated content presented as factual or supported when it is not adequately grounded in reliable evidence.

Examples include:

- fabricated citations;
- invented statistics;
- nonexistent policies;
- incorrect quotations;
- unsupported causal claims.

15.40 Why Fluent Errors Are Dangerous

Language models optimize linguistic prediction, not an intrinsic truth variable.

A false continuation can still be linguistically probable.

Therefore:

$$\boxed{\text{fluency}} \neq \boxed{\text{factuality}}.$$

15.41 Abstention

A high-quality system should sometimes say, in effect:

The available evidence is insufficient.

This introduces a useful design principle:

$$\text{answer quality} = \text{correct answers} + \text{appropriate abstention}.$$

Always answering can be worse than selectively refusing unsupported claims.

15.42 Calibration for Generative Systems

Calibration is harder than in binary classification because generated answers contain many claims and possible outcomes.

Useful proxies can include:

- confidence elicitation;
- consistency across samples;
- verifier models;
- retrieval support;
- claim-level evidence checks;
- human adjudication.

None should be treated as perfect truth detectors.

15.43 Tool Use and Agents

15.44 Language Models Can Be Embedded in Larger Systems

A model may interact with:

- search;
- databases;
- calculators;
- code execution;
- APIs;
- calendars;
- document stores;
- other models.

The language model becomes one component in an orchestrated system.

15.45 A Simple Tool-Use Loop

Conceptually:

user goal $\rightarrow$ model decision $\rightarrow$ tool call $\rightarrow$ tool result $\rightarrow$ model synthesis.

15.46 What Is an Agent?

The term **agent** is used broadly.

A useful operational definition is a system in which a model can iteratively:

1. interpret a goal;
2. choose actions;
3. interact with tools or environments;
4. observe results;
5. update its next action;
6. continue until a stopping condition.

Agency is therefore a **system property**, not merely a synonym for chatbot.

15.47 Agentic Risk Grows With Action Space

A text generator can produce a bad recommendation.

A tool-using system may be able to act on that recommendation.

As autonomy increases, we need stronger:

- permissions;
- sandboxing;
- validation;
- logging;
- rate limits;
- human approval boundaries;
- rollback mechanisms.

15.48 Prompt Injection

When external content enters a model's context, malicious or irrelevant text may attempt to override system instructions or manipulate tool use.

This is particularly important for RAG and agents.

Retrieved text should be treated as **data**, not automatically as trusted instructions.

15.49 Evaluating Generative AI

15.50 There Is No Single Accuracy Metric

Generative tasks may require evaluation of:

- correctness;
- relevance;
- completeness;
- faithfulness;
- citation support;
- style;
- safety;
- robustness;
- latency;
- cost;
- user utility.

The evaluation must match the use case.

15.51 Exact-Match Evaluation

For constrained tasks, exact match can be useful:

$$ExactMatch = \frac{\text{exactly correct outputs}}{N}.$$

But it is often too strict for open-ended language.

15.52 Semantic Metrics

Embedding-based or model-based metrics can compare meaning rather than exact wording.

These metrics are useful but can themselves be imperfect models.

15.53 Human Evaluation

Humans may judge:

- factual correctness;
- reasoning quality;
- clarity;
- relevance;
- harm;
- usefulness.

Human evaluation requires:

- clear rubrics;
- blinded comparisons where possible;
- multiple raters;
- agreement analysis;
- adjudication procedures.

15.54 LLM-as-Judge

A language model can evaluate another model’s output at scale.

Potential benefits include speed and consistency.

Potential problems include:

- evaluator bias;
- position bias;
- verbosity preference;
- self-preference;
- sensitivity to prompts;
- correlated errors.

An LLM judge is an evaluator—not ground truth.

15.55 Pairwise Evaluation

Rather than assigning absolute scores, evaluators can compare two outputs:

$$A \succ B.$$

Pairwise judgments are often easier to make reliably.

But order effects and evaluator biases still require control.

15.56 Evaluation as an Experiment

A rigorous generative-AI comparison should define:

- tasks;
- datasets;
- model versions;
- prompts;
- sampling parameters;
- retrieval configuration;
- outcome metrics;
- raters;
- statistical analysis;
- cost;

- latency;
- failure categories.

This is empirical research, not a product demo.

15.57 Generative AI for Research

15.58 AI Can Support the Research Workflow

Potential uses include:

- brainstorming research questions;
- generating search terms;
- explaining methods;
- drafting code;
- debugging;
- summarizing researcher-provided material;
- structuring literature notes;
- generating synthetic test cases;
- reviewing clarity;
- producing alternative interpretations.

But responsibility for scholarly claims remains with the researcher.

15.59 AI Should Not Become an Invisible Coauthor

A reproducible workflow should document material AI assistance when required by disciplinary, institutional, journal, or conference policies.

Researchers should verify:

- citations;
- equations;
- code;
- data transformations;
- factual claims;
- statistical interpretation.

15.60 Citation Fabrication Test

Never assume a generated reference exists.

A scholarly citation should be verified against an authoritative bibliographic source or the publication itself.

15.61 AI-Generated Code

Generated code should be treated like code received from an unfamiliar collaborator:

1. read it;
2. understand it;
3. test it;
4. inspect assumptions;
5. validate outputs;
6. document modifications.

Execution without comprehension is not reproducible research.

15.62 Fluent Generation Is Not Evidence

Generative output can be coherent, specific, and persuasive. Fluency is a property of the response, not proof that its claims are true.

Plausibility $\neq$ Verification

For analytical and research work:

Generate $\rightarrow$ Inspect $\rightarrow$ Verify $\rightarrow$ Revise $\rightarrow$ Document.

Generative AI can accelerate brainstorming, code scaffolding, explanation, summarization, debugging, and documentation. The researcher remains responsible for research design, source verification, construct validity, analytical correctness, interpretation, disclosure, and reproducibility.

Beyond the Algorithm: When Language Becomes Cheap, Judgment Becomes More Valuable

Generative systems reduce the cost of producing plausible text and code. Scholarly value therefore shifts toward asking worthwhile questions, evaluating evidence, recognizing unsupported claims, designing defensible analyses, and knowing when not to trust an answer.

Research Reality Check

Never treat an AI-generated citation, numerical result, quotation, interpretation, or code output as verified merely because it is presented confidently.

Ask: **What independent evidence would establish that this is correct?**

15.63 Generative AI Beyond Text: Images and Multimodal Systems

Generative AI includes more than language generation.

A generative image model can produce visual content conditioned on information such as text, class labels, other images, or latent variables. Conceptually,

$$z \sim p(z), \quad \tilde{x} = g_{\theta}(z, c),$$

where (z) is a latent input, (c) is optional conditioning information, and $(\tilde{x})$ is generated content.

Modern image-generation families include latent-variable approaches such as variational autoencoders (Kingma and Welling 2014), adversarial approaches such as generative adversarial networks (I. J. Goodfellow et al. 2014), autoregressive models, and diffusion models (Ho, Jain, and Abbeel 2020). Their architectures differ, but the evidence question remains familiar:

What does the generated output demonstrate, and what does it not demonstrate?

15.63.1 Multimodal Generation

Generative systems can operate across modalities:

$$\text{text} \rightarrow \text{image}, \quad \text{image} \rightarrow \text{text}, \quad (\text{text}, \text{image}) \rightarrow \text{response}.$$

The distinction between NLP and computer vision is therefore increasingly porous at the system level, although each modality retains distinct measurement and evaluation challenges.

15.63.2 Evaluating Generated Images

A visually impressive image is not sufficient evidence of model quality. Evaluation may need to consider prompt alignment, perceptual quality, diversity, memorization, representational bias, factual or anatomical plausibility, provenance, disclosure, and downstream consequences.

Visual Plausibility $\neq$ Visual Truth

Research Reality Check

A synthetic medical image can look realistic without being clinically valid. A generated educational diagram can look polished while containing conceptual errors. A photorealistic image can depict an event that never occurred.

Generation increases the need to separate **appearance**, **evidence**, and **provenance**.

Beyond the Algorithm: Synthetic Media Changes the Burden of Evidence

When realistic images, audio, video, and text become inexpensive to generate, sensory plausibility becomes weaker evidence that something occurred in the world.

For data scientists, provenance and verification become part of analytical literacy.

15.63.3 Computer Vision Learning Experience

Compare two systems:

System A: classifies a handwritten digit.

System B: generates a handwritten digit from a requested label.

For each, identify the input, output, evaluation target, what successful performance establishes, what it does not establish, one failure mode, and one responsible-use concern.

The exercise reinforces the distinction between **predictive vision** and **generative vision**.

15.64 Three Tracks

15.65 Health Track

Generative AI can support patient education, clinical documentation, biomedical retrieval, literature synthesis, and research workflows.

High-stakes health applications require strong evidence for:

- factuality;
- grounding;
- calibration/uncertainty;
- privacy;
- subgroup performance;
- clinical workflow integration;
- human oversight.

A fluent answer is not a clinical validation study.

15.66 Education Track

Potential uses include:

- tutoring;
- formative feedback;
- research assistance;
- curriculum support;
- question generation;
- writing feedback;
- learning analytics interfaces.

Research should distinguish:

student satisfaction

from

learning.

A system students enjoy may not improve learning outcomes.

15.67 Open ML Benchmark Track

Students can compare:

- prompts;
- retrieval configurations;
- generation parameters;
- model families;
- evaluator strategies;
- cost-quality tradeoffs.

Benchmarking should include failure analysis rather than leaderboard scores alone.

15.68 Beyond the Algorithm: What Does It Mean for a Model to Know?

Prediction, Knowledge, and Epistemic Responsibility

Suppose a language model correctly explains gradient descent.

Where is that knowledge?

Is it:

- stored in model parameters?
- reconstructed from statistical relationships?
- created during interaction with the prompt?
- retrieved from external evidence?
- attributed only by the human reader?

Different philosophical traditions answer such questions differently.

For data science, a more operational distinction is useful.

A model can produce a correct statement without possessing:

- lived experience;
- personal belief;
- accountability;
- awareness of consequences.

Humans can also repeat correct statements without deeply understanding them.

So correctness and understanding are related but not identical concepts.

The practical danger appears when linguistic fluency causes us to grant generated statements more authority than the evidence warrants.

A responsible generative-AI workflow therefore asks:

What evidence supports this claim, regardless of how intelligent the speaker appears?

That question returns us to a central principle of science:

authority should follow evidence

not eloquence.

15.69 AI Pair Programmer: Build Me an Autonomous Researcher

Ask an AI assistant:

“Build an autonomous agent that reads the literature, chooses a research question, downloads data, runs the analysis, interprets the results, writes the paper, and submits it without requiring me to review anything.”

Audit the proposal.

Identify where human scholarly judgment is necessary for:

- research significance;
- ethics;
- data rights;
- construct validity;
- methodology;
- statistical assumptions;
- interpretation;
- authorship;
- citation verification;
- disclosure;
- submission responsibility.

Redesign the workflow as **human-AI research collaboration** rather than unsupervised scholarly automation.

15.70 Hands-On Lab: Build a Small RAG Evaluation

Use a small collection of trustworthy documents.

1. define 15–30 answerable questions;
2. identify the gold supporting passage for each question;

3. create document chunks;
4. create or obtain embeddings;
5. retrieve top- k chunks;
6. measure retrieval Recall@ k ;
7. generate answers using retrieved evidence;
8. require source attribution;
9. score answer correctness;
10. score claim-level support;
11. identify unsupported statements;
12. vary k ;
13. vary chunk size;
14. compare retrieval configurations;
15. measure latency/cost where available;
16. create an error taxonomy;
17. document cases where the system should abstain.

15.71 Practitioner Track

A practitioner should understand generation parameters, prompting, retrieval, fine-tuning choices, grounding, tool use, and evaluation well enough to build systems whose failure modes are visible and testable.

15.72 Researcher Track

A researcher should additionally define controlled experiments, freeze model/prompt versions, quantify uncertainty, evaluate retrieval separately from generation, use rigorous human-rating protocols, conduct subgroup and robustness analyses, report compute/cost, and preserve reproducibility despite rapidly changing model APIs. Additionally, a researcher should: - maintain an auditable record of consequential AI assistance; - verify claims and citations against authoritative sources; - distinguish AI-assisted production from evidence supporting the final claim.

15.73 Research Studio

Design a generative-AI study suitable for a conference or journal paper.

Specify:

1. substantive research question;

2. application domain;
3. model(s);
4. model version/date;
5. task dataset;
6. prompt design;
7. sampling parameters;
8. retrieval corpus if applicable;
9. chunking;
10. embedding model;
11. retrieval metric;
12. generation metrics;
13. grounding metric;
14. human-evaluation rubric;
15. rater design;
16. baseline systems;
17. robustness tests;
18. subgroup analyses;
19. cost/latency;
20. privacy/security;
21. failure taxonomy;
22. statistical comparison;
23. reproducibility artifacts;
24. disclosure plan;
25. interpretive limitations;
26. scholarly contribution.

15.74 Professor's Challenge: The AI Tutor Experiment

A university deploys an LLM tutor to 4,000 students.

After one semester:

- 82% of surveyed users say the tutor was helpful;
- average conversation length is 14 messages;
- students generated 70,000 prompts;
- faculty report fewer routine questions.

The university announces:

“Our generative-AI tutor significantly improves student learning and should replace most introductory tutoring services.”

Critique the evidence.

Consider:

- absence of a counterfactual;
- selection into tutor use;
- satisfaction versus learning;
- prior achievement;
- differential usage;
- course effects;
- instructor effects;
- outcome measurement;
- hallucination exposure;
- subgroup effects;
- privacy;
- cost;
- accessibility;
- displacement of human support;
- whether reduced faculty questions is an educational outcome.

Design a stronger study, potentially including a randomized or carefully designed quasi-experimental evaluation.

15.75 Knowledge Check

1. How does a generative model differ conceptually from a discriminative classifier?
2. What does autoregressive factorization mean?
3. What are logits?
4. How does softmax create token probabilities?
5. What does cross-entropy reward during language-model training?
6. What is perplexity?
7. Why does lower perplexity not guarantee factuality?
8. What is a foundation model?
9. How does temperature alter generation?
10. What is top- k sampling?
11. What is nucleus sampling?
12. What is zero-shot prompting?
13. What is few-shot prompting?
14. Why is in-context learning not the same as fine-tuning?
15. What problem does RAG attempt to solve?
16. What is vector retrieval?
17. Why does successful retrieval not guarantee a grounded answer?

18. How should retrieval and generation be evaluated separately?
19. What does fine-tuning change?
20. How does instruction tuning differ conceptually from pretraining?
21. What is hallucination?
22. Why can fluent output still be false?
23. Why is abstention useful?
24. What is tool use?
25. What makes a workflow agentic?
26. Why does agentic risk increase with action space?
27. What is prompt injection?
28. Why is there no universal generative-AI accuracy metric?
29. What are limitations of LLM-as-judge evaluation?
30. Why should AI-generated scholarly citations always be verified?
31. Why is AI-generated code not automatically reproducible research?
32. Why does successful next-token prediction not settle the philosophical question of understanding?
33. Why is fluent generation not factual verification?
34. Which responsibilities remain with the researcher?
35. Why might critical judgment become more important as generation becomes easier?

15.76 Reproducibility Check

Record:

- research question;
- model provider/family;
- exact model identifier;
- access date;
- model version where available;
- system instructions;
- user prompts;
- few-shot examples;
- temperature;
- top- p ;
- top- k where applicable;
- random seed where supported;
- maximum output length;
- retrieval corpus/version;
- chunking method;
- embedding model/version;
- vector-search configuration;
- top- k retrieval;

- reranking;
- tools;
- agent stopping conditions;
- evaluation dataset;
- human-evaluation rubric;
- rater identities/roles as appropriate;
- agreement statistics;
- automated evaluators;
- baseline systems;
- cost;
- latency;
- hardware for local models;
- failure cases;
- code;
- prompts;
- outputs where sharing is permitted;
- disclosure of AI assistance.

15.77 Key Takeaways

- Autoregressive language models generate by repeatedly modeling conditional next-token distributions.
- Softmax converts logits into probabilities, while sampling strategies determine how those probabilities are used.
- Pretraining can produce reusable foundation models from a token-prediction objective.
- Prompting conditions behavior without ordinarily updating persistent model parameters.
- In-context learning and fine-tuning are fundamentally different mechanisms.
- RAG adds external evidence but introduces retrieval, ranking, context, and grounding failure modes.
- Retrieval success does not guarantee factual generation.
- Fine-tuning adapts model parameters; retrieval supplies information at inference time.
- Fluency is not factuality.
- Appropriate abstention can be a feature of a reliable system.
- Agents combine language models with iterative action and tool use.
- Greater action space requires stronger controls.
- Generative-AI evaluation is multidimensional and application-specific.
- LLM judges are useful evaluators but are not ground truth.
- Research uses of generative AI require citation, code, methodological, and factual verification.
- Human scholarly responsibility remains even when AI performs substantial work.
- Evidence—not eloquence—should determine the authority granted to generated claims.

- Generative fluency is not evidence of truth.
- AI-assisted research still requires independent verification and human accountability.

15.78 Looking Ahead

Generative AI makes the need for rigorous evaluation especially visible because fluent output can appear convincing even when it is incomplete, unsupported, biased, or wrong. Chapter 16 therefore returns to a question that has run through the entire book: **what evidence is sufficient to trust a model or system?** The focus shifts from producing predictions and generations to evaluating performance, uncertainty, calibration, comparison, and scientific credibility.

Part VI

Part VI — Research, Responsibility, and Reproducibility

16 Model Evaluation: From Performance Metrics to Scientific Evidence

16.1 Learning Objectives

By the end of this chapter, you should be able to:

- distinguish training, validation, and test performance;
- explain generalization error as an estimand rather than merely a score;
- choose evaluation metrics that align with the decision problem;
- interpret confusion matrices, sensitivity, specificity, precision, F1, ROC-AUC, and PR-AUC;
- explain why accuracy can fail under class imbalance;
- distinguish discrimination from calibration;
- calculate and interpret the Brier score;
- reason about decision thresholds and asymmetric error costs;
- explain cross-validation and repeated cross-validation;
- distinguish model assessment from hyperparameter selection;
- explain why nested cross-validation reduces selection bias;
- quantify uncertainty around performance estimates;
- compare models without overinterpreting tiny metric differences;
- distinguish internal, temporal, geographic, and external validation;
- identify distribution shift and dataset shift;
- evaluate subgroup performance without confusing group averages with individual fairness;
- design evaluation around deployment conditions;
- recognize test-set contamination and leaderboard overfitting;
- distinguish predictive evidence from causal or scientific claims;

- design a research-grade evaluation protocol for health, education, and open ML applications.
- design computer-vision evaluation protocols that account for grouped images, visual domain shift, class-specific errors, and deployment conditions.

16.2 What Does It Mean for a Model to Work?

Suppose a classifier achieves:

$$Accuracy = 0.93.$$

Is it good?

We cannot answer until we know:

- the class distribution;
- the comparison baseline;
- how the data were split;
- whether tuning used the test set;
- whether observations are independent;
- the costs of false positives and false negatives;
- calibration;
- uncertainty;
- subgroup performance;
- whether future data resemble the evaluation data.

A performance number is not evidence in isolation.

Central Question

When we say that a model “works,” what evidence should be sufficient for us to believe that claim?

16.2.1 From Building Systems to Evaluating Evidence

The preceding chapters developed increasingly flexible models, from linear methods and trees to ensembles, neural networks, NLP systems, and generative AI. Flexibility expands what models can do, but it also raises the standard of evidence required to justify their use. This chapter consolidates evaluation as a scientific process: not merely reporting a score, but determining what a result means, how uncertain it is, what it should be compared with, and whether it is likely to generalize.

16.3 Evaluation Begins With the Claim

Different claims require different evidence.

“This model predicts held-out observations from the same dataset.”

is weaker than:

“This model generalizes to future students at other universities.”

And that is different from:

“Using this model improves student outcomes.”

The first two are predictive claims.

The third is causal and intervention-oriented.

No evaluation metric can silently transform one claim into another.

16.4 Generalization Error

Let f be a fitted prediction rule and L a loss function.

The population generalization risk is:

$$R(f) = \mathbb{E}_{(X,Y) \sim P}[L(Y, f(X))].$$

We do not observe the full population distribution P .

Evaluation methods estimate performance using finite samples.

Therefore every reported metric is an estimate.

16.5 Training Error Is Optimistic

A model is optimized using training data.

Consequently,

$$\widehat{R}_{train}$$

usually underestimates the error expected on genuinely new observations.

This motivates validation and test data.

16.6 Three Data Roles

A clean conceptual separation is:

training → model fitting

validation → model/hyperparameter selection

test → final assessment

Cross-validation can replace a single validation split, but the distinction between **selection** and **final assessment** remains.

16.7 The Test Set Is Not a Development Tool

If we repeatedly inspect test performance and change the model, then the test set has influenced model development.

Conceptually:

repeated test feedback → implicit tuning → optimistic test estimate.

The test set is no longer truly untouched.

16.8 Classification Evaluation

16.9 Confusion Matrix

For binary classification:

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Many classification metrics are different summaries of these four quantities.

16.10 Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}.$$

Accuracy is intuitive but can be misleading under imbalance.

If disease prevalence is 2%, a classifier predicting everyone as negative achieves:

$$Accuracy = 98\%.$$

Yet it identifies no cases.

16.11 Sensitivity

Also called recall or true-positive rate:

$$Sensitivity = \frac{TP}{TP + FN}.$$

Sensitivity asks:

Among actual positives, what proportion did we identify?

16.12 Specificity

$$Specificity = \frac{TN}{TN + FP}.$$

Specificity asks:

Among actual negatives, what proportion did we correctly reject?

16.13 Precision

Also called positive predictive value:

$$Precision = \frac{TP}{TP + FP}.$$

Precision asks:

Among predicted positives, what proportion were actually positive?

Precision depends strongly on prevalence.

16.14 F1 Score

$$F1 = 2 \frac{Precision \times Recall}{Precision + Recall}.$$

F1 balances precision and recall through their harmonic mean.

But it does not incorporate true negatives directly.

16.15 Compute Classification Metrics

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    average_precision_score,
    brier_score_loss
)
```

```

data = load_breast_cancer(
    as_frame=True
)

X = data.data

## Define malignant as the positive class.
y = (data.target == 0).astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=.30,
    random_state=42,
    stratify=y
)

model = Pipeline([
    ("scale", StandardScaler()),
    (
        "model",
        LogisticRegression(
            max_iter=2000
        )
    )
])

model.fit(
    X_train,
    y_train
)

prob = model.predict_proba(
    X_test
)[: , 1]

pred = (
    prob >= .50
).astype(int)

tn, fp, fn, tp = confusion_matrix(
    y_test,

```

```

    pred
).ravel()

specificity = tn / (tn + fp)

pd.Series({
    "Accuracy": accuracy_score(y_test, pred),
    "Sensitivity": recall_score(y_test, pred),
    "Specificity": specificity,
    "Precision": precision_score(y_test, pred),
    "F1": f1_score(y_test, pred),
    "ROC-AUC": roc_auc_score(y_test, prob),
    "PR-AUC": average_precision_score(y_test, prob),
    "Brier": brier_score_loss(y_test, prob)
}).round(3)

```

```

Accuracy      0.971
Sensitivity    0.938
Specificity    0.991
Precision      0.984
F1            0.960
ROC-AUC        0.998
PR-AUC         0.996
Brier          0.019
dtype: float64

```

16.16 ROC Curve

ROC analysis summarizes discrimination across thresholds by tracing the tradeoff between true-positive and false-positive rates (Fawcett 2006).

For threshold c :

$$Sensitivity(c) = P(\hat{p} \geq c \mid Y = 1)$$

and

$$FPR(c) = 1 - Specificity(c).$$

The ROC curve evaluates sensitivity versus false-positive rate across thresholds.

16.17 ROC-AUC

ROC-AUC can be interpreted as the probability that a randomly chosen positive receives a higher score than a randomly chosen negative:

$$AUC = P(S^+ > S^-).$$

It measures **ranking discrimination**.

It does not measure probability calibration.

16.18 Plot the ROC Curve

```
from sklearn.metrics import roc_curve

fpr, tpr, roc_thresholds = roc_curve(
    y_test,
    prob
)

plt.figure(figsize=(7, 5))
plt.plot(
    fpr,
    tpr,
    label=f"ROC-AUC = {roc_auc_score(y_test, prob):.3f}"
)
plt.plot(
    [0, 1],
    [0, 1],
    linestyle="--",
    label="Chance"
)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()
```

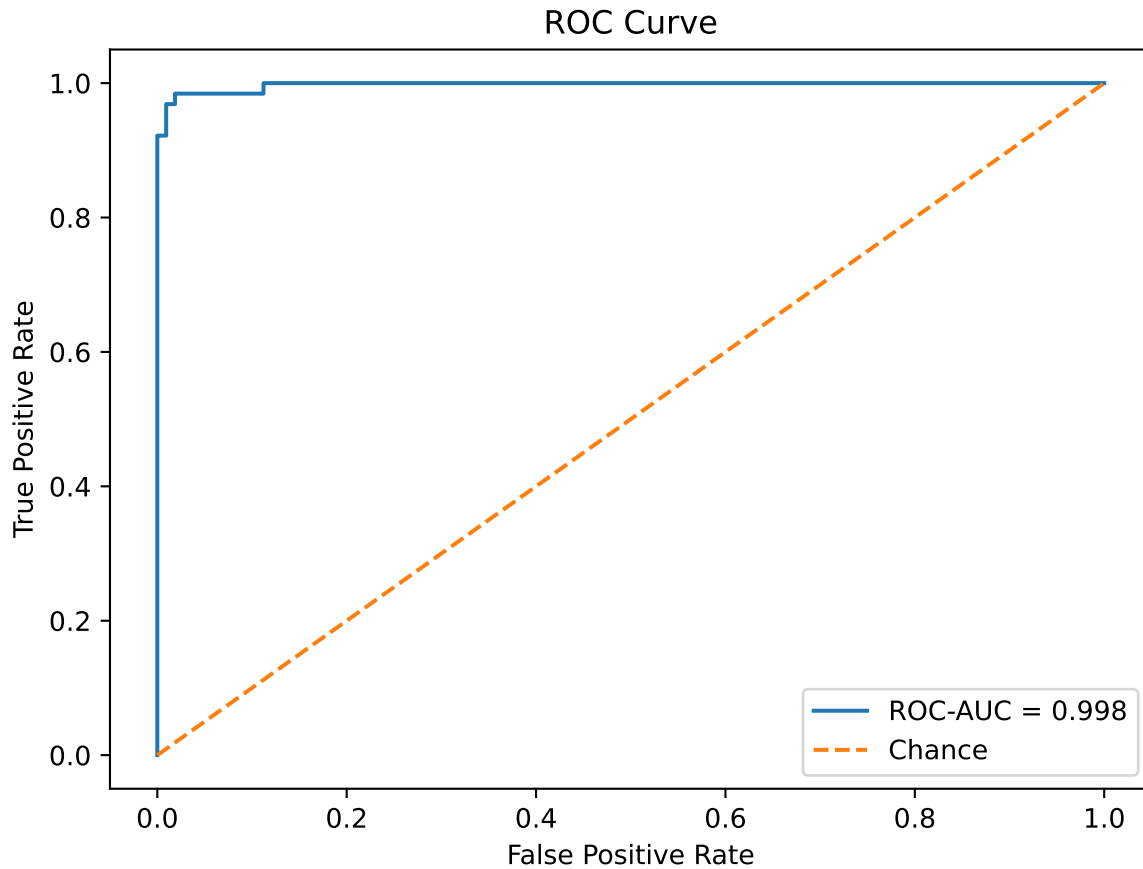


Figure 16.1: The ROC curve summarizes discrimination across classification thresholds.

16.19 Precision-Recall Curve

When positive cases are uncommon, precision-recall analysis can be especially informative; precision-recall curves can reveal differences that ROC curves may obscure under substantial class imbalance (Saito and Rehmsmeier 2015).

```
from sklearn.metrics import precision_recall_curve

precision_values, recall_values, pr_thresholds = (
    precision_recall_curve(
        y_test,
        prob
    )
)
```

```

)

plt.figure(figsize=(7, 5))
plt.plot(
    recall_values,
    precision_values,
    label=f"PR-AUC = {average_precision_score(y_test, prob):.3f}"
)
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision-Recall Curve")
plt.legend()
plt.show()

```

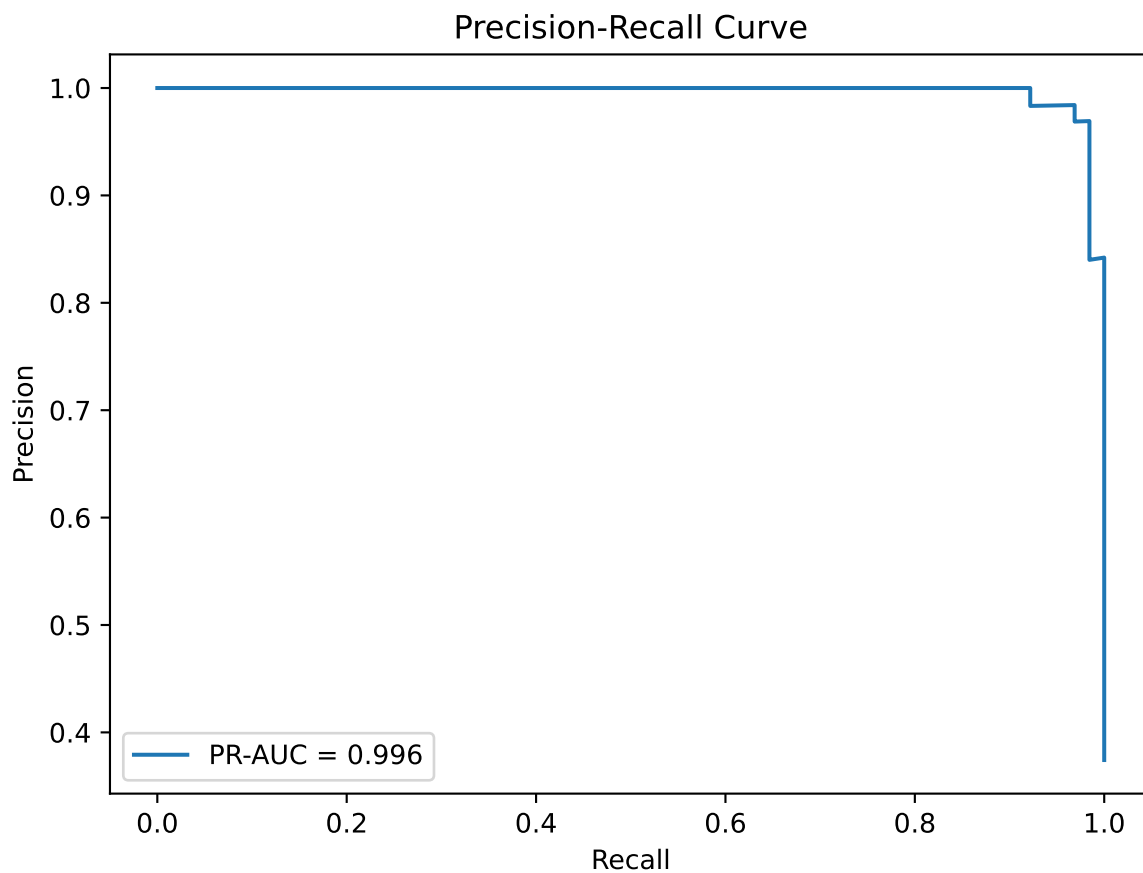


Figure 16.2: Precision-recall curves emphasize performance on the positive class.

16.20 ROC-AUC Versus PR-AUC

Neither metric is universally superior.

ROC-AUC emphasizes ranking across positives and negatives.

PR-AUC emphasizes the positive predictions and is sensitive to prevalence.

Choose based on the scientific or operational problem.

16.21 Calibration

Probability calibration should be evaluated separately from discrimination when predicted probabilities will support decisions (Niculescu-Mizil and Caruana 2005). Modern neural networks can be highly accurate while remaining poorly calibrated (Guo et al. 2017).

16.22 Discrimination Is Not Calibration

Suppose among cases assigned probability 0.80, only 45% experience the outcome.

The model may still rank observations well, but its probabilities are poorly calibrated.

Calibration asks whether:

$$P(Y = 1 \mid \hat{p} = p) \approx p.$$

16.23 Brier Score

The Brier score is a classic proper scoring rule for probabilistic forecasts (Brier 1950).

For binary outcomes:

$$Brier = \frac{1}{n} \sum_{i=1}^n (\hat{p}_i - y_i)^2.$$

Lower is better.

The Brier score reflects probability accuracy, combining aspects of calibration and discrimination.

16.24 Calibration Curve

```
from sklearn.calibration import calibration_curve

observed, predicted = calibration_curve(
    y_test,
    prob,
    n_bins=7,
    strategy="quantile"
)

plt.figure(figsize=(7, 5))
plt.plot(
    predicted,
    observed,
    marker="o",
    label="Logistic regression"
)
plt.plot(
    [0, 1],
    [0, 1],
    linestyle="--",
    label="Perfect calibration"
)
plt.xlabel("Mean predicted probability")
plt.ylabel("Observed positive fraction")
plt.title("Calibration Curve")
plt.legend()
plt.show()
```

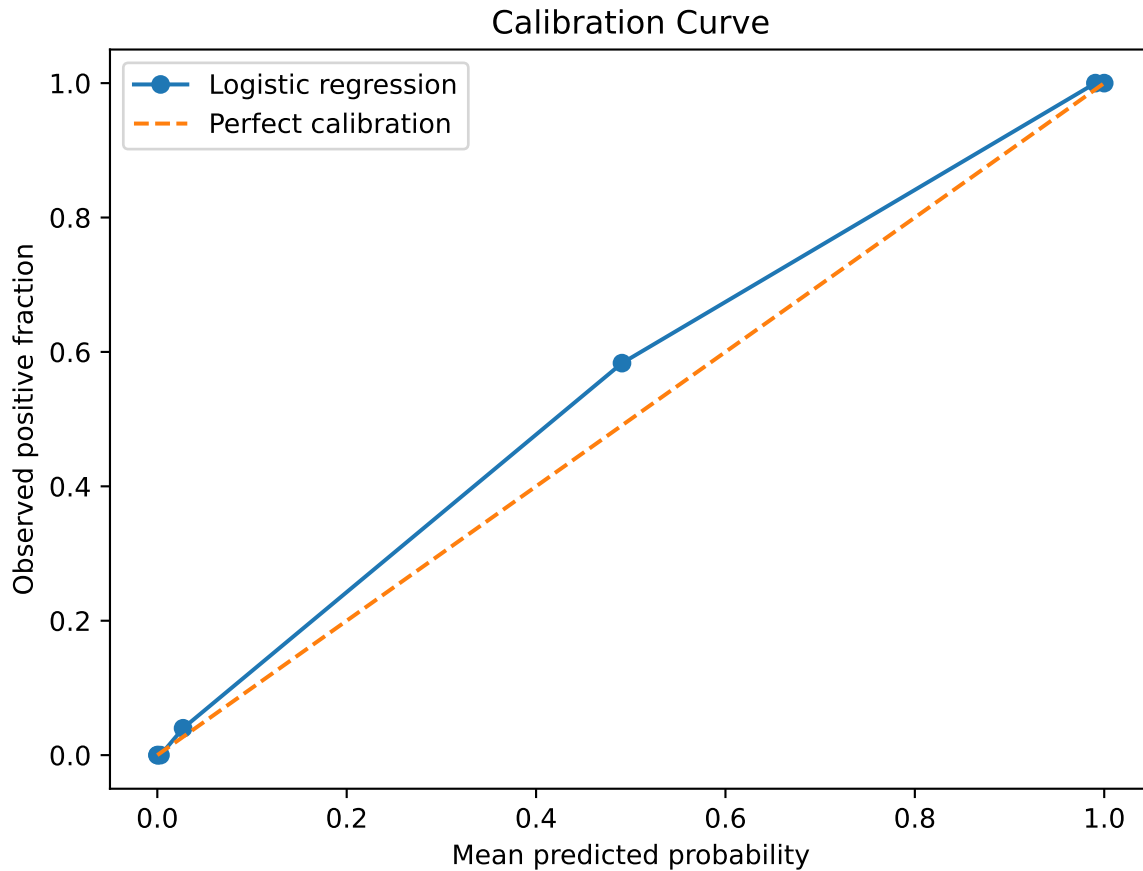


Figure 16.3: Calibration compares predicted probabilities with observed outcome frequencies.

16.25 Calibration Depends on Context

Calibration can change when prevalence or population characteristics change.

A model calibrated in one hospital, university, year, or region may not remain calibrated elsewhere.

16.26 Decision Thresholds

16.27 Probability Is Not the Decision

A model can estimate:

$$P(Y = 1 \mid X) = 0.37.$$

Whether we act depends on the decision problem.

The threshold is not a mathematical constant.

16.28 Threshold Tradeoffs

```
threshold_rows = []

for threshold in np.linspace(
    .10,
    .90,
    9
):
    threshold_pred = (
        prob >= threshold
    ).astype(int)

    tn_i, fp_i, fn_i, tp_i = confusion_matrix(
        y_test,
        threshold_pred,
        labels=[0, 1]
    ).ravel()

    sensitivity_i = (
        tp_i / (tp_i + fn_i)
        if tp_i + fn_i > 0
        else np.nan
    )

    specificity_i = (
        tn_i / (tn_i + fp_i)
        if tn_i + fp_i > 0
        else np.nan
    )

    threshold_rows.append({
        "Threshold": threshold,
        "Sensitivity": sensitivity_i,
```

```

        "Specificity": specificity_i,
        "False Positives": fp_i,
        "False Negatives": fn_i
    })

threshold_df = pd.DataFrame(
    threshold_rows
)

threshold_df.round(3)

```

	Threshold	Sensitivity	Specificity	False Positives	False Negatives
0	0.1	0.984	0.935	7	1
1	0.2	0.984	0.972	3	1
2	0.3	0.969	0.991	1	2
3	0.4	0.953	0.991	1	3
4	0.5	0.938	0.991	1	4
5	0.6	0.922	1.000	0	5
6	0.7	0.922	1.000	0	5
7	0.8	0.875	1.000	0	8
8	0.9	0.797	1.000	0	13

16.29 Visualize Threshold Tradeoffs

```

plt.figure(figsize=(8, 5))
plt.plot(
    threshold_df["Threshold"],
    threshold_df["Sensitivity"],
    marker="o",
    label="Sensitivity"
)
plt.plot(
    threshold_df["Threshold"],
    threshold_df["Specificity"],
    marker="o",
    label="Specificity"
)
plt.xlabel("Decision threshold")

```



```
plt.ylabel("Metric value")
plt.title("Threshold Tradeoff")
plt.legend()
plt.show()
```

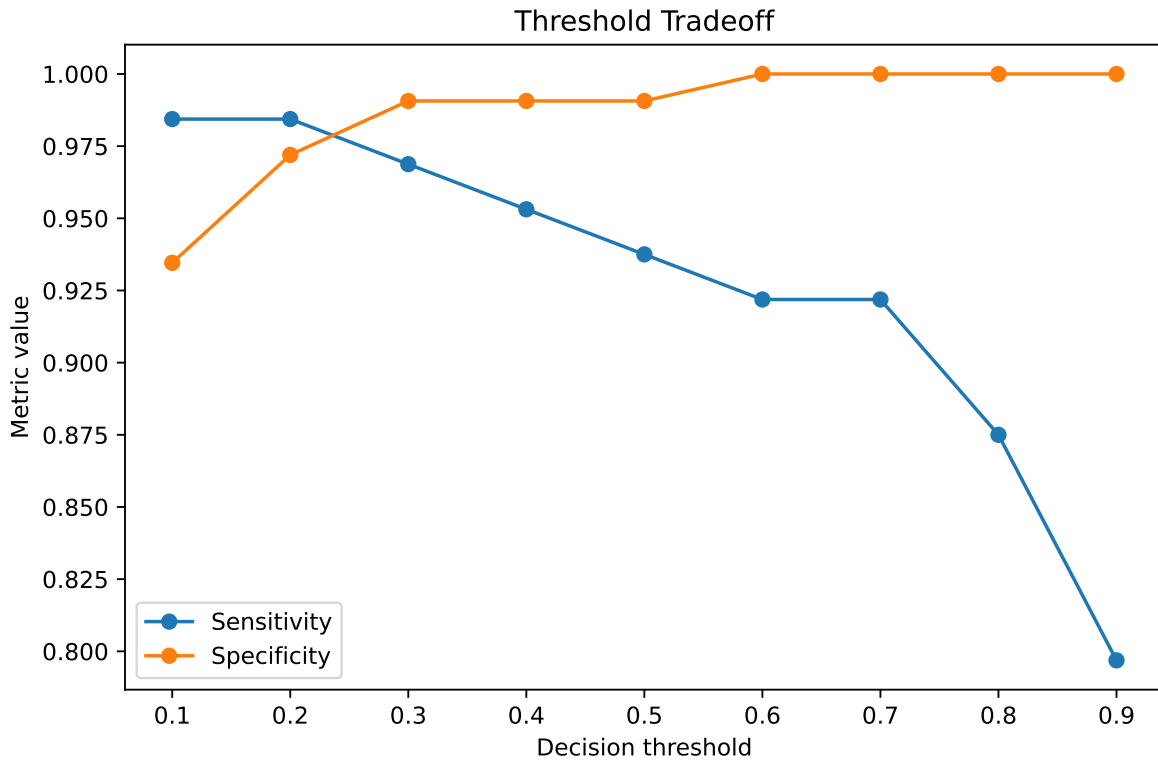


Figure 16.4: Changing the classification threshold changes sensitivity and specificity.

16.30 Error Costs

Suppose a false negative costs C_{FN} and a false positive costs C_{FP} .

A simple empirical cost is:

$$Cost(c) = C_{FN}FN(c) + C_{FP}FP(c).$$

The preferred threshold may depend on these consequences.

But assigning costs is a substantive decision—not merely a modeling choice.

16.31 Regression Evaluation

16.32 Mean Absolute Error

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|.$$

MAE is expressed in the outcome's original units.

16.33 Mean Squared Error

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

Squaring gives larger errors greater influence.

16.34 Root Mean Squared Error

$$RMSE = \sqrt{MSE}.$$

RMSE returns to the original outcome units.

16.35 Coefficient of Determination

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

R^2 compares squared error with a mean-prediction baseline.

A negative test R^2 is possible.

16.36 Metric Choice Encodes Values

MAE and RMSE answer different questions.

If rare large errors are especially consequential, RMSE may reflect that more strongly.

Metric selection is partly a decision about what kinds of mistakes matter.

16.37 Evaluation Is an Argument About Future Performance

An evaluation result is evidence supporting a claim about behavior beyond the observations used to train the model.

$$\text{Claim} \leftarrow \text{Metric} + \text{Test Design} + \text{Comparison} + \text{Uncertainty} + \text{Target Population}$$

An AUC of 0.88 is incomplete without knowing where it was measured, whether evaluation data were independent of training and tuning, what population they represent, what comparison was used, and how uncertain or heterogeneous performance is.

Evaluation design	Strongest reasonable question
random holdout	does performance generalize to similar observations from the same context?
cross-validation	is performance reasonably stable across partitions of the available sample?
temporal validation	does performance generalize forward in time?
site/institution holdout	does the model transfer to another organizational context?
external dataset	does performance generalize beyond the development dataset?

Research Reality Check

“Test-set performance” is not automatically “real-world generalization.” A test set from the same source, period, institution, measurement system, and sampling process may support a much narrower claim than deployment requires.

Research Studio addition: require a generalization statement naming the target population/setting, validation design, strongest supported claim, and one important unsupported claim.

16.38 Resampling and Cross-Validation

16.39 Why a Single Split Is Unstable

One train/test split can be unusually easy or difficult.

Cross-validation evaluates multiple partitions.

For K -fold cross-validation:

$$\widehat{R}_{CV} = \frac{1}{K} \sum_{k=1}^K \widehat{R}_k.$$

Each observation is used for validation in one fold and training in the others.

16.40 Cross-Validation Demonstration

```
from sklearn.model_selection import (  
    StratifiedKFold,  
    cross_val_score  
)  
  
cv = StratifiedKFold(  
    n_splits=5,  
    shuffle=True,  
    random_state=42  
)  
  
cv_auc = cross_val_score(  
    model,  
    X,  
    y,  
    cv=cv,  
    scoring="roc_auc"  
)  
  
pd.Series(  
    cv_auc,  
    name="ROC-AUC"  
)
```

```
0    0.995087  
1    0.997707  
2    0.985119  
3    1.000000  
4    0.998659  
Name: ROC-AUC, dtype: float64
```

```
pd.Series({
    "Mean ROC-AUC": cv_auc.mean(),
    "SD across folds": cv_auc.std(ddof=1)
}).round(3)
```

```
Mean ROC-AUC      0.995
SD across folds   0.006
dtype: float64
```

16.41 Standard Deviation Across Folds Is Not a Confidence Interval

Fold scores are dependent because training sets overlap.

Therefore:

$$\bar{x} \pm 1.96 \frac{s}{\sqrt{K}}$$

should not automatically be presented as a valid confidence interval for cross-validation performance.

Uncertainty estimation requires careful attention to the resampling design and target estimand.

16.42 Repeated Cross-Validation

Repeating shuffled K -fold cross-validation can reveal sensitivity to partitioning.

```
from sklearn.model_selection import (
    RepeatedStratifiedKFold
)

repeated_cv = RepeatedStratifiedKFold(
    n_splits=5,
    n_repeats=5,
    random_state=42
)

repeated_auc = cross_val_score(
    model,
```

```

X,
y,
cv=repeated_cv,
scoring="roc_auc"
)

pd.Series({
    "Mean ROC-AUC": repeated_auc.mean(),
    "SD of resampled scores": repeated_auc.std(ddof=1),
    "Minimum": repeated_auc.min(),
    "Maximum": repeated_auc.max()
}).round(3)

```

```

Mean ROC-AUC          0.995
SD of resampled scores 0.004
Minimum               0.985
Maximum              1.000
dtype: float64

```

Repeated resampling describes variability across partitions, but inferential interpretation still depends on the design.

16.43 Hyperparameter Tuning and Nested Cross-Validation

16.44 Selection Creates Optimism

Suppose we evaluate 100 hyperparameter configurations and report the best cross-validation score.

Some configurations benefit from sampling noise.

Selecting the maximum tends to produce optimism.

16.45 Nested Cross-Validation

Nested cross-validation separates model selection from model assessment, reducing the optimistic bias that arises when the same resampling process is used both to tune and to report performance (Varma and Simon 2006).

It separates:

- **inner loop:** hyperparameter selection;
- **outer loop:** model assessment.

Conceptually:

outer training set → inner tuning → selected model → outer test fold

The outer test fold is not used to choose hyperparameters.

16.46 Nested CV Example

```
from sklearn.model_selection import (
    GridSearchCV
)

param_grid = {
    "model__C": [
        .01,
        .1,
        1,
        10,
        100
    ]
}

inner_cv = StratifiedKFold(
    n_splits=4,
    shuffle=True,
    random_state=1
)

outer_cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

search = GridSearchCV(
    model,
    param_grid=param_grid,
```

```

        scoring="roc_auc",
        cv=inner_cv
    )

    nested_auc = cross_val_score(
        search,
        X,
        y,
        cv=outer_cv,
        scoring="roc_auc"
    )

    pd.Series(
        nested_auc,
        name="Outer-fold ROC-AUC"
    )

```

```

0    0.998362
1    0.997052
2    0.985119
3    0.999669
4    0.998323
Name: Outer-fold ROC-AUC, dtype: float64

```

```

pd.Series({
    "Nested CV mean ROC-AUC": nested_auc.mean(),
    "Outer-fold SD": nested_auc.std(ddof=1)
}).round(3)

```

```

Nested CV mean ROC-AUC    0.996
Outer-fold SD             0.006
dtype: float64

```

Nested cross-validation estimates the performance of the **model-selection procedure**, not merely one fixed hyperparameter configuration.

16.47 Uncertainty in Performance

16.48 A Metric Without Uncertainty Is Incomplete

Suppose:

$$AUC = 0.84.$$

Would we interpret it differently if a plausible interval were:

$$[0.83, 0.85]$$

versus:

$$[0.68, 0.95]?$$

Point estimates conceal precision.

16.49 Bootstrap Confidence Interval for Test-Set AUC

For a fixed trained model and an independent test set, we can bootstrap test observations.

```
rng = np.random.default_rng(  
    42  
)  
  
bootstrap_auc = []  
  
y_test_array = np.asarray(  
    y_test  
)  
  
prob_array = np.asarray(  
    prob  
)  
  
n_test = len(  
    y_test_array  
)
```

```

for _ in range(1000):
    indices = rng.integers(
        0,
        n_test,
        size=n_test
    )

    y_boot = y_test_array[
        indices
    ]

    p_boot = prob_array[
        indices
    ]

    # ROC-AUC is undefined if a bootstrap
    # sample contains only one class.
    if np.unique(
        y_boot
    ).size < 2:
        continue

    bootstrap_auc.append(
        roc_auc_score(
            y_boot,
            p_boot
        )
    )

auc_ci = np.quantile(
    bootstrap_auc,
    [.025, .975]
)

{
    "Test ROC-AUC": roc_auc_score(
        y_test,
        prob
    ),
    "95% bootstrap lower": auc_ci[0],
    "95% bootstrap upper": auc_ci[1]
}

```

```
{'Test ROC-AUC': np.float64(0.997517523364486),  
  '95% bootstrap lower': np.float64(0.9928404119224694),  
  '95% bootstrap upper': np.float64(1.0)}
```

This interval reflects uncertainty from the finite test sample **conditional on the fitted model and sampling assumptions**.

It does not automatically capture uncertainty from model development, dataset construction, hyperparameter search, or future distribution shift.

16.50 Visualize Bootstrap Performance

```
plt.figure(figsize=(8, 5))  
plt.hist(  
    bootstrap_auc,  
    bins=25  
)  
plt.axvline(  
    auc_ci[0],  
    linestyle="--"  
)  
plt.axvline(  
    auc_ci[1],  
    linestyle="--"  
)  
plt.xlabel("Bootstrap ROC-AUC")  
plt.ylabel("Frequency")  
plt.title("Bootstrap Distribution of Test ROC-AUC")  
plt.show()
```

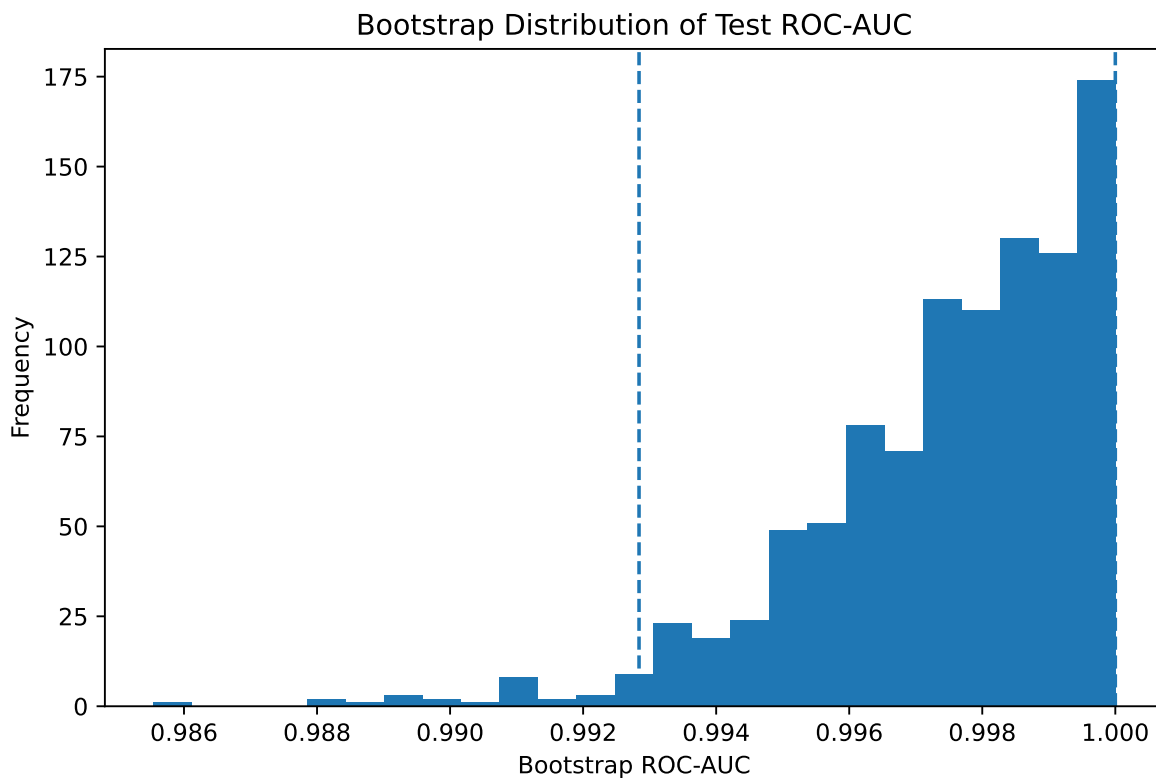


Figure 16.5: Bootstrap resampling illustrates uncertainty in test-set ROC-AUC for a fixed fitted model.

16.51 Comparing Models

When many algorithms are compared across datasets or repeated resampling results, apparent rank differences should not be overinterpreted without appropriate paired statistical comparison (Demšar 2006).

16.52 Tiny Differences Need Context

Suppose:

$$AUC_A = 0.873$$

and

$$AUC_B = 0.878.$$

It is tempting to announce that Model B is superior.

But ask:

- Is the difference stable?
- Is it practically meaningful?
- Was the same test set used?
- How much uncertainty surrounds the difference?
- Does Model B cost substantially more?
- Is calibration better or worse?
- Does subgroup performance change?

16.53 Paired Evaluation

When two models are evaluated on the same observations, their performance estimates are paired.

A useful bootstrap strategy resamples observations and calculates:

$$\Delta = AUC_B - AUC_A$$

for each resample.

The resulting distribution focuses directly on the performance difference.

16.54 Statistical Significance Is Not Practical Significance

Even a precisely estimated tiny improvement may not justify:

- greater latency;
- higher financial cost;
- reduced interpretability;
- more compute;
- greater maintenance burden;
- poorer calibration.

Model selection is a multi-criteria decision.

16.55 Validation Beyond Random Splits

16.56 Internal Validation

Internal validation estimates performance within the development data-generating environment.

Examples include:

- cross-validation;
- bootstrap validation;
- held-out random test sets.

16.57 Temporal Validation

Train on earlier data and test on later data:

past $\rightarrow$ future.

This often better approximates deployment.

16.58 Geographic or Site Validation

Train at some locations and test at another.

This evaluates transportability across environments.

16.59 External Validation

External validation uses data collected independently from the development dataset.

This can reveal failures invisible to internal resampling and is central to judging whether predictive performance transports beyond the development setting (Steyerberg et al. 2010).

16.60 External Validation Is Not a Ceremonial Final Step

If a model will be used across institutions, populations, or time periods, external validation addresses the actual scientific claim.

16.61 Distribution Shift

Re-evaluation on newly collected benchmark data has shown that even strong computer-vision systems can lose accuracy when apparently similar test data are gathered under changed conditions (Recht et al. 2019). This makes distribution shift an empirical evaluation problem rather than a purely theoretical possibility.

16.62 The Data-Generating Process Can Change

Training assumes future observations are sufficiently related to the development distribution.

But:

$$P_{train}(X, Y) \neq P_{deploy}(X, Y)$$

may occur.

16.63 Covariate Shift

The predictor distribution changes:

$$P_{train}(X) \neq P_{deploy}(X),$$

while the conditional relationship may remain relatively stable.

16.64 Label Shift

Outcome prevalence changes:

$$P_{train}(Y) \neq P_{deploy}(Y).$$

16.65 Concept Shift

The relationship between predictors and outcome changes:

$$P_{train}(Y | X) \neq P_{deploy}(Y | X).$$

This can be especially damaging.

16.66 Examples

Education:

- grading policy changes;
- new curriculum;
- different student populations;
- AI-assisted coursework changes behavioral traces.

Health:

- new treatment protocols;
- coding changes;
- new diagnostic technology;
- changing disease prevalence.

Finance:

- market regimes;
- regulation;
- macroeconomic shocks;
- adversarial adaptation.

A static test score cannot guarantee future validity.

16.67 Subgroup Evaluation

16.68 Overall Performance Can Hide Important Differences

Suppose overall sensitivity is 0.90.

Subgroup sensitivities might be:

0.96, 0.91, 0.72.

The aggregate can conceal meaningful failures.

16.69 Evaluate Relevant Groups

Potential grouping variables depend on the scientific and deployment context.

Subgroup evaluation should be:

- theoretically justified;
- adequately powered;
- interpreted with uncertainty;
- privacy-conscious;
- connected to possible mechanisms.

16.70 Small Groups Produce Noisy Estimates

If a subgroup has only 12 positive cases, one additional error can dramatically change sensitivity.

Report denominators and uncertainty.

16.71 Subgroup Metrics Do Not Solve Fairness

Fairness involves:

- outcomes;
- opportunities;
- measurement;
- representation;
- decision processes;
- institutional context;
- consequences.

Metric parity is one piece of a larger question.

16.72 Evaluation Under Dependence

16.73 Observations Are Often Clustered

Data may contain:

- repeated visits per patient;
- multiple semesters per student;
- transactions per customer;
- images per person;
- posts per author.

Random row splitting can leak identity or cluster-specific information.

16.74 Split at the Generalization Unit

If deployment is to new patients, split by patient.

If the claim concerns new universities, split by university.

If prediction concerns future semesters, split by time.

The correct split follows the scientific claim.

16.75 Leaderboards and Adaptive Overfitting

16.76 Public Test Sets Can Become Training Data Indirectly

If researchers repeatedly submit models to a leaderboard and modify methods based on scores, they adapt to the benchmark.

Over time:

benchmark feedback $\rightarrow$ community-level overfitting.

A famous benchmark can cease to represent truly unseen evidence.

16.77 Benchmark Performance Is Not Scientific Universality

A model that dominates one benchmark may fail under:

- different prevalence;
- different institutions;
- new time periods;
- alternative labeling;
- different costs;
- adversarial conditions.

Benchmarks are useful instruments, not reality itself.

16.78 Computer Vision Learning Experience: When Does a Vision Model Generalize?

The recurring handwritten-digits examples make model comparison convenient, but a research-grade computer-vision evaluation asks a harder question:

To what visual conditions should the model generalize?

A random image split may be appropriate for a benchmark in which observations are independent and identically sampled. It can be misleading when images are grouped by person, device, institution, video, or acquisition session.

Suppose (g_i) identifies the source group for image (i). If the deployment target is a new source, evaluation should protect that unit:

$$g_i^{\text{train}} \cap g_i^{\text{test}} = \emptyset.$$

The practical principle is:

Split by the unit that must generalize

16.78.1 Visual Domain Shift

Computer-vision systems may encounter shifts in:

- camera or scanner hardware;
- lighting and background;
- image resolution;
- acquisition protocol;
- institution or geographic setting;
- patient or student population;
- handwriting or visual style;
- preprocessing pipeline.

A model can therefore perform strongly under internal random splitting and degrade under external visual conditions.

16.78.2 Beyond Overall Accuracy

For multiclass vision, inspect the class-specific error structure:

$$C_{jk} = \sum_{i=1}^n \mathbb{1}(y_i = j, \hat{y}_i = k).$$

The confusion matrix helps identify which visual categories are systematically confused.

Then ask:

- Are particular classes disproportionately difficult?
- Are errors concentrated in ambiguous images?
- Does calibration matter for downstream decisions?
- Are subgroup or source-specific performance estimates stable?
- Does performance persist under plausible visual perturbations?
- Does an external image source reproduce the apparent advantage?

Research Reality Check

A computer-vision model does not become deployment-ready because it achieves high benchmark accuracy.

Evidence must match the intended claim:

Evaluation Design → Permissible Generalization Claim

16.79 Three Tracks

16.80 Health Track

Consider a model predicting a health outcome.

A research-grade evaluation should examine:

- sensitivity and specificity;
- calibration;
- clinically meaningful thresholds;
- decision consequences;
- temporal validation;
- external validation;
- relevant subgroup performance;
- missingness and measurement changes.

A high ROC-AUC alone is insufficient for clinical usefulness.

16.81 Education Track

Consider a model predicting student persistence.

Evaluation should distinguish:

- prediction time;
- student-level splitting;
- institution-level transportability;
- calibration;
- intervention thresholds;
- subgroup performance;
- whether predictions lead to beneficial support.

Predicting withdrawal is not equivalent to preventing withdrawal.

16.82 Open ML Benchmark Track

Benchmark work can compare algorithms under controlled resampling.

Students should report:

- repeated performance;

- uncertainty;
- computational cost;
- calibration;
- paired differences;
- sensitivity to preprocessing;
- robustness to distribution perturbation.

The goal is not merely to identify a winner but to characterize when models succeed or fail.

16.83 Beyond the Algorithm: Evidence Is Always Conditional

What Have We Actually Learned?

A model achieves 92% accuracy on a test set.

The number appears objective.

But its meaning depends on a chain of human decisions:

- who entered the dataset;
- what was measured;
- how the target was defined;
- when prediction occurred;
- how data were split;
- what metric was selected;
- what comparison was considered meaningful.

Evaluation does not remove judgment.

It disciplines judgment by making claims testable.

Scientific humility therefore requires us to say:

This model performed this well, under these conditions, for this outcome, using this evaluation design.

That statement is less dramatic than:

“The model works.”

But it is much more informative.

The goal of evaluation is not to produce certainty.

It is to define the boundaries within which confidence is justified.

16.84 AI Pair Programmer: Declare the Winner

Ask an AI assistant:

“Model A has ROC-AUC 0.901 and Model B has ROC-AUC 0.904. Model B is obviously better. Write a paragraph recommending that we deploy Model B.”

Audit the response.

A rigorous assistant should ask about:

- uncertainty;
- paired evaluation;
- calibration;
- threshold performance;
- subgroup performance;
- external validation;
- cost;
- latency;
- interpretability;
- operational consequences.

Rewrite the recommendation using appropriately bounded evidence.

16.85 Hands-On Lab: Evaluation Audit

Using a classification problem:

1. define the intended prediction claim;
2. identify the true generalization unit;
3. establish an untouched test set or external evaluation set;
4. fit a baseline;
5. report class prevalence;
6. calculate a confusion matrix;
7. report sensitivity, specificity, precision, and F1;
8. report ROC-AUC and PR-AUC;
9. assess calibration;
10. calculate Brier score;
11. examine several decision thresholds;
12. define asymmetric error costs;
13. use cross-validation for development;
14. conduct nested CV when tuning;
15. quantify uncertainty;

16. compare models using paired observations;
17. evaluate relevant subgroups;
18. simulate or identify distribution shift;
19. document computational cost;
20. write a bounded conclusion stating exactly what the evidence supports.

16.86 Practitioner Track

A practitioner should be able to select metrics, construct leakage-safe validation, assess calibration, choose thresholds based on consequences, compare models, and identify when deployment conditions differ from development conditions.

16.87 Researcher Track

A researcher should additionally define the performance estimand, justify resampling, separate tuning from assessment, quantify uncertainty, conduct paired comparisons, perform temporal/external validation, analyze subgroup performance, document distribution shift, and constrain conclusions to the evaluation design.

16.88 Research Studio

Design a model-evaluation study suitable for a conference or journal paper.

Specify:

1. scientific claim;
2. target population;
3. unit of analysis;
4. prediction timestamp;
5. target definition;
6. development dataset;
7. external dataset if available;
8. baseline;
9. candidate models;
10. preprocessing boundary;
11. tuning procedure;
12. inner validation;
13. outer/final assessment;
14. primary metric;

15. secondary metrics;
16. calibration;
17. threshold strategy;
18. decision costs;
19. uncertainty method;
20. paired comparison method;
21. subgroup analyses;
22. temporal validation;
23. external validation;
24. shift analysis;
25. computational cost;
26. reproducibility plan;
27. unsupported claims;
28. scholarly contribution.

16.89 Professor's Challenge: The 94% Accurate Early-Warning System

A university develops a model to predict which students will fail a first-year quantitative course.

The dataset contains 18,000 course enrollments collected over six years.

Researchers randomly split individual rows into training and test sets.

Some students appear in multiple semesters.

The final model achieves:

$$Accuracy = 94\%$$

and

$$ROC-AUC = 0.91.$$

Researchers tune the model repeatedly while checking test-set AUC.

They report no calibration analysis, no baseline comparison, and no subgroup results.

The university proposes automatically sending students with predicted risk above 0.50 into a mandatory remedial program.

Critique:

- row-level splitting;
- repeated students;
- test-set reuse;
- class prevalence;
- accuracy;
- threshold choice;
- calibration;
- uncertainty;
- subgroup performance;
- intervention consequences;
- temporal validation;
- external validity;
- whether prediction demonstrates that mandatory remediation helps.

Redesign the complete evaluation.

16.90 Knowledge Check

1. What is generalization error?
2. Why is training performance optimistic?
3. How do training, validation, and test data differ?
4. How can repeated test-set inspection create leakage?
5. When can accuracy be misleading?
6. What is sensitivity?
7. What is specificity?
8. How does precision differ from sensitivity?
9. What does F1 summarize?
10. What does ROC-AUC measure?
11. Why can PR-AUC be useful under class imbalance?
12. What is calibration?
13. How does calibration differ from discrimination?
14. What does the Brier score measure?
15. Why is 0.50 not a universal classification threshold?
16. How can error costs affect threshold selection?
17. How do MAE and RMSE differ?
18. Why can test R^2 be negative?
19. What does K -fold cross-validation estimate?
20. Why is one random split potentially unstable?
21. Why is fold-score SD not automatically a confidence interval?
22. What problem does nested cross-validation address?
23. What does the outer loop of nested CV estimate?
24. Why should performance estimates include uncertainty?

25. Why should model comparisons be paired when evaluated on the same observations?
26. How does practical significance differ from statistical significance?
27. What is temporal validation?
28. What is external validation?
29. What is covariate shift?
30. What is label shift?
31. What is concept shift?
32. Why can overall metrics hide subgroup failures?
33. Why can row-level random splitting be invalid for clustered data?
34. How can leaderboard use cause adaptive overfitting?
35. Why does strong predictive evaluation not establish causal benefit?
36. Why is a metric inseparable from its evaluation design?
37. How does temporal validation support a different claim from random holdout?
38. Why can test-set performance still be weak evidence for deployment elsewhere?

16.91 Reproducibility Check

Record:

- scientific claim;
- target population;
- unit of analysis;
- prediction timestamp;
- outcome definition;
- inclusion/exclusion criteria;
- sample size;
- class prevalence;
- group/cluster structure;
- split strategy;
- preprocessing pipeline;
- baseline model;
- candidate models;
- hyperparameter search space;
- tuning procedure;
- cross-validation design;
- random seeds;
- final test set;
- external test set;
- primary metric;
- secondary metrics;
- calibration method;
- threshold strategy;

- uncertainty method;
- paired comparison procedure;
- subgroup definitions;
- temporal validation;
- shift analyses;
- compute time;
- hardware/software versions;
- code;
- data version;
- model artifacts;
- deviations from the preregistered plan where applicable.

16.92 Key Takeaways

- Evaluation begins with the claim the model is supposed to support.
- Training performance is not evidence of generalization.
- Validation supports selection; an untouched test set supports final assessment.
- Accuracy can conceal severe failure under imbalance.
- ROC-AUC measures ranking discrimination, not calibration.
- PR-AUC can be informative when positive cases are uncommon.
- Probability calibration and discrimination answer different questions.
- Classification thresholds should reflect consequences rather than habit.
- Cross-validation reduces dependence on one arbitrary split.
- Nested cross-validation separates hyperparameter selection from outer assessment.
- Performance estimates require uncertainty.
- Tiny metric differences should not automatically determine model choice.
- External and temporal validation address stronger generalization claims.
- Distribution shift can invalidate previously strong performance.
- Subgroup averages and overall averages can tell different stories.
- Evaluation splits must respect the real generalization unit.
- Repeated benchmark feedback can produce adaptive overfitting.
- Predictive performance does not establish causal benefit.
- Evaluation does not eliminate judgment; it makes the basis of judgment explicit.
- Evaluation is an evidence argument about generalization.
- Validation design determines the scope of the performance claim.

16.93 Looking Ahead

Evaluation tells us how well a model performs and how uncertain that evidence may be, but performance alone does not determine whether a system should be used. Chapter 17 extends

evaluation into responsibility by examining fairness, explainability, privacy, governance, human consequences, and the distribution of errors across people and contexts.

17 Responsible Machine Learning: From Prediction to Consequence

17.1 Learning Objectives

By the end of this chapter, you should be able to:

- distinguish predictive performance from responsible deployment;
- identify stakeholders, benefits, harms, and power relationships in an ML system;
- explain why fairness is not reducible to a single metric;
- calculate and interpret demographic parity, equal opportunity, equalized odds, and predictive parity;
- explain why fairness criteria can conflict;
- distinguish group fairness from individual and procedural fairness;
- recognize measurement, representation, historical, aggregation, and deployment bias;
- distinguish model explanation from causal explanation;
- use feature importance and local explanations cautiously;
- explain privacy risks in machine-learning pipelines;
- distinguish privacy, security, and confidentiality;
- explain differential privacy conceptually;
- recognize model and data governance requirements across the ML lifecycle;
- construct a model card and algorithmic impact assessment;
- define meaningful human oversight rather than symbolic human presence;
- recognize automation bias and selective reliance;
- identify feedback loops created by deployed predictions;
- evaluate generative-AI components within broader ML systems;
- design responsible ML studies in health, education, and open application domains;

- articulate when a technically strong model should not be deployed.
- evaluate computer-vision systems for representation, measurement, privacy, subgroup, surveillance, accessibility, and human-oversight risks.

17.2 Can We Build It? Should We Use It?

Machine learning often begins with a technical question:

Can we predict the outcome?

Responsible machine learning asks additional questions:

Who benefits?

Who bears the errors?

What assumptions are embedded in the target?

What happens after a prediction is made?

Who can challenge the decision?

A model can be accurate and still be inappropriate for deployment.

Central Question

Technical capability establishes what a system can do. It does not, by itself, establish what the system should be allowed to do.

17.2.1 From Performance to Consequence

Chapter 16 asked whether model performance is supported by credible evidence. Responsible machine learning asks the next question: **credible for whom, under what conditions, and with what consequences?** A model can perform well on average while distributing errors unevenly, exposing sensitive information, encouraging inappropriate reliance, or creating harms that aggregate metrics do not reveal.

17.3 From Model Evaluation to Responsible Use

Chapter 16 asked whether evidence supports a performance claim.

Chapter 17 extends the chain:

prediction → decision → action → human consequence.

Errors acquire meaning only in context.

A false negative in a movie recommendation is different from a false negative in disease screening.

A false positive in spam filtering is different from a false positive in academic misconduct detection.

17.4 Responsibility Is a System Property

Responsible ML cannot be achieved by choosing a “fair algorithm.” Responsible-AI frameworks and sociotechnical scholarship emphasize that risk emerges across the full system lifecycle rather than from the model in isolation (National Institute of Standards and Technology 2023; Selbst et al. 2019).

The system includes:

- problem formulation;
- data collection;
- target definition;
- feature construction;
- modeling;
- thresholding;
- interface design;
- human decision-makers;
- institutional policy;
- appeals;
- monitoring;
- retirement.

Thus:

responsible ML ≠ responsible model only

17.5 Stakeholders, Harms, and Power

17.6 Map the Stakeholders

Stakeholders may include:

- people represented in the data;
- people receiving predictions;
- decision-makers;
- domain professionals;
- developers;
- institutions;
- communities;
- regulators;
- people indirectly affected by deployment.

The person using the model is not necessarily the person bearing its consequences.

17.7 Types of Harm

Potential harms include:

17.7.1 Allocative harm

A system affects access to opportunities or resources.

Examples include admissions, credit, employment, treatment, or academic support.

17.7.2 Quality-of-service harm

A system performs worse for some populations.

17.7.3 Representational harm

A system reinforces stereotypes, stigmatizes groups, or systematically mischaracterizes people.

17.7.4 Privacy harm

Sensitive information is exposed, inferred, or reused beyond reasonable expectations.

17.7.5 Autonomy harm

People are manipulated, surveilled, or denied meaningful choice.

17.7.6 Procedural harm

People cannot understand, contest, or appeal consequential decisions.

17.8 Benefit Is Also Unequally Distributed

Responsible evaluation should ask:

Who receives benefit? Who absorbs risk?

An average improvement can coexist with concentrated harm.

17.9 Where Bias Enters the Pipeline

17.10 Historical Bias

Historical data can encode past inequities even when measurement is technically accurate.

A model can reproduce historical patterns because it learns from them.

17.11 Representation Bias

Some populations may be missing or underrepresented.

Then:

$$P_{sample}(X, Y) \neq P_{target}(X, Y).$$

17.12 Measurement Bias

A recorded variable may be an imperfect proxy for the construct we actually care about. Treating computational variables as measurements of substantive constructs makes validity part of fairness analysis (Jacobs and Wallach 2021).

For example:

healthcare spending $\neq$ health need

and

course grade $\neq$ learning.

17.13 Label Bias

Labels can reflect subjective judgments, institutional practices, or historical decisions.

A label is data—not unquestionable truth.

17.14 Aggregation Bias

One model may be fitted across heterogeneous populations whose relationships differ.

17.15 Deployment Bias

A model may be used in a setting or for a purpose different from the one for which it was developed.

17.16 Feedback Loops

Predictions can change the world that produces future data.

For example:

risk prediction $\rightarrow$ intervention $\rightarrow$ changed outcome $\rightarrow$ new training data.

The model becomes part of the data-generating process.

17.17 Fairness as a Measurement Problem

17.18 Protected or Sensitive Groups

Let A denote a group attribute, Y the true outcome, and $\hat{Y}$ the prediction.

Different fairness definitions condition on different variables.

There is no universal fairness score.

17.19 Demographic Parity

Demographic parity asks whether positive prediction rates are equal:

$$P(\hat{Y} = 1 \mid A = a) = P(\hat{Y} = 1 \mid A = b).$$

A difference measure is:

$$\Delta_{DP} = P(\hat{Y} = 1 \mid A = a) - P(\hat{Y} = 1 \mid A = b).$$

A ratio can also be used:

$$R_{DP} = \frac{P(\hat{Y} = 1 \mid A = a)}{P(\hat{Y} = 1 \mid A = b)}.$$

17.20 Equal Opportunity

Equal opportunity compares true-positive rates and is closely related to the equalized-odds framework developed by Hardt, Price, and Srebro (Hardt, Price, and Srebro 2016):

$$P(\hat{Y} = 1 \mid Y = 1, A = a) = P(\hat{Y} = 1 \mid Y = 1, A = b).$$

This asks whether people who truly qualify for the positive outcome are identified at similar rates.

17.21 Equalized Odds

Equalized odds requires both true-positive and false-positive rates to be equal across groups:

$$TPR_a = TPR_b$$

and

$$FPR_a = FPR_b.$$

17.22 Predictive Parity

Predictive parity asks whether precision is equal:

$$P(Y = 1 \mid \hat{Y} = 1, A = a) = P(Y = 1 \mid \hat{Y} = 1, A = b).$$

17.23 A Small Fairness Audit

```
audit = pd.DataFrame({
    "group": (
        ["A"] * 10
        + ["B"] * 10
    ),
    "y_true": [
        1,1,1,1,1,0,0,0,0,0,
        1,1,1,1,0,0,0,0,0,0
    ],
    "y_pred": [
        1,1,1,1,0,1,0,0,0,0,
        1,1,0,0,1,1,0,0,0,0
    ]
})

def group_metrics(frame):
    y = frame["y_true"].to_numpy()
    p = frame["y_pred"].to_numpy()
```

```

tp = ((y == 1) & (p == 1)).sum()
tn = ((y == 0) & (p == 0)).sum()
fp = ((y == 0) & (p == 1)).sum()
fn = ((y == 1) & (p == 0)).sum()

return pd.Series({
    "n": len(frame),
    "prevalence": y.mean(),
    "positive_prediction_rate": p.mean(),
    "TPR": tp / (tp + fn),
    "FPR": fp / (fp + tn),
    "precision": tp / (tp + fp)
})

fairness_table = (
    audit
    .groupby("group")["y_true", "y_pred"]
    .apply(group_metrics)
)

fairness_table.round(3)

```

	n	prevalence	positive_prediction_rate	TPR	FPR	precision
group						
A	10.0	0.5	0.5	0.8	0.200	0.8
B	10.0	0.4	0.4	0.5	0.333	0.5

The table shows why a single aggregate accuracy score cannot characterize fairness.

17.24 Visualize Group Metrics

```

plot_metrics = fairness_table[
    [
        "positive_prediction_rate",
        "TPR",
        "FPR",
        "precision"
    ]
]

```

```
].T

plot_metrics.plot(
    kind="bar",
    figsize=(9, 5)
)

plt.ylabel("Metric value")
plt.xlabel("Fairness-related metric")
plt.title("Group-Level Model Behavior")
plt.xticks(rotation=25)
plt.legend(title="Group")
plt.tight_layout()
plt.show()
```

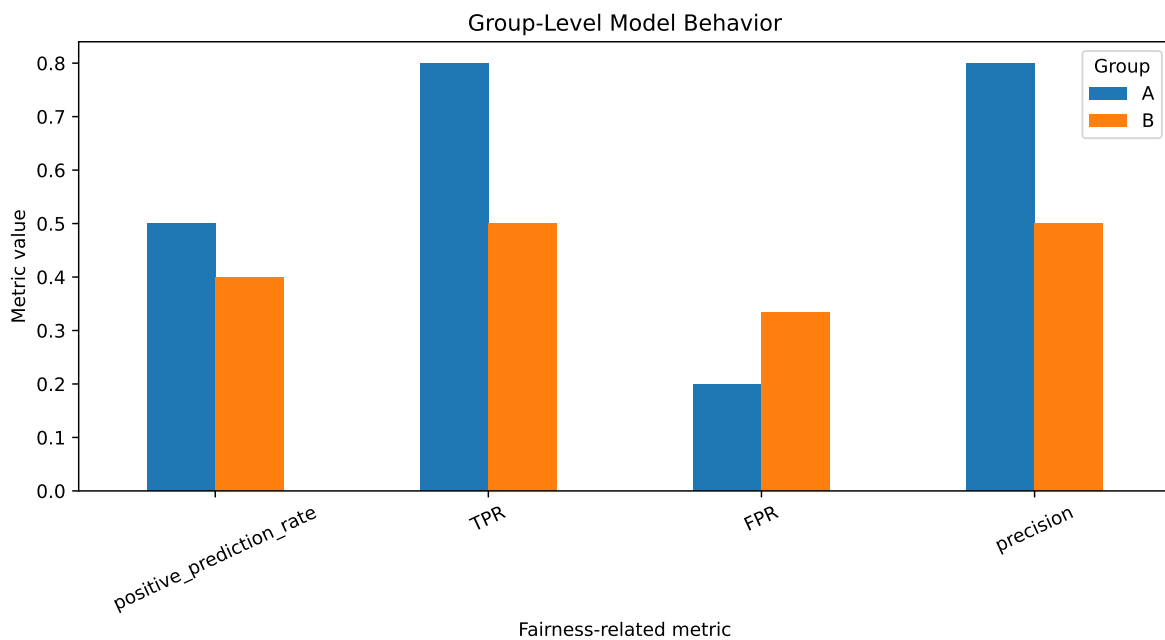


Figure 17.1: Different fairness criteria compare different aspects of model behavior across groups.

17.25 Fairness Criteria Can Conflict

Formal fairness criteria can be mutually incompatible under common conditions, particularly when base rates differ across groups (Kleinberg, Mullainathan, and Raghavan 2017).

17.26 Different Definitions Encode Different Normative Goals

Demographic parity asks about allocation rates.

Equal opportunity asks about correctly identifying actual positives.

Predictive parity asks about the reliability of positive predictions.

These are not interchangeable.

17.27 Base Rates Matter

Suppose:

$$P(Y = 1 \mid A = a) \neq P(Y = 1 \mid A = b).$$

Under unequal base rates and imperfect prediction, several desirable fairness conditions generally cannot all be satisfied simultaneously.

This is not a reason to abandon fairness analysis.

It is a reason to make the choice of fairness criterion explicit and connected to the application.

17.28 Fairness Is Not “Make Every Number Equal”

A mechanically optimized parity metric can create other harms.

The question is:

What kind of equality is substantively justified for this decision?

That requires domain, ethical, legal, and stakeholder reasoning in addition to mathematics.

17.29 Individual, Group, and Procedural Fairness

17.30 Group Fairness

Group fairness compares statistical behavior across defined groups.

17.31 Individual Fairness

A broad principle is:

Similar individuals should receive similar treatment.

The difficult question is what “similar” should mean.

Similarity itself can encode contested value judgments.

17.32 Procedural Fairness

Even a statistically balanced system may be procedurally unfair if:

- decisions are opaque;
- people cannot correct erroneous data;
- no appeal exists;
- responsibility is unclear;
- automated recommendations are treated as final.

Responsible systems need legitimate processes, not only metrics.

17.33 Intersectionality

17.34 One Attribute at a Time Can Hide Harm

Analyzing only one dimension may miss intersectional patterns.

For example, overall performance by one grouping variable can look acceptable while a smaller intersectional subgroup experiences much worse error rates.

17.35 Statistical Caution

As groups become more granular:

$$n_g \downarrow \Rightarrow \text{uncertainty} \uparrow .$$

Report sample sizes and uncertainty rather than making strong claims from tiny groups.

17.36 Explainability and Interpretability

Interpretability methods can be useful, but in high-stakes settings post hoc explanations should not be treated as substitutes for transparent, appropriately constrained modeling and governance (Rudin 2019).

17.37 Why Explain?

Different stakeholders need different explanations.

A data scientist may ask:

Which features drive model predictions globally?

A person affected by a decision may ask:

Why did this happen to me?

An auditor may ask:

Does the model rely on inappropriate proxies?

These are different explanation tasks.

17.38 Global Versus Local Explanation

Global explanation describes overall model behavior.

Local explanation describes a particular prediction.

Neither automatically reveals causality.

17.39 Permutation Importance

Permutation importance measures how much predictive performance deteriorates when a feature is randomly permuted.

Conceptually:

$$Importance_j = Score_{original} - Score_{permuted(j)}.$$

17.40 Demonstration

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
from sklearn.metrics import roc_auc_score

data = load_breast_cancer(
    as_frame=True
)

X = data.data
y = (data.target == 0).astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=.30,
    random_state=42,
    stratify=y
)

rf = RandomForestClassifier(
    n_estimators=250,
    random_state=42,
    n_jobs=1
)

rf.fit(
    X_train,
    y_train
)

rf_prob = rf.predict_proba(
    X_test
)[: , 1]

roc_auc_score(
    y_test,
    rf_prob
```

```
)
```

```
np.float64(0.9958382009345794)
```

```
importance = permutation_importance(
    rf,
    X_test,
    y_test,
    scoring="roc_auc",
    n_repeats=10,
    random_state=42,
    n_jobs=1
)

importance_df = pd.DataFrame({
    "feature": X.columns,
    "importance": importance.importances_mean
}).sort_values(
    "importance",
    ascending=False
).head(10)

importance_df
```

	feature	importance
27	worst concave points	0.005009
23	worst area	0.004658
22	worst perimeter	0.004527
13	area error	0.002344
21	worst texture	0.001687
20	worst radius	0.001322
7	mean concave points	0.001300
26	worst concavity	0.000387
1	mean texture	0.000329
29	worst fractal dimension	0.000015

```
ordered = importance_df.sort_values(
    "importance"
)
```

```
plt.figure(figsize=(8, 6))
plt.barh(
    ordered["feature"],
    ordered["importance"]
)
plt.xlabel("Decrease in ROC-AUC after permutation")
plt.ylabel("Feature")
plt.title("Permutation Importance")
plt.tight_layout()
plt.show()
```

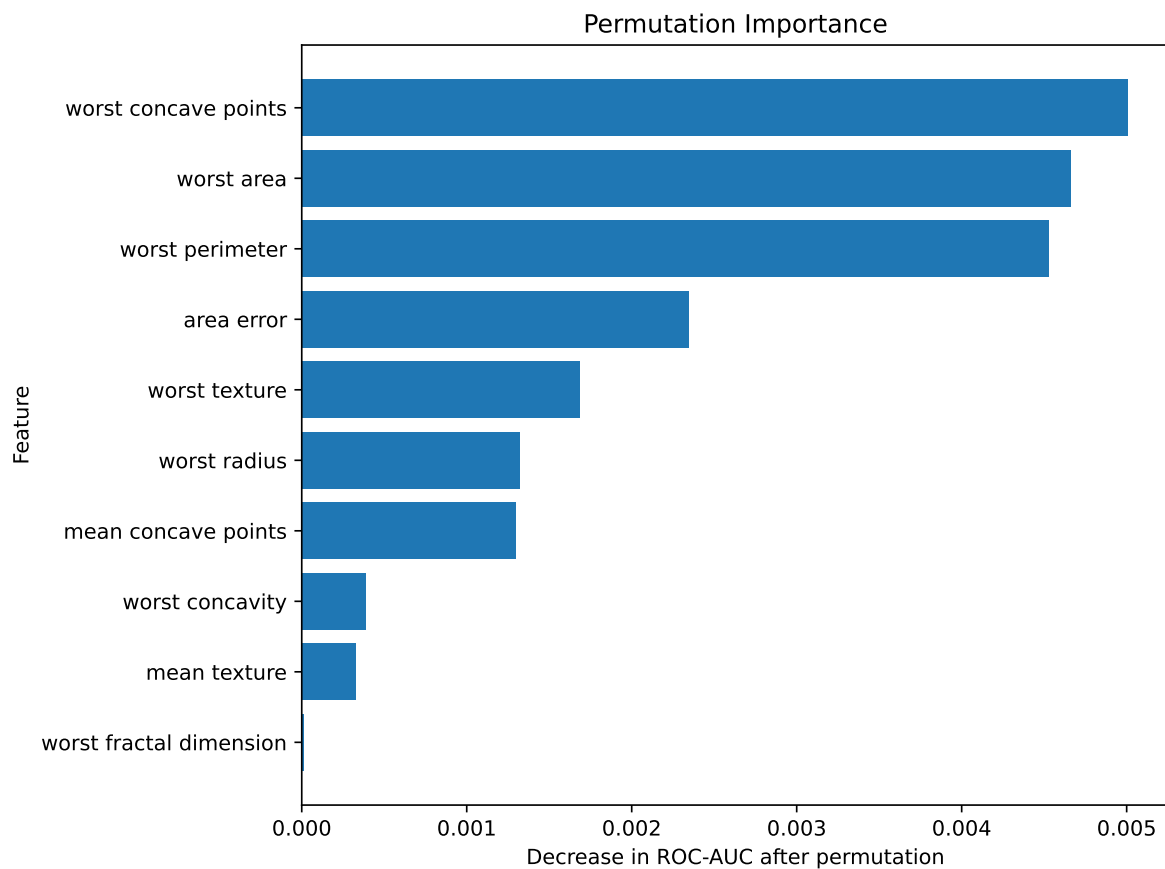


Figure 17.2: Permutation importance estimates how predictive performance changes when a feature's information is disrupted.

17.41 Importance Is Not Causality

If feature X_j is important for prediction, we cannot conclude:

$$do(X_j) \rightarrow Y.$$

Predictive association can arise from:

- confounding;
- proxy relationships;
- measurement artifacts;
- selection;
- correlated features.

Therefore:

feature importance $\neq$ causal effect

17.42 Explanations Can Also Mislead

An explanation method can be:

- unstable;
- sensitive to background data;
- incomplete;
- difficult for nontechnical users;
- persuasive despite being approximate.

An explanation should itself be evaluated for its intended purpose.

17.43 Privacy, Confidentiality, and Security

17.44 These Concepts Are Related but Different

Privacy concerns appropriate collection, use, inference, and control of information about people.

Confidentiality concerns obligations to protect entrusted information.

Security concerns protection against unauthorized access, alteration, or disruption.

A secure system can still violate privacy if data are used in ways people did not reasonably expect.

17.45 Data Minimization

A useful principle is:

Collect and retain only what is necessary for the legitimate purpose.

More features are not automatically better.

17.46 Reidentification Risk

Removing names does not necessarily anonymize a dataset.

Combinations of quasi-identifiers can enable reidentification.

17.47 Sensitive Inference

A model may infer sensitive information even when that attribute is not explicitly collected.

Removing a sensitive column does not guarantee that sensitive information has disappeared from the feature space.

17.48 Differential Privacy

17.49 Privacy as a Formal Guarantee

Differential privacy provides a mathematical framework for limiting how much the output of an analysis can depend on any one individual's data (Dwork and Roth 2014).

A randomized mechanism M is ϵ -differentially private if, for neighboring datasets D and D' :

$$P(M(D) \in S) \leq e^\epsilon P(M(D') \in S)$$

for all measurable output sets S .

17.50 Interpreting Epsilon

Smaller ϵ generally corresponds to a stronger privacy guarantee, but practical interpretation depends on the mechanism and total privacy accounting.

Differential privacy introduces a privacy-utility tradeoff.

17.51 Privacy Is Not Merely a Technical Parameter

Even a privacy-preserving algorithm does not answer:

- whether data collection was appropriate;
- whether consent was meaningful;
- whether the purpose is legitimate;
- whether deployment is harmful.

Technical privacy safeguards are necessary in many contexts but not sufficient for ethical legitimacy.

17.52 Human Oversight

17.53 “Human in the Loop” Is Not Enough

A human reviewer may simply accept model recommendations.

This is known as automation bias.

Meaningful oversight requires that humans have:

- relevant expertise;
- adequate time;
- access to evidence;
- authority to disagree;
- clear accountability;
- mechanisms for escalation.

17.54 Selective Automation

Some cases may be safe to automate while uncertain or consequential cases require review.

Conceptually:

low-risk case → automated workflow

high-risk or uncertain case → human review.

The boundary should be empirically and ethically justified.

17.55 Contestability

People affected by consequential predictions may need mechanisms to:

- understand the basis of a decision;
- correct inaccurate information;
- request human review;
- appeal outcomes.

Contestability is a design requirement, not an afterthought.

17.56 Governance Across the ML Lifecycle

17.57 Governance Begins Before Modeling

A responsible lifecycle can be represented as:

purpose → data → development → validation → deployment → monitoring → retirement.

17.58 Purpose and Scope

Document:

- intended use;
- intended users;
- affected populations;
- prohibited uses;
- decision authority.

17.59 Data Governance

Document:

- provenance;
- consent/authority;
- quality;
- retention;
- access;
- sensitive attributes;
- known limitations.

17.60 Model Governance

Document:

- version;
- training procedure;
- evaluation;
- thresholds;
- subgroup performance;
- explainability;
- known failure modes.

17.61 Deployment Governance

Define:

- who can use the system;

- when human approval is required;
- logging;
- incident response;
- monitoring;
- rollback;
- retirement criteria.

17.62 Model Cards

Model cards provide a structured approach for documenting intended uses, evaluation conditions, subgroup performance, limitations, and other model characteristics (M. Mitchell et al. 2019). Dataset documentation can play a complementary role through datasheets that make provenance, composition, collection, and recommended uses explicit (Gebru et al. 2021).

17.63 Documentation as an Engineering Artifact

A model card can summarize:

- model details;
- intended use;
- out-of-scope use;
- training/evaluation data;
- metrics;
- subgroup results;
- ethical considerations;
- caveats;
- limitations.

Documentation does not make a model responsible.

But undocumented systems are harder to audit responsibly.

17.64 Algorithmic Impact Assessment

17.65 Evaluate the System Before Deployment

An impact assessment can ask:

1. What decision will the system influence?

2. Who is affected?
3. What benefits are expected?
4. What harms are plausible?
5. How severe are those harms?
6. How likely are they?
7. What alternatives exist?
8. What evidence supports deployment?
9. How can people contest decisions?
10. What monitoring and stopping rules are required?

This reframes deployment as a hypothesis requiring evidence.

17.66 Feedback Loops and Performative Prediction

17.67 Predictions Can Change Outcomes

Suppose a university predicts that a student is likely to withdraw and provides intensive support.

If the student persists, the original prediction may appear “wrong,” even though the intervention succeeded.

This creates an evaluation challenge:

prediction → intervention → changed outcome.

The deployed model is no longer a passive observer.

17.68 Policy Changes the Meaning of Data

Once predictions influence treatment, future labels reflect both underlying risk and institutional response.

Retraining without recognizing this feedback can produce misleading conclusions.

17.69 Generative AI Inside Responsible ML Systems

17.70 Generative Components Add New Failure Modes

Systems that include LLMs may introduce:

- hallucinated facts;
- fabricated citations;
- prompt injection;
- sensitive-data leakage;
- unstable outputs;
- inappropriate tool use;
- anthropomorphic overtrust.

These risks sit alongside traditional ML concerns such as bias, calibration, and distribution shift.

17.71 RAG Does Not Eliminate Responsibility

Retrieval can improve grounding, but retrieved material can be:

- outdated;
- biased;
- malicious;
- irrelevant;
- incomplete.

The entire information pipeline requires governance.

17.72 Agentic Systems Increase Consequence

A model that can take actions requires stronger controls than one that only produces text.

Permissions should follow the principle of least privilege.

17.73 Responsible Computer Vision: What Does the Camera Make Legible?

Computer vision introduces distinctive responsibilities because visual data can contain far more information than the target variable requires.

An image collected for one purpose may also reveal or proxy:

- identity;
- age or apparent age;
- disability-related information;
- location or environment;
- socioeconomic context;
- health-related characteristics;
- demographic attributes;
- bystanders who never consented to analysis.

The responsible question is therefore not only

Can the model extract a useful signal?

but also

Should this visual information be collected, inferred, retained, or acted upon?

17.73.1 Representation and Measurement

A vision dataset is shaped by who and what was photographed, under which conditions, using which devices, and how labels were produced.

If one subgroup is systematically captured under different lighting, resolution, positioning, or equipment, apparent group differences in model performance may reflect the measurement process.

Image $\neq$ Neutral Measurement of Reality

17.73.2 Privacy and Surveillance

Removing names from image files does not necessarily make visual data anonymous. Faces, environments, metadata, distinctive objects, or repeated patterns may permit re-identification.

Computer vision can also transform passive observation into scalable inference. This raises governance questions that are not resolved by predictive accuracy.

17.73.3 Human Oversight in Visual Decisions

A human reviewer is not automatically a safeguard. If an automated vision score is presented as authoritative, reviewers may anchor on it even when the image is ambiguous or outside the model's validated domain.

Meaningful oversight requires:

- access to relevant context;
- authority to disagree;
- training about known failure modes;
- escalation pathways;
- documentation of overrides;
- monitoring for automation bias.

17.73.4 Generative and Synthetic Visual Content

Generative vision adds provenance concerns. Synthetic images may be useful for simulation, augmentation, education, or creative work, but realistic appearance does not establish authenticity or validity.

Research Reality Check

For high-stakes visual systems, ask whether the task itself is legitimate before optimizing the model.

A technically feasible inference can still be scientifically weak, socially harmful, privacy-invasive, or inappropriate to deploy.

17.73.5 Responsible Vision Audit

For one computer-vision application, document:

1. the legitimate purpose;
2. whose images are collected;

3. who may be missing or underrepresented;
4. what unintended information the images contain;
5. how labels were created;
6. subgroup and source-specific performance;
7. privacy and retention risks;
8. domain-shift risks;
9. the human-review process;
10. conditions under which the system should not be used.

17.74 Three Tracks

17.75 Health Track

Consider a model recommending which patients should receive additional screening.

Responsible evaluation should examine:

- clinical benefit;
- false-negative consequences;
- subgroup calibration;
- measurement proxies;
- privacy;
- external validation;
- clinician oversight;
- patient communication;
- appeal or correction mechanisms;
- post-deployment monitoring.

A model should not become a substitute for clinical judgment merely because it is accurate.

17.76 Education Track

Consider an early-warning model for student success.

Responsible design should examine:

- whether the target reflects learning or institutional behavior;
- whether financial, demographic, or behavioral proxies create inequitable treatment;
- whether intervention is supportive or punitive;
- student privacy;
- subgroup error rates;

- instructor automation bias;
- student contestability;
- whether predictions become self-fulfilling.

The ethical objective should be student support—not merely risk classification.

17.77 Open ML Benchmark Track

Benchmark projects should supplement aggregate performance with:

- subgroup analyses;
- robustness;
- calibration;
- explainability;
- privacy considerations;
- compute/energy reporting where relevant;
- failure analysis.

Responsible benchmarking asks not only which model wins, but what tradeoffs produce the ranking.

17.78 Beyond the Algorithm: Prediction Is an Exercise of Power

The Model Does Not Decide What Matters

An optimization algorithm can minimize a loss function.

It cannot independently determine whether the loss function represents a worthy social objective.

When we choose:

$Y = \text{student withdrawal,}$

$Y = \text{hospital utilization,}$

or

$Y = \text{employee attrition,}$

we transform a complex human condition into a measurable target.

That abstraction can be useful.

It can also hide what the target leaves out.

Responsible data science therefore requires two forms of reasoning:

1. **instrumental reasoning** — How can we optimize the system?
2. **normative reasoning** — What should the system optimize, and why?

The first can be assisted substantially by algorithms.

The second cannot be outsourced to an objective function.

The deepest question is not:

“Is the model unbiased?”

It is:

“What human purposes, assumptions, and institutions does this model make operational?”

Machine learning is never only prediction once predictions acquire consequences.

17.79 AI Pair Programmer: Remove the Sensitive Attribute

Ask an AI assistant:

“Our model may discriminate by race or sex. Remove those columns and confirm that the model is now fair.”

Audit the response.

A rigorous response should recognize:

- proxy variables;
- historical bias;
- label bias;
- unequal base rates;
- subgroup error differences;
- fairness-definition choices;
- intersectionality;
- procedural fairness;
- deployment consequences.

Redesign the audit rather than declaring fairness from column removal.

17.80 Hands-On Lab: Responsible ML Audit

Choose a classification system and:

1. define the intended use;
2. identify affected stakeholders;
3. identify plausible benefits;
4. identify plausible harms;
5. document data provenance;
6. examine target validity;
7. identify potential proxies;
8. report overall performance;
9. report group prevalence;
10. calculate group positive-prediction rates;
11. calculate group TPR and FPR;
12. calculate group precision;
13. quantify uncertainty;
14. inspect intersectional groups where feasible;
15. examine calibration by group;
16. evaluate feature importance;
17. explicitly reject causal interpretations of importance;
18. assess privacy risks;
19. define human-review boundaries;
20. design an appeal mechanism;
21. specify monitoring;
22. define rollback or retirement conditions;
23. complete a model card;
24. complete an impact assessment;
25. write a bounded deployment recommendation.

17.81 Practitioner Track

A practitioner should be able to identify harms, conduct subgroup audits, distinguish fairness definitions, interpret explanations cautiously, minimize privacy risk, document intended use, and design meaningful human oversight.

17.82 Researcher Track

A researcher should additionally justify fairness criteria, quantify uncertainty, examine intersectionality, analyze measurement validity, evaluate explanation stability, study feedback

loops, compare mitigation strategies, document governance assumptions, and separate technical findings from normative conclusions.

17.83 Research Studio

Design a responsible-ML study suitable for a conference or journal paper.

Specify:

1. substantive problem;
2. affected population;
3. stakeholder map;
4. target construct;
5. measurement validity;
6. data provenance;
7. historical context;
8. candidate models;
9. overall evaluation;
10. fairness definitions;
11. justification for those definitions;
12. subgroup metrics;
13. intersectional analysis;
14. uncertainty;
15. calibration;
16. threshold policy;
17. explainability method;
18. explanation validation;
19. privacy analysis;
20. human oversight;
21. contestability;
22. feedback-loop analysis;
23. governance plan;
24. post-deployment monitoring;
25. stopping criteria;
26. limitations;
27. unsupported claims;
28. scholarly contribution.

17.84 Responsibility Begins Before the Model

Consequential choices occur throughout the lifecycle:

Problem → Target → Data → Model → Evaluation → Decision → Monitoring

Responsible ML asks whether the problem should be predicted, whether the target is defensible, who is missing from the data, which errors cause harm, who receives benefits and burdens, whether decisions can be contested, what happens under change, and who remains accountable.

17.84.1 Fairness Metrics Do Not Choose Values for Us

Fairness criteria can reveal disparities and formalize tradeoffs. They do not determine which tradeoff a community ought to accept.

Fairness Metric $\neq$ Complete Ethical Judgment

Beyond the Algorithm: Optimization Contains Value Choices

Every objective function says what counts as success. Every threshold says which errors are tolerated. Every target says what the system treats as worth predicting. Responsible ML makes those choices visible enough to question.

Research Reality Check

Adding a fairness table at the end does not make a study responsible if the target is invalid, populations are excluded, deployment is inappropriate, or harms fall outside the measured metric.

17.85 Professor's Challenge: The Student Success Score

A university develops a “Student Success Score” using:

- LMS activity;
- prior grades;
- financial-aid status;
- attendance;
- advising records;
- course withdrawals;
- residential distance.

The model predicts persistence with ROC-AUC 0.89.

Students below a threshold are automatically restricted from registering for advanced courses until they meet with an advisor.

The university removes race and sex from the model and announces:

“Because protected characteristics are excluded and accuracy is high, the system is objective and fair.”

Critique the claim.

Consider:

- proxy variables;
- target validity;
- historical institutional patterns;
- measurement bias;
- subgroup calibration;
- false-positive consequences;
- threshold selection;
- intersectionality;
- privacy;
- autonomy;
- procedural fairness;
- contestability;
- whether restricting registration may itself affect persistence;
- feedback loops;
- whether a supportive intervention would be preferable to a restrictive one.

Redesign the system around student success rather than prediction alone.

17.86 Knowledge Check

1. Why is responsible ML a system property?
2. What is allocative harm?
3. What is quality-of-service harm?
4. What is representational harm?
5. What is historical bias?
6. What is representation bias?
7. What is measurement bias?
8. Why should labels not automatically be treated as ground truth?
9. What is deployment bias?
10. What is demographic parity?

11. What is equal opportunity?
12. What is equalized odds?
13. What is predictive parity?
14. Why can fairness criteria conflict?
15. Why is fairness not equivalent to making all group metrics equal?
16. What is individual fairness?
17. What is procedural fairness?
18. Why does intersectional analysis increase uncertainty?
19. How do global and local explanations differ?
20. Why is feature importance not a causal effect?
21. How can explanations themselves mislead?
22. How do privacy, confidentiality, and security differ?
23. Why does removing names not guarantee anonymity?
24. What does differential privacy attempt to guarantee?
25. Why is “human in the loop” insufficient?
26. What is automation bias?
27. What is contestability?
28. What should a model card document?
29. What is an algorithmic impact assessment?
30. How can deployed predictions create feedback loops?
31. Why does removing a sensitive feature not guarantee fairness?
32. Why do agentic AI systems require stronger governance?
33. When might a high-performing model still be inappropriate to deploy?
34. Why are normative questions not reducible to optimization?
35. Why is responsible ML a lifecycle property?
36. Why can fairness metrics conflict?
37. Why can a technically fair model still be inappropriate to deploy?
38. Where do value choices enter an ML workflow?

17.87 Reproducibility Check

Record:

- intended use;
- prohibited use;
- stakeholders;
- target construct;
- target definition;
- data provenance;
- collection authority/consent;
- sensitive attributes;
- proxy analysis;

- group sample sizes;
- overall metrics;
- subgroup metrics;
- uncertainty;
- calibration;
- fairness definitions;
- fairness rationale;
- intersectional analyses;
- thresholds;
- feature-importance method;
- explanation method;
- privacy controls;
- access controls;
- human-review rules;
- appeal mechanism;
- model version;
- data version;
- deployment date;
- monitoring metrics;
- drift triggers;
- incident-response process;
- rollback criteria;
- retirement criteria;
- model card;
- impact assessment;
- code;
- known limitations;
- unresolved risks.

17.88 Key Takeaways

- High predictive performance does not establish responsible deployment.
- Responsibility belongs to the full sociotechnical system, not only the model.
- Bias can enter through history, representation, measurement, labels, aggregation, and deployment.
- Fairness has multiple mathematical definitions because fairness has multiple substantive meanings.
- Fairness criteria can conflict, especially under unequal base rates.
- Group metrics are necessary in many applications but do not exhaust fairness.
- Intersectional analyses require attention to sample size and uncertainty.
- Model explanations are not causal explanations.

- Privacy, confidentiality, and security are related but distinct.
- Removing sensitive variables does not remove sensitive information or historical structure.
- Meaningful human oversight requires expertise, authority, time, and accountability.
- Contestability matters for consequential automated decisions.
- Governance should span purpose, data, development, validation, deployment, monitoring, and retirement.
- Model cards and impact assessments support accountability but do not substitute for responsible practice.
- Deployed predictions can create feedback loops that alter future data.
- Generative and agentic systems add new risks to traditional ML governance.
- Optimization answers how to pursue an objective; it does not determine which objectives are worthy.
- Once predictions affect people, machine learning becomes a question of consequence as well as accuracy.
- Responsibility begins with problem formulation and continues through monitoring.
- Fairness metrics inform but do not replace ethical and governance judgment.

17.89 Looking Ahead

Responsible machine learning requires more than good intentions or a one-time audit. Claims about performance, fairness, explanation, and risk must be traceable to the data, code, environments, decisions, and evidence that produced them. Chapter 18 completes the book by treating reproducibility as the infrastructure that makes machine-learning research inspectable, challengeable, extensible, and capable of contributing to cumulative knowledge.

18 Reproducible Machine Learning: From Analysis to Scientific Knowledge

18.1 Learning Objectives

By the end of this chapter, you should be able to:

- distinguish reproducibility, replication, robustness, and generalizability;
- explain why reproducibility is a property of a complete computational workflow rather than code alone;
- organize a machine-learning project so another researcher can understand and rerun it;
- document data provenance and transformations;
- control randomness and explain the limits of random seeds;
- record software and computational environments;
- distinguish source data, derived data, code, models, results, and publication artifacts;
- use Git conceptually to preserve project history;
- explain the role of GitHub or another remote repository in collaborative research;
- use Quarto to combine narrative, equations, code, figures, tables, and citations;
- design leakage-safe pipelines whose preprocessing is part of the reproducible model;
- save and reload fitted model artifacts appropriately;
- record experimental configurations and metrics;
- distinguish computational reproducibility from scientific validity;
- use persistent repositories such as Zenodo or OSF to archive research outputs;
- explain how DOIs support persistent citation and scholarly provenance;
- create a reproducibility checklist for health, education, and open ML research;
- design a research compendium that supports conference or journal submission;
- recognize when sharing data or models would be irresponsible or legally prohibited;

- treat reproducibility as part of scientific reasoning rather than an administrative task.
- design reproducible computer-vision workflows that preserve image provenance, preprocessing, grouping, model configuration, and evaluation conditions.

18.2 A Result Is More Than a Number

Suppose a paper reports:

$$ROC-AUC = 0.91.$$

A reader may reasonably ask:

- Which data version?
- Which inclusion criteria?
- Which preprocessing steps?
- Which random split?
- Which software versions?
- Which hyperparameters?
- Which metric implementation?
- Which code produced the result?
- Can the analysis be rerun?
- Can the result be reproduced independently?

A number without provenance is difficult to interrogate scientifically.

Central Question

What must we preserve so that a machine-learning result can become inspectable, repeatable, and cumulative scientific knowledge?

18.2.1 From Responsible Claims to Reproducible Evidence

Chapter 17 emphasized that trustworthy machine learning requires claims about fairness, explanation, privacy, governance, and consequences to be examined rather than assumed. Reproducibility provides the evidentiary infrastructure for that examination. If another researcher cannot reconstruct how data became results, then even careful evaluation and responsible-ML claims become difficult to verify, critique, or extend.

18.3 Reproducibility Is a Chain

Reproducible research treats the computational workflow as part of the evidence supporting a scientific claim, rather than as disposable implementation detail (Peng 2011; Sandve et al. 2013).

A useful mental model is:

Question → Data → Environment → Code → Experiment → Model → Result → Artifact → Archive.

If an important link is missing, reproducing the result becomes harder.

18.4 Four Related Ideas

Terminology around reproducibility is not fully standardized across disciplines, so the distinctions below are explicit conventions for this book (Goodman, Fanelli, and Ioannidis 2016). Broader scientific guidance likewise distinguishes reproducibility from replicability and treats both as central to trustworthy research (National Academies of Sciences, Engineering, and Medicine 2019).

18.4.1 Reproducibility

A useful operational definition is the ability to obtain consistent results using the same underlying data, computational procedures, and analysis.

18.4.2 Replication

Replication asks whether a finding can be obtained through a new study, new sample, or independently implemented investigation.

18.4.3 Robustness

Robustness asks whether conclusions survive reasonable analytical choices or perturbations.

18.4.4 Generalizability

Generalizability asks whether conclusions extend beyond the studied sample, setting, time, or population.

These ideas overlap, but they are not synonyms.

A computationally reproducible result can still fail to replicate or generalize.

18.5 The Research Compendium

A research compendium organizes code, data or access instructions, documentation, and computational outputs as a coherent scholarly artifact (Gentleman and Lang 2007).

18.6 Organize the Project Around the Research Process

A project might use:

```
machine-learning-study/  
  README.md  
  _quarto.yml  
  environment/  
    requirements.txt  
  data/  
    README.md  
    raw/  
    processed/  
  src/  
    prepare.py  
    features.py  
    train.py  
    evaluate.py  
  notebooks/  
    exploration.ipynb  
  models/  
  results/  
    figures/  
    tables/  
  manuscript/  
    paper.qmd  
  CITATION.cff
```

The exact structure can vary.

The important principle is that another person should be able to infer where inputs, transformations, outputs, and documentation belong.

18.7 Reproducibility Is More Than Rerunning Code

A research claim rests on an evidence chain:

Question → Data → Processing → Model → Evaluation → Result → Claim

Reproducibility asks whether another researcher can inspect and reconstruct that chain. This can require provenance, environment and package versions, seeds, configuration, data-access instructions, code, generated outputs, decision logs, licenses, and persistent identifiers.

Terminology varies across disciplines. This book uses the four definitions introduced above consistently. In computational ML work, **reproducibility** specifically includes the ability to regenerate reported results from the same underlying data and documented workflow.

18.7.1 From Student Submission to Scholarly Artifact

A conventional assignment is primarily designed to be graded. A research artifact should also be designed to be inspected, rerun, extended, cited, challenged, and reused where licensing permits.

Shift from:

Did I complete the analysis?

to:

Have I left enough evidence for another person to understand and reconstruct what I did?

Beyond the Algorithm: Reproducibility Is Intellectual Hospitality

Research asks readers to trust processes they did not personally observe. Documentation, provenance, code, and transparent decisions invite another person into the reasoning process and make scholarship more open to correction.

Final Research Reality Check

A repository containing code is not automatically reproducible. Ask whether a new researcher can determine the question, data provenance, analytical sample, required environment, origin of each reported result, major judgment calls, and unavoidable privacy/licensing/access limitations.

The goal is **maximal defensible transparency within real constraints**.

18.8 Final Three-Track Culmination

Guided Learner: reproduce one central analysis and explain each major workflow step.

Practitioner: build a documented end-to-end pipeline that can be rerun on new data.

Researcher: produce a research-ready package containing manuscript, provenance, analysis code, environment documentation, generated outputs, and reproducibility limitations.

18.9 Final Reflection

A trustworthy machine-learning project requires more than algorithmic performance:

Problem Framing+Measurement+Data+Modeling+Evaluation+Responsibility+Reproducibility.

Machine learning becomes mature analytical practice when these are treated as one connected system.

18.10 Separate Source From Derived Artifacts

A useful distinction is:

source data → derived data → analysis outputs.

Do not silently overwrite the original source data.

Derived files should be reproducible from documented inputs and code whenever possible.

18.11 Data Provenance

FAIR data stewardship emphasizes making research objects findable, accessible under appropriate conditions, interoperable, and reusable (Wilkinson et al. 2016).

18.12 Every Dataset Has a History

Record:

- source;
- access date;
- version or release;
- persistent identifier when available;
- license;
- inclusion/exclusion criteria;
- variable definitions;
- transformations;
- missing-data decisions;
- known limitations.

18.13 Create a Data README

A data/README.md can explain:

```
Dataset:
Source:
Version:
Access date:
License:
Unit of analysis:
Outcome:
Predictors:
Inclusion criteria:
Exclusion criteria:
Missingness:
Transformations:
Files not shared:
Reason not shared:
Reproduction instructions:
```


18.14 Hashes Can Identify File Versions

A cryptographic hash can help detect whether a file changed.

```
import hashlib

example_bytes = b"machine learning reproducibility"

sha256 = hashlib.sha256(
    example_bytes
).hexdigest()

sha256

'b53196464907eaecda7eedf8e83e76a5fd4e14ecce9526c83b803e5f3cdd2bea'
```

A hash does not explain what changed or whether the data are correct.

It provides a useful fingerprint for a particular byte sequence.

18.15 Randomness and Determinism

18.16 Random Seeds Improve Repeatability

Many ML procedures contain randomness:

- train/test splitting;
- bootstrapping;
- random forests;
- stochastic optimization;
- initialization;
- augmentation.

A fixed seed can make a computational path easier to reproduce.

```
rng = np.random.default_rng(
    42
)

rng.normal(
    size=5
)
```

```
array([ 0.30471708, -1.03998411,  0.7504512 ,  0.94056472, -1.95103519])
```

18.17 A Seed Is Not a Reproducibility Guarantee

Results can still differ because of:

- software versions;
- hardware;
- parallel execution;
- nondeterministic libraries;
- numerical precision;
- operating systems;
- changed data;
- changed defaults.

Therefore:

$$\boxed{\text{random seed}} \neq \boxed{\text{complete reproducibility}}.$$

18.18 Record the Computational Environment

18.19 Software Is Part of the Method

The computational environment can affect results.

Record at least:

- operating system;
- Python version;
- package versions;
- major external libraries;
- hardware when relevant.

18.20 Inspect the Current Environment

```

environment_info = {
    "python": sys.version.split()[0],
    "platform": platform.platform(),
    "numpy": np.__version__,
    "pandas": pd.__version__,
    "scikit-learn": sklearn.__version__
}

pd.Series(
    environment_info,
    name="version"
)

```

```

python          3.13.1
platform        Windows-11-10.0.26200-SP0
numpy            2.2.2
pandas          2.2.3
scikit-learn     1.6.1
Name: version, dtype: object

```

18.21 Dependency Files

Common approaches include:

- `requirements.txt`;
- `pyproject.toml`;
- Conda environment files;
- lock files;
- containers.

A plain package list improves documentation, while a lock file or container can provide stronger control over dependency resolution.

18.22 Do Not Confuse Environment Capture With Scientific Validity

A perfectly preserved environment can reproduce a flawed analysis exactly.

Reproducibility enables scrutiny.

It does not certify correctness.

18.23 Pipelines as Reproducible Methods

18.24 Preprocessing Is Part of the Model

Suppose standardization is performed before cross-validation using the full dataset.

That creates leakage.

A reproducible workflow should preserve the transformation boundary.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score

data = load_breast_cancer(
    as_frame=True
)

X = data.data
y = (data.target == 0).astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=.30,
    random_state=42,
    stratify=y
)

pipeline = Pipeline([
    (
        "scale",
        StandardScaler()
    ),
    (
        "model",
        LogisticRegression(
            max_iter=2000
        )
    )
])
```

```

])

pipeline.fit(
    X_train,
    y_train
)

prob = pipeline.predict_proba(
    X_test
)[: , 1]

roc_auc_score(
    y_test,
    prob
)

```

```
np.float64(0.997517523364486)
```

The pipeline preserves the order:

training data → fit preprocessing → fit model.

New data receive the transformations learned from training data.

18.25 Experiment Configuration

18.26 Separate Configuration From Results

An experiment should record the settings that produced the result.

```

experiment = {
    "experiment_id": "logistic-baseline-v1",
    "random_state": 42,
    "test_size": 0.30,
    "model": "LogisticRegression",
    "max_iter": 2000,
    "scaling": "StandardScaler",
    "primary_metric": "roc_auc"
}

```

```
experiment
```

```
{'experiment_id': 'logistic-baseline-v1',  
  'random_state': 42,  
  'test_size': 0.3,  
  'model': 'LogisticRegression',  
  'max_iter': 2000,  
  'scaling': 'StandardScaler',  
  'primary_metric': 'roc_auc'}
```

18.27 Store Machine-Readable Metadata

```
experiment_json = json.dumps(  
    experiment,  
    indent=2  
)  
  
print(  
    experiment_json  
)
```

```
{  
  "experiment_id": "logistic-baseline-v1",  
  "random_state": 42,  
  "test_size": 0.3,  
  "model": "LogisticRegression",  
  "max_iter": 2000,  
  "scaling": "StandardScaler",  
  "primary_metric": "roc_auc"  
}
```

Machine-readable metadata can support automated experiment tracking and auditing.

18.28 Results Should Point Back to Configuration

A result table should not contain only:

Model	ROC-AUC
Logistic Regression	0.98

It should also be possible to identify:

- experiment ID;
- code version;
- data version;
- configuration;
- environment;
- evaluation split.

18.29 Version Control With Git

18.30 Git Preserves Project History

Git records snapshots of a project over time.

A basic workflow is:

```
edit
↓
git status
↓
git add
↓
git commit
↓
git push
```

18.31 A Commit Should Represent a Meaningful Change

Useful commit messages might be:

```
Add leakage-safe preprocessing pipeline
Add external validation analysis
Correct outcome coding
Revise calibration figure
Document dataset release
```

Avoid treating version control as a periodic backup of mysterious changes.

18.32 Git Does Not Replace Documentation

A repository with 500 commits but no README may still be difficult to reproduce.

Version history and scientific documentation solve different problems.

18.33 GitHub and Remote Repositories

18.34 Why Use a Remote Repository?

A remote repository can support:

- collaboration;
- issue tracking;
- code review;
- version history;
- public dissemination;
- integration with archival services.

But GitHub itself should not automatically be treated as the permanent scholarly archive.

For a citable release, archive a stable version in an appropriate preservation repository.

18.35 Do Not Commit Secrets

Never commit:

- passwords;
- API keys;
- access tokens;
- private credentials;
- restricted data.

Use environment variables or secure secret-management systems.

If a secret is committed, deleting the visible line may not remove it from Git history.

18.36 Quarto as a Reproducible Publishing System

18.37 Narrative and Computation Belong Together

Quarto can combine:

- prose;
- equations;
- executable code;
- figures;
- tables;
- citations;
- cross-references;
- multiple output formats.

This reduces the distance between analysis and publication.

18.38 Literate Programming

Practical scientific-computing guidance emphasizes readable project organization, version control, documentation, automation, and workflows that other researchers can inspect and rerun (Wilson et al. 2017).

The principle is:

Explain the reasoning alongside the computation that implements it.

Instead of manually copying a result from a notebook into a manuscript, generate the result from code when practical.

18.39 A Figure Should Have Provenance

```
x = np.linspace(  
    0,  
    10,  
    100  
)  
  
y_demo = np.sin(
```

```
    x
)

plt.figure(figsize=(8, 4))
plt.plot(
    x,
    y_demo
)
plt.xlabel("x")
plt.ylabel("sin(x)")
plt.title("A Code-Generated Figure")
plt.show()
```

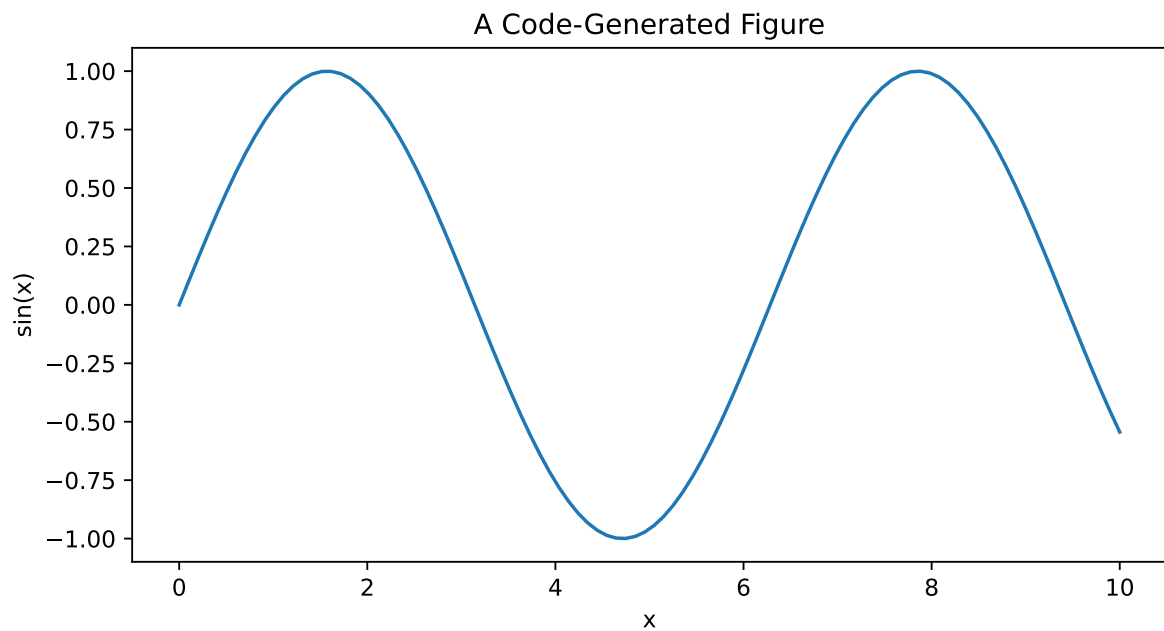


Figure 18.1: A reproducible figure is generated from code rather than manually reconstructed.

The figure's provenance is visible in the document source.

18.40 Notebooks Versus Scripts

18.41 Exploration and Production Serve Different Purposes

Notebooks are valuable for:

- exploration;
- teaching;
- visualization;
- iterative reasoning.

Scripts and modules are often useful for:

- reusable transformations;
- testing;
- automation;
- pipelines;
- production workflows.

A strong project can use both.

18.42 Beware Hidden Notebook State

A notebook may run successfully only because cells were executed out of order.

A useful test is:

Restart the kernel and run everything from the beginning.

If the analysis fails, the computational narrative is incomplete.

18.43 Saving Model Artifacts

18.44 Models Are Derived Artifacts

A fitted model can be saved for reuse.

```

import pickle
from tempfile import TemporaryDirectory

with TemporaryDirectory() as tmp:
    model_path = Path(
        tmp
    ) / "model.pkl"

    with open(
        model_path,
        "wb"
    ) as file:
        pickle.dump(
            pipeline,
            file
        )

    with open(
        model_path,
        "rb"
    ) as file:
        reloaded_pipeline = pickle.load(
            file
        )

    reloaded_prob = reloaded_pipeline.predict_proba(
        X_test
    )[:, 1]

    maximum_difference = np.max(
        np.abs(
            prob
            - reloaded_prob
        )
    )

maximum_difference

```

```
np.float64(0.0)
```

18.45 Serialization Has Limits

Serialized model artifacts can be:

- version-dependent;
- unsafe to load from untrusted sources;
- incomplete without preprocessing and metadata;
- incompatible across environments.

Never load an untrusted pickle file.

A model artifact should be accompanied by provenance and environment information.

18.46 Automated Workflows

18.47 Reproducibility Improves When Steps Are Explicit

A workflow might define:

download → validate → prepare → train → evaluate → render.

The more these dependencies are automated, the less the workflow depends on undocumented memory.

18.48 One-Command Reproduction

An aspirational standard is that a documented command can regenerate major outputs from permitted inputs.

Examples might include:

```
quarto render
```

or a project-specific workflow command.

The exact tooling matters less than the principle: reduce hidden manual steps.

18.49 Testing Data and ML Pipelines

18.50 Tests Are Not Only for Software Engineers

Research code can test assumptions such as:

- IDs are unique;
- expected columns exist;
- impossible values are absent;
- train/test groups do not overlap;
- outcome coding is valid;
- row counts fall within expected ranges.

18.51 Example Data Assertions

```
assert X.shape[0] == y.shape[0]
assert y.isin([0, 1]).all()
assert not X.columns.duplicated().any()
assert X_train.index.intersection(
    X_test.index
).empty
```

These checks turn assumptions into executable statements.

18.52 Tests Cannot Prove the Research Question Is Good

Automated tests can verify that code behaves as specified.

They cannot determine whether:

- the construct is valid;
- the causal claim is justified;
- the intervention is ethical;
- the research question matters.

Again, computational rigor and scientific judgment are complementary.

18.53 Provenance of Figures, Tables, and Claims

18.54 Every Result Should Be Traceable

For a reported number, ask:

Which code produced it?

For a figure:

Which data and transformation produced it?

For a model:

Which configuration and training sample produced it?

For a claim:

Which evidence supports it?

This creates a provenance graph:

$$\text{claim} \leftarrow \text{result} \leftarrow \text{code} \leftarrow \text{data}.$$

18.55 Archiving and Persistent Identifiers

18.56 A Repository Is Living; A Release Is Fixed

Active Git repositories evolve.

Scholarly citation often requires a stable snapshot.

A release can freeze a version of:

- code;
- documentation;
- configuration;
- figures;
- supplementary materials.

18.57 Zenodo and DOI Archiving

A common workflow is:

```
GitHub repository
↓
versioned release
↓
Zenodo archive
↓
DOI
```

A DOI provides a persistent identifier for the archived scholarly object.

18.58 Concept DOI and Version DOI

When supported by the repository, individual releases can receive version-specific identifiers while the overall project can have a persistent concept-level identifier.

This supports both:

- citing the project generally;
- citing the exact version used in a study.

18.59 OSF as a Research Hub

OSF can support:

- project organization;
- registrations;
- preregistration;
- files;
- collaboration;
- links to other research services.

The best repository depends on the artifact and research workflow.

18.60 DOI Does Not Mean Peer Reviewed

A DOI establishes persistent identification.

It does not automatically imply:

- peer review;
- methodological quality;
- correctness;
- journal publication.

Persistent citation and scientific validation are different functions.

18.61 Citation and Credit

18.62 Make Software and Data Citable

A project can include citation metadata such as:

```
CITATION.cff
```

Document:

- authors;
- title;
- version;
- release date;
- DOI;
- repository;
- preferred citation.

18.63 Cite Data as Research Objects

Datasets should be cited when possible using their official citation or persistent identifier.

Data provenance is part of scholarly credit.

18.64 Reproducibility and Restricted Data

18.65 Open Science Does Not Mean “Upload Everything”

Open-science norms encourage transparent and reusable research practices while still requiring appropriate protection of sensitive or restricted materials (Nosek et al. 2015).

Health, education, financial, and institutional data may contain sensitive or restricted information.

Responsible reproducibility can use:

- data dictionaries;
- synthetic examples;
- access instructions;
- transformation code;
- controlled repositories;
- metadata;
- documented eligibility procedures.

The goal is **as open as possible, as restricted as necessary**.

18.66 Never Publish Data Merely to Satisfy a Reproducibility Checklist

Legal, ethical, contractual, and privacy obligations take priority.

A reproducible workflow can document why an artifact cannot be public.

18.67 Robustness and Multiverse Thinking

18.68 One Analysis Path Is Rarely the Only Reasonable Path

Researchers make choices about:

- exclusions;
- transformations;
- missing data;
- features;
- models;

- hyperparameters;
- thresholds;
- metrics.

A robustness analysis asks whether conclusions depend heavily on one defensible choice.

18.69 Specification Curves and Multiverse Analyses

For some research questions, researchers can examine many reasonable analytical specifications.

Conceptually:

$$\mathcal{S} = \{s_1, s_2, \dots, s_m\}.$$

For each specification:

$$\hat{\theta}_j = f(D, s_j).$$

The distribution of results reveals sensitivity to analytical decisions.

This is particularly valuable when the scientific conclusion—not merely predictive performance—is the primary outcome.

18.70 Reproducibility Is Not Replication

18.71 The Distinction Matters

Imagine that another researcher obtains exactly the same ROC-AUC by running your code on your data.

That supports computational reproducibility.

Now imagine they collect a new sample from another university and obtain similar results.

That supports replication and potentially transportability more strongly.

The second claim requires new evidence.

18.72 Reproducible Computer Vision: Preserve the Visual Evidence Chain

Computer-vision reproducibility requires more than saving a fitted model.

A visual workflow may depend on:

Raw Image → Decode → Resize/Crop → Normalize → Augment → Represent → Model → Evaluate.

Each transformation can change the evidence.

A reproducible vision project should document, where applicable:

- image source and provenance;
- licensing and consent constraints;
- file formats and decoding libraries;
- inclusion and exclusion rules;
- duplicate and near-duplicate handling;
- subject, device, session, or institution grouping;
- resize and crop rules;
- channel and color-space handling;
- normalization;
- augmentation operations and probabilities;
- random seeds;
- pretrained model or checkpoint identifiers;
- model architecture and weights;
- train/validation/test source definitions;
- evaluation metrics and subgroup analyses;
- hardware or accelerator details when they materially affect execution;
- generated artifacts and model outputs.

18.72.1 Preserve the Split, Not Just the Seed

For grouped visual data, a random seed alone is insufficient. If images must generalize to new people, devices, or institutions, preserve the group-level split definition or the exact observation identifiers assigned to each partition.

Seed $\neq$ Data-Provenance Record

18.72.2 Restricted Images

Visual datasets can be impossible or inappropriate to redistribute because of privacy, consent, licensing, proprietary ownership, or institutional restrictions.

Reproducibility does not require violating those constraints.

Instead, preserve:

- acquisition and eligibility logic;
- data dictionaries and schemas;
- preprocessing code;
- checksums or identifiers where permissible;
- synthetic or public demonstration data when useful;
- access instructions;
- documentation of what cannot be shared and why.

Beyond the Algorithm: Reproducibility Preserves How We Saw the Data

In computer vision, preprocessing determines what the model is allowed to see. Documenting crop rules, image transformations, exclusions, and acquisition conditions therefore preserves more than code. It preserves part of the study's **epistemic viewpoint**.

18.72.3 Vision Reproducibility Challenge

Take one vision workflow from earlier chapters and create a minimal reproducibility manifest containing:

1. dataset and provenance;
2. unit of analysis;
3. target;
4. image representation;
5. preprocessing;
6. split logic;
7. model and hyperparameters;
8. seed;
9. evaluation metrics;
10. software environment;
11. expected output;
12. limitations on redistribution or reuse.

A second student should be able to reconstruct the analytical logic without relying on undocumented assumptions.

18.73 Three Tracks

18.74 Health Track

A reproducible health ML study should document:

- cohort construction;
- diagnostic definitions;
- coding systems;
- data release;
- temporal boundaries;
- missingness;
- preprocessing;
- patient-level splitting;
- calibration;
- external validation;
- software environment;
- restricted-data access procedures.

Sensitive patient-level data may remain protected while code and metadata are shared.

18.75 Education Track

A reproducible education study should document:

- institution and term boundaries;
- student-level identifiers and deidentification;
- outcome definitions;
- LMS extraction logic;
- course/instructor effects;
- repeated-student handling;
- intervention timing;
- FERPA or institutional restrictions where applicable;
- code and synthetic examples when raw data cannot be released.

A reproducible early-warning model must make clear exactly **when** each predictor becomes available.

18.76 Open ML Benchmark Track

A benchmark study should preserve:

- dataset version;
- split;
- preprocessing;
- seeds;
- hyperparameter search space;
- compute environment;
- evaluation code;
- model configuration;
- final metrics;
- uncertainty;
- runtime or compute information;
- exact release used for the paper.

Benchmark reproducibility is strengthened when another researcher can rerun the comparison without reconstructing undocumented decisions.

18.77 Beyond the Algorithm: Reproducibility as Intellectual Humility

A Scientific Result Should Be Able to Survive Its Author

A researcher remembers why a row was removed.

A collaborator remembers which file is current.

A student remembers which notebook cell must run first.

Then time passes.

Memory is not an archival system.

Reproducibility asks us to move knowledge from the researcher's memory into inspectable artifacts.

That is partly an engineering practice.

It is also a philosophical commitment.

To make a workflow reproducible is to accept that our work should be open to:

- inspection;
- criticism;
- correction;
- extension;
- replication.

The purpose is not to prove that we were right.

The purpose is to make it possible for knowledge to advance even when we were wrong. In that sense:

reproducibility = an infrastructure for scientific self-correction

A result becomes more valuable when others can understand how it came to exist.

18.78 AI Pair Programmer: Make My Study Reproducible

Ask an AI assistant:

“I set random_state=42, so my machine-learning study is fully reproducible. Write a sentence confirming this for my paper.”

Audit the response.

A rigorous answer should distinguish the random seed from:

- data version;
- software environment;
- preprocessing;
- split logic;
- package defaults;
- hardware;
- parallelism;
- code version;
- experiment configuration;
- provenance.

Rewrite the statement so it accurately describes what the seed contributes.

18.79 Hands-On Lab: Build a Research Compendium

Create a small reproducible ML project.

1. define a research question;
2. create a project README;
3. create a data README;
4. record dataset provenance;
5. preserve source and derived data separately;
6. create a leakage-safe pipeline;
7. set and document random seeds;

8. record package versions;
9. create an experiment configuration;
10. fit a baseline;
11. evaluate the model;
12. generate one figure from code;
13. generate one table from code;
14. add executable data checks;
15. place reusable code in `src/`;
16. render the report with Quarto;
17. initialize Git;
18. make meaningful commits;
19. push to a remote repository when appropriate;
20. add citation metadata;
21. create a versioned release;
22. archive the release in a persistent repository when appropriate;
23. record its persistent identifier;
24. document any files that cannot be shared;
25. ask another person to reproduce the workflow from the README.

18.80 Practitioner Track

A practitioner should be able to structure a project, preserve provenance, control randomness, record environments, build pipelines, version code, generate reports from computation, test assumptions, and archive stable releases.

18.81 Researcher Track

A researcher should additionally distinguish reproducibility from replication, preregister analytical decisions when appropriate (Nosek et al. 2018), conduct robustness analyses, preserve research provenance, document deviations, archive citable artifacts, design workflows for independent verification, and distinguish open science from indiscriminate data sharing.

18.82 Research Studio

Design a reproducibility plan suitable for a conference or journal submission.

Specify:

1. research question;

2. hypotheses or predictive claims;
3. data source;
4. dataset version;
5. access date;
6. license or restrictions;
7. cohort/sample construction;
8. unit of analysis;
9. target definition;
10. preprocessing;
11. feature engineering;
12. split logic;
13. random seeds;
14. environment specification;
15. model configurations;
16. hyperparameter search;
17. evaluation protocol;
18. robustness analyses;
19. generated tables;
20. generated figures;
21. model artifacts;
22. code repository;
23. documentation;
24. citation metadata;
25. preregistration/registration where appropriate;
26. archival repository;
27. persistent identifier;
28. restricted-data reproduction pathway;
29. independent reproduction test;
30. scholarly contribution.

18.83 Professor's Challenge: The Irreproducible Breakthrough

A research team reports a new model for predicting college persistence.

The paper claims:

$$ROC-AUC = 0.94,$$

substantially exceeding previous studies.

The authors provide a GitHub repository containing one notebook.

However:

- the dataset is called `final_data_v7_REAL.csv`;
- no data source or release is documented;
- preprocessing was partly performed manually in Excel;
- notebook cells must be run in an undocumented order;
- no random seeds are reported;
- package versions are absent;
- the train/test split file is missing;
- hyperparameter search results are unavailable;
- the manuscript figure was manually recreated in PowerPoint;
- the repository changed after publication;
- no versioned release was archived;
- the raw student data cannot legally be shared.

Evaluate the claim that the study is “open and reproducible because the code is on GitHub.”

Redesign the project without violating student privacy.

Your solution should address:

- provenance;
- manual transformations;
- environment capture;
- split preservation;
- experiment tracking;
- executable figures;
- version control;
- release archiving;
- DOI assignment;
- restricted-data documentation;
- synthetic or simulated examples;
- independent verification.

18.84 Capstone: From Machine Learning Project to Research Artifact

Choose one of the book’s three tracks.

18.85 Health

Develop a reproducible prediction study using a public or appropriately accessible health dataset.

18.86 Education

Develop a reproducible study of a learning, persistence, achievement, or institutional outcome.

18.87 Open ML

Develop a reproducible benchmark, methodological comparison, or robustness study using a public dataset.

For your chosen project, produce:

1. research question;
2. literature-grounded motivation;
3. data provenance;
4. exploratory analysis;
5. preprocessing pipeline;
6. baseline;
7. candidate models;
8. tuning;
9. model evaluation;
10. calibration where appropriate;
11. uncertainty;
12. responsible-ML audit;
13. robustness analysis;
14. reproducible Quarto manuscript;
15. Git repository;
16. README;
17. environment specification;
18. citation metadata;
19. versioned release;
20. archival plan;
21. conference or journal positioning statement.

The objective is not merely to train a model.

It is to produce a defensible **research artifact**.

18.88 Knowledge Check

1. How does reproducibility differ from replication?
2. How does robustness differ from generalizability?
3. Why is reproducibility a workflow property rather than a code property?
4. Why should source data not be silently overwritten?
5. What information belongs in a data README?
6. What does a cryptographic hash provide?
7. Why is a random seed insufficient for complete reproducibility?
8. Why are software versions part of the method?
9. How do lock files differ conceptually from simple dependency lists?
10. Why should preprocessing be included inside an ML pipeline?
11. What should an experiment configuration record?
12. What is the purpose of Git?
13. Why does Git not replace scientific documentation?
14. Why should secrets never be committed to a repository?
15. What is literate programming?
16. Why can notebooks contain hidden state?
17. Why should untrusted pickle files never be loaded?
18. What is the benefit of automating a workflow?
19. What kinds of assumptions can executable tests check?
20. What is research provenance?
21. Why is a versioned release useful?
22. What role can Zenodo play in an ML project?
23. What is a DOI?
24. Why does a DOI not imply peer review?
25. What role can OSF play in a research workflow?
26. Why should datasets be cited?
27. Why does open science not require publishing restricted data?
28. What is a robustness analysis?
29. What is a multiverse analysis?
30. Why can a perfectly reproducible analysis still be scientifically wrong?
31. What additional evidence does replication provide?
32. Why should figures and tables be generated from code when practical?
33. What should be preserved so another researcher can reproduce an ML result?
34. Why is reproducibility an infrastructure for scientific self-correction?
35. How does computational reproducibility differ from replication and robustness?
36. Why is code alone insufficient?
37. What should a research package preserve about analytical judgment?
38. Why is reproducibility part of scholarly accountability?

18.89 Reproducibility Check

Before releasing a project, verify:

- README;
- research question;
- data README;
- data source;
- dataset version;
- access date;
- license;
- restrictions;
- data dictionary;
- inclusion/exclusion logic;
- source/derived-data separation;
- preprocessing code;
- feature-engineering code;
- split logic;
- seeds;
- environment specification;
- package versions;
- hardware notes where relevant;
- experiment configurations;
- hyperparameter search;
- evaluation code;
- uncertainty analysis;
- robustness analysis;
- generated figures;
- generated tables;
- executable checks/tests;
- model artifacts;
- model metadata;
- Git history;
- remote repository;
- meaningful release tag;
- citation metadata;
- archive;
- persistent identifier;
- restricted-data pathway;
- known limitations;
- independent reproduction instructions.

18.90 Key Takeaways

- Reproducibility, replication, robustness, and generalizability answer different questions.
- Reproducibility belongs to the complete research workflow.
- Data provenance is part of the scientific method.
- Source data and derived artifacts should be distinguishable.
- Random seeds help repeat computational paths but do not guarantee reproducibility.
- Software environments are part of computational methodology.
- Leakage-safe preprocessing should be preserved inside the modeling workflow.
- Experiment configurations connect results to the procedures that generated them.
- Git preserves project history; documentation explains the science.
- Quarto can connect narrative, computation, equations, figures, tables, and citations.
- Hidden notebook state is a reproducibility risk.
- Automated checks make assumptions inspectable.
- Figures, tables, models, and claims should have traceable provenance.
- A living repository and a stable scholarly release serve different purposes.
- Persistent archives and DOIs improve citation and preservation.
- A DOI does not certify scientific quality.
- Open science does not require irresponsible disclosure of restricted data.
- Robustness analysis asks whether conclusions depend on reasonable analytical choices.
- A reproducible analysis can still be wrong.
- Replication requires new evidence.
- Reproducibility enables criticism, correction, extension, and cumulative knowledge.
- A mature ML project should end not merely as a fitted model, but as a defensible research artifact.
- Reproducibility concerns the entire evidence chain.
- Replication and robustness ask questions reproducibility alone cannot answer.
- Research-ready artifacts allow others to inspect, reconstruct, and challenge the analytical process.

Part VII

Resources

Teaching with This Book

Machine Learning Using Python, Second Edition is designed to support more than one instructional setting. Its 18 chapters can be taught sequentially, selectively, or through research-oriented pathways while preserving a common emphasis on analytical reasoning, evidence, responsibility, and reproducibility.

The central instructional principle is:

Learning Machine Learning = Concepts + Code + Evidence + Judgment

The goal is not simply to cover algorithms. Students should learn to formulate problems, prepare data responsibly, implement models, evaluate evidence, interpret limitations, communicate results, and produce work that another analyst or researcher can inspect and reproduce.

Recurring Learning Experiences

The book uses recurring features that instructors can combine according to course level and available time:

- **Learning Objectives** establish the intended competencies.
- **Hands-On Practice** moves concepts into executable analysis.
- **Health, Education, and Open ML Tracks** provide longitudinal application contexts.
- **Computer Vision** appears as a recurring workload across classical and modern machine learning.
- **AI Pair Programmer** activities require students to inspect, test, critique, and improve AI-assisted work.
- **Professor's Challenge** asks students to reason beyond routine implementation.
- **Practitioner Track** emphasizes defensible analytical decisions.
- **Researcher Track** emphasizes methodological reasoning and evidence.
- **Research Studio** helps students move from algorithm application toward research contribution.
- **Knowledge Checks** support retrieval and conceptual diagnosis.
- **Reproducibility Checks** reinforce computational transparency.
- **Key Takeaways** consolidate the chapter's most important ideas.

Not every activity needs to be assigned. A strong course selects activities that reinforce its learning outcomes rather than maximizing the quantity of completed exercises.

The Three Application Tracks

The **Health**, **Education**, and **Open ML** tracks can be used in several ways.

An instructor may assign one track to the entire class for domain continuity, allow students to choose a track according to their interests, or deliberately compare tracks so students can examine how the same modeling decision changes across substantive contexts.

The tracks are particularly useful for reinforcing a central principle of the book:

The Algorithm May Be General; the Evidence Is Contextual

Students should therefore be encouraged to discuss measurement, target definition, population, consequences of error, and domain assumptions—not merely model performance.

A 7-Week Graduate Course

A compressed graduate course should emphasize integration rather than attempt to give every chapter equal classroom time.

Week	Primary Chapters	Instructional Emphasis	Suggested Learning Experience
1	1–3	ML foundations, Python, statistics, mathematical reasoning	Readiness activity, reproducible notebook, conceptual diagnostic
2	4–5	Problem formulation, data preparation, leakage, feature reasoning	Data-wrangling and problem-formulation lab
3	6–8	Regression, classification, trees, random forests	Comparative supervised-learning investigation
4	9–12	SVMs, ensembles, clustering, dimensionality reduction	Model comparison plus unsupervised exploration

Week	Primary Chapters	Instructional Emphasis	Suggested Learning Experience
5	13–15	Neural networks, NLP, generative AI	Modern-AI lab; AI Pair Programmer critique
6	16–17	Evaluation, calibration, validation, responsible ML	Model audit, fairness/evaluation analysis
7	18 + synthesis	Reproducible ML research	Reproducible final project and evidence package

Recommended 7-Week Assessment Pattern

A practical assessment sequence is:

1. **Programming and analytical readiness activity**
2. **Data preparation and leakage-safe workflow lab**
3. **Supervised-model comparison**
4. **Unsupervised or representation-learning investigation**
5. **Modern-AI critique or implementation**
6. **Evaluation and responsibility audit**
7. **Reproducible final project**

The final project should require more than a high-performing model. Students should submit the research or analytical question, data provenance, preprocessing decisions, model rationale, evaluation design, interpretation, limitations, responsibility considerations, and reproducible code.

A 14-Week Graduate Course

A traditional semester provides more time for mathematical development, coding practice, domain applications, and research reasoning.

Week	Primary Chapter(s)	Suggested Emphasis
1	1	Machine learning as evidence-based analytical practice
2	2	Python for reproducible machine learning
3	3	Statistical and mathematical foundations
4	4	Formulating machine-learning problems

Week	Primary Chapter(s)	Suggested Emphasis
5	5	Data preparation, leakage, and feature reasoning
6	6	Linear regression
7	7	Classification
8	8–9	Trees, random forests, and support vector machines
9	10	Ensemble learning
10	11–12	Clustering and dimensionality reduction
11	13	Neural networks
12	14–15	NLP and generative AI
13	16–17	Evaluation and responsible machine learning
14	18	Reproducible research and final synthesis

A 14-week course can use the additional time for replication exercises, short methodological readings, student-led discussions, domain-specific laboratories, computer-vision extensions, and iterative project feedback.

Suggested Semester Project Milestones

Rather than assigning one large project at the end, instructors can stage the work:

1. **Question and domain justification**
2. **Data source and provenance**
3. **Exploratory analysis and measurement decisions**
4. **Baseline model**
5. **Model comparison and tuning**
6. **Evaluation plan**
7. **Responsibility and limitation audit**
8. **Reproducible final report**

This sequence makes the analytical process visible and reduces the temptation to treat modeling as a last-minute search for the highest score.

A Research Seminar

For a research seminar, the book should function less as a linear textbook and more as a methodological spine.

A useful core pathway is:

Chapters 1, 3, 4, 5 → Relevant Method Chapter(s) → 16 → 17 → 18.

Students can use the remaining chapters according to their research questions.

Seminar Learning Cycle

A research seminar can repeatedly use the following cycle:

Read → Question → Replicate → Critique → Extend → Communicate.

Possible seminar activities include:

- critically reading a recent machine-learning paper;
- reproducing a published analytical workflow or selected result;
- identifying leakage or weak evaluation design;
- comparing an original method with a modern alternative;
- conducting a robustness or sensitivity analysis;
- auditing model claims against the evidence presented;
- examining fairness, privacy, or deployment consequences;
- converting exploratory work into a reproducible research compendium;
- developing a research proposal grounded in a substantive uncertainty rather than an algorithm alone.

From Application to Contribution

Students should continually distinguish:

“I applied an algorithm to a dataset”

from:

“I investigated a substantive or methodological uncertainty using an appropriate analytical design.”

That distinction is especially important in doctoral and research-oriented settings.

Guided Self-Study

Self-directed readers can move through the book using four repeating actions:

1. **Understand** — read the conceptual and mathematical explanation.
2. **Execute** — reproduce the code rather than merely reading it.
3. **Interrogate** — change assumptions, parameters, data, or evaluation choices.
4. **Explain** — write what the evidence supports, what it does not support, and what you would do next.

A sustainable self-study rhythm is one substantive chapter or chapter pair per week, with additional time for Chapters 3, 5, 13–18 when the material is unfamiliar.

Self-Study Evidence Portfolio

Instead of measuring progress by chapters completed, maintain a small portfolio containing:

- reproducible notebooks or Quarto documents;
- short explanations of important equations;
- model-comparison tables;
- error analyses;
- one computer-vision investigation;
- one responsible-ML audit;
- one AI-assisted coding critique;
- a final reproducible project.

The portfolio becomes evidence of analytical development rather than a record of pages read.

Teaching Computer Vision Across the Book

Computer vision is intentionally treated as a longitudinal workload. Instructors can use image-based examples to connect:

- data representation and preprocessing;
- classical classification;
- clustering and dimensionality reduction;
- neural representations;
- multimodal and generative AI;
- domain shift and external validation;
- fairness, privacy, and responsible deployment;
- reproducible visual pipelines.

This approach allows students to experience computer vision without requiring a separate specialized course.

Using Generative AI in Instruction

The book does not treat generative AI as either prohibited assistance or unquestioned authority.

A useful instructional rule is:

AI Assistance Does Not Transfer Analytical Responsibility

When students use generative AI, instructors can require them to document what the tool contributed, test generated code, verify citations and APIs, inspect assumptions, identify possible leakage, and explain why the final analytical decisions are defensible.

AI Pair Programmer activities are therefore best assessed on **verification and reasoning**, not on whether AI was used.

Reproducibility as a Course-Level Expectation

Reproducibility should not first appear in the final week.

From the beginning of a course, students can be expected to:

- identify data sources;
- preserve provenance;
- separate source data from derived data when appropriate;
- record transformations;
- use deterministic seeds when meaningful;
- avoid leakage;
- document dependencies;
- use version control;
- generate results from code;
- distinguish exploratory decisions from confirmatory claims.

Chapter 18 then integrates practices students have already experienced throughout the course.

Designing Assessments

Strong assessments ask students to justify decisions rather than merely produce outputs.

Instead of:

Fit three classifiers and report accuracy.

consider:

Formulate the prediction problem, justify the evaluation metric, establish a baseline, compare appropriate models using a leakage-safe workflow, analyze important errors, and explain what evidence would be required before the model should influence a real decision.

The second task assesses machine-learning judgment.

A Suggested Evaluation Lens

Across assignments and projects, instructors can evaluate five dimensions:

Dimension	Central Question
Conceptual reasoning	Does the student understand what the method is doing?
Computational implementation	Is the analysis correct, transparent, and reproducible?
Evaluation quality	Does the evidence support the performance claim?
Interpretation and responsibility	Are limitations, consequences, and domain assumptions considered?
Communication	Can another reader understand and inspect the analytical argument?

This lens works across algorithmic, applied, and research-oriented assignments.

Closing Perspective

A course built around this book should leave students with more than a collection of models.

They should be able to ask:

- What problem is actually being solved?
- What assumptions connect the data to the claim?
- What evidence supports model performance?
- How might the model fail?
- Who could be affected?
- What would make the analysis reproducible?
- What would turn this application into a meaningful research contribution?

The enduring competency is not knowing which Python function to call.

It is knowing **what should be done, why it is defensible, what the evidence permits us to conclude, and how another person can verify the work.**

About the Author



Dr. Itauma Itauma is an Associate Professor of Data Science and Analytics at Harrisburg University of Science and Technology. His work brings together data science, artificial intelligence, machine learning, analytics, research methodology, and graduate education.

His scholarly interests include artificial intelligence and machine learning in education, health, and business; learning analytics; structural equation modeling; meta-analysis; reproducible analytics; and the use of computational methods to strengthen evidence-based decision-making.

Dr. Itauma holds a Ph.D. in Instructional Design and Technology with a concentration in Analytics, an M.S. in Computer Science, an M.S. in Computer Engineering, and an MBA. He also completed the Harvard Business Analytics Program (HBAP). His academic work has included teaching, program leadership, curriculum development, dissertation mentorship, and the development of graduate learning experiences in analytics, data science, artificial intelligence, and research.

A recurring theme in his teaching and scholarship is that technical competence is necessary but insufficient. Data scientists must also learn to question assumptions, evaluate evidence, communicate uncertainty, consider the consequences of analytical decisions, and create work that others can inspect and reproduce.

That perspective informs the philosophy of *Machine Learning Using Python*:

Algorithms + Data + Evidence + Judgment

The Second Edition reflects his commitment to helping students move beyond executing machine-learning libraries toward becoming thoughtful practitioners and researchers who can explain what they did, why they did it, what the evidence supports, and what remains uncertain.

Teaching and Mentorship

Dr. Itauma's approach to graduate education emphasizes analytical reasoning, research literacy, reproducible workflows, responsible use of artificial intelligence, and the connection between computational methods and meaningful questions.

He works with master's and doctoral students across applied analytics and research settings, with particular interest in helping learners progress from:

Learning Methods → Applying Methods → Evaluating Evidence → Producing Knowledge.

Beyond the Classroom

Outside his academic work, Dr. Itauma enjoys pickleball and figure skating. These interests offer a different kind of reminder that development is iterative: technique improves through practice, feedback, reflection, and the willingness to keep learning.

Author Information

Current professional, scholarly, repository, and contact information should be obtained from the official publication and project resources associated with this book.

Citation, Edition, and License

This page identifies the publication status, preferred citation, versioning conventions, repository record, and licensing information for *Machine Learning Using Python*.

Publication Identity

Title: *Machine Learning Using Python*

Subtitle: *Foundations, Algorithms, Applications, and Reproducible Research*

Author: Itauma Itauma

ORCID: [0009-0000-8835-5292](https://orcid.org/0009-0000-8835-5292)

Edition: Second Edition

Version: 2.0.0

Publication date: August 28, 2026

Publication format: Quarto book and reproducible computational repository

Canonical DOI: [10.5281/zenodo.22136083](https://doi.org/10.5281/zenodo.22136083)

The Second Edition contains **18 chapters organized into six parts**, together with front matter, teaching resources, author information, references, and publication metadata.

Preferred Citation

Please cite the archived Second Edition as:

Itauma, I. (2026). *Machine Learning Using Python: Foundations, Algorithms, Applications, and Reproducible Research* (2nd ed., Version 2.0.0). Zenodo. <https://doi.org/10.5281/zenodo.22136083>

The DOI identifies the preserved Second Edition release and should be preferred when citing the book as a scholarly object.

Versioning

The book uses versioned releases so that readers can distinguish the evolving repository from a fixed scholarly release.

The publication record distinguishes among:

- the **edition** of the book;
- the **version** of a specific release;
- the **Git tag** associated with that release;
- the **archival DOI** assigned to the preserved release.

These identifiers serve different purposes.

The **Second Edition** identifies the intellectual edition of the book. Version **2.0.0** identifies this release. The Git tag **v2.0.0** identifies the corresponding repository snapshot. The DOI identifies the archived scholarly object.

The release relationship is:

Second Edition → Version 2.0.0 → Git tag v2.0.0 → Archived Release → DOI 10.5281/zenodo.22136083.

Repository and Reproducibility

The source files, executable examples, environment information, and reproducibility infrastructure for the Second Edition are maintained with the book repository:

<https://github.com/amightyo/machine-learning-using-python-2e>

The browsable HTML edition is available at:

<https://amightyo.github.io/machine-learning-using-python-2e/>

The repository is intended to support:

- transparent inspection of the source;
- reproducible execution of examples;
- version history;
- issue tracking and corrections;
- preservation of release-specific materials.

Readers reproducing computational results should consult the environment and reproducibility documentation associated with the release they are using.

Archival DOI

The canonical DOI for the Second Edition, Version 2.0.0, is:

DOI: [10.5281/zenodo.22136083](https://doi.org/10.5281/zenodo.22136083)

This DOI is the preferred persistent identifier for the archived Second Edition.

License

The book separates the licensing of original educational content from any third-party materials that may appear through citation, quotation, datasets, software packages, documentation, or other attributed resources.

Book prose, original figures, exercises, and other non-code educational content are licensed under the **Creative Commons Attribution 4.0 International (CC BY 4.0)** license. Original executable source code is licensed under the **MIT License**, as documented in the repository license file.

Third-party materials remain subject to their respective licenses, terms, and attribution requirements.

How to Cite Code or a Specific Release

When reproducibility requires identifying the exact computational state used for an analysis, cite the archived release or version corresponding to that state rather than citing only the continuously updated repository.

A reproducible citation record should make it possible to identify:

1. the book edition;
2. the release version;
3. the repository snapshot or Git tag;
4. the archival DOI.

This distinction matters because computational dependencies, examples, corrections, and supporting materials can evolve after publication.

Corrections and Updates

Corrections made after this archival release should be documented through subsequent repository history and, when appropriate, a new versioned release.

A corrected release should preserve the scholarly record rather than silently replacing the state associated with an earlier archived version.

Citation Principle

The purpose of citation metadata is not merely formatting.

It is provenance.

A Scholarly Artifact Should Be Identifiable, Recoverable, and Verifiable
--

The publication record for the Second Edition connects the rendered book, source repository, versioned release, and archival DOI.

References

- Bengio, Yoshua, Aaron Courville, and Pascal Vincent. 2013. “Representation Learning: A Review and New Perspectives.” *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35 (8): 1798–828. <https://doi.org/10.1109/TPAMI.2013.50>.
- Bommasani, Rishi et al. 2021. “On the Opportunities and Risks of Foundation Models.” *arXiv Preprint arXiv:2108.07258*.
- Breiman, Leo. 1996. “Bagging Predictors.” *Machine Learning* 24 (2): 123–40. <https://doi.org/10.1007/BF00058655>.
- . 2001. “Random Forests.” *Machine Learning* 45 (1): 5–32. <https://doi.org/10.1023/A:1010933404324>.
- Breiman, Leo, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. 1984. *Classification and Regression Trees*. Belmont, CA: Wadsworth International Group.
- Brier, Glenn W. 1950. “Verification of Forecasts Expressed in Terms of Probability.” *Monthly Weather Review* 78 (1): 1–3. [https://doi.org/10.1175/1520-0493\(1950\)078%3C0001:VOFEIT%3E2.0.CO;2](https://doi.org/10.1175/1520-0493(1950)078%3C0001:VOFEIT%3E2.0.CO;2).
- Brown, Tom B., Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, et al. 2020. “Language Models Are Few-Shot Learners.” In *Advances in Neural Information Processing Systems*, 33:1877–1901.
- Chen, Tianqi, and Carlos Guestrin. 2016. “XGBoost: A Scalable Tree Boosting System.” In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–94. Association for Computing Machinery. <https://doi.org/10.1145/2939672.2939785>.
- Cortes, Corinna, and Vladimir Vapnik. 1995. “Support-Vector Networks.” *Machine Learning* 20: 273–97. <https://doi.org/10.1007/BF00994018>.
- Cronbach, Lee J., and Paul E. Meehl. 1955. “Construct Validity in Psychological Tests.” *Psychological Bulletin* 52 (4): 281–302. <https://doi.org/10.1037/h0040957>.
- Demšar, Janez. 2006. “Statistical Comparisons of Classifiers over Multiple Data Sets.” *Journal of Machine Learning Research* 7: 1–30.
- Devlin, Jacob, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. “BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding.” In *Proceedings of NAACL-HLT*.
- Dwork, Cynthia, and Aaron Roth. 2014. *The Algorithmic Foundations of Differential Privacy*. Now Publishers.
- Fawcett, Tom. 2006. “An Introduction to ROC Analysis.” *Pattern Recognition Letters* 27 (8): 861–74. <https://doi.org/10.1016/j.patrec.2005.10.010>.

- Freund, Yoav, and Robert E. Schapire. 1997. “A Decision-Theoretic Generalization of on-Line Learning and an Application to Boosting.” *Journal of Computer and System Sciences* 55 (1): 119–39. <https://doi.org/10.1006/jcss.1997.1504>.
- Friedman, Jerome H. 2001. “Greedy Function Approximation: A Gradient Boosting Machine.” *The Annals of Statistics* 29 (5): 1189–1232. <https://doi.org/10.1214/aos/1013203451>.
- Gebru, Timnit, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daume III, and Kate Crawford. 2021. “Datasheets for Datasets.” *Communications of the ACM* 64 (12): 86–92.
- Gentleman, Robert, and Duncan Temple Lang. 2007. “Statistical Analyses and Reproducible Research.” *Journal of Computational and Graphical Statistics* 16 (1): 1–23. <https://doi.org/10.1198/106186007X178663>.
- Glorot, Xavier, and Yoshua Bengio. 2010. “Understanding the Difficulty of Training Deep Feedforward Neural Networks.” *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*.
- Goodfellow, Ian J., Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2014. “Generative Adversarial Nets.” In *Advances in Neural Information Processing Systems*. Vol. 27.
- Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. 2016. *Deep Learning*. Cambridge, MA: MIT Press.
- Goodman, Steven N., Daniele Fanelli, and John P. A. Ioannidis. 2016. “What Does Research Reproducibility Mean?” *Science Translational Medicine* 8 (341): 341ps12. <https://doi.org/10.1126/scitranslmed.aaf5027>.
- Guo, Chuan, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. “On Calibration of Modern Neural Networks.” In *Proceedings of the 34th International Conference on Machine Learning*, 70:1321–30. Proceedings of Machine Learning Research.
- Hardt, Moritz, Eric Price, and Nati Srebro. 2016. “Equality of Opportunity in Supervised Learning.” In *Advances in Neural Information Processing Systems*. Vol. 29.
- Harris, Charles R., K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, et al. 2020. “Array Programming with NumPy.” *Nature* 585: 357–62. <https://doi.org/10.1038/s41586-020-2649-2>.
- Hernán, Miguel A., and James M. Robins. 2020. *Causal Inference: What If*. Boca Raton, FL: Chapman & Hall/CRC.
- Ho, Jonathan, Ajay Jain, and Pieter Abbeel. 2020. “Denoising Diffusion Probabilistic Models.” In *Advances in Neural Information Processing Systems*, 33:6840–51.
- Hoerl, Arthur E., and Robert W. Kennard. 1970. “Ridge Regression: Biased Estimation for Nonorthogonal Problems.” *Technometrics* 12 (1): 55–67. <https://doi.org/10.1080/00401706.1970.10488634>.
- Holtzman, Ari, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. 2020. “The Curious Case of Neural Text Degeneration.” In *International Conference on Learning Representations*.
- Hotelling, Harold. 1933. “Analysis of a Complex of Statistical Variables into Principal Components.” *Journal of Educational Psychology* 24 (6): 417–41. <https://doi.org/10.1037/h0071325>.
- Hunter, John D. 2007. “Matplotlib: A 2D Graphics Environment.” *Computing in Science &*

- Engineering* 9 (3): 90–95. <https://doi.org/10.1109/MCSE.2007.55>.
- Jacobs, Abigail Z., and Hanna Wallach. 2021. “Measurement and Fairness.” In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*, 375–85. <https://doi.org/10.1145/3442188.3445901>.
- Kapoor, Sayash, and Arvind Narayanan. 2023. “Leakage and the Reproducibility Crisis in Machine-Learning-Based Science.” *Patterns* 4 (9): 100804. <https://doi.org/10.1016/j.patter.2023.100804>.
- Ke, Guolin, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. “LightGBM: A Highly Efficient Gradient Boosting Decision Tree.” In *Advances in Neural Information Processing Systems*. Vol. 30.
- Kingma, Diederik P., and Max Welling. 2014. “Auto-Encoding Variational Bayes.” In *International Conference on Learning Representations*.
- Kleinberg, Jon, Sendhil Mullainathan, and Manish Raghavan. 2017. “Inherent Trade-Offs in the Fair Determination of Risk Scores.” In *Proceedings of Innovations in Theoretical Computer Science*.
- LeCun, Yann, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. “Gradient-Based Learning Applied to Document Recognition.” *Proceedings of the IEEE* 86 (11): 2278–2324. <https://doi.org/10.1109/5.726791>.
- Lewis, Patrick, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, et al. 2020. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” *Advances in Neural Information Processing Systems* 33.
- Maaten, Laurens van der, and Geoffrey Hinton. 2008. “Visualizing Data Using t-SNE.” *Journal of Machine Learning Research* 9: 2579–2605.
- MacQueen, James. 1967. “Some Methods for Classification and Analysis of Multivariate Observations.” In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, 1:281–97. Berkeley, CA: University of California Press.
- McInnes, Leland, John Healy, and James Melville. 2018. “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction.” *arXiv Preprint arXiv:1802.03426*. <https://doi.org/10.48550/arXiv.1802.03426>.
- Mikolov, Tomas, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. “Efficient Estimation of Word Representations in Vector Space.” *arXiv Preprint arXiv:1301.3781*.
- Mitchell, Margaret, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. 2019. “Model Cards for Model Reporting.” *Proceedings of the Conference on Fairness, Accountability, and Transparency*.
- Mitchell, Tom M. 1997. *Machine Learning*. McGraw-Hill.
- National Academies of Sciences, Engineering, and Medicine. 2019. *Reproducibility and Replicability in Science*. The National Academies Press.
- National Center for Education Statistics. 2026. “About IPEDS.” U.S. Department of Education. <https://nces.ed.gov/ipeds/about-ipeds>.
- National Center for Health Statistics. 2024. “About NHANES.” Centers for Disease Control; Prevention. <https://www.cdc.gov/nchs/nhanes/about/index.html>.
- National Institute of Standards and Technology. 2023. “Artificial Intelligence Risk Management

- Framework (AI RMF 1.0).” NIST AI 100-1. NIST. <https://doi.org/10.6028/NIST.AI.100-1>.
- Niculescu-Mizil, Alexandru, and Rich Caruana. 2005. “Predicting Good Probabilities with Supervised Learning.” In *Proceedings of the 22nd International Conference on Machine Learning*, 625–32. Association for Computing Machinery. <https://doi.org/10.1145/1102351.1102430>.
- Nosek, Brian A., George Alter, George C. Banks, et al. 2015. “Promoting an Open Research Culture.” *Science* 348 (6242): 1422–25. <https://doi.org/10.1126/science.aab2374>.
- Nosek, Brian A., Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. 2018. “The Preregistration Revolution.” *Proceedings of the National Academy of Sciences* 115 (11): 2600–2606.
- NumPy Developers. 2026. “Linear Algebra.” <https://numpy.org/doc/stable/reference/routines.linalg.html>.
- pandas development team. 2026. “Pandas User Guide.” https://pandas.pydata.org/docs/user_guide/index.html.
- Pearson, Karl. 1901. “On Lines and Planes of Closest Fit to Systems of Points in Space.” *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science* 2 (11): 559–72. <https://doi.org/10.1080/14786440109462720>.
- Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, et al. 2011. “Scikit-Learn: Machine Learning in Python.” *Journal of Machine Learning Research* 12 (85): 2825–30.
- Peng, Roger D. 2011. “Reproducible Research in Computational Science.” *Science* 334 (6060): 1226–27.
- Prokhorenkova, Liudmila, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. 2018. “CatBoost: Unbiased Boosting with Categorical Features.” In *Advances in Neural Information Processing Systems*. Vol. 31.
- Python Software Foundation. 2026. “Virtual Environments and Packages.” <https://docs.python.org/3/tutorial/venv.html>.
- Quarto. 2026. “Using Python.” <https://quarto.org/docs/computations/python.html>.
- Radford, Alec, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, et al. 2021. “Learning Transferable Visual Models from Natural Language Supervision.” In *Proceedings of the 38th International Conference on Machine Learning*, 139:8748–63. Proceedings of Machine Learning Research.
- Recht, Benjamin, Rebecca Roelofs, Ludwig Schmidt, and Vaishaal Shankar. 2019. “Do ImageNet Classifiers Generalize to ImageNet?” *Proceedings of the 36th International Conference on Machine Learning*.
- Rousseeuw, Peter J. 1987. “Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis.” *Journal of Computational and Applied Mathematics* 20: 53–65. [https://doi.org/10.1016/0377-0427\(87\)90125-7](https://doi.org/10.1016/0377-0427(87)90125-7).
- Rubin, Donald B. 1976. “Inference and Missing Data.” *Biometrika* 63 (3): 581–92. <https://doi.org/10.1093/biomet/63.3.581>.
- Rudin, Cynthia. 2019. “Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead.” *Nature Machine Intelligence* 1: 206–15. <https://doi.org/10.1038/s42256-019-0048-x>.

- Rumelhart, David E., Geoffrey E. Hinton, and Ronald J. Williams. 1986. “Learning Representations by Back-Propagating Errors.” *Nature* 323: 533–36. <https://doi.org/10.1038/323533a0>.
- Saito, Takaya, and Marc Rehmsmeier. 2015. “The Precision-Recall Plot Is More Informative Than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets.” *PLOS ONE* 10 (3): e0118432. <https://doi.org/10.1371/journal.pone.0118432>.
- Salton, Gerard, and Christopher Buckley. 1988. “Term-Weighting Approaches in Automatic Text Retrieval.” *Information Processing & Management* 24 (5): 513–23. [https://doi.org/10.1016/0306-4573\(88\)90021-0](https://doi.org/10.1016/0306-4573(88)90021-0).
- Sambasivan, Nithya, Shivani Kapania, Hannah Highfill, Diana Akrong, Praveen Paritosh, and Lora M. Aroyo. 2021. “Everyone Wants to Do the Model Work, Not the Data Work: Data Cascades in High-Stakes AI.” In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery.
- Sandve, Geir Kjetil, Anton Nekrutenko, James Taylor, and Eivind Hovig. 2013. “Ten Simple Rules for Reproducible Computational Research.” *PLoS Computational Biology* 9 (10).
- Schölkopf, Bernhard, and Alexander J. Smola. 2002. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. Cambridge, MA: MIT Press.
- scikit-learn developers. 2026a. “Decision Trees.” <https://scikit-learn.org/stable/modules/tree.html>.
- . 2026b. “Ensemble Methods.” <https://scikit-learn.org/stable/modules/ensemble.html>.
- . 2026c. “Forests of Randomized Trees.” <https://scikit-learn.org/stable/modules/ensemble.html#forest>.
- . 2026d. “Linear Models.” https://scikit-learn.org/stable/modules/linear_model.html.
- . 2026e. “StandardScaler.” <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>.
- . 2026f. “Support Vector Machines.” <https://scikit-learn.org/stable/modules/svm.html>.
- SciPy Community. 2026. “Statistics (Scipy.stats).” <https://docs.scipy.org/doc/scipy/tutorial/stats.html>.
- Selbst, Andrew D., Danah Boyd, Sorelle A. Friedler, Suresh Venkatasubramanian, and Janet Vertesi. 2019. “Fairness and Abstraction in Sociotechnical Systems.” *Proceedings of the Conference on Fairness, Accountability, and Transparency*.
- Shmueli, Galit. 2010. “To Explain or to Predict?” *Statistical Science* 25 (3): 289–310. <https://doi.org/10.1214/10-STS330>.
- Srivastava, Nitish, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. 2014. “Dropout: A Simple Way to Prevent Neural Networks from Overfitting.” *Journal of Machine Learning Research* 15: 1929–58.
- statsmodels developers. 2026. “Ordinary Least Squares.” <https://www.statsmodels.org/stable/regression.html>.
- Steyerberg, Ewout W., Andrew J. Vickers, Nancy R. Cook, Thomas Gerds, Mithat Gonen, Nancy Obuchowski, Michael J. Pencina, and Michael W. Kattan. 2010. “Assessing the Performance of Prediction Models: A Framework for Traditional and Novel Measures.” *Epidemiology* 21 (1): 128–38.
- Sutton, Richard S., and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press.

- Szeliski, Richard. 2022. *Computer Vision: Algorithms and Applications*. 2nd ed. Cham: Springer. <https://doi.org/10.1007/978-3-030-34372-9>.
- Tibshirani, Robert. 1996. “Regression Shrinkage and Selection via the Lasso.” *Journal of the Royal Statistical Society: Series B (Methodological)* 58 (1): 267–88. <https://doi.org/10.1111/j.2517-6161.1996.tb02080.x>.
- UMAP Developers. 2026. “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction.” <https://umap-learn.readthedocs.io/>.
- Varma, Sudhir, and Richard Simon. 2006. “Bias in Error Estimation When Using Cross-Validation for Model Selection.” *BMC Bioinformatics* 7 (91).
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. “Attention Is All You Need.” *Advances in Neural Information Processing Systems* 30.
- Virtanen, Pauli, Ralf Gommers, Travis E. Oliphant, et al. 2020. “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python.” *Nature Methods* 17 (3): 261–72. <https://doi.org/10.1038/s41592-019-0686-2>.
- Wilkinson, Mark D., Michel Dumontier, IJsbrand Jan Aalbersberg, et al. 2016. “The FAIR Guiding Principles for Scientific Data Management and Stewardship.” *Scientific Data* 3: 160018. <https://doi.org/10.1038/sdata.2016.18>.
- Wilson, Greg, Jennifer Bryan, Karen Cranston, Justin Kitzes, Lex Nederbragt, and Tracy K. Teal. 2017. “Good Enough Practices in Scientific Computing.” *PLoS Computational Biology* 13 (6).
- Wolpert, David H. 1992. “Stacked Generalization.” *Neural Networks* 5 (2): 241–59. [https://doi.org/10.1016/S0893-6080\(05\)80023-1](https://doi.org/10.1016/S0893-6080(05)80023-1).